Introduction to Machine Learning and Data Science

Instructor: Tan Bui
Fall · The University of Texas at Austin · COE 371

Scribe: Tan Bui providing last year handwritten lecture notes, providing instructions for Claude, and revising Claude's outputs.

Collaborator: Claude drafting the latex document following guidance from the instructor.

tanbui@oden.utexas.edu
Website: users.oden.utexas.edu/~tanbui

Contents

I Deterministic Foundations 8

Chapter 1 The Learning Problem Page 9

1.1 Three regimes of learning 9
1.1.1 Supervised learning 9
1.1.2 Unsupervised learning 10
1.1.3 Reinforcement learning 10
1.2 The supervised learning problem, formally 11
1.2.1 Targets: regression versus classification 12
1.2.2 Feature maps 12
1.3 The three ingredients 13
1.4 Exercises 14

Chapter 2 Optimization Foundations Page 16

2.1 The objective function 16
2.2 Minimization on the real line 17
2.2.1 Why the derivative points the wrong way 18
2.3 Minimization in $\mathbb{R}^d$ 20
2.3.1 Second-order conditions 21
2.4 Exercises 22

Chapter 3 Linear and Polynomial Regression Page 25

3.1 The model class and the loss 25
3.2 The gradient, three ways 26
3.2.1 Componentwise 26
3.2.2 Two matrix identities 26
3.2.3 Matrix form 27
3.3 The normal equations 27
3.3.1 Certifying that this is a minimum 27
3.4 Polynomial regression 28
3.5 Exercises 29

Chapter 4 Gradient Descent Page 31

4.1 The steepest descent direction 31
4.2 The step size 32
4.2.1 Backtracking 32
4.3 The algorithm 33
4.3.1 Stopping criteria 33

4.4 What the iteration converges to 33
4.4.1 Zig-zagging, and why steepest descent is slow 34
4.4.2 Dependence on the initial guess 34
4.5 Momentum 35
4.6 Exercises 37

II Probability and Statistical Learning 39

Chapter 5 Probability Page 40
5.1 Why probability 40
5.2 The probability space 41
5.3 Combining events 42
5.3.1 The sum rule 42
5.3.2 The union bound 42
5.3.3 Conditional probability and independence 43
5.4 Random variables 45
5.5 Expectation, variance, covariance 46
5.6 The law of large numbers 48
5.6.1 How fast? The Monte Carlo rate 50
5.7 Exercises 51

Chapter 6 Generalization and Regularization Page 53
6.1 The data distribution 53
6.2 What makes a model good 54
6.3 Why supervised learning works 54
6.4 Overfitting and underfitting 56
6.5 Regularization 58
6.5.1 Ridge regression: L_2 regularization 59
6.6 The constrained formulation 60
6.7 Lasso: L_1 regularization and sparsity 60
6.7.1 Why L_1 produces sparsity 61
6.8 Choosing λ 62
6.8.1 The validation set 62
6.8.2 The L-curve, as a diagnostic 63
6.9 Exercises 64

Chapter 7 The Probabilistic View Page 68
7.1 The learning problem, as empirical risk minimization 68
7.2 Learning the distribution instead 69
7.2.1 Distributions over data 69
7.2.2 The three ingredients, restated 70
7.3 Monte Carlo estimation 70
7.4 Kullback–Leibler divergence 71
7.5 From KL to maximum likelihood 72
7.6 Generative versus predictive modeling 73
7.6.1 Conditional maximum likelihood 73

7.7 Conditional maximum likelihood recovers the L_2 loss 74
7.8 Bayesian inference 75
7.9 MAP estimation and ridge regression 75
7.10 The full Bayesian predictive model 76
7.11 Exercises 77

Chapter 8 Classification Page 79

8.1 Why linear regression is the wrong tool 79
8.2 The logistic function 80
8.3 The logistic model 81
8.3.1 Cross-entropy loss 81
8.4 Prediction and the decision boundary 82
8.5 Multi-class classification and softmax 84
8.5.1 Identifiability 84
8.5.2 Decision boundaries 84
8.6 Exercises 84

Chapter 9 Generative Classification: Gaussian Discriminant Analysis Page 87

9.1 The postulates 88
9.2 The induced marginal is a Gaussian mixture 89
9.3 Maximum likelihood 89
9.3.1 The mixture weights 90
9.3.2 The Gaussian parameters 90
9.4 Shared or separate covariances 90
9.5 The unlabeled case: expectation–maximization 92
9.5.1 Why the closed forms disappear 92
9.5.2 Responsibilities 92
9.5.3 The algorithm 93
9.5.4 Why it works 93
9.6 Exercises 94

Chapter 10 Stochastic Gradient Descent Page 97

10.1 The objective, and what it approximates 97
10.2 The cost of an exact gradient 97
10.3 The mini-batch gradient 98
10.4 The algorithm 100
10.5 What the iterates converge to 100
10.6 Mini-batching as a cost–variance trade 103
10.7 Adaptive step sizes 104
10.7.1 RMSProp 104
10.7.2 Adam 106
10.7.3 How these differ from what came before 107
10.8 What this chapter does not cover 108
10.9 Exercises 108

III Deep Learning 112

Chapter 11 Neural Networks: What They Can Represent Page 113
11.1 The neuron and the network 113
11.1.1 Layers 114
11.2 ReLU networks represent piecewise-linear functions exactly 115
11.2.1 Building blocks 115
11.2.2 The general construction 115
11.3 Piecewise-linear interpolation 116
11.4 The error bound 117
11.5 How many neurons? 118
11.6 Exercises 119

Chapter 12 Training Deep Networks Page 121
12.1 Backpropagation 121
12.2 Vanishing and exploding gradients 123
12.3 ResNet 124
12.3.1 From ResNet to neural ODE 124
12.3.2 Stability, and what it forbids 125
12.3.3 Other discretizations, other architectures 126
12.3.4 The adjoint equation, or backpropagation in the limit 126
12.4 Convolutional networks 127
12.4.1 What weight sharing buys 127
12.4.2 A filter is a stencil 128
12.4.3 Equivariance, and what pooling does to it 128
12.4.4 Depth and the receptive field 129
12.5 Recurrent networks 129
12.5.1 Long short-term memory 130
12.6 Exercises 130

IV Dimension Reduction and Generative Models 133

Chapter 13 Constrained Optimization, PCA, and Reduced-Order Modeling Page 134
13.1 Lagrange multipliers 134
13.2 PCA: the first principal direction 135
13.3 PCA: r principal directions 137
13.4 Feature extraction and storage 137
13.5 Singular value decomposition 138
13.6 Proper orthogonal decomposition and reduced-order modeling 139
13.7 Autoencoders 141
13.8 Exercises 142

Chapter 14 Generative Models Page 145
14.1 From autoencoder to VAE 145
14.2 The evidence lower bound 146

14.3 Training the VAE 147
14.3.1 The reparameterization trick 147
14.3.2 Generation 148
14.4 Hierarchical VAE 148
14.5 Variational diffusion models 148
14.5.1 Decomposing the ELBO 149
14.6 What the network actually learns 150
14.6.1 The KL reduces to matching means 150
14.6.2 Three equivalent objectives 150
14.7 Exercises 151

Chapter A Linear Algebra and Matrix Calculus Reference Page 154

Chapter B A JAX Primer Page 156
B.1 Optimizers with `optax` 157

Chapter C The Computing Environment: Google Colab Page 158
C.1 Why Colab, and where it falls short 158
C.2 A reproducible first cell 159
C.3 Precision: the trap in this course 159
C.4 When you need an accelerator 159
C.5 Getting the course datasets 160
C.6 Reproducibility 161
C.7 Working locally instead 162
C.8 What to submit 162

These notes accompany COE 371, *Introduction to Machine Learning and Data Science*, at the University of Texas at Austin.

The course takes a deliberately mathematical route through machine learning. Every method is introduced through the same three-part structure — a model class, an objective function, and an optimizer — and every algorithm is derived rather than asserted. Where a result is stated without proof, this is flagged explicitly and usually left as an exercise.

The notes are organized into four parts:

Part I: Foundations. The supervised learning problem, the optimization theory needed to solve it, linear and polynomial regression in closed form, regularization, and gradient descent.

Part II: The Probabilistic View. The same problems recast in terms of probability distributions: maximum likelihood, Bayesian inference, classification, and mixture models.

Part III: Deep Learning. What neural networks can represent, and how they are trained.

Part IV: Dimension Reduction and Generative Models. Constrained optimization, PCA, proper orthogonal decomposition and reduced-order modeling, autoencoders, variational autoencoders, and diffusion models.

Two streams of exercises run through the notes. *Exercises* are mathematical: proofs, derivations, and counterexamples. *Programming Exercises* are computational, written in JAX and intended to be run in Google Colab against real data. The two are meant to be done together — several programming exercises exist specifically to make a theoretical result visible on a screen.

A note on notation

The notation is fixed once and used consistently throughout:

Note:-

Why the parentheses. The sample index is written $\boldsymbol{x}^{(i)}$ rather than $\boldsymbol{x}^i$ because the superscript slot is also where *powers* live, and the two meet constantly: the polynomial feature map of Chapter 3 sends a sample $x^{(i)}$ to $\left(1, x^{(i)}, (x^{(i)})^2, \dots\right)$. Parentheses make the reading unambiguous. Bare superscripts elsewhere are *not* sample indices: they denote gradient-descent iterates $\boldsymbol{\theta}^j$ (Chapter 4), layers $\boldsymbol{y}^\ell$ (Chapter 12), and diffusion timesteps appear as subscripts $\boldsymbol{x}_t$ (Chapter 14).

Symbol	Meaning
$\boldsymbol{x}^{(i)}$	the i-th data sample (parenthesized superscript)
x_j	the j-th component of a vector (subscript indexes components)
$x_j^{(i)}$	component j of sample i; its k-th power is $\left(x_j^{(i)}\right)^k$
n	number of samples
d	number of features; parameters lie in $\mathbb{R}^{d+1}$ with $x_0 = 1$
$\mathcal{D}$	the dataset $\{(\boldsymbol{x}^{(i)}, y^{(i)})\}_{i=1}^n$
$\boldsymbol{\varphi}$	feature map, $\boldsymbol{\varphi} : \mathbb{R}^d \to \mathbb{R}^p$
$\boldsymbol{\Phi}$	design matrix, with i-th row $\boldsymbol{\varphi}(\boldsymbol{x}^{(i)})^\mathsf{T}$
$\mathcal{M}$	model class
$\boldsymbol{\theta} \in \Theta$	parameters
$J(\boldsymbol{\theta})$	objective (loss) function
λ	regularization weight

The L_2 loss carries the prefactor $\frac{1}{2n}$ everywhere; the factor of $\frac{1}{2}$ is there so that differentiation does not leave a stray 2 behind, and the $\frac{1}{n}$ so that the loss does not grow with the size of the dataset.

Part I

Deterministic Foundations

Chapter 1

The Learning Problem

Idea

In traditional programming we supply inputs together with the *rules* that act on them, and the machine returns answers. Machine learning inverts this: we supply inputs together with the *answers*, and the machine returns the rules. ms. p.1

Everything in these notes follows from taking that inversion seriously. If the rules are what we are solving for, then the rules must be the unknowns of some problem — and the bulk of this course is about what that problem is and how to solve it.

§ 1.1 Three regimes of learning

1.1.1 Supervised learning

We are given a dataset of *labeled* training data

$$\mathcal{D} = \{(\boldsymbol{x}^{(i)}, y^{(i)})\}_{i=1}^{n},$$

train a model to produce accurate predictions on this dataset, and require that the model remain accurate when it is shown new, similar data. Written as a procedure:

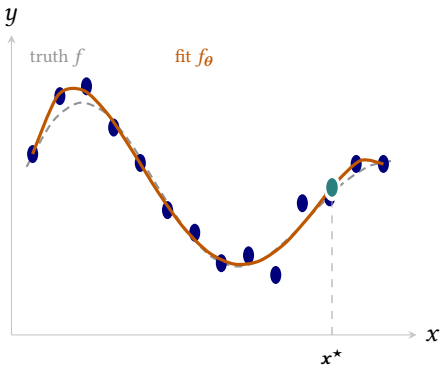
(i) collect $\mathcal{D} = \{(\boldsymbol{x}^{(i)}, y^{(i)})\}_{i=1}^{n}$, where $y^{(i)}$ is the label;

(ii) train f so that $y \approx f(\boldsymbol{x})$;

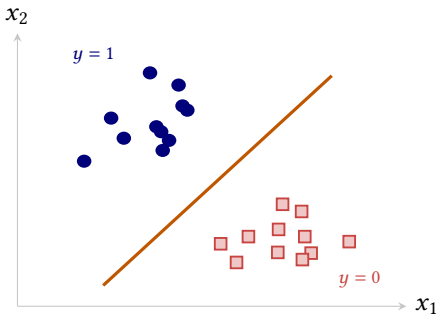
(iii) given new input $\boldsymbol{x}^{\star}$, predict $y^{\star} = f(\boldsymbol{x}^{\star})$.

Applications include medical diagnostics, language translation, and autonomous driving. ms. p.1

Let us see this geometrically. Each data point arrives with its answer attached, so we can see how far any candidate rule is from the truth. Fitting means choosing the curve that sits closest to the points; predicting means reading that curve at a new input.



(a) Regression: y is continuous.



(b) Classification: y is discrete; the orange line is the decision boundary.

Figure 1.1: **Supervised learning.** In both panels every data point (navy, and red in (b)) arrives with its answer attached, so the error of any candidate rule can be measured directly — which is what makes the objective function of Chapter 2 possible. **(a)** We never see the true function f (gray, dashed); we see only noisy samples of it, and recover a rule f_θ (orange) from those. Evaluating f_θ at an unseen $x^\star$ gives the prediction (teal). Note that f_θ tracks the samples more faithfully than it tracks the truth — it has partly fitted the noise, which is exactly the overfitting problem of Chapter 6. **(b)** When the answer is a label rather than a number, the rule becomes a surface separating the classes instead of a curve through the points.

1.1.2 Unsupervised learning

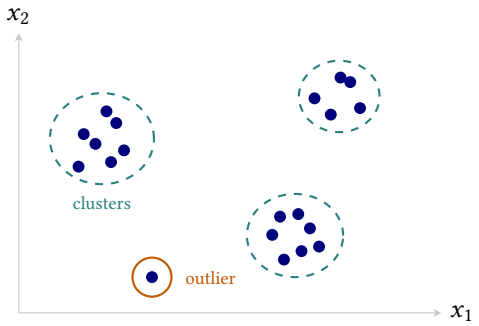
Here the dataset carries no labels,

$$\mathcal{D} = \{x^{(i)}\}_{i=1}^{n},$$

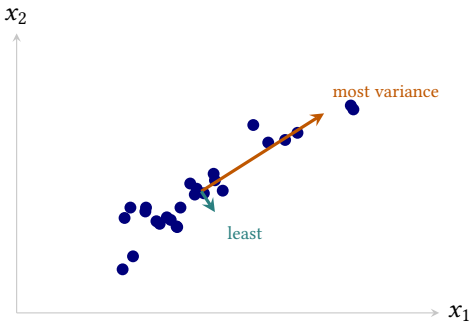
and the task is to find structure that is present but not annotated: clusters hidden in the data, outliers, or a useful signal buried in noise. Applications include recommendation systems, anomaly detection, and signal denoising.

ms. p.1

Let us see this geometrically. Strip the answers away and the picture changes completely. Nothing marks one point as different from another, so there is no error to measure — yet structure is plainly visible, and finding it is the whole task.



(a) Clustering and outlier detection.



(b) Low-dimensional structure.

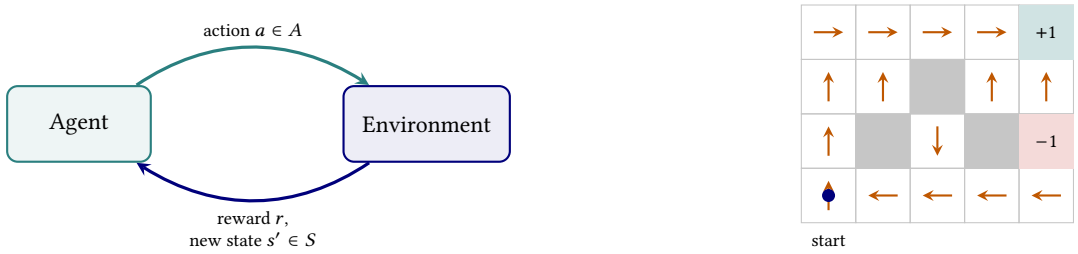
Figure 1.2: **Unsupervised learning.** In both panels every point is the same kind of object — there are no labels to distinguish them, so both coordinates are input. Contrast Figure 1.1a, where the vertical coordinate was an answer supplied to us. **(a)** The three clusters and the outlier are not given; they are what the algorithm must recover. **(b)** A different kind of hidden structure: the cloud is nominally two-dimensional but lies close to a single line, so almost all of its variability is captured by one coordinate rather than two. Finding that line is principal component analysis, and it is the subject of Chapter 13.

1.1.3 Reinforcement learning

An agent interacts with an environment over time and is given rewards. At each step the agent observes a state $s \in S$, takes an action $a \in A$, receives a reward r, and finds itself in a new state $s' \in S$. Applications include the discovery of new drug compounds, games such as Chess and Go, and control problems such as regulating the cooling of a data center.

ms. p.1

Let us see this geometrically. Here there is no fixed dataset at all. The agent's own actions determine what it sees next, so the data is generated by the very rule we are trying to learn — a loop rather than a one-way pipeline.



(a) The interaction loop.

(b) A learned policy on a grid world.

Figure 1.3: **Reinforcement learning. (a)** The agent acts, the environment responds with a reward and a new state, and the cycle repeats. Unlike Figures 1.1 and 1.2 there is no dataset handed over in advance: the agent's own behavior determines which data it ever observes. **(b)** What is learned is a *policy* — an action for every state. Here the states are cells, gray cells are walls, and the two shaded cells carry a reward on entry. The arrows are the policy the agent settles on: from the start it routes around the walls toward +1 while steering clear of −1. Note that no single step is ever labeled correct or incorrect; only the eventual reward is observed, which is why the credit for a good outcome must be propagated back over many steps.

Note:-

These notes concentrate on supervised learning through Chapter 8, turn to unsupervised learning in Chapters 13 and 14, and do not treat reinforcement learning.

§ 1.2 The supervised learning problem, formally

Definition 1.2.1: Model

A model *is a function* $f : \mathcal{X} \to \mathcal{Y}$ *mapping inputs* $\boldsymbol{x} \in \mathcal{X}$ *to targets* $y \in \mathcal{Y}$*. Models are typically parameterized by* $\boldsymbol{\theta} \in \Theta$*, and we write* $f_{\boldsymbol{\theta}}$*.*

Definition 1.2.2: Model class

The model class *is the set of models under consideration,*

$$\mathcal{M} \coloneqq \{ f \mid f : \mathcal{X} \to \mathcal{Y},\ \boldsymbol{\theta} \in \Theta \},$$

where Θ *is some parameter set.* *ms. p.2*

Example 1.2.1 (Affine models)

With $\mathcal{X} = \mathbb{R}$, the class $f_{\boldsymbol{\theta}}(x) = \theta_1 x + \theta_0$ is the familiar straight-line model, and $\Theta = \mathbb{R}^2$.

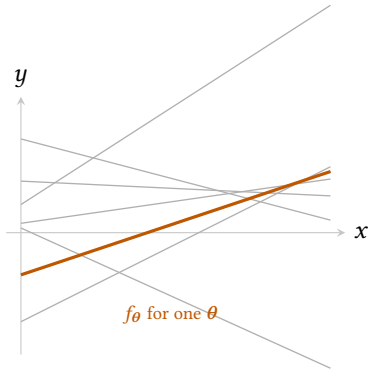
Example 1.2.2 (A trigonometric model class)

$$f_{\boldsymbol{\theta}}(x) = \sum_{i=0}^{d} \theta_i \sin(ix), \qquad \boldsymbol{\theta} \in \mathbb{R}^{d+1}.$$

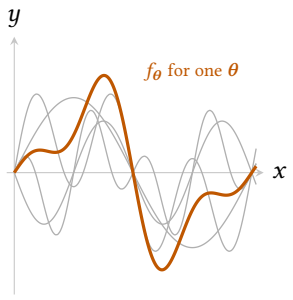
Each choice of $\boldsymbol{\theta}$ gives a different function. Which one should we choose? That question is the subject of Section 1.3. *ms. p.4*

Let us see this geometrically. A model class is a *set* of functions, not a function. Each point of Θ picks out one member. Drawing several members at once is the honest picture: the class is everything faintly drawn, and learning is the act of

choosing one of them.



(a) $f_\theta(x) = \theta_1 x + \theta_0$, so $\Theta = \mathbb{R}^2$.



(b) $f_\theta(x) = \sum_{i=0}^{d} \theta_i \sin(ix)$, so $\Theta = \mathbb{R}^{d+1}$.

Figure 1.4: **Two model classes.** Gray curves are members of the class, one per choice of $\boldsymbol{\theta}$; the orange curve is one particular member. In (a) the class contains every straight line and nothing else — it cannot bend. In (b) the class contains only functions vanishing at $x = 0$ and repeating with period 2π, so it can bend a great deal but can never represent a trend. Neither class is better; each encodes an assumption, and choosing the class is choosing what you are willing to assume.

1.2.1 Targets: regression versus classification

The two panels of Figure 1.1 are already an example of this distinction, and it is worth returning to them now that we have the vocabulary to name it.

Regression. The target y is continuous. We are fitting a curve that approximates the shape of the dataset. This is Figure 1.1a: the target is a height on the vertical axis, and the rule f_θ is a curve passing among the data.

Classification. The target y is discrete. Each value corresponds to a class, and we are looking for a hypersurface — in
ms. p.2 the simplest case a hyperplane — separating the classes. This is Figure 1.1b: the target is one of two labels, and the rule is a line with one class on either side.

Notice what does *not* change between the two panels. Both have labeled data, both fix a model class, and both choose a member of that class by making some measure of misfit small. Only the shape of $\mathcal{Y}$ differs — continuous in one case, a finite set in the other — and that single change is what turns a curve into a boundary.

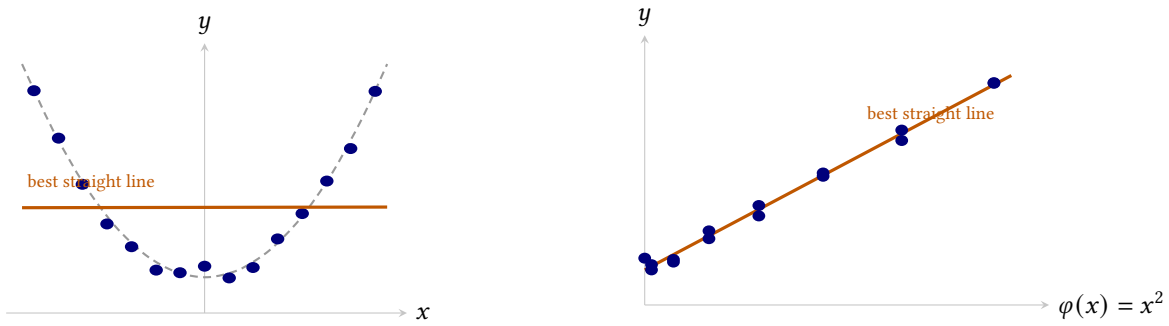
1.2.2 Feature maps

Raw inputs are rarely the right coordinates. A *feature map*

$$\boldsymbol{\varphi} : \mathbb{R}^d \to \mathbb{R}^p, \qquad \boldsymbol{\varphi}(\boldsymbol{x}^{(i)}) = \begin{bmatrix} \varphi_1(\boldsymbol{x}^{(i)}) \\ \varphi_2(\boldsymbol{x}^{(i)}) \\ \vdots \\ \varphi_p(\boldsymbol{x}^{(i)}) \end{bmatrix},$$

re-expresses each sample in a space where the relationship we are looking for is easier to represent — often, as we will
ms. p.2 see in Chapter 3, *linear*.

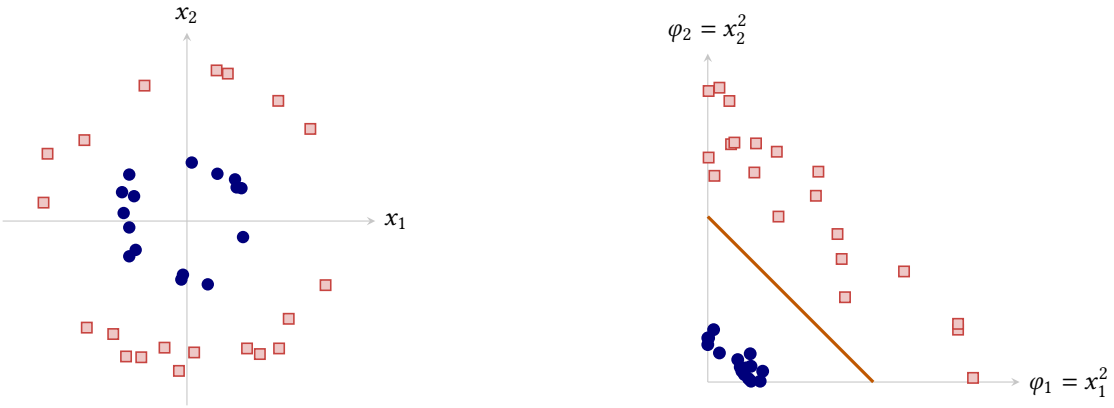
Let us see this geometrically. The claim is worth seeing rather than believing. In both figures below, the data is unchanged between the panels — only the coordinates it is drawn in have changed. A relationship that no straight line could capture on the left becomes a straight line on the right.



(a) Input space. No line fits.

(b) Feature space. A line fits.

Figure 1.5: **A feature map for regression.** The same fifteen points in both panels. On the left the relationship is quadratic and the best straight line (orange) is nearly flat — useless. On the right the horizontal coordinate is $\boldsymbol{\varphi}(x) = x^2$ and the identical data lies on a line. The model $\theta_1 x^2 + \theta_0$ is nonlinear in x but linear in $\boldsymbol{\theta}$, which is the only kind of linearity Chapter 3 requires.



(a) Input space. No line separates them.

(b) Feature space. The line $\varphi_1 + \varphi_2 = c$ separates them.

Figure 1.6: **A feature map for classification.** Two concentric rings, the same points in both panels. No straight line in the input space can separate them. Under $\boldsymbol{\varphi}(\boldsymbol{x}) = (x_1^2, x_2^2)$ each ring maps to a segment of a line $\varphi_1 + \varphi_2 = r^2$, and the two classes fall on opposite sides of the straight boundary drawn in orange. The decision rule is a hyperplane *in feature space*; pulled back to the input space it is a circle.

Key idea

A model that is nonlinear in $\boldsymbol{x}$ may still be linear in $\boldsymbol{\theta}$. Everything in Chapter 3 depends on linearity in the *parameters*, not in the inputs. This is why polynomial regression is a special case of linear regression rather than a departure from it.

§ 1.3 The three ingredients

Every method in these notes is assembled from exactly three parts. ms. p.2

1. **Model class** $\mathcal{M}$: the set of models we are willing to consider.
2. **Objective function** J: a rule defining how good a model is.
3. **Optimizer**: a procedure that finds the best model in $\mathcal{M}$ according to J.

Let us see this geometrically. The three are not a checklist to be worked through in order. They are coupled: the optimizer searches the model class, but it can only tell which way to move because the objective scores what it finds, and the objective can only score anything because the data is there to compare against.

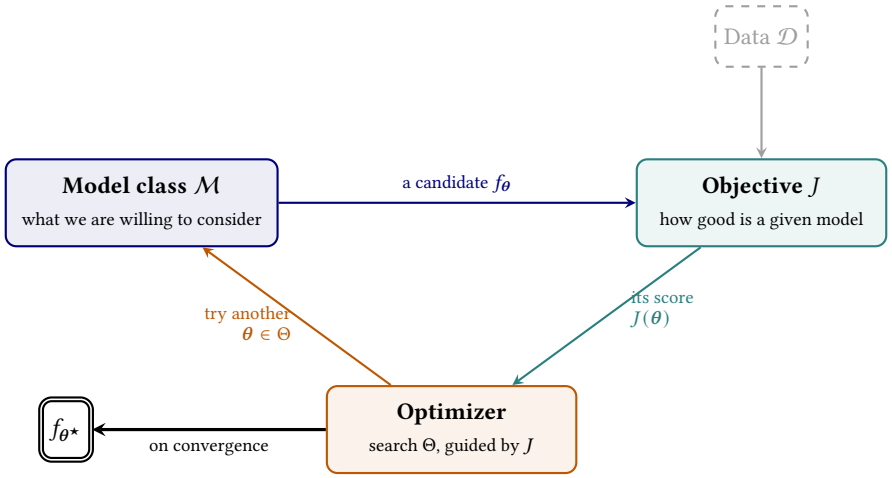


Figure 1.7: **How the three ingredients couple.** The optimizer proposes a parameter, the model class turns it into a function, the objective scores that function against the data, and the score sends the optimizer somewhere new. The loop is the whole of training. Changing any one box changes what the other two are doing — which is why the rest of these notes can be read as a catalog of choices for each: model classes in Chapters 3 and 11, objectives in Chapters 6 and 7, and optimizers in Chapters 4 and 12.

Take-home message

dataset + algorithm ⟶ predictive model

Training is optimization.

Algorithms for training *are* algorithms for minimization. Once the model class and the objective function are fixed, nothing about learning remains except solving a minimization problem — which is why Chapter 2 is about minimization and not about machine learning at all.

ms. p.2

§ 1.4 Exercises

The datasets referenced below are free to download. The two engineering sets are hosted by the UCI Machine Learning Repository and are also available through the `ucimlrepo` package (`pip install ucimlrepo`).

- **Airfoil Self-Noise** (NASA; 1503 samples, 5 features → scaled sound pressure level in dB) — `https://archive.ics.uci.edu/dataset/291/airfoil+self+noise`, or `fetch_ucirepo(id=291)`.
- **Combined Cycle Power Plant** (9568 samples, 4 ambient variables → net hourly electrical output in MW) — `https://archive.ics.uci.edu/dataset/294/combined+cycle+power+plant`, or `fetch_ucirepo(id=294)`.
- **Palmer Penguins** (344 samples, morphological measurements and species labels) — `https://allisonhorst.github.io/palmerpenguins/`.
- **California Housing** — bundled with scikit-learn as `sklearn.datasets.fetch_california_housing`.

Warm-up

1. [Theory] Classify each of the following as supervised, unsupervised, or reinforcement learning, and state what plays the role of x, y, and $\mathcal{D}$ in each: (a) predicting the drag coefficient of an airfoil from its geometry; (b) identifying recurring flow structures in a turbulence simulation; (c) tuning the control inputs of a chemical reactor to maximize yield.

2. [Theory] For each of the four panels 1.1a, 1.1b, 1.2a and 1.2b, write down what $\mathcal{X}$ is, what $\mathcal{Y}$ is (or state that there is none), and how many coordinates of each plotted point are inputs.

3. [Theory] Write down the model class of affine functions on $\mathbb{R}^d$ in the notation of Definition 1.2.2, and state $\dim \Theta$. How does $\dim \Theta$ grow with d?

4. [Theory] Name the three ingredients of Section 1.3 for the following procedure: "among all straight lines, pick the one minimizing the sum of squared vertical distances to the data, by solving a linear system."

5. [Code] Download the Airfoil Self-Noise data and report, in JAX: the shape of $\boldsymbol{X}$ and $\boldsymbol{y}$, the name and physical unit of each feature, and the minimum and maximum of the target. State which of the three regimes of Section 1.1 this dataset supports, and why.

6. [Code] Evaluate the polynomial feature map $\boldsymbol{\varphi}(x) = (1, x, x^2, x^3)^\mathsf{T}$ at $x = 0.5$ and $x = 2$ by hand, then reproduce both with `jnp.vander`. Confirm they agree.

Core

7. [Theory] Let $\boldsymbol{\varphi}(x) = (1, x, x^2, \dots, x^d)^\mathsf{T}$. Show that the model class $\{\boldsymbol{\theta}^\mathsf{T}\boldsymbol{\varphi}(x) : \boldsymbol{\theta} \in \mathbb{R}^{d+1}\}$ contains every polynomial of degree at most d, and that it is a vector space. Is the class $\{\sin(\theta x) : \theta \in \mathbb{R}\}$ a vector space? Justify your answer.

8. [Theory] A model class is *linear in the parameters* if $f_{\boldsymbol{\theta}} = \boldsymbol{\theta}^\mathsf{T}\boldsymbol{\varphi}(\boldsymbol{x})$ for some fixed $\boldsymbol{\varphi}$. Show that polynomial regression is linear in the parameters even though it is nonlinear in x, and explain why this is the property that matters for Chapter 3 — as anticipated by the Key idea in Section 1.2.2.

9. [Theory] Take the relation $y = c\,e^{kx}$ with c, k unknown and positive. Exhibit a transformation of the data after which the relation is linear in the unknowns, and say precisely what the feature map is. What has the transformation done to the noise, if the original data was $y^{(i)} = c\,e^{kx^{(i)}} + \varepsilon^{(i)}$?

10. [Code] Load the Combined Cycle Power Plant data and produce, in JAX: (a) the shape of the data matrix and the target vector; (b) per-feature means and standard deviations; (c) a scatter plot of the target against each of the four features. Which features look as though an affine model would capture them, and which do not?

11. [Code] For the same dataset, standardize every feature to zero mean and unit variance. Explain why this is not a change of model class but a change of feature map, and write down the $\boldsymbol{\varphi}$ that implements it.

12. [Code] Load the Palmer Penguins data, drop rows with missing values, and scatter bill length against flipper length with points colored by species. By eye, could a single straight line separate any one species from the other two? Relate your answer to Figure 1.1b.

Challenge

13. [Theory] Section 1.3 claims that fixing a model class and an objective function leaves nothing but a minimization problem. Give an example of a model class and an objective function for which that minimization has no solution, and one for which it has infinitely many. What does each failure tell you about the model class?

14. [Theory] Reinforcement learning is sometimes described as "supervised learning where the labels arrive late." Argue for and against this description, using the fact that in Figure 1.3b no individual move is ever labeled correct or incorrect.

15. [Theory] Let $\mathcal{X} = \mathbb{R}$ and suppose the true relation is $y = \mathbf{1}[x > a]$ for some unknown threshold a. Show that no model class of the form $\{\boldsymbol{\theta}^\mathsf{T}\boldsymbol{\varphi}(x)\}$ with $\boldsymbol{\varphi}$ continuous and $\dim \boldsymbol{\theta}$ finite can represent this exactly, but that it can be approximated to any accuracy. (You will prove a sharp version of this in Chapter 11.)

16. [Code] Write a JAX function `feature_map(x, d)` returning the polynomial features $(1, x, \dots, x^d)$ for a vector of inputs, using `jax.vmap` rather than a Python loop. Verify against `jnp.vander` and time both for $n = 10^6$, $d = 10$.

17. [Code] For the Airfoil Self-Noise data, build the design matrix $\boldsymbol{\Phi}$ for polynomial feature maps of degree 1 through 8 applied to the frequency column, and compute the condition number of $\boldsymbol{\Phi}$ at each degree. Plot it on a log scale. You have not fitted anything yet — but the plot already predicts trouble, and Chapter 3 explains what kind.

18. [Code] Take the Palmer Penguins measurements, *discard* the species labels, and plot the two measurements you find most informative. Does the cluster structure you see match the species you threw away? This is the difference between Figures 1.1 and 1.2, on one dataset.

Chapter 2

Optimization Foundations

Goal of this chapter

Chapter 1 ended by claiming that once a model class and an objective function are fixed, learning *is* minimization. This chapter supplies the minimization theory that claim rests on, and it answers three questions:

1. What is a minimizer?
2. What conditions must a minimizer necessarily satisfy?
3. What conditions are *enough* to guarantee one?

No data appears anywhere in this chapter. That is deliberate: the results here are facts about functions, and they hold whatever the function came from.

§ 2.1 The objective function

Definition 2.1.1: Objective function

An objective function *(or* loss function*) is a map*

$$J : \mathcal{M} \to [0, \infty)$$

describing how well a model f fits the data $\mathcal{D}$. When f is parameterized by $\theta \in \Theta$, this becomes

$$J : \Theta \to [0, \infty), \qquad \theta \mapsto J(f_\theta) =: J(\theta).$$

ms. p.3

Example 2.1.1 (Mean squared error, or L_2 loss)

$$J(\theta) \coloneqq \frac{1}{2n} \sum_{i=1}^{n} \left(f_\theta(\boldsymbol{x}^{(i)}) - y^{(i)}\right)^2.$$

ms. p.3

Example 2.1.2 (Mean absolute error, or L_1 loss)

$$J(\boldsymbol{\theta}) \coloneqq \frac{1}{n}\sum_{i=1}^{n}\left|f_{\boldsymbol{\theta}}(\boldsymbol{x}^{(i)}) - y^{(i)}\right|.$$

ms. p.3

The learning problem is then

$$\boldsymbol{\theta}^\star = \arg\min_{\boldsymbol{\theta}\in\Theta} J(\boldsymbol{\theta}),$$

and a best model is $f_{\boldsymbol{\theta}^\star}$. ms. p.4

Claim 2.1.1 Scale invariance

For every $\alpha \in \mathbb{R}_+$, $\arg\min_{\boldsymbol{\theta}\in\Theta} J(\boldsymbol{\theta}) = \arg\min_{\boldsymbol{\theta}\in\Theta} \alpha J(\boldsymbol{\theta})$. ms. p.4

Claim 2.1.2 Maximization reduces to minimization

$\arg\max_{\boldsymbol{\theta}\in\Theta} J(\boldsymbol{\theta}) = \arg\min_{\boldsymbol{\theta}\in\Theta}\big(-J(\boldsymbol{\theta})\big)$. *Consequently it suffices to develop learning algorithms for minimization alone.* ms. p.5

Both claims are proved in the exercises.

§ 2.2 Minimization on the real line

Take $J : \mathbb{R} \to [0, \infty)$, $\theta \mapsto J(\theta)$.

Definition 2.2.1: Local minimizer

A point $\theta^\star$ *is a* local minimizer *of* J *if there exists* $\delta > 0$ *such that*

$$J(\theta) \geq J(\theta^\star) \qquad \text{for all } \theta \in (\theta^\star - \delta,\ \theta^\star + \delta).$$

ms. p.5

Let us see this geometrically. The word doing the work in Definition 2.2.1 is *local*. The inequality is only required to hold inside some interval around $\theta^\star$ — however small — and says nothing whatever about what J does elsewhere.

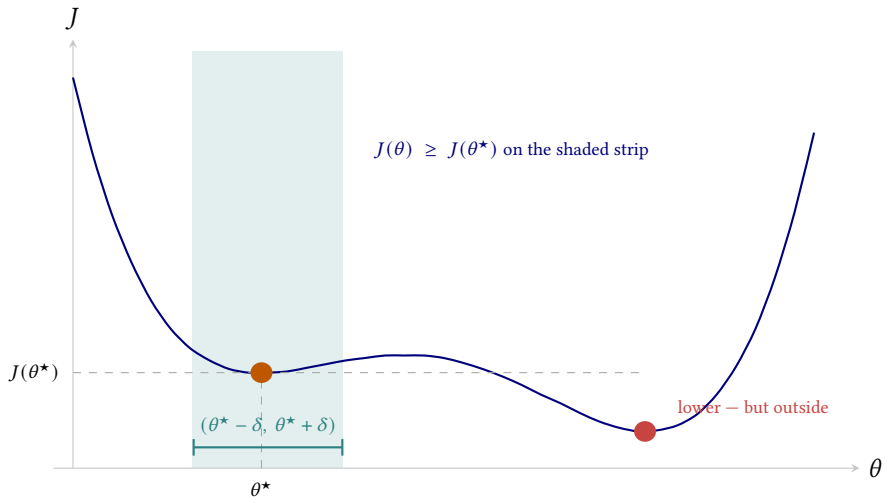


Figure 2.1: **A local minimizer.** Inside the shaded interval no point of the curve falls below the dashed level, so $\theta^\star$ satisfies Definition 2.2.1. The red point is lower, but it lies outside the interval and is therefore irrelevant to the definition: local minimality is a statement about a neighborhood, not about the whole line. Shrinking δ only makes the condition easier to satisfy.

2.2.1 Why the derivative points the wrong way

Fix θ' and take a small step h. Taylor expansion gives

$$J(\theta' + h) = J(\theta') + h\frac{dJ}{d\theta}(\theta') + O(h^2).$$

If $\frac{dJ}{d\theta}(\theta') > 0$ then a positive step increases J and a negative step decreases it; if $\frac{dJ}{d\theta}(\theta') < 0$ the situation reverses. In both cases, stepping *opposite* to the sign of the derivative decreases the objective. Equivalently: choose h such that

$$h\,\frac{dJ}{d\theta}(\theta') < 0 \qquad \Longrightarrow \qquad \text{we go downhill.}$$

ms. p.5 This single observation is the whole of gradient descent, and we return to it in Chapter 4.

Let us see this geometrically. Two points, one on each side of the minimum. At both, the tangent line tells you which way is downhill — and at both, the answer is *against* the sign of the derivative.

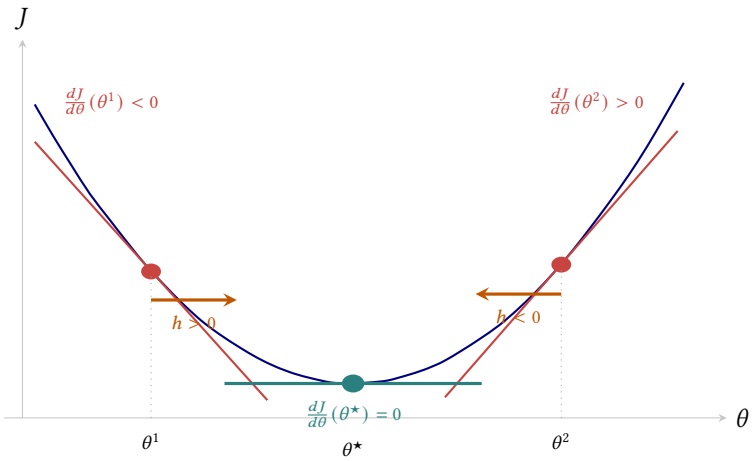


Figure 2.2: **The derivative points uphill; the minimizer is where it vanishes.** At θ^1 the slope is negative, so a *positive* step decreases J; at θ^2 the slope is positive, so a *negative* step does. In both cases the productive step has $h\frac{dJ}{d\theta} < 0$: move against the derivative. Squeezed between the two, the tangent at $\theta^\star$ is horizontal — which is Theorem 2.2.1. The theorem's proof is exactly the observation that if the tangent were *not* horizontal, one of the two orange arrows would still be available, and $\theta^\star$ could not have been a minimizer.

Theorem 2.2.1 First-order necessary condition, one dimension

If $\theta^\star$ is a local minimizer of a differentiable J, then

$$\frac{dJ}{d\theta}(\theta^\star) = 0.$$

Theorem 2.2.2 Second-order necessary condition, one dimension

If $\theta^\star$ is a local minimizer of a twice continuously differentiable J, then in addition to Theorem 2.2.1,

$$\frac{d^2J}{d\theta^2}(\theta^\star) \geq 0.$$

Note the weak inequality.

Proof: By contradiction. Taylor expansion about $\theta^\star$ gives, for ε small,

$$J(\theta^\star + \varepsilon) = J(\theta^\star) + \varepsilon\frac{dJ}{d\theta}(\theta^\star) + \tfrac{1}{2}\varepsilon^2\frac{d^2J}{d\theta^2}(\gamma),$$

with γ between $\theta^\star$ and $\theta^\star + \varepsilon$.

Suppose $\frac{dJ}{d\theta}(\theta^\star) < 0$. For $\varepsilon > 0$ sufficiently small the quadratic remainder is dominated by the linear term,

$$\left|\tfrac{1}{2}\varepsilon^2\frac{d^2J}{d\theta^2}(\gamma)\right| \ll \left|\varepsilon\frac{dJ}{d\theta}(\theta^\star)\right|,$$

so that

$$J(\theta^\star + \varepsilon) \approx J(\theta^\star) + \varepsilon\frac{dJ}{d\theta}(\theta^\star) < J(\theta^\star),$$

contradicting Definition 2.2.1.

The case $\frac{dJ}{d\theta}(\theta^\star) > 0$ is symmetric and is left as Exercise 9; take $\varepsilon < 0$. Hence $\frac{dJ}{d\theta}(\theta^\star) = 0$. ∎

§ 2.3 Minimization in $\mathbb{R}^d$

Definition 2.3.1: Neighborhood

For $r > 0$, $N_{\theta^\star}(r) := \{\theta \in \mathbb{R}^d : \|\theta - \theta^\star\| < r\}$, where $\|z\| := \left(\sum_i z_i^2\right)^{1/2}$ is the Euclidean norm.

Definition 2.3.2: Local minimizer

ms. p.10

$\theta^\star$ is a local minimizer of $J : \mathbb{R}^d \to [0, \infty)$ if there exists $r > 0$ with $J(\theta) \geq J(\theta^\star)$ for all $\theta \in N_{\theta^\star}(r)$.

Theorem 2.3.1 First-order necessary condition

If $\theta^\star$ is a local minimizer of a differentiable J, then

$$\nabla_\theta J(\theta^\star) = \begin{bmatrix} \frac{\partial J}{\partial \theta_1}(\theta^\star) \\ \vdots \\ \frac{\partial J}{\partial \theta_d}(\theta^\star) \end{bmatrix} = \mathbf{0}.$$

Proposition 2.3.2 Reduction to one dimension

Fix a unit direction $\boldsymbol{p} \in \mathbb{R}^d$, $\|\boldsymbol{p}\| = 1$, and define the scalar function

$$F(\delta; \boldsymbol{p}) := J(\theta^\star + \delta \boldsymbol{p}).$$

If $\theta^\star$ is a local minimizer of J, then $\delta = 0$ is a local minimizer of $F(\cdot\,; \boldsymbol{p})$ — for every *direction $\boldsymbol{p}$.*

Note:-

This is the workhorse of the whole section. Every d-dimensional statement below is obtained by slicing the surface along a direction, applying the one-dimensional result of Section 2.2 to the slice, and then observing that the direction was arbitrary. Nothing new has to be proved in $\mathbb{R}^d$; the work was already done on the line.

Let us see this geometrically. Picture J as a surface over $\mathbb{R}^d$. Each direction $\boldsymbol{p}$ cuts a vertical plane through $\theta^\star$, and $F(\delta; \boldsymbol{p})$ is the curve where that plane meets the surface. If $\theta^\star$ is the bottom of the surface, then $\delta = 0$ is the bottom of every one of those curves.

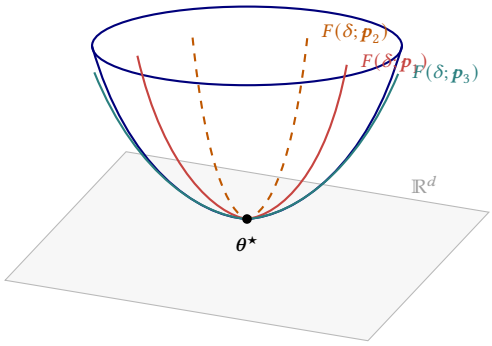


Figure 2.3: **Slicing reduces $\mathbb{R}^d$ to $\mathbb{R}$.** The surface is J over the base plane $\mathbb{R}^d$. Each direction $\boldsymbol{p}_i$ cuts a vertical plane through $\theta^\star$, and the curve of intersection is the scalar function $F(\delta; \boldsymbol{p}_i)$ of Proposition 2.3.2. If $\theta^\star$ minimizes J, then $\delta = 0$ minimizes every such curve, whichever way we look. This is why the one-dimensional theory of Section 2.2 suffices.

Let us see this geometrically. In $\mathbb{R}^2$ the picture is a contour map. The neighborhood $N_{\theta^\star}(r)$ is a disc rather than an interval, and the gradient — which points uphill, perpendicular to the contour through it — must vanish at the bottom, because at the bottom there is no uphill direction to point in.

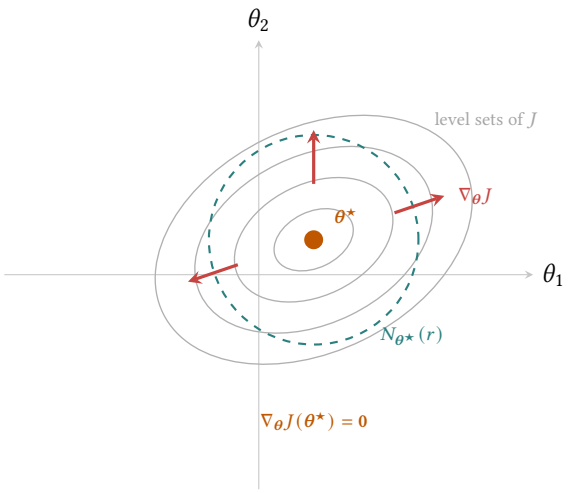


Figure 2.4: **A local minimizer in $\mathbb{R}^2$, and why the gradient must vanish there.** Gray curves are level sets of J; the dashed circle is the neighborhood $N_{\theta^\star}(r)$ of the definition, and every point of it has $J(\boldsymbol{\theta}) \geq J(\boldsymbol{\theta}^\star)$. The gradient (red) is perpendicular to the level set through its foot and points uphill, so it is nonzero wherever the level sets have not closed up. At $\boldsymbol{\theta}^\star$ they have, and the gradient is $\mathbf{0}$ — which is Theorem 2.3.1. Note that the same picture with the arrows reversed would show a *maximum*, and the gradient would vanish there too: this is precisely why the condition is necessary and not sufficient.

Theorem 2.3.3 Second-order necessary condition

If $\boldsymbol{\theta}^\star$ is a local minimizer of a twice continuously differentiable J, then in addition to Theorem 2.3.1,

$$\boldsymbol{H}(\boldsymbol{\theta}^\star) \succeq 0,$$

that is, $\boldsymbol{z}^\top \boldsymbol{H}(\boldsymbol{\theta}^\star)\boldsymbol{z} \geq 0$ for every $\boldsymbol{z} \in \mathbb{R}^d$. Positive semi-definite, not definite.

Note:-

Compare Theorem 2.3.3 with Theorem 2.3.4 carefully; the difference is one symbol and it is the whole story of this section. A minimizer *must* have $\boldsymbol{H} \succeq 0$; a point with $\boldsymbol{H} \succ 0$ *must* be a minimizer. Between the two lies the semi-definite case, where neither theorem decides — and Example 2.3.1 and Figure 2.5b live exactly there.

Note:-

The condition is *necessary*, not sufficient. It is satisfied by maxima and saddle points as well. In practice, finding a minimizer therefore requires two things: ms. p.7

1. solve the system $\nabla_\theta J(\boldsymbol{\theta}) = \mathbf{0}$;
2. supply at least one additional piece of information certifying that the solution is in fact a minimizer — for instance that J is convex, or a curvature condition of the kind developed next.

2.3.1 Second-order conditions

Definition 2.3.3: Hessian

The Hessian *of J at $\boldsymbol{\theta}$ is*

$$\boldsymbol{H}(\boldsymbol{\theta}) \coloneqq \nabla_\theta\big(\nabla_\theta J(\boldsymbol{\theta})\big) \in \mathbb{R}^{d\times d}, \qquad \boldsymbol{H}_{jk}(\boldsymbol{\theta}) \coloneqq \frac{\partial^2 J}{\partial\theta_j \partial\theta_k}(\boldsymbol{\theta}).$$

ms. p.11

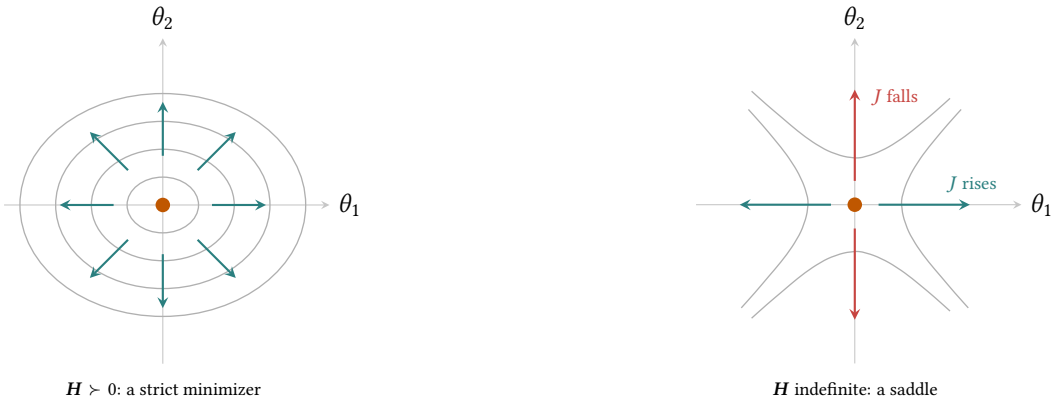
Definition 2.3.4: Positive definite matrix

ms. p.11 $\boldsymbol{A} \in \mathbb{R}^{d\times d}$ *is* positive definite, *written* $\boldsymbol{A} \succ 0$, *if* $z^\top \boldsymbol{A} z > 0$ *for every* $z \in \mathbb{R}^d$ *with* $z \neq \mathbf{0}$.

Theorem 2.3.4 Second-order sufficient condition

ms. p.11 *If (a)* $\nabla_\theta J(\boldsymbol{\theta}^\star) = \mathbf{0}$ *and (b)* $\boldsymbol{H}(\boldsymbol{\theta}^\star) \succ 0$, *then* $\boldsymbol{\theta}^\star$ *is a strict local minimizer of* J.

Let us see this geometrically. Condition (a) alone cannot tell a valley from a mountain pass: the gradient vanishes at both. Condition (b) is what distinguishes them, and the two panels below are the same stationary point seen under the two hypotheses.



(a) Condition (b) holds.

(b) Condition (b) fails.

Figure 2.5: **Why condition (b) cannot be dropped.** Both panels show a point at which $\nabla_\theta J = \mathbf{0}$, so condition (a) of Theorem 2.3.4 holds in each. **(a)** With $\boldsymbol{H} \succ 0$ the level sets are closed curves around the point and J increases in *every* direction: a strict local minimizer. **(b)** For $J = \theta_1^2 - \theta_2^2$ the level sets are hyperbolas, J rises along θ_1 and falls along θ_2, and the origin is a saddle — a stationary point that is no minimizer at all. A one-dimensional version of the same failure, with $\boldsymbol{H}$ merely positive *semi*-definite rather than indefinite, is Example 2.3.1 below.

Example 2.3.1 (The condition really must be strict)

Let $J(\theta) = 3\theta^2 + 2\theta + 1$. Then $J'(\theta) = 6\theta + 2$ and $\boldsymbol{H} = 6 > 0$, so the stationary point $\theta^\star = -\frac{1}{3}$ is a minimizer. Now let $J(\theta) = \theta^3$. At $\theta^\star = 0$ we have $J'(0) = 0$ and $\boldsymbol{H}(0) = 0$: the Hessian is positive *semi*-definite, and $\theta^\star = 0$ is
ms. p.11 not a minimizer. This is precisely why Theorem 2.3.4 demands definiteness rather than semi-definiteness.

Common pitfall

Solving $\nabla_\theta J(\boldsymbol{\theta}) = \mathbf{0}$ and stopping there is the single most common error in this material. Without a curvature or convexity argument you have found a *stationary point*, not a minimizer.

§ 2.4 Exercises

Nothing in this chapter needs data, and the programming problems need nothing beyond `jax.grad`, `jax.hessian`, and `jnp.linalg.eigvalsh`. Enable double precision before you start — eigenvalues of a Hessian are exactly the quantity that `float32` reports badly:

```
import jax
jax.config.update("jax_enable_x64", True)
```

See Appendix C if that line is unfamiliar.

Warm-up

1. [Theory] Prove Claim 2.1.1.

2. [Theory] For each of the following, compute $\frac{dJ}{d\theta}$ and $\frac{d^2J}{d\theta^2}$, find every stationary point, and classify it as a local minimizer, a local maximizer, or neither: (a) $J(\theta) = \theta^2 - 4\theta + 7$; (b) $J(\theta) = \theta^4$; (c) $J(\theta) = \theta^3 - 3\theta$. For which of them does Theorem 2.3.4 settle the question, and for which does it not?

3. [Theory] Let $J(\boldsymbol{\theta}) = \frac{1}{2}\boldsymbol{\theta}^\top \boldsymbol{A}\boldsymbol{\theta} - \boldsymbol{b}^\top\boldsymbol{\theta}$ with $\boldsymbol{A} = \left[\begin{smallmatrix} 4 & 1 \\ 1 & 3 \end{smallmatrix}\right]$ and $\boldsymbol{b} = \left[\begin{smallmatrix} 1 \\ -2 \end{smallmatrix}\right]$. Compute $\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$ and $\boldsymbol{H}(\boldsymbol{\theta})$ by hand, solve $\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \mathbf{0}$, and decide using Theorem 2.3.4 whether the solution is a minimizer.

4. [Theory] Take $J(\theta) = \theta^2$ and $\theta^\star = 0$. Verify Definition 2.2.1 directly by exhibiting a δ that works. Does every $\delta > 0$ work? Now take the function of Figure 2.1 and explain why no δ larger than about 3 would do.

5. [Theory] Using Figure 2.5b, write down a direction along which $J = \theta_1^2 - \theta_2^2$ *decreases* away from the origin, and one along which it increases. What does this say about $\boldsymbol{H}(\mathbf{0})$ without computing it?

6. [Code] Define $J(\theta) = \theta^2 - 4\theta + 7$ in JAX. Use `jax.grad` and `jax.hessian` to evaluate J' and J'' at $\theta = 0, 1, 2, 3$, and check the four values against your hand computation from Exercise 2.

Core

7. [Theory] Show by counterexample that Claim 2.1.1 fails if α is allowed to be negative, and state what $\arg\min_{\boldsymbol{\theta}} \alpha J(\boldsymbol{\theta})$ equals for $\alpha < 0$.

8. [Theory] Prove Claim 2.1.2.

9. [Theory] Complete the proof of Theorem 2.2.1 by treating the case $\frac{dJ}{d\theta}(\theta^\star) > 0$.

10. [Theory] Prove Theorem 2.3.1 from Theorem 2.2.1. *Hint:* for a fixed direction $\boldsymbol{v}$, consider the scalar function $g(t) = J(\boldsymbol{\theta}^\star + t\boldsymbol{v})$ and apply the one-dimensional result at $t = 0$. Where does the argument use that $\boldsymbol{v}$ was arbitrary?

11. [Theory] Show that if $\boldsymbol{A} \succ 0$ then $\boldsymbol{A}$ is invertible, and that all of its eigenvalues are strictly positive. Is the converse true for non-symmetric $\boldsymbol{A}$?

12. [Theory] Give a function $J : \mathbb{R}^2 \to \mathbb{R}$ with a stationary point at which the Hessian is indefinite, and describe the geometry of J near that point. Sketch its level sets and compare with Figure 2.5b.

13. [Code] Implement $J(\theta) = \theta^3$, $J(\theta) = 3\theta^2 + 2\theta + 1$, and $J(\boldsymbol{\theta}) = \theta_1^2 - \theta_2^2$ in JAX. Use `jax.grad` and `jax.hessian` to locate the stationary points numerically and classify each by inspecting the eigenvalues of the Hessian with `jnp.linalg.eigvalsh`. Confirm that the classification matches the analysis of Section 2.3.1.

Challenge

14. [Theory] Prove that if J is convex and $\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}^\star) = \mathbf{0}$, then $\boldsymbol{\theta}^\star$ is a *global* minimizer, not merely a local one. This is the convexity escape route promised in the note after Theorem 2.3.1, and it is the reason Chapter 3 can stop after solving one linear system.

15. [Theory] Is the converse of Theorem 2.3.4 true — must a strict local minimizer have $\boldsymbol{H} \succ 0$? Settle it with a proof or a counterexample. *Hint:* consider $J(\theta) = \theta^4$.

16. [Theory] Exhibit $J : \mathbb{R}^2 \to \mathbb{R}$ with $\nabla_{\boldsymbol{\theta}} J(\mathbf{0}) = \mathbf{0}$ and $\boldsymbol{H}(\mathbf{0}) \succeq 0$ (positive *semi*-definite) for which the origin is nonetheless not a local minimizer. This is the two-dimensional analogue of Example 2.3.1, and it shows the gap in Theorem 2.3.4 is not an artifact of one dimension.

17. [Theory] A function is *coercive* if $J(\boldsymbol{\theta}) \to \infty$ whenever $\|\boldsymbol{\theta}\| \to \infty$. Prove that a continuous coercive J attains its minimum somewhere on $\mathbb{R}^d$. Which of the losses in Section 2.1 are coercive, and what goes wrong for the ones that are not?

18. [Code] Write a function that, given a scalar J and a point θ', verifies the descent property numerically: sample h over several orders of magnitude and plot $J(\theta' + h) - J(\theta')$ against $h\,J'(\theta')$ on log axes. Confirm that the two agree to first order and identify the h at which the quadratic term takes over. Repeat in `float32` and explain the second regime that appears at small h.

19. [Code] Reproduce Figure 2.5 numerically. On a grid over $[-2, 2]^2$, contour $J_a(\boldsymbol{\theta}) = \theta_1^2 + 1.6\,\theta_2^2$ and $J_b(\boldsymbol{\theta}) = \theta_1^2 - \theta_2^2$. For each, confirm $\nabla_{\boldsymbol{\theta}} J(\mathbf{0}) = \mathbf{0}$ with `jax.grad`, compute the two Hessian eigenvalues, and print their signs. Then, for each function, evaluate J at 200 random points on a small circle about the origin and report the minimum: for one function it exceeds $J(\mathbf{0})$ and for the other it does not. Explain which hypothesis of Theorem 2.3.4 that experiment is testing.

Chapter 3

Linear and Polynomial Regression

Goal of this chapter

Chapter 2 produced two tools: a necessary condition $\nabla_\theta J(\boldsymbol{\theta}) = \mathbf{0}$, and a sufficient condition $\boldsymbol{H} \succ 0$. This chapter uses both, on the one problem in the course where the minimizer can be written down in closed form.
By the end we will have

1. computed $\nabla_\theta J(\boldsymbol{\theta})$ for the L_2 loss, three different ways;
2. solved $\nabla_\theta J(\boldsymbol{\theta}) = \mathbf{0}$ exactly, obtaining the *normal equations*;
3. certified the solution with a Hessian argument, and identified exactly when it fails to be unique;
4. found that "linear regression" fits curves as happily as lines, provided we are careful about what *linear* refers to.

The last section adds a warning: the closed form we derive is, numerically, one of the worst ways to compute the answer.

§ 3.1 The model class and the loss

Take the affine model class

$$\mathcal{M} = \Big\{ f_\theta(\boldsymbol{x}) = \sum_{j=0}^{d} \theta_j x_j \;:\; \boldsymbol{\theta} \in \mathbb{R}^{d+1} \Big\}, \qquad x_0 \equiv 1,$$

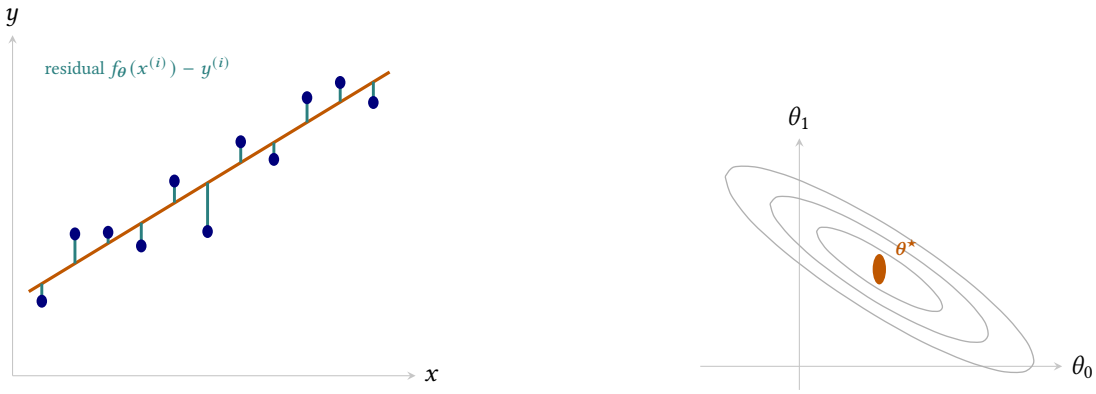
so that θ_0 is the intercept. With the L_2 loss,

$$J(\boldsymbol{\theta}) = \frac{1}{2n} \sum_{i=1}^{n} \big(f_\theta(\boldsymbol{x}^{(i)}) - y^{(i)} \big)^2 = \frac{1}{2n} \sum_{i=1}^{n} \Big(\sum_{j=0}^{d} \theta_j x_j^{(i)} - y^{(i)} \Big)^2 . \tag{3.1}$$

ms. p.7

Two steps remain: compute $\nabla_\theta J(\boldsymbol{\theta})$, then solve $\nabla_\theta J(\boldsymbol{\theta}) = \mathbf{0}$.

Let us see this geometrically. The loss measures the total squared *vertical* distance from the data to the model. Every choice of $\boldsymbol{\theta}$ moves the line and changes that total, so $J(\boldsymbol{\theta})$ is a surface over the parameter plane — and by Chapter 2 we already know what kind of surface to expect.



(a) Data space: the loss adds up the squared teal segments.

(b) Parameter space: level sets of $J(\boldsymbol{\theta})$.

Figure 3.1: **The same problem, seen twice. (a)** A line in data space; the L_2 loss is $\frac{1}{2n}$ times the sum of the squared teal residuals. **(b)** The same eleven points, now viewed as a function of $\boldsymbol{\theta}$: each point of the parameter plane is one line in (a), and the level sets of $J(\boldsymbol{\theta})$ close around a single $\boldsymbol{\theta}^\star$. These are exactly the ellipses of Figure 2.5a — the loss is a convex quadratic, so Theorem 2.3.4 applies and the stationary point is a strict minimizer. Their elongation is the condition number of $\boldsymbol{\Phi}^\mathsf{T}\boldsymbol{\Phi}$, and it is what Section 3.4 returns to.

§ 3.2 The gradient, three ways

3.2.1 Componentwise

Differentiating (3.1) with respect to θ_k and using the chain rule,

$$\frac{\partial J}{\partial \theta_k}(\boldsymbol{\theta}) = \frac{1}{2n}\sum_{i=1}^{n} 2\Big(\sum_{j=0}^{d}\theta_j x_j^{(i)} - y^{(i)}\Big)\frac{\partial}{\partial \theta_k}\Big(\sum_{j=0}^{d}\theta_j x_j^{(i)} - y^{(i)}\Big) = \frac{1}{n}\sum_{i=1}^{n}\Big(\sum_{j=0}^{d}\theta_j x_j^{(i)} - y^{(i)}\Big)x_k^{(i)}.$$

Stacking the components,

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \frac{1}{n}\sum_{i=1}^{n}\Big(\sum_{j=0}^{d}\theta_j x_j^{(i)} - y^{(i)}\Big)\boldsymbol{x}^{(i)}. \tag{3.2}$$

ms. p.8

3.2.2 Two matrix identities

Rather than repeat that computation for every new loss, it pays to record two identities once.

Lemma 3.2.1 Gradient of a linear form

For fixed $\boldsymbol{x} \in \mathbb{R}^m$, $\nabla_{\boldsymbol{\theta}}\big(\boldsymbol{\theta}^\mathsf{T}\boldsymbol{x}\big) = \nabla_{\boldsymbol{\theta}}\big(\boldsymbol{x}^\mathsf{T}\boldsymbol{\theta}\big) = \boldsymbol{x}$.

Lemma 3.2.2 Gradient of a quadratic form

For $\boldsymbol{A} \in \mathbb{R}^{m\times m}$,

$$\nabla_{\boldsymbol{\theta}}\big(\boldsymbol{\theta}^\mathsf{T}\boldsymbol{A}\boldsymbol{\theta}\big) = \boldsymbol{A}\boldsymbol{\theta} + \boldsymbol{A}^\mathsf{T}\boldsymbol{\theta},$$

which reduces to $2\boldsymbol{A}\boldsymbol{\theta}$ *when* $\boldsymbol{A}$ *is symmetric.*

ms. p.8,10 Both are proved in the exercises.

3.2.3 Matrix form

Collect the targets and the data into

$$\boldsymbol{y} \coloneqq \begin{bmatrix} y^{(1)} \\ \vdots \\ y^{(n)} \end{bmatrix} \in \mathbb{R}^n, \qquad \boldsymbol{\Phi} \coloneqq \begin{bmatrix} - & (\boldsymbol{x}^{(1)})^{\mathsf{T}} & - \\ & \vdots & \\ - & (\boldsymbol{x}^{(n)})^{\mathsf{T}} & - \end{bmatrix} \in \mathbb{R}^{n \times (d+1)}.$$

Writing $\boldsymbol{d} \coloneqq \boldsymbol{\Phi}\boldsymbol{\theta}$, whose i-th entry is $\boldsymbol{x}^{(i)} \cdot \boldsymbol{\theta}$,

$$J(\boldsymbol{\theta}) = \frac{1}{2n} \sum_{i=1}^{n} (d_i - y^{(i)})^2 = \frac{1}{2n} (\boldsymbol{d} - \boldsymbol{y}) \cdot (\boldsymbol{d} - \boldsymbol{y}) = \frac{1}{2n} \|\boldsymbol{\Phi}\boldsymbol{\theta} - \boldsymbol{y}\|^2. \tag{3.3}$$

ms. p.9

Expanding,

$$J(\boldsymbol{\theta}) = \frac{1}{2n} \big(\boldsymbol{\theta}^{\mathsf{T}} \boldsymbol{\Phi}^{\mathsf{T}} \boldsymbol{\Phi} \boldsymbol{\theta} - 2\, \boldsymbol{\theta}^{\mathsf{T}} \boldsymbol{\Phi}^{\mathsf{T}} \boldsymbol{y} + \boldsymbol{y}^{\mathsf{T}} \boldsymbol{y} \big),$$

and applying Lemma 3.2.1 and Lemma 3.2.2 with the symmetric matrix $\boldsymbol{A} = \boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi}$,

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \frac{1}{2n} \big[2\boldsymbol{\Phi}^{\mathsf{T}} \boldsymbol{\Phi} \boldsymbol{\theta} - 2\boldsymbol{\Phi}^{\mathsf{T}} \boldsymbol{y} \big] = \frac{1}{n} \big[\boldsymbol{\Phi}^{\mathsf{T}} \boldsymbol{\Phi} \boldsymbol{\theta} - \boldsymbol{\Phi}^{\mathsf{T}} \boldsymbol{y} \big]. \tag{3.4}$$

ms. p.12

§ 3.3 The normal equations

Setting (3.4) to zero gives the *normal equations*

$$\boldsymbol{\Phi}^{\mathsf{T}} \boldsymbol{\Phi} \boldsymbol{\theta} = \boldsymbol{\Phi}^{\mathsf{T}} \boldsymbol{y},$$

and, provided $\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi}$ is invertible,

$$\boxed{\boldsymbol{\theta}^{\star} = (\boldsymbol{\Phi}^{\mathsf{T}} \boldsymbol{\Phi})^{-1} \boldsymbol{\Phi}^{\mathsf{T}} \boldsymbol{y}} \tag{3.5}$$

ms. p.12

Theorem 3.3.1 Invertibility

$\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi} \succ 0$ if and only if $\boldsymbol{\Phi}$ has full column rank. In that case $\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi}$ is invertible and (3.5) is the unique stationary point.

Proof: [Proof sketch] For $\boldsymbol{z} \neq \boldsymbol{0}$, $\boldsymbol{z}^{\mathsf{T}}\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi}\boldsymbol{z} = \|\boldsymbol{\Phi}\boldsymbol{z}\|^2 \geq 0$, with equality if and only if $\boldsymbol{\Phi}\boldsymbol{z} = \boldsymbol{0}$. So $\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi} \succ 0$ exactly when $\boldsymbol{\Phi}$ has trivial null space, that is, when its columns are linearly independent. The details are Exercise 9. ■

3.3.1 Certifying that this is a minimum

Differentiating (3.4) once more,

$$\boldsymbol{H}(\boldsymbol{\theta}) = \nabla_{\boldsymbol{\theta}} \big(\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) \big) = \frac{1}{n} \boldsymbol{\Phi}^{\mathsf{T}} \boldsymbol{\Phi},$$

which is independent of $\boldsymbol{\theta}$ and, by Theorem 3.3.1, positive definite whenever $\boldsymbol{\Phi}$ has full column rank. By Theorem 2.3.4, $\boldsymbol{\theta}^{\star}$ is a minimizer — and since the Hessian is positive definite everywhere, J is strictly convex and the minimizer is global. ms. p.12

Let us see this geometrically. There is a second reading of the normal equations, and it is the one worth carrying away. Rewrite $\boldsymbol{\Phi}^{\mathsf{T}}(\boldsymbol{\Phi}\boldsymbol{\theta}^{\star} - \boldsymbol{y}) = \boldsymbol{0}$: it says the residual vector is orthogonal to every column of $\boldsymbol{\Phi}$. Least squares is a projection.

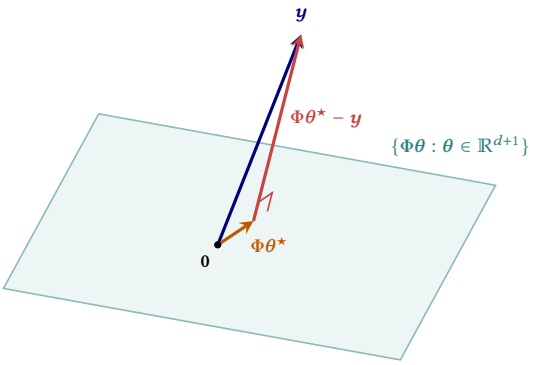


Figure 3.2: **Least squares is an orthogonal projection.** The set of all achievable predictions $\Phi\boldsymbol{\theta}$ is a subspace — the column space of Φ, shaded. The data $\boldsymbol{y}$ generally does not lie in it. The normal equations say $\Phi^{\mathsf{T}}(\Phi\boldsymbol{\theta}^\star - \boldsymbol{y}) = \mathbf{0}$, that is, the residual is orthogonal to every column of Φ, so $\Phi\boldsymbol{\theta}^\star$ is the point of the subspace closest to $\boldsymbol{y}$. This is why the minimizer exists whatever the data, and also why it is unique exactly when the columns are independent: if they are not, several $\boldsymbol{\theta}$ name the same point of the subspace.

Key idea

The full argument used both halves of Chapter 2: the first-order condition produced the candidate, and the second-order condition certified it. Neither alone would have been enough.

Note:-

When Φ is rank deficient. If Φ does not have full column rank there are infinitely many solutions to the normal equations. Chapter 6 shows that adding a regularization term repairs this unconditionally.

ms. p.12

§ 3.4 Polynomial regression

Nothing above used the fact that the entries of $\boldsymbol{x}^{(i)}$ were the raw inputs. Replacing $\boldsymbol{x}^{(i)}$ by $\boldsymbol{\varphi}^i := \boldsymbol{\varphi}(x^{(i)})$ throughout, with

$$\boldsymbol{\varphi}(x) := \begin{bmatrix} 1 \\ x \\ x^2 \\ \vdots \\ x^d \end{bmatrix},$$

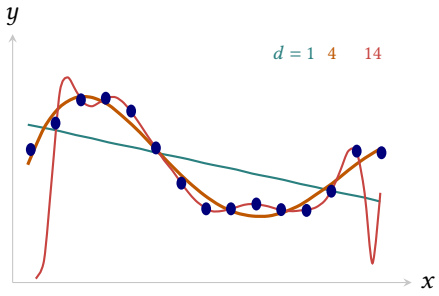
gives the model

$$f_{\boldsymbol{\theta}}(x) = \boldsymbol{\theta}^{\mathsf{T}}\boldsymbol{\varphi}(x) = \sum_{j=0}^{d} \theta_j x^j,$$

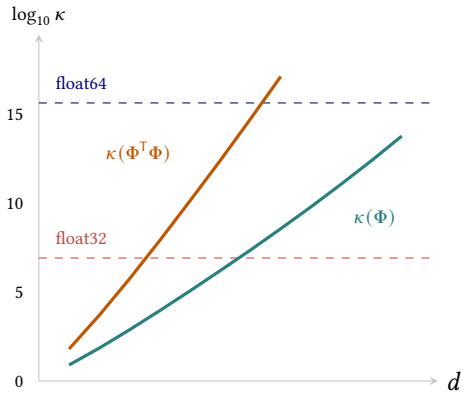
which is d-order polynomial regression. The loss is $J(\boldsymbol{\theta}) = \frac{1}{2n}\|\Phi\boldsymbol{\theta} - \boldsymbol{y}\|^2$ with Φ the Vandermonde-type matrix whose i-th row is $\boldsymbol{\varphi}(x^{(i)})^{\mathsf{T}}$, and the solution is again (3.5).

ms. p.13,14

Let us see this geometrically. Two things go wrong as the degree grows, and they are not the same thing. The fit starts chasing noise, and the linear algebra starts losing digits. The first is a modeling failure; the second is a numerical one.



(a) Fits of degree 1, 4 and 14 to fifteen points.



(b) Conditioning against degree.

Figure 3.3: **Two different failures of high-degree polynomial regression. (a)** Degree 1 (teal) cannot follow the data; degree 4 (orange) does; degree 14 (red) passes near every point and oscillates wildly between them — it has fitted the noise. That is *overfitting*, and it is a modeling failure that Chapter 6 addresses. **(b)** Separately, the condition number of the design matrix grows geometrically with the degree, and forming $\Phi^\top\Phi$ squares it. Once κ crosses the reciprocal of the unit roundoff, the computed solution has no correct digits: in `float32` that happens at degree 3 for the normal equations, and in `float64` at about degree 7. That is a *numerical* failure, and no amount of regularization fixes it — one changes the algorithm instead.

Common pitfall

For large d the Vandermonde matrix is severely ill-conditioned, and forming $\Phi^\top\Phi$ squares that condition number. Solving the normal equations directly is numerically the worst way to do this; use a QR or SVD based least-squares solver instead. Programming Exercise 13 and Figure 3.3b make the effect visible.

§ 3.5 Exercises

The code problems use the two engineering datasets introduced in Chapter 1: **Airfoil Self-Noise** (`https://archive.ics.uci.edu/dataset/291/airfoil+self+noise`, or `fetch_ucirepo(id=291)`) and **Combined Cycle Power Plant** (`https://archive.ics.uci.edu/dataset/294/combined+cycle+power+plant`, or `fetch_ucirepo(id=294)`). Appendix C covers loading them, and Section C.3 explains why several of these problems will give the wrong answer without `jax_enable_x64`.

Warm-up

1. [Theory] Prove Lemma 3.2.1.
2. [Theory] Write out the design matrix Φ explicitly for the three samples $(x^{(1)}, y^{(1)}) = (1, 2)$, $(x^{(2)}, y^{(2)}) = (2, 3)$, $(x^{(3)}, y^{(3)}) = (3, 5)$ with the affine model, and solve (3.5) by hand.
3. [Theory] Take the fitted line of Figure 3.1a. Which quantity does the L_2 loss square and sum — the vertical segments, or the perpendicular distances to the line? Why does the distinction matter, and what would change if we minimized the other one?
4. [Theory] Using Figure 3.2, state in one sentence why a minimizer of the L_2 loss exists for *every* dataset, even when Φ is rank deficient, and why uniqueness is the property that can fail.
5. [Theory] For $\varphi(x) = (1, x, x^2)^\top$ and the samples $x = 0, 1, 2$, write out Φ and compute $\Phi^\top\Phi$. Is it invertible? What would go wrong if two of the samples coincided?
6. [Code] Load the Combined Cycle Power Plant data, fit the affine model with `jnp.linalg.lstsq`, and report the four coefficients and the intercept. Which feature has the largest effect on output, and does the sign agree with physical intuition?

Core

7. [Theory] Prove Lemma 3.2.2, and verify the symmetric case by direct computation for a 2×2 example.

8. [Theory] Show that (3.2) and (3.4) agree.

9. [Theory] Prove Theorem 3.3.1 in full, in both directions.

10. [Theory] Show that the residual $\Phi\theta^\star - \boldsymbol{y}$ is orthogonal to every column of Φ, and interpret (3.5) geometrically as the orthogonal projection drawn in Figure 3.2.

11. [Theory] Repeat the derivation of this chapter for the weighted loss $J(\boldsymbol{\theta}) = \frac{1}{2n}\sum_i w_i(f_{\boldsymbol{\theta}}(\boldsymbol{x}^{(i)}) - y^{(i)})^2$ with fixed $w_i > 0$. What replaces $\Phi^\mathsf{T}\Phi$? Under what condition on the w_i does the invertibility argument still go through?

12. [Theory] Explain why the L_1 loss of Chapter 2 does not admit a closed-form minimizer by this route. Which step of the argument fails, and what does the loss surface look like near its minimum?

13. [Code] Generate $n = 200$ points from a degree-5 polynomial plus Gaussian noise. For $d = 1, \dots, 15$, fit by (a) explicitly forming $(\Phi^\mathsf{T}\Phi)^{-1}\Phi^\mathsf{T}\boldsymbol{y}$ with `jnp.linalg.inv`, (b) `jnp.linalg.solve` on the normal equations, and (c) `jnp.linalg.lstsq`. Plot the condition number of Φ and of $\Phi^\mathsf{T}\Phi$ against d, reproducing Figure 3.3b. At what d does method (a) fail in `float32`? In `float64`? Do your crossings match the dashed lines in the figure?

Challenge

14. [Theory] When Φ is rank deficient the normal equations have infinitely many solutions. Show that they form an affine subspace, and that exactly one of them has minimal $\|\boldsymbol{\theta}\|$. (That one is what `lstsq` returns, and Chapter 6 will reach it by a different route.)

15. [Theory] Prove that $\kappa(\Phi^\mathsf{T}\Phi) = \kappa(\Phi)^2$ for Φ of full column rank, where κ is the ratio of largest to smallest singular value. This is the precise statement behind the squaring visible in Figure 3.3b.

16. [Theory] Suppose one sample is duplicated, so two rows of Φ are identical. Does $\boldsymbol{\theta}^\star$ change? Now suppose one *feature* is duplicated, so two columns are identical. Does $\boldsymbol{\theta}^\star$ change? Explain the asymmetry using Figure 3.2.

17. [Theory] Show that centering the data — replacing $x^{(i)}$ by $x^{(i)} - \bar{x}$ and $y^{(i)}$ by $y^{(i)} - \bar{y}$ — makes the intercept vanish and decouples it from the slope. What does this do to the level sets of Figure 3.1b, and hence to the conditioning?

18. [Code] Fit polynomial models of degree 1 through 20 to the Airfoil Self-Noise frequency column, holding out 30% of the data. Plot training and held-out loss against degree on the same axes. Identify the degree at which the two curves separate, and reproduce qualitatively the three regimes of Figure 3.3a. Note that you have just used a held-out set without any theory justifying it — that gap is filled in Chapter 6.

19. [Code] Implement $J(\boldsymbol{\theta})$ in JAX and confirm that `jax.grad` reproduces (3.4) to machine precision. Then confirm that `jax.hessian` returns $\Phi^\mathsf{T}\Phi/n$ and that it does not depend on $\boldsymbol{\theta}$. Finally, compute its eigenvalues for the power plant data before and after standardizing the features, and relate the change to Exercise 19.

Chapter 4

Gradient Descent

Goal of this chapter

Chapter 2 characterized minimizers; Chapter 3 found one in closed form. That was the exception. For every model class from Chapter 11 onward, $\nabla_\theta J(\theta) = \mathbf{0}$ cannot be solved by hand, and the minimizer must be reached iteratively. This chapter turns the theory of Chapter 2 into an algorithm. It answers:

1. Which *direction* should we step in, and in what sense is it best?
2. How far should we step, given that the theory is only local?
3. When should we stop?
4. What does the resulting sequence converge to — and what does it not?

The first question has a clean answer, from Cauchy–Schwarz. The remaining three are where the engineering lives.

§ 4.1 The steepest descent direction

Taylor expansion of J about a current iterate θ^0 in a direction $\boldsymbol{p}$ reads

$$J(\theta^0 + \delta \boldsymbol{p}) = J(\theta^0) + \delta\, \boldsymbol{p}^\top \nabla_\theta J(\theta^0) + \frac{\delta^2}{2}\, \boldsymbol{p}^\top \boldsymbol{H}(\theta^0)\boldsymbol{p} + \cdots$$

For $0 < \delta \ll 1$ the first-order truncation

$$J(\theta^0 + \delta \boldsymbol{p}) \approx J(\theta^0) + \delta\, \boldsymbol{p}^\top \nabla_\theta J(\theta^0) \tag{4.1}$$

is accurate.

ms. p.16

We want more than a direction that decreases J: we want the one that decreases it *most*. That is,

$$\min_{\boldsymbol{p}}\ J(\theta^0 + \delta \boldsymbol{p}) - J(\theta^0) \qquad \Longleftrightarrow \qquad \min_{\boldsymbol{p}}\ \boldsymbol{p}^\top \nabla_\theta J(\theta^0),$$

the equivalence following from (4.1) and $\delta > 0$.

Theorem 4.1.1 Steepest descent direction

Among all $\boldsymbol{p}$ of a fixed length, $\boldsymbol{p}^\top \nabla_\theta J(\theta^0)$ is minimized by

$$\boxed{\boldsymbol{p} = -\nabla_\theta J(\theta^0)}$$

Proof: By the Cauchy–Schwarz inequality,

$$|\boldsymbol{p}^\top \nabla_\theta J(\theta^0)| \ \le\ \|\boldsymbol{p}\|\, \|\nabla_\theta J(\theta^0)\|,$$

equivalently

$$-\|\boldsymbol{p}\|\,\|\nabla_\theta J(\boldsymbol{\theta}^0)\| \;\le\; \boldsymbol{p}^{\mathsf{T}}\nabla_\theta J(\boldsymbol{\theta}^0) \;\le\; \|\boldsymbol{p}\|\,\|\nabla_\theta J(\boldsymbol{\theta}^0)\|.$$

The left-hand bound is attained exactly when $\boldsymbol{p}$ is antiparallel to $\nabla_\theta J(\boldsymbol{\theta}^0)$, and $\boldsymbol{p} = -\nabla_\theta J(\boldsymbol{\theta}^0)$ is such a choice. ■

Note:-

Equivalently, for vectors $\boldsymbol{a}\cdot\boldsymbol{b} = \|\boldsymbol{a}\|\|\boldsymbol{b}\|\cos\alpha$, minimized when $\alpha = \pi$ — when $\boldsymbol{a}$ is parallel to $\boldsymbol{b}$ but points the opposite way. Figure 2.2 showed the one-dimensional case of exactly this: step against the sign of the derivative.

Key idea

The gradient points uphill and is perpendicular to the level set through its foot (Figure 2.4). Its negative is therefore the most downhill direction available. Note the word *available*: steepest descent is optimal only among directions, and only to first order. It is not the fastest route to the minimizer, as Figure 4.2 will show.

§ 4.2 The step size

The direction is settled; the distance is not. Taylor expansion is a local statement, so (4.1) is trustworthy only for small δ:
ms. p.16 too large a step can overshoot and *increase* the loss.

Let us see this geometrically. Take a step from $\boldsymbol{\theta}^0$ along $-\nabla_\theta J(\boldsymbol{\theta}^0)$. A short step lands lower. A long step can cross the valley and land higher than where it started. Nothing in the derivative tells us which is which, because the derivative is local information only.

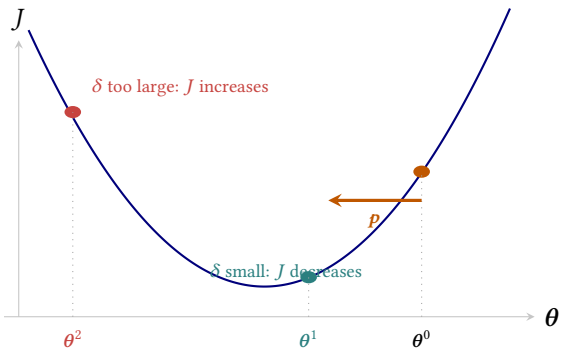


Figure 4.1: **The step size matters as much as the direction.** From $\boldsymbol{\theta}^0$ the direction $\boldsymbol{p} = -\nabla_\theta J(\boldsymbol{\theta}^0)$ is downhill. A modest step reaches $\boldsymbol{\theta}^1$, below $\boldsymbol{\theta}^0$. A step that is too long reaches $\boldsymbol{\theta}^2$, *above* it, even though the direction was correct. The derivative is local information and cannot warn us; the remedy is to try a step and check.

4.2.1 Backtracking

The practical answer is to attempt a full step and shrink it until the loss actually decreases.

Algorithm 1: Backtracking line search

Input: current iterate $\boldsymbol{\theta}^0$, initial step $\delta = 1$
Output: a next iterate $\boldsymbol{\theta}^1$ with $J(\boldsymbol{\theta}^1) < J(\boldsymbol{\theta}^0)$

$\boldsymbol{\theta}^1 \leftarrow \boldsymbol{\theta}^0 - \delta\,\nabla_\theta J(\boldsymbol{\theta}^0)$
while $J(\boldsymbol{\theta}^1) \ge J(\boldsymbol{\theta}^0)$ **do**
  $\delta \leftarrow \delta/2$
  $\boldsymbol{\theta}^1 \leftarrow \boldsymbol{\theta}^0 - \delta\,\nabla_\theta J(\boldsymbol{\theta}^0)$
return $\boldsymbol{\theta}^1$

Note:-

Algorithm 1 always terminates when $\nabla_\theta J(\theta^0) \neq \mathbf{0}$, because (4.1) guarantees a decrease for all sufficiently small δ — and halving reaches "sufficiently small" in finitely many steps. It does *not* terminate at a stationary point, which is one reason the outer loop needs its own stopping test.

§ 4.3 The algorithm

Algorithm 2: Steepest gradient descent

Input: initial guess θ^0, tolerances $\tau_1, \ldots, \tau_4 > 0$
Output: approximate minimizer $\theta^\dagger$

```
j ← 0
while stopping criteria not met do
    determine δ at θ^j by Algorithm 1
    θ^{j+1} ← θ^j − δ ∇_θ J(θ^j)
    j ← j + 1
θ^† ← θ^j
```

ms. p.16

4.3.1 Stopping criteria

The natural test is $\|\nabla_\theta J(\theta^j)\| < \tau_1$, since Theorem 2.3.1 says the gradient vanishes at a minimizer. It is natural but by no means the best, and it is usually combined with others: ms. p.16

$$\begin{aligned}
&\text{(i) absolute gradient} && \|\nabla_\theta J(\theta^j)\| < \tau_1, \\
&\text{(ii) relative gradient} && \frac{\|\nabla_\theta J(\theta^j)\|}{\|\nabla_\theta J(\theta^0)\|} < \tau_2, \\
&\text{(iii) incremental movement} && \frac{\|\theta^{j+1} - \theta^j\|}{\|\theta^j\|} < \tau_3, \\
&\text{(iv) incremental improvement} && \frac{|J(\theta^{j+1}) - J(\theta^j)|}{|J(\theta^j)|} < \tau_4.
\end{aligned}$$

In practice one stops when *at least one* is met.

Note:-

The relative forms (ii)–(iv) are scale invariant, and that is their point: criterion (i) alone reports differently for J and $100J$, even though Claim 2.1.1 says the two problems have the same minimizer. A smaller τ_1 brings $\theta^\dagger$ closer to $\theta^\star$ but costs more iterations.

Common pitfall

Criteria (iii) and (iv) can both fire because backtracking has driven δ toward zero, with the iterate nowhere near a stationary point. Only (i) and (ii) actually test the condition of Theorem 2.3.1. Reporting convergence on the strength of (iii) alone is a common and silent error.

§ 4.4 What the iteration converges to

Proposition 4.4.1 Limit points are stationary

Suppose the sequence $\{\theta^j\}$ generated by Algorithm 2 converges to some $\hat{\theta}$, and that $\nabla_\theta J$ is continuous. Then $\nabla_\theta J(\hat{\theta}) = \mathbf{0}$.

Proof: Take limits in $\boldsymbol{\theta}^{j+1} = \boldsymbol{\theta}^{j} - \delta\,\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}^{j})$. The left side tends to $\hat{\boldsymbol{\theta}}$; on the right, $\boldsymbol{\theta}^{j} \to \hat{\boldsymbol{\theta}}$ and, by continuity, ms. p.17 $\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}^{j}) \to \nabla_{\boldsymbol{\theta}} J(\hat{\boldsymbol{\theta}})$. Hence $\hat{\boldsymbol{\theta}} = \hat{\boldsymbol{\theta}} - \delta\,\nabla_{\boldsymbol{\theta}} J(\hat{\boldsymbol{\theta}})$, and since $\delta > 0$ the gradient vanishes. ■

Common pitfall

Proposition 4.4.1 delivers Theorem 2.3.1 and nothing more. A stationary point may be a maximizer or a saddle (Figure 2.5b); certifying a minimizer still requires Theorem 2.3.4. What the algorithm does supply is monotone decrease — backtracking enforces it — which rules out converging to a maximizer, but not to a saddle.

4.4.1 Zig-zagging, and why steepest descent is slow

Let us see this geometrically. The search direction is proportional to the gradient, and the gradient is perpendicular to the level set. So every step crosses the contours at right angles — and when the contours are elongated, that forces the path to bounce from side to side rather than travel down the valley.

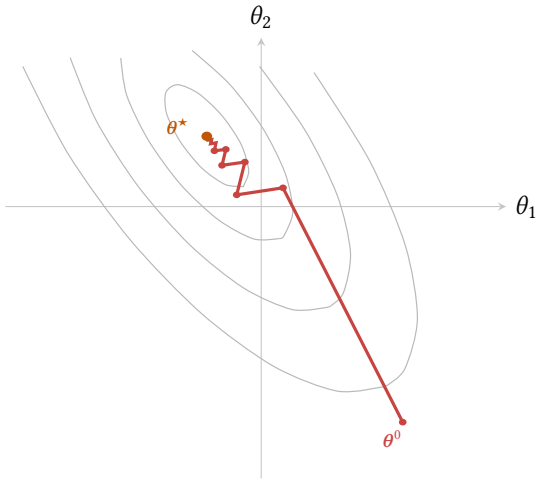


Figure 4.2: **Steepest descent zig-zags.** The path taken by Algorithm 2 on $J(\boldsymbol{\theta}) = 2\theta_1^2 + 2\theta_1\theta_2 + \theta_2^2 + \theta_1 - \theta_2$, whose minimizer is $\boldsymbol{\theta}^\star = (-1,\ 1.5)$. Each search direction is orthogonal to the level set it leaves, so consecutive steps are perpendicular to one another and the path oscillates across the valley instead of running along it. Convergence is linear, and the rate degrades as the level sets become more elongated — that is, as the condition number of $\boldsymbol{H}$ grows. Here $\kappa(\boldsymbol{H}) \approx 6.9$ and 24 iterations were needed.

Note:-

This inefficiency motivates better algorithms. The *conjugate gradient* method uses the history of past gradients to move more directly toward the optimum; *Newton-like* methods use the Hessian itself, converging much faster near a minimizer at the cost of forming or approximating $\boldsymbol{H}$.

4.4.2 Dependence on the initial guess

Let us see this geometrically. Gradient descent only ever consults local information, so it can only find a local minimizer — and which one it finds is decided by where it starts.

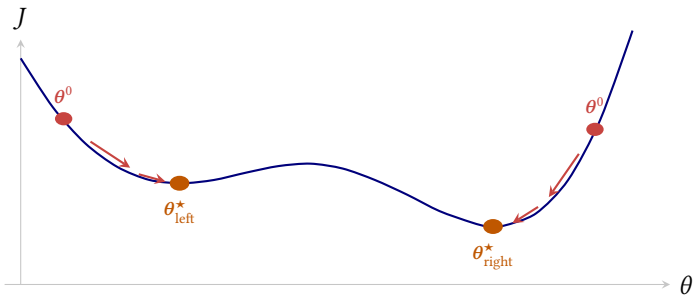


Figure 4.3: **Gradient descent is a local method.** Applied to a loss with two local minimizers, the algorithm converges to whichever basin contains the initial guess. Neither run has any way of knowing the other minimum exists, and the right-hand one here is the lower of the two. This is the practical content of the word *local* in Definition 2.2.1, and it is why initialization is a modeling decision rather than a detail.

§ 4.5 Momentum

Figure 4.2 diagnosed the problem: consecutive steps are orthogonal, so the iterate crosses and re-crosses the valley instead of running along it. The successive steps largely cancel in the direction we want to move and reinforce in the direction we do not. That description also suggests the cure — *remember* the previous step.

Definition 4.5.1: Gradient descent with momentum

Given $\alpha > 0$ *and a* momentum coefficient $\beta \in [0, 1)$, *set* $\boldsymbol{v}^0 = \mathbf{0}$ *and iterate*

$$\boldsymbol{v}^{j+1} = \beta\, \boldsymbol{v}^j - \alpha\, \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}^j), \qquad \boldsymbol{\theta}^{j+1} = \boldsymbol{\theta}^j + \boldsymbol{v}^{j+1}.$$

Taking $\beta = 0$ *recovers the plain iteration of Section 5.3.*

The vector $\boldsymbol{v}^j$ accumulates past steps with geometrically decaying weights: unrolling the recursion gives

$$\boldsymbol{v}^{j+1} = -\alpha \sum_{i=0}^{j} \beta^{j-i}\, \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}^i),$$

so the update is a weighted average of every gradient seen so far, with the most recent counting most.

Key idea

Averaging is the mechanism. Components of the gradient that keep pointing the same way add up over successive terms; components that alternate in sign — precisely the zig-zag — cancel. Momentum therefore damps oscillation across the valley while accelerating progress along it, which is exactly the asymmetry Figure 4.2 exhibits.

Note:-

The name comes from a physical reading: $\boldsymbol{v}$ is a velocity, $-\nabla_{\boldsymbol{\theta}} J$ a force, and β a friction coefficient. A ball rolling down a narrow valley does not stop and re-aim at every step; it carries speed through the oscillations. This is why the scheme is also called the *heavy ball* method.

Algorithm 3: Gradient descent with momentum (heavy ball)

Input: initial guess $\boldsymbol{\theta}^0$, step $\alpha > 0$, momentum $\beta \in [0, 1)$
Output: approximate minimizer $\boldsymbol{\theta}^\dagger$

$\boldsymbol{v} \leftarrow \mathbf{0}$; $\quad j \leftarrow 0$
while *stopping criteria not met* **do**
  $\boldsymbol{v} \leftarrow \beta \boldsymbol{v} - \alpha \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}^j)$
  $\boldsymbol{\theta}^{j+1} \leftarrow \boldsymbol{\theta}^j + \boldsymbol{v}$
  $j \leftarrow j + 1$
$\boldsymbol{\theta}^\dagger \leftarrow \boldsymbol{\theta}^j$

Let us see this geometrically. On the very problem of Figure 4.2, with the same step size and the same starting point, adding momentum turns the staircase into a curve.

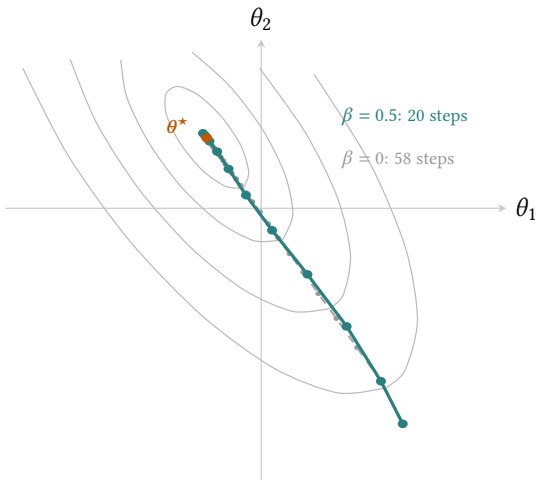


Figure 4.4: **Momentum straightens the path.** The same objective, starting point and step size α as Figure 4.2, with and without momentum. Plain descent (gray) creeps along the valley floor in 58 iterations; with $\beta = 0.5$ (teal) the accumulated velocity carries the iterate through the oscillation and reaches the same tolerance in 20. Note the slight overshoot past $\boldsymbol{\theta}^\star$ near the end — the ball has inertia, and stopping it takes a moment.

Common pitfall

More momentum is not better. On the problem above, $\beta = 0.85$ takes 87 iterations — worse than plain gradient descent — because the iterate acquires enough speed to sail past the minimizer and orbit it. There is an optimal β, and for a quadratic with condition number κ it is $\left((\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1)\right)^2$; Exercise 21 works this out.

Note:-

A close relative, *Nesterov accelerated gradient*, evaluates the gradient at the *look-ahead* point $\boldsymbol{\theta}^j + \beta \boldsymbol{v}^j$ rather than at $\boldsymbol{\theta}^j$ — asking where the ball is heading before deciding how to push it. On convex problems this improves the worst-case rate from $O(1/j)$ to $O(1/j^2)$.

Note:-

Stochastic gradient descent, mini-batching, RMSProp and Adam all rest on treating $\nabla_{\boldsymbol{\theta}} J$ as a *random estimate* of the true gradient, which needs the expectation and variance of Chapter 5. They are therefore deferred to Chapter 10, immediately after that material. Momentum, being purely deterministic, belongs here.

Take-home message

Direction from Cauchy–Schwarz; step size by trial; stopping by tolerance.

Only the first has a clean answer. Gradient descent converges to a stationary point of whichever basin it starts in, at

a rate set by the conditioning of the Hessian — and every refinement in the rest of these notes attacks one of those three weaknesses.

§ 4.6 Exercises

The programming problems need only `jax.grad` and `jax.jit`; The datasets of Chapter 1 appear in the last two. Enable `x64` first (Section C.3).

Warm-up

1. [Theory] For $J(\boldsymbol{\theta}) = \theta_1^2 + 4\theta_2^2$ at $\boldsymbol{\theta}^0 = (1, 1)$, compute $\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}^0)$ and write down the steepest descent direction. Sketch it against the level set through $\boldsymbol{\theta}^0$ and confirm it is perpendicular to that set.

2. [Theory] Run Algorithm 1 by hand on $J(\theta) = \theta^2$ from $\theta^0 = 1$. How many halvings does it take? Now from $\theta^0 = 10$. Explain the difference.

3. [Theory] Which of the four stopping criteria of Section 5.3.1 are scale invariant, in the sense of Claim 2.1.1? Verify your answer by replacing J with $100J$.

4. [Theory] Using Figure 4.3, explain why reporting "the minimizer found by gradient descent" without also reporting $\boldsymbol{\theta}^0$ is incomplete.

5. [Theory] In Figure 4.1, the direction at $\boldsymbol{\theta}^0$ is correct but the step to $\boldsymbol{\theta}^2$ increases the loss. Which approximation has been violated, and at what order in δ?

6. [Theory] Unroll the momentum recursion of Definition 4.5.1 for $j = 0, 1, 2$ and confirm the weighted-sum formula stated after it. What is the total weight $\sum_i \beta^{j-i}$ as $j \to \infty$, and why does that matter for choosing α?

7. [Code] Implement Algorithm 1 in JAX for $J(\theta) = \theta^2$ and print δ at each halving. Confirm your hand answer to Exercise 2.

Core

8. [Theory] Prove Theorem 4.1.1 in full, stating where Cauchy–Schwarz is used and why a fixed length for $\boldsymbol{p}$ is needed for the statement to make sense.

9. [Theory] Apply the iteration with *fixed* step α to $J(\boldsymbol{\theta}) = \frac{1}{2}\boldsymbol{\theta}^\mathsf{T}\boldsymbol{A}\boldsymbol{\theta}$ with $\boldsymbol{A} \succ 0$ symmetric. Show $\boldsymbol{\theta}^j = (\mathrm{I} - \alpha\boldsymbol{A})^j\boldsymbol{\theta}^0$ and deduce convergence if and only if $0 < \alpha < 2/\lambda_{\max}(\boldsymbol{A})$. Show the best rate occurs at $\alpha = 2/(\lambda_{\min} + \lambda_{\max})$ and depends on $\kappa = \lambda_{\max}/\lambda_{\min}$.

10. [Theory] Show that consecutive steepest-descent directions produced by an *exact* line search are orthogonal. This is the zig-zag of Figure 4.2, proved.

11. [Theory] Prove that Algorithm 1 terminates in finitely many halvings whenever $\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}^0) \neq \mathbf{0}$, and exhibit what happens when the gradient does vanish.

12. [Theory] Proposition 4.4.1 assumes the sequence converges. Give a smooth J bounded below for which the fixed-step iteration diverges, and reconcile this with Algorithm 2, which cannot diverge.

13. [Theory] Show that criteria (iii) and (iv) of Section 5.3.1 can both be satisfied at a point where $\|\nabla_{\boldsymbol{\theta}} J\|$ is large, by constructing a scalar example. This is the pitfall stated there.

14. [Theory] Show that momentum with β close to 1 behaves, over many iterations, like plain gradient descent with an effective step size $\alpha/(1-\beta)$. Use this to explain why increasing β without decreasing α can destabilize the iteration, as it did for $\beta = 0.85$ in Section 4.5.

15. [Code] Implement Algorithm 2 in JAX with `jax.jit` and apply it to the linear regression problem of Chapter 3. Verify it converges to (3.5). Plot loss against iteration for several fixed α, including one that diverges, and confirm the threshold predicted by Exercise 9.

Challenge

16. [Theory] For the quadratic of Exercise 9, show the error contracts by a factor $(\kappa-1)/(\kappa+1)$ per iteration in the worst case. How many iterations are needed for one digit when $\kappa = 10$? When $\kappa = 10^4$? Relate this to Figure 3.3b.

17. [Theory] Steepest descent is optimal to *first* order. Derive the Newton step by minimizing the second-order Taylor model instead, and state what must be true of $\boldsymbol{H}$ for the step to be a descent direction at all.

18. [Theory] (*Defer until after Chapter 6.*) Show that gradient descent applied to the ridge objective (6.1) is equivalent to gradient descent on the unregularized loss with an extra shrinkage of $\boldsymbol{\theta}$ at each step. What does this say about the effect of λ on the admissible step size?

19. [Theory] Prove that if J is convex, every limit point of Algorithm 2 is a global minimizer. Which chapter's result does this combine with Proposition 4.4.1?

20. [Code] Reproduce Figure 4.2. Implement Algorithms 1 and 2 for $J = 2\theta_1^2 + 2\theta_1\theta_2 + \theta_2^2 + \theta_1 - \theta_2$ from $\boldsymbol{\theta}^0 = (2.6, -4.6)$, overlay the path on the contours, and check numerically that consecutive steps are orthogonal. Report the iteration count and compare with $\kappa(\boldsymbol{H})$.

21. [Theory] For the quadratic $J(\boldsymbol{\theta}) = \tfrac{1}{2}\boldsymbol{\theta}^{\mathsf{T}}\boldsymbol{A}\boldsymbol{\theta}$, write the momentum iteration as a linear recursion on the stacked vector $(\boldsymbol{\theta}^{j+1}, \boldsymbol{\theta}^{j})$ and show that convergence is governed by the spectral radius of the resulting $2d \times 2d$ matrix. Deduce that the optimal parameters are $\beta = \big((\sqrt{\kappa}-1)/(\sqrt{\kappa}+1)\big)^2$ and $\alpha = 4/(\sqrt{\lambda_{\max}} + \sqrt{\lambda_{\min}})^2$, giving a rate depending on $\sqrt{\kappa}$ rather than κ. Compare with Exercise 19 and say how much is gained when $\kappa = 10^4$.

22. [Code] Reproduce Figure 4.4. Implement Algorithm 3 on the same quadratic and starting point, and plot iteration count against β for $\beta \in \{0, 0.1, \dots, 0.95\}$ at fixed α. Identify the minimum empirically and compare it with the optimal β of Exercise 21. Explain the sharp rise at large β.

Part II

Probability and Statistical Learning

Chapter 5

Probability

Goal of this chapter

Everything so far has been deterministic: a fixed dataset, a fixed loss, an iteration that converges or does not. But the data in Chapter 3 was noisy, and we never said what noise *is*. From here on the notes need a language for uncertainty, and this chapter supplies it.
It is the hinge of the course. Everything before it treats $\mathcal{D}$ as given; everything after it treats $\mathcal{D}$ as a sample.
It builds, in order:

1. probability spaces, and the rules for combining events;
2. random variables, distributions, and densities;
3. expectation, variance, and covariance;
4. the law of large numbers — which says that averaging samples works, and how fast.

The last is the one the rest of the notes leans on hardest: it is the license for every $\approx$ in Chapter 7, and the reason Monte Carlo estimation converges at all.

Note:-

Some sections are marked **Going deeper** and set in a shaded box. They give the measure-theoretic formulation, which is the right language if you continue into uncertainty quantification, inverse problems, or Markov chain Monte Carlo. Nothing later in these notes depends on them, and they can be skipped on a first reading without any loss of continuity.

§ 5.1 Why probability

Deterministic models cannot express uncertainty, and there is more than one theory that can — decision theory and fuzzy logic among them. We use probability, for the reason Maxwell gave: it is the language of science.

Definition 5.1.1: Deterministic and random events

An event is deterministic *if its outcome is completely determined or known. An event is* random *if its outcome is not completely known.*

Example 5.1.1 (Which is which)

Deterministic: today is Wednesday, so "tomorrow is Thursday"; a coin has a head and a tail; rolling a die gives a number at most six. Random: which face the die shows; whether tomorrow is colder.

§ 5.2 The probability space

Probability is set theory with a normalization condition. Before the definitions, it is worth seeing the correspondence in a case you already know.

Set theory, with $\mathbb{N}$	Probability
the set $\mathbb{N}$	the *sample space* Ω
singletons $\{1\}, \{2\}, \ldots$	elementary outcomes ω
a subset such as $\{1, 2, 3\}$	an *event*
the collection $2^{\mathbb{N}}$ of all subsets	a *σ-algebra* $\mathcal{F}$ of events
a map $f : \mathbb{N} \to \{0, 1\}$	a *probability measure* $\mathbb{P}$

Nothing in probability is new machinery; it is sets, subsets, and a function that assigns each subset a number.

Definition 5.2.1: Probability space

$(\Omega, \mathcal{F}, \mathbb{P})$ *is a* probability space *when*

1. Ω, *the* sample space, *contains all possible outcomes ω of an elementary event;*
2. $\mathcal{F}$, *the* σ-algebra, *contains all events $A \subseteq \Omega$ that can be assigned a probability;*
3. $\mathbb{P}$, *the* probability measure, *is a map $\mathbb{P} : \mathcal{F} \to [0, 1]$ with*

$$\mathbb{P}[\emptyset] = 0, \qquad \mathbb{P}[\Omega] = 1,$$

and, for disjoint (mutually exclusive) events $A_1, \ldots, A_n$,

$$\mathbb{P}\Big[\bigcup_{i=1}^{n} A_i\Big] = \sum_{i=1}^{n} \mathbb{P}[A_i]. \tag{5.1}$$

Example 5.2.1 (Flipping a fair coin)

$\Omega = \{H, T\}$, $\mathcal{F} = \{\emptyset, \{H\}, \{T\}, \Omega\}$, and

$$\mathbb{P}[\emptyset] = 0, \quad \mathbb{P}[H] = \tfrac{1}{2}, \quad \mathbb{P}[T] = \tfrac{1}{2}, \quad \mathbb{P}[\Omega] = 1.$$

Going deeper — optional

The σ-algebra $\mathcal{F}$ is required to be closed under two operations:

$$\text{(a) complementation: } A \in \mathcal{F} \implies A' \in \mathcal{F}, \qquad \text{(b) countable union: } A_i \in \mathcal{F} \implies \Big(\bigcup_{i=1}^{\infty} A_i\Big) \in \mathcal{F}.$$

Closure is what makes the axioms consistent: if we can speak of A, we must be able to speak of "not A", and of "at least one of the A_i". Additivity (5.1) then extends from finite to countable unions. For finite or countable Ω one may simply take $\mathcal{F} = 2^{\Omega}$; the subtlety only arises for continuous Ω, where not every subset can be assigned a length.

Lemma 5.2.1 Complement rule

For $A \in \mathcal{F}$, $\mathbb{P}[A'] = 1 - \mathbb{P}[A]$.

Proof: $A' \cap A = \emptyset$ and $A' \cup A = \Omega$, so by (5.1) and $\mathbb{P}[\Omega] = 1$, $\mathbb{P}[A'] + \mathbb{P}[A] = \mathbb{P}[A' \cup A] = \mathbb{P}[\Omega] = 1$. ∎

§ 5.3 Combining events

5.3.1 The sum rule

Lemma 5.3.1 Sum rule

For any $A, B \in \mathcal{F}$,

$$\mathbb{P}[A \cup B] = \mathbb{P}[A] + \mathbb{P}[B] - \mathbb{P}[A \cap B].$$

Proof: Write $A \cup B = A \cup (B \setminus (A \cap B))$, a union of disjoint sets, so

$$\mathbb{P}[A \cup B] = \mathbb{P}[A] + \mathbb{P}[B \setminus (A \cap B)]. \tag{5.2}$$

Separately $B = (A \cap B) \cup (B \setminus (A \cap B))$, again disjoint, so

$$\mathbb{P}[B] = \mathbb{P}[A \cap B] + \mathbb{P}[B \setminus (A \cap B)]. \tag{5.3}$$

Subtracting (5.3) from (5.2) gives the result. ∎

Let us see this geometrically. The rule is visible in a Venn diagram: adding the areas of A and B counts the overlap twice, so it must be subtracted once.

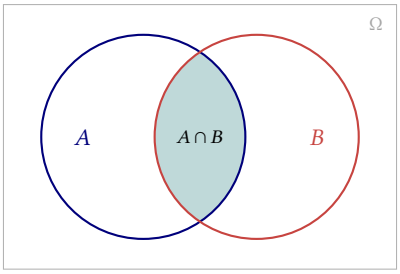


Figure 5.1: **The sum rule.** Read $\mathbb{P}$ as normalized area, with $\mathbb{P}[\Omega] = 1$. Adding the area of A to the area of B counts the shaded overlap twice, so $\mathbb{P}[A \cap B]$ must be subtracted once. When the events are disjoint the overlap is empty and (5.1) is recovered.

5.3.2 The union bound

Since $\mathbb{P}[A \cap B] \geq 0$, Lemma 5.3.1 immediately gives $\mathbb{P}[A \cup B] \leq \mathbb{P}[A] + \mathbb{P}[B]$. The general statement below is the engine of Theorem 6.3.1 in the next chapter, and is worth more attention than its one-line proof suggests.

Lemma 5.3.2 Union bound

For $A_1, \ldots, A_n \in \mathcal{F}$,

$$\mathbb{P}\Big[\bigcup_{i=1}^{n} A_i\Big] \leq \sum_{i=1}^{n} \mathbb{P}[A_i].$$

Proof: Build disjoint sets carrying the same union:

$$\bar{A}_1 = A_1, \quad \bar{A}_2 = A_2 \setminus A_1, \quad \ldots, \quad \bar{A}_i = A_i \setminus \Big(\bigcup_{j=1}^{i-1} A_j\Big).$$

By construction the $\bar{A}_i$ are disjoint, $\bigcup_i \bar{A}_i = \bigcup_i A_i$, and $\bar{A}_i \subseteq A_i$. Hence by (5.1)

$$\mathbb{P}\Big[\bigcup_i A_i\Big] = \mathbb{P}\Big[\bigcup_i \bar{A}_i\Big] = \sum_i \mathbb{P}[\bar{A}_i] \leq \sum_i \mathbb{P}[A_i],$$

the inequality because $\bar{A}_i \subseteq A_i$. ∎

Key idea

The union bound is crude — it ignores all overlap — and that is exactly why it is useful: it needs no independence assumption whatsoever. This is what will make Theorem 6.3.1 possible in Chapter 6, where the H bad models may overlap in arbitrarily complicated ways and we still get a bound.

Example 5.3.1 (How crude is "crude"?)

A launch vehicle carries 200 components whose individual failure probabilities over one mission are each 10^{-4}. What is the probability that *something* fails?

The union bound answers immediately: $\mathbb{P}[\text{some failure}] \le 200 \times 10^{-4} = 0.0200$, and it answers *without knowing how the components interact* — they may share a power bus, a thermal environment, a manufacturing lot.

If they happened to be independent the exact answer would be $1 - (1 - 10^{-4})^{200} = 0.019802$. The bound is loose by a factor of 1.0100. This is the typical situation when the individual probabilities are small: the union bound gives up almost nothing, and in exchange it gives up the independence assumption entirely — which in a real vehicle would have been the hardest part to justify.

Keep this example in mind at Theorem 6.3.1. There the $\mathbb{P}[A_i]$ are the probabilities that particular bad models fit the training data, the events overlap in ways nobody can characterize, and the same crudeness is what rescues the argument.

5.3.3 Conditional probability and independence

Definition 5.3.1: Conditional probability

For $\mathbb{P}[B] \neq 0$, the probability of A given that B has happened is

$$\mathbb{P}[A \mid B] := \frac{\mathbb{P}[A \cap B]}{\mathbb{P}[B]}.$$

(Kolmogorov.)

Example 5.3.2 (Rolling a fair die)

Let A be "the face is 6" and B "the face exceeds 4". Then $\mathbb{P}[B] = \frac{2}{6}$, $\mathbb{P}[A] = \frac{1}{6}$, and $\mathbb{P}[A \cap B] = \mathbb{P}[A] = \frac{1}{6}$ since $A \subset B$. So

$$\mathbb{P}[A \mid B] = \frac{1/6}{2/6} = \frac{1}{2}.$$

Intuitively: once B is known, only 5 and 6 remain, and they are equally likely.

Note:-

Conditioning is nothing more than probability restricted to a smaller sample space. Given B, the pair $(B, B \cap \mathcal{F})$ is a probability space in its own right, with measure $\tilde{\mathbb{P}}[C] = \mathbb{P}[C]/\mathbb{P}[B]$ — the division being exactly the renormalization needed to make $\tilde{\mathbb{P}}[B] = 1$.

Definition 5.3.2: Independent events

$A, B \in \mathcal{F}$ are independent *if* $\mathbb{P}[A \cap B] = \mathbb{P}[A]\,\mathbb{P}[B]$.

Comparing with Definition 5.3.1, independence says exactly $\mathbb{P}[A \mid B] = \mathbb{P}[A]$: knowing B tells us nothing about A.

Common pitfall

It is tempting to multiply probabilities whenever two events "look" unrelated. Take A and B both to be "the coin shows heads", each with probability $\frac{1}{2}$.

- *Two separate coins:* $\mathbb{P}[A \cap B] = \frac{1}{4} = \mathbb{P}[A]\mathbb{P}[B]$. Independent.
- *One coin, one toss:* $A = B$, so $\mathbb{P}[A \cap B] = \mathbb{P}[A] = \frac{1}{2} \neq \frac{1}{4}$.
- *Two coins welded tails-out:* $\mathbb{P}[A \cap B] = 0$.

Same events by description, three different answers. Independence is a property of the probability measure, not of how the events are worded.

Example 5.3.3 (Causally connected yet independent)

Roll a fair four-faced die with faces labeled HH, HT, TH, TT, each of probability $\frac{1}{4}$. Let A be "first letter is H" and B "second letter is H". Then $\mathbb{P}[A] = \mathbb{P}[B] = \frac{1}{2}$ and $\mathbb{P}[A \cap B] = \mathbb{P}[HH] = \frac{1}{4} = \mathbb{P}[A]\mathbb{P}[B]$. The two events are independent even though they are produced by a single physical act.

Theorem 5.3.3 Law of total probability

Let $\Omega = \bigcup_{i=1}^{n} A_i$ with the A_i mutually disjoint. Then for any $B \in \mathcal{F}$,

$$\mathbb{P}[B] = \sum_{i=1}^{n} \mathbb{P}[B \mid A_i]\, \mathbb{P}[A_i].$$

Proof: By Definition 5.3.1, $\mathbb{P}[B \cap A_i] = \mathbb{P}[A_i]\mathbb{P}[B \mid A_i]$. Summing and using that the sets $B \cap A_i$ are mutually disjoint,

$$\sum_i \mathbb{P}[B \mid A_i]\mathbb{P}[A_i] = \sum_i \mathbb{P}[B \cap A_i] = \mathbb{P}\Big[\bigcup_i (B \cap A_i)\Big] = \mathbb{P}\Big[B \cap \bigcup_i A_i\Big] = \mathbb{P}[B \cap \Omega] = \mathbb{P}[B].$$

■

Example 5.3.4 (Where the risk actually comes from)

A component's failure rate depends on the flight regime. Suppose a vehicle spends 70% of its time in cruise, 20% climbing and 10% descending, with per-mission failure probabilities 10^{-5}, 5×10^{-5} and 2×10^{-5} respectively. Partitioning by regime,

$$\mathbb{P}[\text{failure}] = 0.70(10^{-5}) + 0.20(5 \times 10^{-5}) + 0.10(2 \times 10^{-5}) = 1.9 \times 10^{-5}.$$

Notice what the decomposition exposes: climb accounts for a fifth of the time but 53% of the risk. An average failure rate quoted without the partition would have hidden that, and the engineering effort would have gone to the wrong regime.

This is a mixture, and it is the same object Chapter 9 works with: there the A_i are class labels, $\mathbb{P}[A_i]$ are class priors, and $\mathbb{P}[B \mid A_i]$ become class-conditional densities.

Example 5.3.5 (A positive test that means very little)

Welds in a structure are inspected ultrasonically. Historically 1% of welds carry a crack. The inspection detects an existing crack with probability 0.95, and flags a sound weld with probability 0.05.

A weld tests positive. What is the probability it is cracked? By Theorem 5.3.3 the chance of a positive reading at all is

$$\mathbb{P}[+] = 0.95(0.01) + 0.05(0.99) = 0.0590,$$

and then by Definition 5.3.1,

$$\mathbb{P}[\text{crack} \mid +] = \frac{0.95 \times 0.01}{0.0590} = 0.161.$$

A positive result from a test that catches 95% of cracks leaves a 16% chance the weld is actually cracked. The reason is that sound welds outnumber cracked ones 99 to 1, so the 5% of them falsely flagged (0.0495) swamp the 0.0095 of true detections. To reach even a coin-flip posterior the false-positive rate would have to fall from 0.05 to 0.0096. *The base rate is not a detail.* A classifier evaluated only on sensitivity looks excellent here and is nearly useless. This example is worth remembering through Chapter 8, where accuracy on imbalanced classes is exactly this trap, and Chapter 9, which makes the prior an explicit parameter rather than something to be forgotten.

Note:-

Theorem 5.3.3 is used in Chapter 9 to show that the class priors and class-conditional densities of Gaussian discriminant analysis combine into a Gaussian mixture. Combined with Definition 5.3.1 it also yields Bayes' formula, which Chapter 7 takes as its starting point.

§ 5.4 Random variables

Definition 5.4.1: Random variable

A random variable *is a map* $M : \Omega \longrightarrow S$ *from the sample space to a* state space S*,*

$$\omega \;\mapsto\; M(\omega) \in S.$$

That is, a random variable is a function whose input *is random.*

Key idea

A random variable is neither random nor a variable: it is an ordinary deterministic function. All the randomness lives in its argument ω. The name is historical and misleads every year.

Example 5.4.1 (Summing two dice)

Roll two fair dice and let M be the sum. The sample space $\Omega = \{(1,1),(1,2),\dots,(6,6)\}$ has 36 equally likely elements, and the state space is $S = \{2,3,\dots,12\}$. For $A = \{11\}$ the inverse image is $M^{-1}(A) = \{(5,6),(6,5)\}$, so $\mathbb{P}[M^{-1}(A)] = \frac{2}{36} = \frac{1}{18}$. For $A = \{M : M < 5\}$ the inverse image is $\{(1,1),(1,2),(1,3),(2,1),(2,2),(3,1)\}$, of probability $\frac{1}{6}$.

The example shows the pattern: to assign a probability to a set A in the state space, take the probability of its inverse image in Ω, since A happens exactly when $M^{-1}(A)$ does.

Going deeper — optional

Formally, M must be *measurable*: the inverse image of every measurable set in S is measurable in Ω. (Compare the topological definition of continuity, where inverse images of open sets are open.) The assignment above then defines the *probability distribution* or *law* of M,

$$\mu_M[A] \coloneqq \mathbb{P}\big[\{\omega : M(\omega) \in A\}\big], \qquad A \subseteq S,$$

and one checks directly that μ_M satisfies Definition 5.2.1: it is the *push-forward* of $\mathbb{P}$ through M, written $\mu_M = M_{\#}\mathbb{P}$. When a density exists, $\mu_M[A] = \int_A \pi(m)\,\mathrm{d}m$, and $\pi = \mathrm{d}\mu_M/\mathrm{d}m$ is the Radon–Nikodym derivative — the generalization of the fundamental theorem of calculus that says μ_M is absolutely continuous with respect to Lebesgue measure.

Definition 5.4.2: Density and mass

For a continuous *random variable, the* probability density $\pi(m)$ *satisfies*

$$\mathbb{P}[M \in A] = \int_A \pi(m)\,\mathrm{d}m, \qquad A \subseteq S.$$

For a discrete *random variable one works instead with the* probability mass $\mathbb{P}[M = m]$ *of each individual outcome.*

Example 5.4.2 (Two densities we will use constantly)

Uniform: $M \sim \mathcal{U}[a,b]$ has $\pi(m) = 1/(b-a)$ on $[a,b]$.
Gaussian: $M \sim \mathcal{N}(\bar{m}, \sigma^2)$ has

$$\pi(m) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{1}{2}\left(\frac{m-\bar{m}}{\sigma}\right)^2\right).$$

Note:-

Once π is known the sample space can be forgotten entirely: every probability of interest is an integral of π over the state space. This is why applied work rarely mentions Ω at all, and why these notes will not either after this chapter.

§ 5.5 Expectation, variance, covariance

Definition 5.5.1: Expectation

The expectation *(mean, average) of a random variable M is*

$$\mathbb{E}[M] := \int_S m\,\pi(m)\,\mathrm{d}m \qquad \textit{(continuous)}, \qquad \mathbb{E}[M] := \sum_{i=1}^{n} m_i\,\mathbb{P}[M = m_i] \qquad \textit{(discrete)}.$$

Example 5.5.1 (Rolling a die)

$S = \{1, \ldots, 6\}$ with equal masses, so $\mathbb{E}[M] = (1+2+3+4+5+6)/6 = \frac{7}{2}$. Summing two dice gives $\mathbb{E}[M] = 7$.

Lemma 5.5.1 Linearity of expectation

For $a, b \in \mathbb{R}$ *and random variables* M, N,

$$\mathbb{E}[aM + bN] = a\mathbb{E}[M] + b\mathbb{E}[N].$$

Note:-

Linearity holds whether or not M and N are independent, which is what makes it so useful. It follows immediately from linearity of the integral in the continuous case, and from the law of total probability in the discrete case.

Proposition 5.5.2 Expectation of a product

If X *and* Y *are independent, then* $\mathbb{E}[XY] = \mathbb{E}[X]\mathbb{E}[Y]$.

Proof: In the discrete case, independence gives $\mathbb{P}[X = x, Y = y] = \mathbb{P}[X = x]\mathbb{P}[Y = y]$, so

$$\mathbb{E}[XY] = \sum_x \sum_y xy\,\mathbb{P}[X = x]\mathbb{P}[Y = y] = \Big(\sum_x x\mathbb{P}[X = x]\Big)\Big(\sum_y y\mathbb{P}[Y = y]\Big) = \mathbb{E}[X]\mathbb{E}[Y].$$

■

Common pitfall

Independence is essential here. Let X be one die roll. Then

$$\mathbb{E}[X \cdot X] = \mathbb{E}[X^2] = \tfrac{1}{6}(1^2 + \cdots + 6^2) = \tfrac{91}{6} \approx 15.2,$$

whereas $(\mathbb{E}[X])^2 = (7/2)^2 = 12.25$. A variable is emphatically not independent of itself, and the gap between the two numbers is exactly its variance.

Example 5.5.2 (Counting failures without knowing the dependence)

Return to the 200 components. Let $X_i = 1$ if component i fails and 0 otherwise, so $\mathbb{E}[X_i] = 10^{-4}$, and let $S = \sum_i X_i$ count the failures. By Lemma 5.5.1,

$$\mathbb{E}[S] = \sum_{i=1}^{200} \mathbb{E}[X_i] = 0.0200,$$

and *no independence was used.* The expected number of failures is the same whether the components are independent, share a single power supply, or come from one bad production batch.

The variance is another matter entirely. If the X_i are independent, $\mathrm{Var}[S] = \sum_i p_i(1-p_i) = 0.019998$. If instead the components fail all-or-none — one common cause takes out everything with probability 10^{-4} — then S is 200 with probability 10^{-4} and 0 otherwise, giving $\mathrm{Var}[S] = 3.9996$, larger by a factor of 200. Same mean, wildly different risk.

This separation is worth holding on to: Lemma 5.5.1 is free, and everything that needs independence is expensive. Chapter 10 leans on exactly this. Proposition 10.3.1 says a mini-batch gradient is unbiased, and its proof uses only linearity — so it holds however the samples in a batch are chosen. Proposition 10.3.2, the companion statement about variance, is where independence finally has to be paid for.

Definition 5.5.2: Covariance and variance

$$\mathrm{Cov}(X,Y) := \mathbb{E}\big[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])\big], \qquad \mathrm{Var}[M] := \mathbb{E}\big[(M - \bar{m})^2\big],$$

where $\bar{m} = \mathbb{E}[M]$. *Thus* $\mathrm{Var}[M] = \mathrm{Cov}(M, M)$.

Proposition 5.5.3 Computational form

$\mathrm{Cov}(X,Y) = \mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y]$, *and in particular* $\mathrm{Var}[M] = \mathbb{E}[M^2] - (\mathbb{E}[M])^2$.

Proof: Expand the product and apply Lemma 5.5.1, using that $\mathbb{E}[X]$ and $\mathbb{E}[Y]$ are constants:

$$\mathrm{Cov}(X,Y) = \mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y] - \mathbb{E}[X]\mathbb{E}[Y] + \mathbb{E}[X]\mathbb{E}[Y] = \mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y].$$

■

Let us see this geometrically. Covariance is a single number, but what it measures is the *tilt* of a cloud of points. Three samples from the same marginals, differing only in how the two coordinates are coupled, make the sign and the magnitude visible.



(a) $\mathrm{Cov}(X, Y) = +0.71$ (b) $\mathrm{Cov}(X, Y) = +0.07$ (c) $\mathrm{Cov}(X, Y) = -0.80$

Figure 5.2: **What covariance measures.** Each panel shows 95 draws from a bivariate normal with the same marginals — both coordinates have mean 0 and variance 1 — so the three clouds differ only in how X and Y are coupled. Sample covariances are computed from the points shown, not chosen. **(a)** Large positive covariance: the cloud tilts along the diagonal, and knowing X is above its mean makes it likely Y is too. **(b)** Independent coordinates. By Proposition 5.5.2 and Proposition 5.5.3 the true covariance is 0; the measured value is +0.07 rather than exactly zero because 95 samples estimate it only to $O(1/\sqrt{n})$ — an instance of Theorem 5.6.2. **(c)** Large negative covariance: the same coupling with the sign reversed. What none of the three panels can show is that a cloud may be strongly *dependent* and still have covariance zero; covariance sees linear coupling only.

Combining Proposition 5.5.3 with Proposition 5.5.2: independent variables have zero covariance. The converse is false, and Exercise 14 asks for a counterexample.

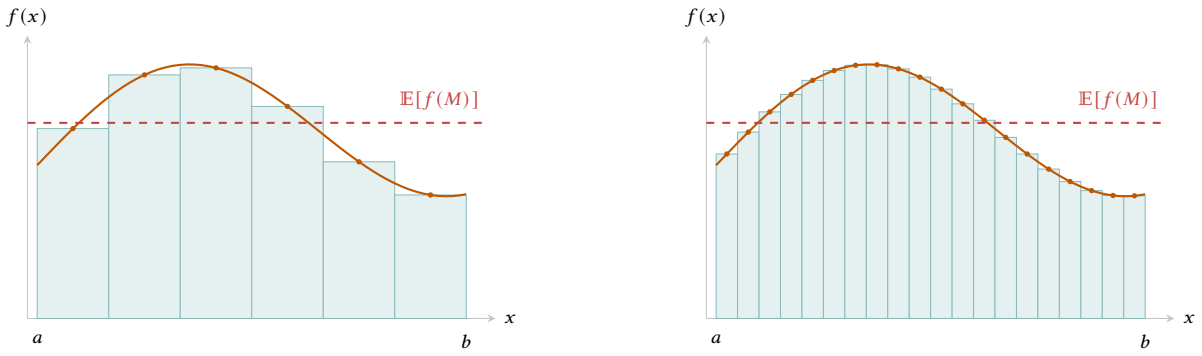
§ 5.6 The law of large numbers

We now reach the result the rest of these notes depends on. It answers: if we cannot compute an expectation, may we estimate it by averaging samples?

Let us see this geometrically. The connection between averaging and integration is older than probability. Subdivide $[a, b]$ into n pieces of width $\Delta x = (b - a)/n$ with midpoints ξ_i. The average of f at those midpoints is

$$\hat{f} = \frac{1}{n} \sum_{i=1}^{n} f(\xi_i) = \frac{1}{b-a} \sum_{i=1}^{n} f(\xi_i)\, \Delta x \xrightarrow[n\to\infty]{} \frac{1}{b-a} \int_a^b f(x)\, \mathrm{d}x,$$

by Riemann integrability. But $1/(b - a)$ is precisely the density of $M \sim \mathcal{U}[a, b]$, so the right-hand side is $\mathbb{E}[f(M)]$. An elementary average has turned into an expectation.



(a) $n = 6$: average 1.4711.

(b) $n = 20$: average 1.4681.

Figure 5.3: **An average is an integral in disguise.** The function $f(x) = 1.15 + 0.62\sin(2.4x) + 0.20x$ on $[a, b] = [0, 2]$, sampled at the midpoints of n equal subintervals. The shaded rectangles have total area $\sum_i f(\xi_i)\Delta x$, so their *mean height* is exactly the average $\hat{f} = \frac{1}{n}\sum_i f(\xi_i)$. As n grows the rectangles fill the region under the curve and that mean height converges to the dashed line, which is $\frac{1}{b-a}\int_a^b f = \mathbb{E}[f(M)] = 1.46786$ for $M \sim \mathcal{U}[a, b]$. **(a)** Six subintervals already give 1.4711, an error of 3.2×10^{-3}. **(b)** Twenty give 1.4681, an error of 2.8×10^{-4}. Fitting the error over $n = 10$ to 320 gives order -2.00, the midpoint rule's rate. The point is not the rate but the identification: nothing probabilistic has happened here, yet an elementary average has produced an expectation. Theorem 5.6.1 says the same identification survives when the ξ_i are drawn at random rather than placed on a grid — and Figure 7.1 shows what that costs.

Definition 5.6.1: Independent and identically distributed

$M_1, \ldots, M_N$ *are* i.i.d. *draws from* π *if each* M_i *is distributed according to* π *and the draws are mutually independent.*

Theorem 5.6.1 Strong law of large numbers

Let M_i *be i.i.d. from* π *with finite first two moments,* $\bar{m} = \mathbb{E}[M] < \infty$ *and* $\sigma^2 = \mathbb{E}[(M - \bar{m})^2] < \infty$. *Define the* sample average

$$S_N := \frac{1}{N}\sum_{i=1}^{N} M_i.$$

Then $S_N \to \bar{m}$ *as* $N \to \infty$, *with probability one.*

Going deeper — optional

"With probability one" means *almost sure* convergence:

$$\mathbb{P}\Big[\{\omega : \lim_{N\to\infty} S_N(\omega) = \bar{m}\}\Big] = 1,$$

that is, the set of sample paths on which the average fails to converge has measure zero. Such sets are not empty — for $M \sim \mathcal{U}[0, 1]$ the countable set $\{1/n\}_{n=1}^{\infty}$ has probability zero yet contains infinitely many points — they are merely negligible.

A full proof needs techniques beyond these notes. Assuming in addition a bounded fourth moment, $\mathbb{E}[M_i^4] < \infty$, one can argue as follows (take $\bar{m} = 0$ without loss of generality). Expanding the fourth power and discarding the terms that vanish by independence and zero mean,

$$\mathbb{E}\Big[\Big(\sum_{i=1}^{n} M_i\Big)^4\Big] = n\mathbb{E}[M_1^4] + 3(n^2 - n)\big(\mathbb{E}[M_1^2]\big)^2 \le n^2 C.$$

Markov's inequality then gives, for any $\varepsilon > 0$,

$$\mathbb{P}\Big[\Big|\tfrac{1}{n}\sum_i M_i\Big| \ge \varepsilon\Big] = \mathbb{P}\Big[\Big(\sum_i M_i\Big)^4 \ge (\varepsilon n)^4\Big] \le \frac{1}{(\varepsilon n)^4}\,\mathbb{E}\Big[\Big(\sum_i M_i\Big)^4\Big] \le \frac{C}{\varepsilon^4 n^2},$$

which is summable in n; the Borel–Cantelli lemma upgrades this to almost sure convergence.

5.6.1 How fast? The Monte Carlo rate

The law of large numbers is asymptotic. For finite N we want the error, and here the computation is elementary — it needs only Lemma 5.5.1 and independence.

Theorem 5.6.2 Mean square error of the sample average

Under the hypotheses of Theorem 5.6.1,

$$\mathbb{E}\big[(S_N-\bar m)^2\big]=\operatorname{Var}[S_N]=\frac{\sigma^2}{N},\qquad \textit{so}\qquad \operatorname{MSE}(S_N)=\frac{\sigma}{\sqrt{N}}.$$

Proof: By the definition of S_N,

$$\mathbb{E}\big[(S_N-\bar m)^2\big]=\frac{1}{N^2}\mathbb{E}\Big[\sum_i (M_i-\bar m)\sum_j (M_j-\bar m)\Big]=\frac{1}{N^2}\Big(\underbrace{\mathbb{E}\Big[\sum_i (M_i-\bar m)^2\Big]}_{N\sigma^2}+\mathbb{E}\Big[\sum_{i\neq j}(M_i-\bar m)(M_j-\bar m)\Big]\Big).$$

For $i\neq j$ the variables are independent, so by Proposition 5.5.2

$$\mathbb{E}\big[(M_i-\bar m)(M_j-\bar m)\big]=\mathbb{E}[M_i-\bar m]\,\mathbb{E}[M_j-\bar m]=(\bar m-\bar m)^2=0,$$

and every cross term vanishes. Hence $\mathbb{E}[(S_N-\bar m)^2]=N\sigma^2/N^2=\sigma^2/N$. ∎

Take-home message

The Monte Carlo error decays like $\sigma/\sqrt{N}$ — and the rate does not depend on dimension.

More generally, for any function f,

$$S_N=\frac{1}{N}\sum_{i=1}^{N} f(m_i)\;\longrightarrow\;\mathbb{E}[f(M)],\qquad \operatorname{MSE}(S_N)=\frac{\sqrt{\operatorname{Var}[f]}}{\sqrt{N}}.$$

The rate is slow: a further correct digit costs a hundredfold more samples. But it is *dimension independent*, which is why Monte Carlo remains the method of choice in high dimensions, where every grid-based quadrature rule fails — including the midpoint rule of Figure 5.3, whose order -2 in one dimension degrades to $-2/d$ in d. Figure 7.1 measures the $\sigma/\sqrt{N}$ rate on the estimator itself. Every $\approx$ in Chapter 7 is an appeal to this theorem, and $\sqrt{\operatorname{Var}[f]}$ is the constant that decides whether the appeal is cheap or expensive.

Example 5.6.1 (The crossover, in points)

The claim that dimension independence matters is worth making concrete. Suppose we want an integral to an RMS accuracy of 10^{-2}, with $\sqrt{\operatorname{Var}[f]}=1$.
Monte Carlo needs $N=(\sigma/\varepsilon)^2=10^4$ samples. That number does not change with d.
A grid rule of order 2 in one dimension — the midpoint rule of Figure 5.3 — converges at $N^{-2/d}$ once the grid is laid over d dimensions, so it needs $N=\varepsilon^{-d/2}$:

d	grid rule	Monte Carlo
1	10^1	10^4
2	10^2	10^4
4	10^4	10^4
10	10^{10}	10^4
20	10^{20}	10^4

The crossover is at $d=4$. Below it the grid rule wins easily and nobody sensible uses Monte Carlo; above it the

grid rule becomes impossible with a speed that is hard to overstate. At $d = 10$ it needs a million times more points, and at $d = 20$ the number exceeds the count of atoms in a gram of matter.
Every model in these notes lives above the crossover. A neural network in Chapter 12 has d in the millions, and Chapter 10 estimates an expectation over that space thousands of times per second by averaging over a handful of samples. It is the only method available, and this theorem is why it works.

Example 5.6.2 (Estimating π)
Draw $X_i, Y_i \sim \mathcal{U}[0,1]$ independently and set $Z_i = 1$ if $X_i^2 + Y_i^2 \le 1$ and 0 otherwise. Then

$$\mathbb{E}[Z] = \mathbb{P}\left[X^2 + Y^2 \le 1\right] = \iint_{x^2+y^2\le 1} \mathrm{d}x\,\mathrm{d}y = \text{area of the quarter disc} = \frac{\pi}{4},$$

so by Theorem 5.6.1, $\frac{1}{N}\sum_i Z_i \to \pi/4$. This is the *indicator function* trick: the probability of an event is the expectation of its indicator, $\mathbb{E}[\mathbf{1}_A] = \mathbb{P}[A]$.

§ 5.7 Exercises

The code problems need only `jax.random`; see Section C.3 on seeding and reproducibility.

Warm-up

1. [Theory] Give an example of two events that are independent but not mutually exclusive. Then give two that are mutually exclusive but not independent. What does this say about the two notions?

2. [Theory] A fair die is rolled. Compute $\mathbb{P}[B]$ for B the event that the face exceeds 4, using Lemma 5.3.1 explicitly and identifying each term.

3. [Theory] Verify Lemma 5.2.1 on the coin example, and use it to show $\mathbb{P}[A] \le 1$ for every $A \in \mathcal{F}$.

4. [Theory] For M the sum of two fair dice, compute $\mathbb{P}[M = 2]$, $\mathbb{P}[M = 3]$ and $\mathbb{P}[M = 7]$ from first principles by counting inverse images, and confirm $\mathbb{E}[M] = 7$.

5. [Theory] State which of Ω, $\mathcal{F}$, $\mathbb{P}$, S and π you would need in order to compute $\mathbb{E}[f(M)]$, and which you could dispense with. Justify against Definition 5.5.1.

6. [Theory] In Example 5.3.1, the union bound gave $\mathbb{P}[\text{some failure}] \le 0.0200$ for $m = 200$ components at 10^{-4} each. Recompute the bound and, assuming independence, the exact value for $m = 2000$ and $m = 5000$. At what m does the bound stop saying anything at all, and what is the exact probability there?

7. [Theory] In Example 5.3.4 the vehicle spends 70/20/10 percent of its time in cruise, climb and descent. Suppose a revised mission doubles the climb fraction at the expense of cruise. Recompute the total failure probability, and recompute climb's share of the risk. Which would reduce total risk more: halving the climb failure rate, or halving the cruise one?

8. [Code] Simulate 10^5 rolls of a fair die in JAX with an explicit PRNG key and estimate $\mathbb{E}[M]$. Repeat with three different seeds and report the spread. Compare with the exact value $7/2$.

Core

9. [Theory] Prove Lemma 5.3.2 a second way, by induction on n using Lemma 5.3.1.

10. [Theory] Prove that $\mathbb{P}\left[\bigcap_{i=1}^n A_i\right] \ge 1 - \sum_{i=1}^n \mathbb{P}[A_i']$. *Hint:* De Morgan, then Lemma 5.3.2.

11. [Theory] (What rescues a screening test.) Let A be "tests positive" and B "has the disease", with $\mathbb{P}[A \mid B] = 0.99$, $\mathbb{P}[A' \mid B'] = 0.9999$ and $\mathbb{P}[B] = 0.01$. Compute $\mathbb{P}[B \mid A]$, and for a population of 10 000 count how many are sick, how many sick people test negative, and how many healthy people test positive.

Now compare with Example 5.3.5, which has the same prevalence and a similar sensitivity yet yields a posterior of only 0.161. Identify the single parameter responsible for the difference and say by what factor it changed. Then hold the prevalence at 0.01 and find the largest false-positive rate for which $\mathbb{P}[B \mid A] \geq 0.9$.

12. [Theory] Prove Lemma 5.5.1 for discrete random variables using Theorem 5.3.3.

13. [Theory] Show that $\operatorname{Var}[aM] = a^2 \operatorname{Var}[M]$ and, for independent M_i, $\operatorname{Var}\left[\sum_i a_i M_i\right] = \sum_i a_i^2 \operatorname{Var}[M_i]$. Where is independence used?

14. [Theory] Proposition 5.5.2 shows independence implies zero covariance. Show the converse fails: construct X and Y with $\operatorname{Cov}(X, Y) = 0$ that are not independent. *Hint:* try $Y = X^2$ with X symmetric about zero.

15. [Theory] Example 5.5.2 contrasts independent failures with an all-or-none common cause. Interpolate between them: let each $X_i = 1$ with probability p, but suppose with probability q a common cause forces every $X_i = 1$ at once, and otherwise the X_i are independent with probability p' chosen so the marginal stays p. Find p', then compute $\mathbb{E}[S]$ and $\operatorname{Var}[S]$ as functions of q. Confirm that $\mathbb{E}[S]$ does not depend on q and that $\operatorname{Var}[S]$ grows without bound as $q \to p$. Which of the two does a reliability requirement actually care about?

16. [Theory] A weld in Example 5.3.5 is inspected twice, by instruments whose errors are independent given the true state of the weld. Both readings are positive. Compute $\mathbb{P}[\text{crack} \mid ++]$ and compare with the 0.161 from a single reading. Then show that no finite number of such repetitions drives the posterior to 1, and identify what it converges to.

17. [Code] Estimate π by the method of Example 6.7.1: draw N points uniformly in the unit square and count those inside the quarter disc. Report the N needed for two correct digits, and plot the error against N on log–log axes alongside the line $N^{-1/2}$.

Challenge

18. [Theory] Prove the Cauchy–Schwarz inequality for random variables, $|\operatorname{Cov}(M, Y)| \leq \sqrt{\operatorname{Var}[M] \operatorname{Var}[Y]}$. *Hint:* consider $\operatorname{Var}[M + tY] \geq 0$ as a quadratic in t and use the discriminant. Compare with the use of Cauchy–Schwarz in Theorem 4.1.1.

19. [Theory] Let $G \sim \mathcal{N}(0, 1)$. Using the bound $\mathbb{P}[G > a] \leq \frac{1}{\sqrt{2\pi}} e^{-a^2/2}$ for $a > 1$, apply Lemma 5.3.2 to show $\mathbb{P}[|G| \leq a] \geq 1 - \sqrt{2/\pi}\, e^{-a^2/2}$. This is a *concentration* statement: a Gaussian is almost never far from its mean.

20. [Theory] Example 5.6.1 works out the Monte Carlo versus quadrature crossover for a rule of order $p = 2$ and finds it at $d = 4$. Generalize: for a tensor-product rule of order p, whose error is $O(N^{-p/d})$, derive the crossover dimension as a function of p and show that it is $2p$, independent of the target accuracy ε. Why does ε drop out, and what *does* depend on it? Finally, explain why no practical increase in p rescues the grid rule for the dimensions of Chapter 14.

21. [Theory] Prove $\mathbb{E}[\mathbf{1}_A] = \mathbb{P}[A]$ from Definition 5.5.1, and use it to express $\mathbb{P}[M > c]$ as an expectation. Then derive Markov's inequality, $\mathbb{P}[M \geq c] \leq \mathbb{E}[M]/c$ for $M \geq 0$.

22. [Code] Estimate π a second way, from $\pi/4 = \int_0^1 \sqrt{1 - x^2}\, \mathrm{d}x = \mathbb{E}[\sqrt{1 - X^2}]$ with $X \sim \mathcal{U}[0, 1]$. Report the N needed for two correct digits and compare with the previous exercise. Which converges faster, and why? Compute $\operatorname{Var}[f]$ for both estimators and check that the ratio of sample counts matches the ratio of variances predicted by Theorem 5.6.2.

23. [Code] Test the crossover of Example 5.6.1 directly. Integrate $f(\boldsymbol{x}) = \exp(-\|\boldsymbol{x}\|^2)$ over $[0, 1]^d$ for $d = 1, 2, 4, 8$, by a tensor-product midpoint rule and by Monte Carlo, and for each method plot error against the *number of function evaluations*, not against the grid size. Report the evaluation count each method needs to reach 10^{-2}, and identify the dimension at which the ordering reverses. Does it land where the example says? *Warning:* the tensor-product rule at $d = 8$ with a modest grid is already tens of millions of points — estimate the cost before running it.

24. [Code] Verify Theorem 5.6.2 empirically. For $M \sim \mathcal{U}[0, 1]$, estimate $\mathbb{E}[M]$ from N samples, repeating 500 times for each N to estimate the mean square error. Plot MSE against N on log–log axes, fit a slope, and compare with the predicted $-1/2$. Then repeat with a heavy-tailed distribution having infinite variance and explain what you observe.

Chapter 6

Generalization and Regularization

Goal of this chapter

Chapter 3 fitted data. It never said what fitting was *for*. This chapter supplies the missing purpose — we want models that work on data we have not seen — and then confronts the fact that minimizing training loss does not, on its own, deliver it.

The chapter answers four questions:

1. What does it mean for a model to be good, if not "small training loss"?
2. Why does supervised learning work at all? (There is a theorem.)
3. How does it fail? (Overfitting, and its opposite.)
4. What do we do about the failure? (Regularization.)

Chapter 5 supplied the language; this chapter is its first real use. The shift is deliberate: the quantity we can compute — training loss — and the quantity we care about — performance on data we have not seen — are not the same, and the gap between them is the subject.

§ 6.1 The data distribution

Everything so far has treated the dataset $\mathcal{D}$ as a fixed list of numbers. To say anything about data we have *not* seen we must say where data comes from in the first place.

Definition 6.1.1: Data distribution

We assume the dataset is sampled from a probability distribution $\mathbb{P}$ *in the sense of Definition 5.2.1, called the* data distribution*, written*

$$\boldsymbol{x}, y \sim \mathbb{P}.$$

The training set $\mathcal{D} = \{(\boldsymbol{x}^{(i)}, y^{(i)}) \mid i = 1, \ldots, n\}$ *consists of* independent and identically distributed *(IID) samples from* $\mathbb{P}$.

Note:-

Two assumptions are hiding here, and both can fail in practice. *Identically distributed* fails when deployment conditions differ from training conditions — an airfoil model trained at low Reynolds number and applied at high. *Independent* fails for time series, where consecutive samples are correlated by construction. Chapter 12 returns to the second when it treats recurrent networks.

§ 6.2 What makes a model good

Definition 6.2.1: Hold-out dataset

A hold-out dataset

$$\dot{\mathcal{D}} = \{(\dot{\boldsymbol{x}}^{(i)}, \dot{y}^{(i)}) \mid i = 1, \dots, m\}$$

is another dataset sampled IID from the same *distribution* $\mathbb{P}$ *as the training set* $\mathcal{D}$*, and disjoint from it.*

Definition 6.2.2: Accurate model

Fix a tolerance $\varepsilon > 0$*. A sample* $\boldsymbol{x}, y \sim \mathbb{P}$ *is* ε-badly predicted *by a model* f *if* $L\big(y, f(\boldsymbol{x})\big) > \varepsilon$*, and the* risk *of* f *being* ε*-bad at* $\boldsymbol{x}, y$ *is the probability that this happens,*

$$R_\varepsilon(f) \coloneqq \mathbb{P}\Big[\, L\big(y, f(\boldsymbol{x})\big) > \varepsilon \,\Big], \qquad \boldsymbol{x}, y \sim \mathbb{P}.$$

Given a second parameter $\gamma \in (0, 1)$*, the model is* (ε, γ)-accurate *if* $R_\varepsilon(f) \le \gamma$*, and* inaccurate *otherwise.*

Note:-

Read the definition carefully: $R_\varepsilon(f)$ is a property of the model and the distribution, and *no dataset appears in it.* It is not about held-out data in particular. By Definition 6.1.1 the training samples are themselves IID draws from the same $\mathbb{P}$, so R_ε governs a training point exactly as it governs a fresh one.
That coincidence is not a technicality — it is the only reason evidence gathered on training data can say anything about $\mathbb{P}$, and Theorem 6.3.1 is built on it. A hold-out set earns its place for a different reason, given in Definition 6.2.1: it lets us *estimate* R_ε without the circularity that §6.4 is about.

Common pitfall

The two parameters do different jobs and are easy to conflate. ε says *how wrong* a single prediction may be before we call it bad; γ says *how often* we tolerate a bad one. A model can be excellent on 99% of the distribution and catastrophic on the rest, and only γ sees that. Everything below controls γ; nothing below controls what happens on the bad γ-fraction.

Key idea

Note what has changed. The training loss $J(\boldsymbol{\theta})$ is a number we can compute. Accuracy is a statement about a *random* sample we have never seen, so it is not computable — only estimable. Every technique in the rest of this chapter exists because those two quantities differ.

§ 6.3 Why supervised learning works

It is not obvious that fitting training data should tell us anything about unseen data. Under two simplifying assumptions it does, and the reason is short enough to prove.

Say that a model ε-*fits* a sample if the loss there is at most ε; this is the training-set counterpart of the tolerance in Definition 6.2.2. We make two simplifying assumptions:

1. the model class $\mathcal{M}$ is *finite*, containing $H < \infty$ models;
2. the class is *realizable*: at least one member ε-fits every training point.

Both can be relaxed, at a technical cost beyond this course.

Theorem 6.3.1 Supervised learning works

Fix $\varepsilon > 0$ *and* $\gamma \in (0, 1)$*. Under the two assumptions above, suppose the learning algorithm returns some* $f \in \mathcal{M}$ *that*

ε-fits all n training samples. Then

$$\mathbb{P}\big[\, f \text{ is not } (\varepsilon,\gamma)\text{-accurate} \,\big] \;\le\; H\,(1-\gamma)^n \;\le\; H\,e^{-n\gamma},$$

which decays exponentially in the training set size n.

Proof: Fix one particular inaccurate model $f \in \mathcal{M}$, chosen *before any data is drawn.* By Definition 6.2.2 inaccuracy means $R_\varepsilon(f) > \gamma$, so on a draw from $\mathbb{P}$ this model is ε-bad with probability exceeding γ, and ε-fits with probability $1 - R_\varepsilon(f) < 1-\gamma$.

Now apply that to a training point. By Definition 6.1.1 each training sample is itself a draw from $\mathbb{P}$, and f was fixed without reference to it, so the two are independent and the same $1 - R_\varepsilon(f)$ applies. The n samples are mutually independent, so the probability that this one bad model ε-fits all of them is at most $(1-\gamma)^n$.

The independence just used is exactly what the algorithm destroys. It does not return a model fixed in advance; it returns whichever member of $\mathcal{M}$ happened to fit this particular dataset, so the returned model depends on the data and $1 - R_\varepsilon$ is no longer its probability of fitting. The repair is to avoid conditioning on the algorithm's output at all: bound instead the probability that *some* inaccurate model — each of them fixed in advance — ε-fits all n samples. There are at most H of them, and the probability of a union is at most the sum of the probabilities, by Lemma 5.3.2 — giving $H(1-\gamma)^n$. Finally $1 - \gamma \le e^{-\gamma}$, which yields the second bound. ∎

Common pitfall

It is tempting to summarize the proof as "a bad model is unlikely to fit n samples, so the model we got is probably good". That skips the step the theorem exists to supply. The estimate $(1-\gamma)^n$ is valid only for a model named *before* the data is seen. The model the algorithm hands you was named *after*, by a procedure that searched for one that fits — and for that model $\mathbb{P}[\text{it fits all } n]$ is close to 1 by construction, not small. The union bound is what buys back a statement about the chosen model, and the price is the factor H.

Note:-

Only γ appears in the bound. The tolerance ε enters through the *hypothesis*, since it decides what counts as fitting a training point and hence which models the algorithm is allowed to return. Demanding a smaller ε shrinks the set of admissible models, which may make the realizability assumption fail — that is the price, and it is paid before the theorem is applied rather than inside it.

Key idea

The union bound is the whole content of the theorem, and it is where the assumption $H < \infty$ earns its keep. A single fixed bad model is unlikely to fit n samples by chance; but if we may search over enormously many models, one of them probably will. *Learning is safe only when the search space is small relative to the data.* That sentence is the reason regularization works, and it will be worth remembering in Chapter 11, where the model classes become very large indeed.

Corollary 6.3.2 Sample complexity

To make the failure probability at most δ it suffices to take

$$n \;\ge\; \frac{1}{\gamma}\,\log\frac{H}{\delta}.$$

The logarithm is the good news: doubling the data squares the reliability, and a model class ten times larger costs only $\log 10$ extra samples per unit of $1/\gamma$. The proof is Exercise 11.

Example 6.3.1 (How much data does a grid search need?)

Corollary 6.3.2 looks abstract until H is something countable. Suppose we tune a model by grid search: six hyper-

parameters, each tried at ten values. That is $H = 10^6$ candidate models, and the algorithm returns whichever fits the training data.
Asking for $\gamma = 0.05$ and $\delta = 0.05$ — at most a 5% chance of returning a model that is badly wrong on more than 5% of the distribution — the corollary requires

$$n \geq \frac{1}{0.05} \log \frac{10^6}{0.05} \approx 336.$$

Three hundred and thirty-six samples to certify a search over a million models. The logarithm is doing enormous work here.
Push it: refine every grid to a hundred values, so $H = 10^{12}$. The requirement rises only to 613 — squaring the search space does not square the data requirement, it doubles it. Halving γ to 0.01 instead costs far more, taking n to 1681, because γ enters as $1/\gamma$ and not as a logarithm.
Searching wider is cheap; demanding more accuracy is expensive. That asymmetry is the practical content of Theorem 6.3.1, and it is worth remembering when a search space feels extravagant.

§ 6.4 Overfitting and underfitting

Theorem 6.3.1 appears to have just promised that a model fitting its training data is probably accurate. This section is about a model that fits its training data *perfectly* and is badly wrong. Something has to give, and it is worth being precise about what, because the answer is the whole reason the rest of this chapter exists.

Nothing gives. The theorem is not contradicted; it simply says nothing about this case. Its bound is $He^{-n\gamma}$, and it is informative only when that quantity is below 1 — that is, when $\log H$ is small compared with $n\gamma$. Fitting fifteen points with a degree-14 polynomial puts us at the opposite extreme.

Example 6.4.1 (What the theorem says about a degree-14 fit)

Strictly, Theorem 6.3.1 does not apply at all: the degree-14 polynomials form a continuum, so $H = \infty$ and $He^{-n\gamma}$ is infinite.
Be charitable and discretize. Store each coefficient in a float64, so a degree-d class holds $H = 2^{64(d+1)}$ models, and ask Corollary 6.3.2 how much data that needs at $\gamma = \delta = 0.05$:

degree	H	n required
1	2^{128}	1,834
5	2^{384}	5,383
14	2^{960}	13,368

We have $n = 15$. The degree-14 class would need about 900 times more data before the theorem certified anything. Turn it around and the point is starker still. With $n = 15$ and $\gamma = 0.05$, the bound $He^{-n\gamma}$ drops below 1 only for $H < 0.106$ — fewer than one model. At these sample sizes the theorem is not merely weak, it is *empty*: it bounds a probability by a number vastly larger than 1, which is true and useless.

Key idea

Theorem 6.3.1 was never a certificate you could apply to a fitted model. It is a statement about a *mechanism*: learning is safe when the search space is small relative to the data, and the quantity that decides this is $\log H/n$. Overfitting is simply what that ratio being large looks like from the inside — the model class was rich enough to contain something that matched the noise, and the algorithm found it.
Notice the table still ranks the degrees correctly: degree 14 needs 2.5 times the data of degree 5. The theorem's numbers are useless in absolute terms and informative in relative ones, and every method in the rest of this chapter is a way of acting on that ranking without needing the absolute figure.

So the realizability assumption cuts both ways. It guaranteed that some model ε-fits every training point — but a class rich enough to fit anything is rich enough to fit noise, and that same richness is what sends $\log H$ past $n\gamma$ and empties the

guarantee.

Definition 6.4.1: Generalization

Generalization *is the property of a predictive model of performing well on new, held-out data distinct from the training set.*

Definition 6.4.2: Overfitting

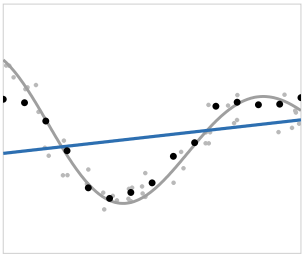
A model overfits *when it is very expressive, fits the training data almost perfectly, and makes wildly incorrect predictions off it — so it does not generalize.*

Definition 6.4.3: Underfitting

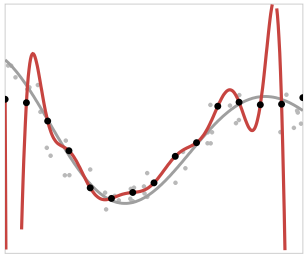
A model underfits *when it is too small to fit the training data well. Since held-out data resembles training data, it will not be accurate there either.*

Figure 3.3a already showed both: degree 1 underfits, degree 14 overfits, degree 4 does neither. What the figure could not show is how to *detect* which case you are in, since all three look like curves through points. Definition 6.2.1 supplies the answer: measure the loss on held-out data. Training loss decreases monotonically with model capacity and so tells you nothing; held-out loss turns around.

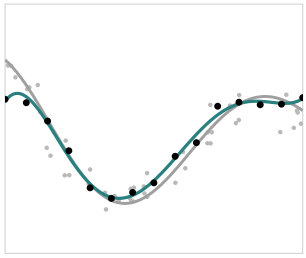
Let us see this geometrically. Before measuring the two failures, it is worth seeing them. The same fifteen training points are fitted three times, at a degree that is too low, far too high, and about right.



(a) Degree 1: $J_{\text{out}} = 0.0705$.



(b) Degree 14: $J_{\text{out}} = 1.1767$.



(c) Degree 5: $J_{\text{out}} = 0.0106$.

Figure 6.1: **The two ways to fail, and the middle.** Fifteen training points (black) drawn from the target curve (gray) with noise $\sigma = 0.12$, together with a held-out sample from the same distribution (small gray dots). Each panel fits a polynomial of one degree and reports the measured held-out loss. **(a)** Degree 1 cannot bend, so it misses the shape everywhere; training loss is 4.5×10^{-2} and held-out loss is no better. The model is wrong in a way that is visible *without* held-out data. **(b)** Degree 14 passes through all fifteen points exactly — training loss is 5×10^{-25}, zero to machine precision — and oscillates so violently between them that the curve leaves the frame twice. Held-out loss is 1.18, more than a hundred times worse than the best fit. **(c)** Degree 5 follows the target closely and lands at 0.0106, within striking distance of the noise floor $\sigma^2/2 = 0.0072$ that no model can beat. The contrast between **(a)** and **(b)** is the point. Underfitting announces itself on the training data; overfitting does the opposite, reporting a *perfect* training score at the moment it becomes useless. Only the held-out numbers separate them, which is what Figure 6.2 plots across every degree at once — the three panels here are its degrees 1, 14 and 5.

Let us see this geometrically. Both failures are invisible if you look only at the training data — a model that underfits and a model that overfits both produce a curve through the points. Plot the two losses against capacity and they separate immediately.

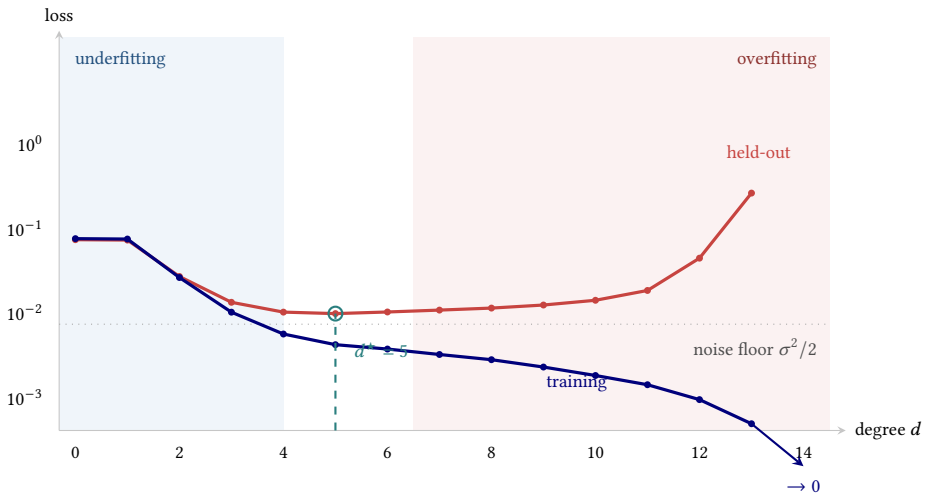


Figure 6.2: **Overfitting, made measurable.** Polynomial fits to the fifteen noisy points of Figure 3.3a, with the loss $J(\boldsymbol{\theta}) = \frac{1}{2n}\|\boldsymbol{\Phi\theta} - \boldsymbol{y}\|^2$ evaluated on the training set (navy) and on held-out data (red), averaged over 400 independent noise draws so that the curves are not an accident of one sample. Training loss falls *monotonically* with degree and reaches exactly zero at $d = 14$, where a degree-14 polynomial interpolates fifteen points — a perfect score that certifies nothing. Held-out loss falls only until $d^\star = 5$, then turns and rises by a factor of 512 by $d = 14$. The vertical gap between the curves is the quantity Definition 6.4.2 names, and it is what Definition 6.2.1 lets you measure. Note that the best achievable held-out loss is not zero but the noise floor $\sigma^2/2$ (dotted): no model can predict the noise, and a model that beats this line on training data is fitting it.

Key idea

Training loss and held-out loss both fall while the model is learning signal. Only training loss keeps falling once it starts learning noise. The gap between the two curves *is* overfitting, made measurable.

Note:-

Two remedies for underfitting: enlarge the model class, or build richer features for the data. Two remedies for overfitting: shrink the model class, or modify the objective to penalize complex models. This chapter develops the second, which is the one that does not require guessing the right size in advance.

§ 6.5 Regularization

Definition 6.5.1: Regularization

Regularization trains with an augmented objective $J : \mathcal{M} \to \mathbb{R}$,

$$J(f) = \frac{1}{n}\sum_{i=1}^{n} L\big(y^{(i)}, f(\boldsymbol{x}^{(i)})\big) \; + \; \lambda \cdot R(f),$$

built from three pieces:

- *a* loss $L(y, f(\boldsymbol{x}))$, *such as the squared error;*
- *a* regularizer $R : \mathcal{M} \to \mathbb{R}_+$, *penalizing models that are overly complex;*
- *a* regularization parameter $\lambda > 0$, *setting how much the penalty counts.*

When f is parameterized by $\boldsymbol{\theta}$ we write $R(\boldsymbol{\theta})$ and $J(\boldsymbol{\theta})$.

Note:-

The regularizer encodes what we mean by "complex". Different choices of R give genuinely different methods, and the next two sections work out the two standard ones. Nothing forces R to involve $\boldsymbol{\theta}$ at all — it is a function on the model class.

6.5.1 Ridge regression: L_2 regularization

Take $R(\boldsymbol{\theta}) = \|\boldsymbol{\theta}\|^2$ with the squared-error loss:

$$J(\boldsymbol{\theta}) = \frac{1}{2n}\|\boldsymbol{\Phi\theta} - \boldsymbol{y}\|^2 + \lambda\|\boldsymbol{\theta}\|^2, \qquad \lambda \in \mathbb{R}_+. \tag{6.1}$$

Differentiating and setting the result to zero gives

$$\boxed{\boldsymbol{\theta}^\star = \left(\boldsymbol{\Phi}^\mathsf{T}\boldsymbol{\Phi} + \lambda \mathrm{I}\right)^{-1}\boldsymbol{\Phi}^\mathsf{T}\boldsymbol{y}} \tag{6.2}$$

ms. p.14

Proposition 6.5.1 Unconditional invertibility

For every $\lambda > 0$, the matrix $\boldsymbol{\Phi}^\mathsf{T}\boldsymbol{\Phi} + \lambda \mathrm{I}$ is invertible, regardless of the rank of $\boldsymbol{\Phi}$.

Key idea

Proposition 6.5.1 is the structural payoff. Ordinary least squares required $\boldsymbol{\Phi}$ to have full column rank; ridge regression requires nothing at all. The rank-deficient case that Chapter 3 had to set aside is now covered.

Note:-

Why should small parameters mean a simple model? Fitting fifteen points with a degree-30 polynomial requires enormous coefficients — of order 10^5 and up — because the curve must swing violently to pass through every point. Forcing the coefficients to stay small forbids the swinging. That is the entire intuition, and Exercise 15 makes it quantitative.

Example 6.5.1 (Two sensors that measure the same thing)

A rig carries two strain gauges mounted close together, so their readings x_1 and x_2 agree to within instrument noise — in the data below, $\mathrm{corr}(x_1, x_2) = 0.9998$. The response actually depends on the strain alone, with true coefficient 3.0 spread between them however you like, since $x_1 \approx x_2$.

Least squares must still pick a single pair. On $n = 60$ observations it returns $\boldsymbol{\theta} = (0.88,\ 2.11)$. Repeat the experiment — same rig, same physics, fresh noise — and it returns $(2.12,\ 0.85)$. The two coefficients have essentially swapped. What is stable is their *sum*: 2.984 and 2.967 respectively, both close to the 3.0 that physics fixes.

The diagnosis is Proposition 6.5.1 read in reverse. Here $\kappa(\boldsymbol{\Phi}^\mathsf{T}\boldsymbol{\Phi}) = 1.2 \times 10^4$: not singular, so least squares has a unique answer, but close enough that the answer is determined by noise in the fourth digit of the data. Adding $\lambda\|\boldsymbol{\theta}\|^2$ with $\lambda = 10^{-2}$ gives $(1.474,\ 1.493)$ — the sum preserved at 2.967, but now split almost evenly, and stable across resamples.

Two lessons. First, an unstable coefficient is not the same as a wrong prediction: both fits predict well, since only the sum enters. Second, if the coefficients are what you plan to *interpret* — to decide which gauge matters — then least squares has told you nothing here, and no amount of extra data at the same two locations will fix it. Only regularization, or moving a gauge, will.

§ 6.6 The constrained formulation

Theorem 6.6.1 Penalty and constraint are equivalent

The penalized problem and the constrained problem

$$\min_{\theta\in\Theta} L(\theta) + \lambda R(\theta) \qquad \text{and} \qquad \min_{\theta\in\Theta} L(\theta) \ \text{ subject to } \ R(\theta) \le r$$

ms. p.14 *have the same solution set, for a correspondence between $\lambda > 0$ and $r > 0$ that depends on the problem.*

So there are two equivalent ways to say the same thing: penalize complexity, or forbid it beyond a threshold. Each value of λ implicitly sets a constraint level r, and conversely. The machinery justifying this — Lagrange multipliers — is developed in Chapter 13; Exercise 19 asks you to close the loop once it is in hand.

Key idea

The constrained form is the one to picture. It turns regularization into a geometry problem: minimize L over a set, rather than minimize a sum. The shape of that set is what distinguishes the two regularizers below.

§ 6.7 Lasso: L_1 regularization and sparsity

Replacing the squared Euclidean penalty by

$$J(\theta) = \frac{1}{2n}\|\Phi\theta - y\|^2 + \lambda\|\theta\|_1, \qquad \|\theta\|_1 := \sum_{j=0}^{d}|\theta_j|, \tag{6.3}$$

ms. p.15 gives the *lasso*. The constrained form now confines θ to the cross-polytope $\|\theta\|_1 \le r$ rather than a ball.

Definition 6.7.1: Sparsity

A vector is sparse *if many of its entries are exactly zero. A model with sparse θ uses only a few features, so fitting it also performs* feature selection.

Note:-

$\|\theta\|_1$ is not differentiable at any point where a component vanishes — which is precisely where its solutions like to sit. The lasso therefore has no closed form and cannot be attacked with Chapter 2's machinery directly; it needs methods for non-smooth objectives.

Example 6.7.1 (Choosing which channels to instrument)

A test article can be fitted with $d = 40$ candidate sensor channels, but each one costs money, mass and a wiring run. After $n = 80$ test articles, which channels actually predict the response?

Least squares cannot answer. It returns 40 nonzero coefficients for 80 observations, every channel apparently contributing something, and offers no principle for dropping any of them.

Lasso answers directly. On data where only channels 3, 10 and 18 carry real signal (coefficients 2.5, −1.8, 1.1, with noise $\sigma = 0.5$):

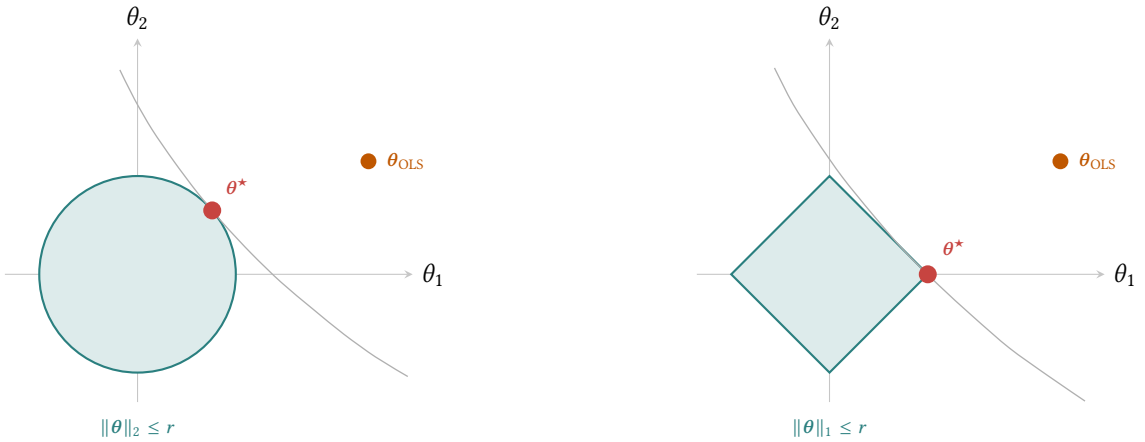
λ	channels kept	true channels recovered
0.05	15	all three
0.1	5	all three
0.2	3	all three
0.4	3	all three

At $\lambda = 0.2$ the answer is exactly the three real channels and nothing else. The other 37 are not merely small — by Definition 6.7.1 they are *exactly* zero, which is what makes this a procurement decision rather than a judgment

call about which small coefficients to round down.
Note what λ controls here. It is not accuracy; it is how much evidence a channel must present before it is kept. Pushing λ too high would begin discarding real channels, and §6.8 is about choosing it honestly rather than by eye.

6.7.1 Why L_1 produces sparsity

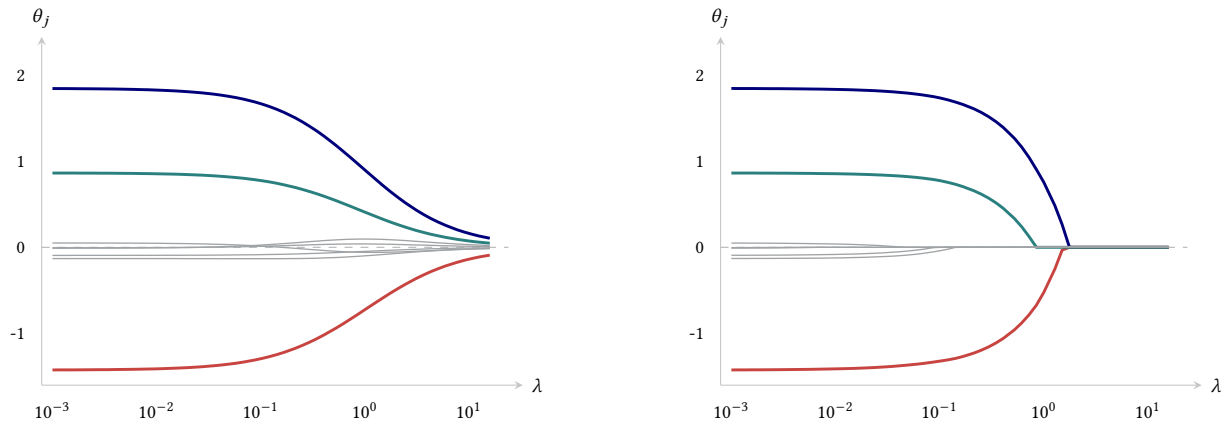
Let us see this geometrically. The geometric argument is the whole explanation, and it is visible in one picture. The unconstrained loss has elliptical level sets centered on the least-squares solution. The constrained minimizer lies where the smallest such ellipse first touches the constraint set — so the shape of that set decides where solutions land.



(a) L_2: contact in the interior of a smooth face.

(b) L_1: contact at a vertex, so $\theta_2 = 0$.

Figure 6.3: **Why L_1 selects features and L_2 does not.** In both panels the gray curve is a level set of the unregularized loss, the orange point is the unconstrained least-squares solution $\boldsymbol{\theta}_{\text{OLS}}$, and the shaded region is the constraint set. The regularized solution $\boldsymbol{\theta}^\star$ (red) is where the smallest level set first touches that region. **(a)** The ball is smooth and has no distinguished directions, so contact happens at a generic boundary point and both components of $\boldsymbol{\theta}^\star$ are nonzero: L_2 shrinks. **(b)** The cross-polytope has vertices *on the coordinate axes*, and those corners stick out, so contact generically happens at one — where all but one coordinate is exactly zero. L_1 selects. Both points here were computed, not drawn by eye: the L_1 solution really is $(1.00,\ 0.00)$.



(a) Ridge: paths approach zero, never arrive.

(b) Lasso: paths hit zero and stay.

Figure 6.4: **Shrinking is not selecting.** Coefficient paths for the same penalized least-squares problem with eight features, of which only three carry signal (colored; the five noise features are pale gray), traced as λ sweeps four decades. **(a)** Ridge. Every path approaches the axis but none reaches it: over the whole sweep the ridge solution contains *zero* exact zeros, and even at $\lambda = 16$ the smallest coefficient is still 5.6×10^{-3}. Ridge shrinks all eight features together and keeps all eight. **(b)** Lasso. Each path meets the axis at a finite λ and stays there — the five noise features are all exactly zero by $\lambda = 0.155$, while the three signal features survive to $\lambda = 0.85$ and beyond. At $\lambda = 0.1$ ridge retains 8 features and lasso retains 4. This is Figure 6.3 seen from the other side: there the vertex of the cross-polytope explained *why* a coordinate is set to zero, and here we watch it happen as the constraint tightens.

Key idea

The corners are the mechanism. A smooth constraint set has no reason to prefer the axes; a polytope with vertices on them does. Sparsity is not a numerical accident but a consequence of the shape of $\{\|\boldsymbol{\theta}\|_1 \le r\}$.

§ 6.8 Choosing λ

The regularization parameter is not determined by the training data: raising λ always increases training loss, so training loss alone would always choose $\lambda = 0$. We need a criterion outside the training set — and Section 6.1 has already supplied one.

6.8.1 The validation set

Definition 6.8.1: Validation set

Partition the available data into three disjoint parts, each IID from $\mathbb{P}$*: a* training set *used to fit* $\boldsymbol{\theta}$ *for a given* λ*; a* validation set *used to choose* λ*; and a* test set*, touched once, used only to report final performance.*

Algorithm 4: Choosing λ by validation

Input: candidate weights $\lambda_1, \dots, \lambda_m$; disjoint sets $\mathcal{D}_{\text{train}}$, $\mathcal{D}_{\text{val}}$, $\mathcal{D}_{\text{test}}$
Output: a choice of λ and an honest estimate of performance

for $i = 1, \dots, m$ **do**
  fit $\boldsymbol{\theta}_i^\star$ on $\mathcal{D}_{\text{train}}$ with weight λ_i
  record the *unregularized* loss of $\boldsymbol{\theta}_i^\star$ on $\mathcal{D}_{\text{val}}$
$\lambda^\star \leftarrow$ the λ_i of smallest validation loss
optionally refit on $\mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{val}}$ using $\lambda^\star$
report the loss on $\mathcal{D}_{\text{test}}$

Common pitfall

The validation set has been used to make a choice, so validation loss is an optimistic estimate of performance — we selected the λ that happened to look best on it. This is why a third, untouched set is needed for the number you report. Selecting on the test set and then reporting the test loss is the most common methodological error in applied machine learning, and it is undetectable from the outside.

Note:-

When data is scarce, splitting it three ways is expensive. *k-fold cross-validation* recycles it: partition the non-test data into k folds, train on $k-1$ and validate on the remaining one, and average over the k choices. The cost is k fits instead of one.

Example 6.8.1 (Why the winner always looks better than it is)

Here is an experiment with a deliberately rigged setup. Take m candidate models that are all *exactly equally good* — each has true loss 1.0 — and give each an unbiased validation estimate with noise of standard deviation 0.10. Now pick the one with the lowest validation loss and report its validation loss as your performance.

candidates m	reported loss	optimism
1	1.000	0.000
5	0.884	0.116
20	0.814	0.186
100	0.749	0.251

With a hundred candidates the reported figure is 25% better than the truth, and *no model in the experiment is any better than any other.* The entire gap is selection: the winner is whichever model got the luckiest validation draw, and its estimate is biased low by roughly $\sigma\sqrt{2\log m}$.

This is the mechanism behind the pitfall above, made quantitative. It is also why the discipline is worth taking literally. Every time a validation set is consulted — to pick λ, to pick a feature set, to decide whether to add a layer — m goes up by one, and the final validation number drifts a little further from the truth. The test set is the only honest estimate left, and only because it was touched once.

6.8.2 The L-curve, as a diagnostic

There is a second, older criterion that uses no held-out data at all. For each λ_i solve the regularized problem and plot the penalty $\|\boldsymbol{\theta}_i^\star\|^2$ against the residual $\frac{1}{2n}\|\boldsymbol{\Phi}\boldsymbol{\theta}_i^\star - \boldsymbol{y}\|^2$. The resulting curve is typically L-shaped, and the corner — the "elbow" — marks the point past which a large increase in penalty buys only a small reduction in residual. Points to the left of it are over-regularized and underfit; points to the right are under-regularized and overfit. ms. p.15

Let us see this geometrically. Plot the two competing quantities against each other as λ sweeps from tiny to large. Neither axis is the thing we want, but the shape of the trade-off between them has a corner, and the corner is informative.

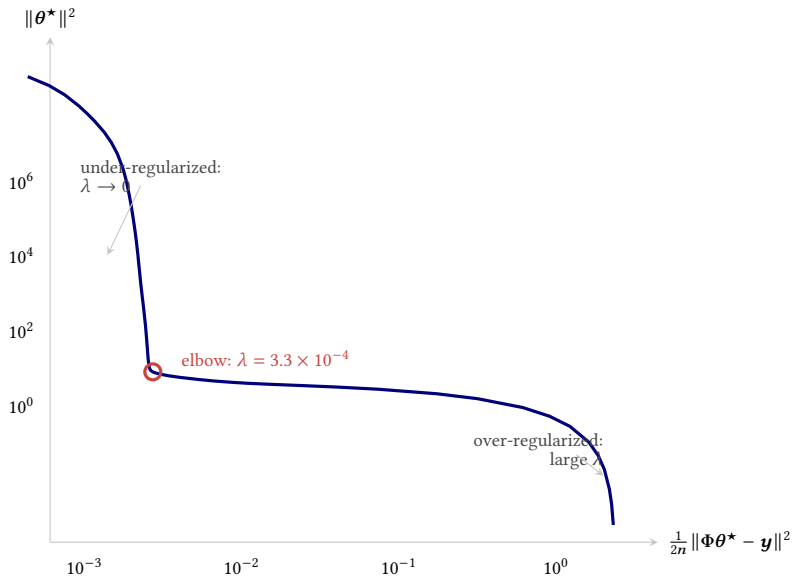


Figure 6.5: **The L-curve.** Ridge solutions for the degree-14 polynomial design of Figure 3.3a, whose Gram matrix has condition number 3.5×10^{12}, with λ swept over fourteen decades. Each point is one solved problem; the axes are the two terms of (6.1) plotted against each other, and λ increases from upper left to lower right. The vertical arm is the under-regularized regime: the residual is already as small as it will get, and decreasing λ further buys nothing while inflating $\|\boldsymbol{\theta}^\star\|^2$ through seven orders of magnitude — the coefficients grow large and opposite in sign, which is exactly the degree-14 pathology of Figure 3.3a. The horizontal arm is the over-regularized regime, where the penalty is nearly satisfied and further increases in λ only spoil the fit. The corner marked in red is where the curvature of the log–log curve is maximal, which is the standard criterion; here it selects $\lambda = 3.3 \times 10^{-4}$. Note what the picture does *not* use: no held-out data appears anywhere, which is why this diagnostic survives in the inverse-problem setting of Chapter 13.

Note:-

Prefer validation when you can afford to hold data out; it estimates the quantity we actually care about, whereas the elbow is a heuristic about the shape of a trade-off curve. The L-curve earns its place in inverse problems, where held-out data may not exist at all — as in the reduced-order modeling of Chapter 13.

Take-home message

Training loss measures fit. Held-out loss measures learning.

The two diverge exactly when a model starts memorizing rather than generalizing, and every technique in this chapter — hold-out sets, the sample complexity bound, the penalty $\lambda R(\boldsymbol{\theta})$, the validation split — exists to manage that divergence. Chapter 7 will re-derive (6.1) from an entirely different starting point and arrive at the same formula, which is some evidence that we have the right one.

§ 6.9 Exercises

The code problems use the datasets of Chapter 1; Appendix C, Section C.5 covers loading them and `train_test_split`.

Warm-up

1. [Theory] State Definition 6.2.1 in your own words, and explain why the hold-out set must be drawn from the *same* distribution as the training set. What breaks if it is not?

2. [Theory] A model f incurs loss 0.2 on a region carrying probability 0.95 and loss 50 on the remaining 0.05; a second model g incurs loss 1.2 everywhere.

(a) Compute $R_\varepsilon(f)$ and $R_\varepsilon(g)$ for $\varepsilon = 1$ and for $\varepsilon = 0.1$.

(b) For $\varepsilon = 1$, give the smallest γ for which each model is (ε, γ)-accurate, or say that none exists.

(c) Compute $\mathbb{E}[L]$ for both. One model is preferred by Definition 6.2.2 at $\varepsilon = 1$ and the other by expected loss. Say in one sentence why the two criteria disagree, and which one Theorem 6.3.1 controls.

3. [Theory] Example 6.3.1 finds $n \geq 336$ for $H = 10^6$ at $\gamma = \delta = 0.05$. Recompute for a search one thousand times larger, $H = 10^9$, and separately for the original H with γ halved to 0.025. Which change costs more data, and by how much? State the reason in terms of where each parameter sits in Corollary 6.3.2.

4. [Theory] In Example 6.5.1 least squares returned (0.88, 2.11) on one sample and (2.12, 0.85) on another, while the sum stayed near 2.98 both times. Explain why the sum is the stable quantity by writing the prediction $\boldsymbol{\Phi\theta}$ when $x_1 = x_2$ exactly. In that exact case, how many solutions does least squares have, and which one does ridge select?

5. [Theory] Derive (6.2) from (6.1).

6. [Theory] In Theorem 6.3.1, what happens to the bound as $H \to \infty$? As $n \to \infty$ with H fixed? Explain in one sentence each why both behaviors are what you would expect.

7. [Theory] A student reports: "my training loss is 0.001, so the model is excellent." Identify the error using Definition 6.4.2 and Definition 6.4.1.

8. [Theory] Using Figure 6.3, explain why $\theta_2 = 0$ exactly in panel (b) but only approximately in panel (a), without writing down any equations.

9. [Code] Load the Combined Cycle Power Plant data and split it 60/20/20 into training, validation, and test sets with `train_test_split`. Confirm the three parts are disjoint and report their sizes and per-feature means. Are the means similar across the three? Should they be?

Core

10. [Theory] Prove Proposition 6.5.1. *Hint:* show that $\boldsymbol{z}^\mathsf{T}(\boldsymbol{\Phi}^\mathsf{T}\boldsymbol{\Phi} + \lambda \mathrm{I})\boldsymbol{z} \geq \lambda\|\boldsymbol{z}\|^2$ for all $\boldsymbol{z}$.

11. [Theory] Prove Corollary 6.3.2 from Theorem 6.3.1.

12. [Theory] Express the ridge solution in terms of the singular value decomposition $\boldsymbol{\Phi} = \boldsymbol{U\Sigma V}^\mathsf{T}$. Show that the effect of λ is to replace each singular value σ_j by $\sigma_j/(\sigma_j^2 + \lambda)$, and explain why this suppresses the directions associated with small σ_j — the very directions that Figure 3.3b showed were badly conditioned.

13. [Theory] Consider the lasso in one dimension with $\boldsymbol{\Phi} = \mathrm{I}$, that is $\min_\theta \frac{1}{2}(\theta - y)^2 + \lambda|\theta|$. Solve it exactly and show that the solution is the soft-thresholding operator $\theta^\star = \mathrm{sgn}(y)\max(|y| - \lambda, 0)$. Note explicitly where $\theta^\star = 0$ on a set of positive measure — this is sparsity in its simplest form.

14. [Theory] Show that adding $\lambda\|\boldsymbol{\theta}\|^2$ shifts every eigenvalue of the Hessian $\boldsymbol{\Phi}^\mathsf{T}\boldsymbol{\Phi}/n$ up by λ, and deduce the effect on the condition number. Relate this to the elongated level sets of Figure 3.1b.

15. [Theory] For n points fitted exactly by a degree-$(n-1)$ polynomial, argue that the coefficients must generally be large, and explain why penalizing $\|\boldsymbol{\theta}\|^2$ therefore forbids the wild oscillation of Figure 3.3a.

16. [Code] Implement ridge regression in JAX for the polynomial fitting problem of Chapter 3. Using Algorithm 4, sweep λ over 10^{-8} to 10^2 logarithmically and select $\lambda^\star$ by validation loss. Report the test loss. Then plot training, validation, and test loss against λ on one set of axes and mark $\lambda^\star$.

Challenge

17. [Code] Extend Example 6.7.1 into a procurement question. With $d = 40$ candidate channels and only three carrying signal, suppose each channel costs one unit to install and a prediction error above 0.6 costs ten units. Sweep λ, and for each value record the number of channels retained and the held-out error. Find the λ minimizing total cost. Does it coincide with the λ that minimizes held-out error alone? Explain the gap in terms of what Definition 6.5.1 does and does not know about.

18. [Theory] Example 6.8.1 shows validation optimism growing like $\sigma\sqrt{2\log m}$. Combine this with Corollary 6.3.2, which also depends on the number of candidates only logarithmically. Are these the same logarithm? Argue either that one implies the other or that the resemblance is coincidental, being explicit about what is random in each case.

19. [Theory] (*Defer until after Chapter 13.*) Using Lagrange multipliers, prove Theorem 6.6.1 for the ridge case, and give the relationship between λ and r. Why does the correspondence depend on the data?

20. [Theory] Theorem 6.3.1 assumed realizability. Suppose instead that no model in $\mathcal{M}$ ε-fits every training point. State what you would expect the bound to become, and explain why the proof no longer applies verbatim. (You are describing the difference between the realizable and agnostic settings.)

21. [Theory] Show that ridge regression is equivalent to ordinary least squares on an augmented dataset: append $d+1$ artificial rows to Φ and appropriate zeros to $\boldsymbol{y}$. What are the artificial rows? This gives a second proof of Proposition 6.5.1.

22. [Theory] Criticize the L-curve. Construct a problem in which the residual–penalty curve has no clear corner, and explain what a practitioner should do. Then argue for a case where the L-curve is preferable to validation despite this.

23. [Code] On the Airfoil Self-Noise data, plot the coefficient paths — each θ_j against λ — for ridge and for lasso side by side. Verify that lasso coefficients reach exactly zero at finite λ while ridge coefficients only approach it asymptotically, and identify which features survive longest. (Use `optax` with proximal updates, or implement coordinate descent directly, for the lasso.)

24. [Code] Investigate Theorem 6.3.1 numerically. Fix a finite model class by discretizing the parameters of a simple family, choose a tolerance ε and a level γ, and for a range of n estimate by Monte Carlo the probability that a model which ε-fits all training points fails to be (ε,γ)-accurate on fresh samples. Plot the estimate against n on a log scale alongside the bound $He^{-n\gamma}$. Is the bound tight, and does the gap grow or shrink with H?

Reproducing the figures and examples

The numerical figures and examples of this chapter are measurements, and the problems below ask you to make them again and check the numbers. Figures 6.2, 6.5 and the polynomial parts of Figure 3.3a share one setup:

$$n = 15 \text{ points on } [0.3,\ 6.1], \qquad \sigma = 0.12, \qquad \text{degrees 0 through 14}, \tag{6.4}$$

with a smooth non-polynomial target and the loss $J = \frac{1}{2n}\|\Phi\boldsymbol{\theta} - \boldsymbol{y}\|^2$. Curves that are averaged over noise draws say so; a single draw will be visibly rougher, and that difference is itself worth seeing.

25. [Code] **Figure 6.2.** Using (6.4), fit polynomials of every degree from 0 to 14 and record training and held-out loss, averaging over 400 independent noise draws. Confirm that held-out loss is minimized at degree 5 and rises by a factor of about 512 by degree 14, and that training loss at degree 14 is zero to machine precision. Explain that last fact exactly — why *must* it be zero, and what would change with 16 points? Finally, plot the noise floor $\sigma^2/2$ and confirm no held-out curve goes below it.

26. [Code] **Figure 6.3.** The caption claims the L_1 solution is exactly $(1.00,\ 0.00)$ rather than something merely close. Reproduce it: choose a two-dimensional quadratic loss and a radius r, then solve $\min J(\boldsymbol{\theta})$ subject to $\|\boldsymbol{\theta}\|_1 \le r$ numerically, and separately subject to $\|\boldsymbol{\theta}\|_2 \le r$. Report both minimizers to six decimals. Then perturb the loss slightly and rerun: which of the two minimizers moves continuously, and which stays pinned at a vertex until the perturbation is large enough? This is Definition 6.7.1 as a computational fact rather than a picture.

27. [Code] **Figure 6.4.** With 8 features of which three carry signal $(2.0,\ -1.4,\ 0.9)$, $n = 60$ and $\sigma = 0.5$, trace ridge and lasso coefficients over four decades of λ. Count *exact* zeros in each path — not coefficients below a threshold, but values equal to zero in floating point. You should find none for ridge anywhere, and several hundred for lasso. Report the λ at which the last noise feature dies and the λ at which the first signal feature does, and say what the gap between them means for feature selection.

28. [Code] **Figure 6.5.** On the degree-14 design of (6.4), whose Gram matrix has $\kappa(\Phi^{\mathsf{T}}\Phi) = 3.5\times10^{12}$, sweep λ over fourteen decades and plot $\|\boldsymbol{\theta}^\star\|^2$ against the residual on log–log axes. Locate the elbow by maximum curvature of the log–log curve rather than by eye, and confirm $\lambda \approx 3.3\times10^{-4}$. Then compare against the λ that a validation split would have chosen on the same data. Do they agree, and which would you trust when held-out data exists?

29. [Code] **Example 6.5.1.** Reproduce it. Generate $n = 60$ samples with $x_2 = x_1 + 0.02\varepsilon$, response $3x_1$ plus noise 0.3, and fit by least squares. Repeat over 200 independent draws and report the standard deviation of θ_1, of θ_2, and of $\theta_1 + \theta_2$. Then repeat with ridge at $\lambda = 10^{-2}$. Which of the three standard deviations does regularization actually reduce, and which was already small?

30. [Code] **Example 6.8.1.** Reproduce it. Draw m unbiased estimates of a common true loss 1.0 with noise 0.10, take the minimum, and average over many repetitions for $m = 1, 5, 20, 100$. Confirm the optimism values 0, 0.116, 0.186, 0.251, and compare against $\sigma\sqrt{2\log m}$. Then answer the question that matters: if you select among m models on validation and then report the winner's *test* loss, is that estimate biased?

31. [Theory] **Example 6.3.1.** Reproduce the three sample sizes 336, 613 and 1681 from Corollary 6.3.2, then plot n against $\log_{10} H$ for fixed γ, and against γ for fixed H. One plot is a straight line and the other is a hyperbola. Which is which, and which of the two shapes is the reason a large hyperparameter search is affordable?

32. [Code] **Example 6.7.1.** Reproduce the table: $d = 40$ channels, $n = 80$ samples, signal on three channels with coefficients $(2.5, -1.8, 1.1)$ and $\sigma = 0.5$. Confirm that $\lambda = 0.2$ retains exactly three channels and that they are the right three. Then make it harder: raise σ until lasso starts either dropping a real channel or keeping a spurious one, and report the noise level at which that happens. How does the answer change if n is halved to 40?

Chapter 7

The Probabilistic View

Goal of this chapter

Until now a loss function was something we *chose*. Chapter 2 offered L_1 and L_2 with no argument for either; Chapter 6 added a penalty $\lambda R(\boldsymbol{\theta})$ and left λ to a validation sweep.
This chapter removes the arbitrariness. It opens by looking back: the objective every earlier chapter minimized is an *empirical risk*, a Monte Carlo estimate of an expectation over the data distribution (§7.1). Seeing it that way makes a different move available — stop choosing a function and a loss, and learn the distribution itself (§7.2). Once we do, the loss is *derived* rather than chosen, and three results the notes have already used fall out as consequences:

1. the L_2 loss is the negative log-likelihood of Gaussian noise;
2. ridge regression is maximum *a posteriori* estimation with a Gaussian prior, and $\lambda = \sigma^2/\gamma^2$ is a ratio of variances rather than a free knob;
3. the parameter $\boldsymbol{\theta}$ is itself an artifact, and integrating it away gives a prediction with error bars instead of a point.

The tool throughout is the Kullback–Leibler divergence, and the license to replace every expectation by an average is Theorem 5.6.2.

§ 7.1 The learning problem, as empirical risk minimization

Chapter 2 handed us a loss and Chapter 3 minimized it, both without ever saying where the objective came from. Every chapter since has minimized an average over the training set. Before replacing that objective with something better, it is worth saying what the average is an average *of* — because the answer is what the rest of this chapter builds on.

What we actually want is a model that predicts well on data we have not seen, and Definition 6.1.1 already says where such data comes from: the training set is n IID samples from a distribution $\mathbb{P}$.

With $\mathbb{P}$ in hand the goal becomes a single number. The quantity we would like to minimize is the *risk*: the loss we expect to incur on a fresh sample drawn from $\mathbb{P}$,

$$\mathcal{R}(\boldsymbol{\theta}) \coloneqq \mathbb{E}_{\boldsymbol{x},y\sim\mathbb{P}}\Big[\, L\big(y, f_{\boldsymbol{\theta}}(\boldsymbol{x})\big) \,\Big]. \tag{7.1}$$

Every earlier chapter has been minimizing something else. The reason is immediate: (7.1) is an expectation with respect to a distribution we do not know. We have never been given $\mathbb{P}$ — only n samples drawn from it.

Key idea

This is exactly the situation of Section 5.6. We want an expectation we cannot evaluate, and we hold IID draws from the distribution it is taken over. Theorem 5.6.1 says the sample average of $L(y^{(i)}, f_{\boldsymbol{\theta}}(\boldsymbol{x}^{(i)}))$ converges to $\mathcal{R}(\boldsymbol{\theta})$ with probability one, and Theorem 5.6.2 says the error decays like $\sigma/\sqrt{n}$.

So we do the only thing available: replace the expectation by the Monte Carlo estimate built from the data we have.

Writing $\ell_i(\boldsymbol{\theta}) := L\big(y^{(i)}, f_{\boldsymbol{\theta}}(\boldsymbol{x}^{(i)})\big)$ for the loss on the i-th sample,

$$\mathcal{R}(\boldsymbol{\theta}) \;\approx\; \frac{1}{n}\sum_{i=1}^{n} \ell_i(\boldsymbol{\theta}) \;=:\; J(\boldsymbol{\theta}), \tag{7.2}$$

and minimizing J in place of $\mathcal{R}$ is called *empirical risk minimization*. Equation (7.2) is the objective every algorithm in these notes has been given, from the least squares of Chapter 3 onward. Nothing about it has changed; what Chapter 5 adds is the knowledge that it was an *approximation* all along, to a quantity we can now name, with a rate for how good an approximation it is.

Note:-

The $\approx$ in (7.2) is doing real work and it is not free. J and $\mathcal{R}$ have different minimizers, and the gap between them is what Chapter 6 was about: a model chosen to minimize the sample average can be much worse on $\mathbb{P}$ than the average suggests.

§ 7.2 Learning the distribution instead

Look again at what (7.1) needs and what (7.2) does about it. The risk is an expectation against $\mathbb{P}$. We cannot evaluate it, so we approximate the expectation by a sample average and go on choosing a *function* $f_{\boldsymbol{\theta}}$ out of a model class, judged by a loss we picked by hand.

There is a second thing one could do with the same difficulty. The obstruction was never the function; it was not knowing $\mathbb{P}$. So rather than approximate the expectation and keep guessing at losses, aim directly at the distribution: take the model class to be a family of *distributions* $\mathbb{P}_{\boldsymbol{\theta}}(\boldsymbol{x}, y)$ and choose the member closest to $\mathbb{P}_{\text{data}}$.

Key idea

The two programs differ in what the model class contains.

	Chapters 1–6	This chapter
model class	functions $f_{\boldsymbol{\theta}} : \mathcal{X} \to \mathcal{Y}$	distributions $\mathbb{P}_{\boldsymbol{\theta}}(\boldsymbol{x}, y)$
loss	chosen $(L_1, L_2, \dots)$	derived: a distance between distributions
fitted to	the n samples	the distribution the samples came from

The payoff is that the loss stops being an assumption. Once the class contains distributions there is a natural notion of "close", and the squared error of Chapter 3 will drop out of it as a special case rather than being posited — that is §7.7.

This does not discard (7.2); it reuses it. The new objective will also be an expectation against $\mathbb{P}_{\text{data}}$ that we cannot evaluate, and we will again replace it by an average over the same n samples, licensed by the same Theorem 5.6.2. What changes is *which* expectation, not the method of estimating it.

7.2.1 Distributions over data

A data distribution is a map $\mathbb{P}(\boldsymbol{x}, y) : \mathcal{X} \times \mathcal{Y} \to [0, 1]$ satisfying ms. p.17

$$\text{(i) } 0 \le \mathbb{P}(\boldsymbol{x}, y) \le 1, \qquad \text{(ii) } \sum_{\boldsymbol{x} \in \mathcal{X},\, y \in \mathcal{Y}} \mathbb{P}(\boldsymbol{x}, y) = 1.$$

The *marginal* of $\boldsymbol{x}$ is $\mathbb{P}(\boldsymbol{x}) = \sum_{y \in \mathcal{Y}} \mathbb{P}(\boldsymbol{x}, y)$, and the *conditional* is

$$\mathbb{P}(y \mid \boldsymbol{x}) = \frac{\mathbb{P}(\boldsymbol{x}, y)}{\sum_{y \in \mathcal{Y}} \mathbb{P}(\boldsymbol{x}, y)},$$

the probability that the label is y given that the input is $\boldsymbol{x}$. ms. p.17

7.2.2 The three ingredients, restated

We assume $\{(\boldsymbol{x}^{(i)}, y^{(i)})\}_{i=1}^{n} \sim \mathbb{P}_{\text{data}}(\boldsymbol{x}, y)$, a distribution we do not know except through the samples. The three ingredients
ms. p.18 of Chapter 1 reappear:

1. data $\{(\boldsymbol{x}^{(i)}, y^{(i)})\}_{i=1}^{n}$;
2. model class $\mathcal{M} = \{\mathbb{P}_{\boldsymbol{\theta}}(\boldsymbol{x}, y) : \boldsymbol{\theta} \in \Theta\}$, with $0 < \mathbb{P}_{\boldsymbol{\theta}}$ and $\sum \mathbb{P}_{\boldsymbol{\theta}} = 1$;
3. a learning algorithm, comprising a loss $J(\boldsymbol{\theta}) = D_{\text{KL}}(\mathbb{P}_{\text{data}} \,\|\, \mathbb{P}_{\boldsymbol{\theta}})$ and the rule $\boldsymbol{\theta}^{\star} = \arg\min_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$.

§ 7.3 Monte Carlo estimation

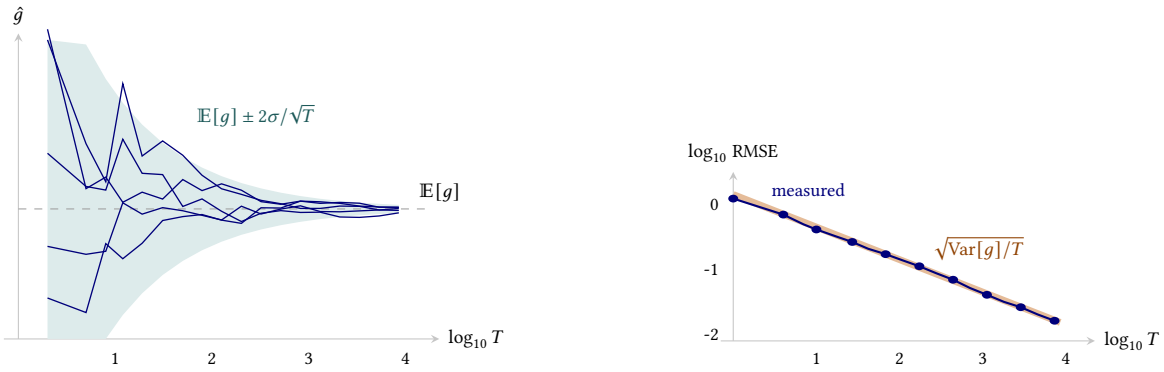
Every method in this chapter reduces an optimization over a *distribution* to an optimization over a *sample*, and the tool that licenses this is Monte Carlo estimation — which Chapter 5 has already supplied. We restate it here only to fix notation.

Given $g : \mathcal{X} \to \mathbb{R}$ and T i.i.d. samples $x^{(1)}, \dots, x^{(T)} \sim \mathbb{P}$, the *Monte Carlo estimator* of $\mathbb{E}_{x\sim\mathbb{P}}[g(x)]$ is

$$\hat{g} \coloneqq \frac{1}{T} \sum_{t=1}^{T} g(x^{(t)}). \tag{7.3}$$

ms. p.17 By Theorem 5.6.1 it converges to $\mathbb{E}[g(x)]$ as $T \to \infty$, and by Theorem 5.6.2 its mean square error is $\sqrt{\text{Var}[g]}/\sqrt{T}$.

Let us see this geometrically. Two things are being claimed at once: that the average converges to the expectation at all, and that it does so at a definite rate. Both are visible in a single experiment — run the estimator repeatedly and watch.



(a) Five independent runs of the same estimator.

(b) Error against sample count, log–log.

Figure 7.1: **Monte Carlo estimation, and its price.** The estimator (7.3) applied to $g(x) = x^2$ with $x \sim \mathcal{N}(0, 1)$, so that $\mathbb{E}[g] = 1$ and $\text{Var}[g] = 2$. **(a)** Five independent runs. Each wanders badly for small T and settles toward the dashed true value — this is Theorem 5.6.1. Note that no run is monotone: a Monte Carlo estimate can get worse before it gets better, and a single run that looks converged may not be. **(b)** Root mean square error over 400 repetitions, against the reference line $\sqrt{\text{Var}[g]/T}$ of Theorem 5.6.2. The measured slope is $-\frac{1}{2}$: each additional correct digit costs a hundredfold increase in T. That is the price of the $\approx$ we are about to use everywhere.

Key idea

Every appearance of the symbol $\approx$ in the derivations below is an application of (7.3): an expectation we cannot evaluate is replaced by an average over the data we happen to have. The $T^{-1/2}$ rate of Theorem 5.6.2 is what makes the replacement legitimate, and $\sqrt{\text{Var}[g]}$ is the constant that decides whether it is cheap.

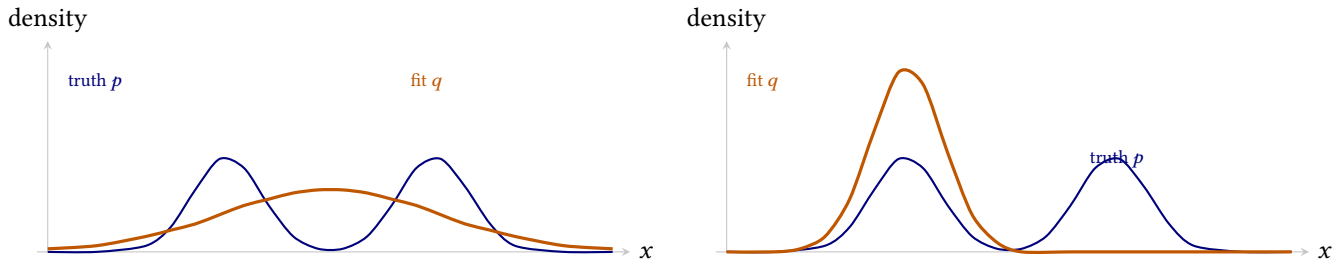
§ 7.4 Kullback–Leibler divergence

Definition 7.4.1: KL divergence

$$D_{\mathrm{KL}}(\mathbb{P}_{\text{data}} \,\|\, \mathbb{P}_{\boldsymbol{\theta}}) \coloneqq \sum_{\boldsymbol{x},y} \mathbb{P}_{\text{data}}(\boldsymbol{x},y) \log \frac{\mathbb{P}_{\text{data}}(\boldsymbol{x},y)}{\mathbb{P}_{\boldsymbol{\theta}}(\boldsymbol{x},y)}.$$

ms. p.18

Let us see this geometrically. Two things about Definition 7.4.1 are worth seeing before proving anything. It is not symmetric — $D_{\mathrm{KL}}(p \,\|\, q)$ and $D_{\mathrm{KL}}(q \,\|\, p)$ are different numbers — and the asymmetry is not a technicality. It decides what a fitted model looks like when the family is too small to contain the truth, which is the usual case.



(a) $\min_q D_{\mathrm{KL}}(p \,\|\, q)$: mass-covering. (b) $\min_q D_{\mathrm{KL}}(q \,\|\, p)$: mode-seeking.

Figure 7.2: **KL divergence is not symmetric, and the asymmetry matters.** A bimodal truth p (navy) approximated by the best single Gaussian q (orange), where "best" means minimizing KL — in the two available orders. **(a)** Minimizing $D_{\mathrm{KL}}(p \,\|\, q)$ penalizes putting *low* q where p is high, so q stretches to cover both modes and puts most of its mass where p has none. **(b)** Minimizing $D_{\mathrm{KL}}(q \,\|\, p)$ penalizes putting *high* q where p is low, so q collapses onto one mode and ignores the other entirely. Both fits are optimal; they answer different questions. Maximum likelihood, developed next, is the first of these — which is why a badly-specified model tends to hedge rather than commit.

Proposition 7.4.1 Properties

(i) $D_{\mathrm{KL}}(\mathbb{P}_{\text{data}} \,\|\, \mathbb{P}_{\boldsymbol{\theta}}) \geq 0$;

(ii) $D_{\mathrm{KL}}(\mathbb{P}_{\text{data}} \,\|\, \mathbb{P}_{\boldsymbol{\theta}}) = 0$ *if and only if* $\mathbb{P}_{\text{data}}(\boldsymbol{x},y) = \mathbb{P}_{\boldsymbol{\theta}}(\boldsymbol{x},y)$ *for all* $\boldsymbol{x}, y$.

ms. p.18,20

Proof: Both parts follow from one elementary inequality:

$$\log t \;\leq\; t-1 \quad \text{for all } t>0, \qquad \text{with equality if and only if } t=1. \tag{7.4}$$

To see (7.4), let $h(t) = t - 1 - \log t$. Then $h'(t) = 1 - 1/t$, which is negative on $(0,1)$ and positive on $(1,\infty)$, so h decreases to $h(1) = 0$ and increases thereafter. Hence $h \geq 0$ everywhere, and $h(t) = 0$ only at $t = 1$.

Write $z = (\boldsymbol{x}, y)$ and abbreviate $p = \mathbb{P}_{\text{data}}$, $q = \mathbb{P}_{\boldsymbol{\theta}}$. Apply (7.4) with $t = q(z)/p(z)$ at each z with $p(z) > 0$:

$$\log \frac{q(z)}{p(z)} \;\leq\; \frac{q(z)}{p(z)} - 1.$$

Multiply by $p(z) > 0$, sum over those z, and negate:

$$D_{\mathrm{KL}}(p \,\|\, q) = -\sum_z p(z) \log \frac{q(z)}{p(z)} \;\geq\; -\sum_z \Big(q(z) - p(z)\Big) = -\Big(\sum_z q(z) - \sum_z p(z)\Big) = -(1-1) = 0,$$

using that both are distributions. That is (i). Terms with $p(z) = 0$ contribute nothing, under the convention $0 \log 0 = 0$, and dropping them from the middle sum only decreases it, so the inequality is unaffected.

For (ii), equality throughout forces equality in (7.4) at every z with $p(z) > 0$, hence $q(z) = p(z)$ there. The remaining mass agrees too: $\sum_{z:p(z)>0} q(z) = \sum_{z:p(z)>0} p(z) = 1$, so q assigns zero to every z with $p(z) = 0$. Therefore $p = q$ everywhere. The converse is immediate, since $\log 1 = 0$. ∎

Note:-

Part (i) also follows in one line from Jensen's inequality applied to the convex function $-\log$, which is the usual textbook route. The argument above is longer but self-contained, and it delivers (ii) at the same time — with Jensen one still has to chase the equality condition separately, and that condition is what Theorem 7.5.1 actually relies on.

§ 7.5 From KL to maximum likelihood

Theorem 7.5.1 Minimizing KL is maximizing likelihood

$$\boldsymbol{\theta}^\star = \arg\min_{\boldsymbol{\theta}} D_{\mathrm{KL}}(\mathbb{P}_{\text{data}} \,\|\, \mathbb{P}_{\boldsymbol{\theta}}) = \arg\max_{\boldsymbol{\theta}} \mathbb{E}_{(\boldsymbol{x},y)\sim\mathbb{P}_{\text{data}}}\big[\log \mathbb{P}_{\boldsymbol{\theta}}(\boldsymbol{x}, y)\big].$$

Proof: Split the logarithm:

$$D_{\mathrm{KL}}(\mathbb{P}_{\text{data}} \,\|\, \mathbb{P}_{\boldsymbol{\theta}}) = -\sum_{\boldsymbol{x},y} \mathbb{P}_{\text{data}} \log \mathbb{P}_{\boldsymbol{\theta}} + \sum_{\boldsymbol{x},y} \mathbb{P}_{\text{data}} \log \mathbb{P}_{\text{data}}.$$

The second term does not involve $\boldsymbol{\theta}$. Since $\arg\min_x f(x) = \arg\min_x [f(x) + m]$ for any constant m, it may be dropped,
ms. p.18 and Claim 2.1.2 converts the remaining minimization of $-\sum \mathbb{P}_{\text{data}} \log \mathbb{P}_{\boldsymbol{\theta}}$ into a maximization. ∎

Applying the Monte Carlo estimator (7.3) to the expectation gives the practical form

$$\boldsymbol{\theta}^\star = \arg\max_{\boldsymbol{\theta}} \frac{1}{n} \sum_{i=1}^{n} \log \mathbb{P}_{\boldsymbol{\theta}}(\boldsymbol{x}^{(i)}, y^{(i)}) = \arg\max_{\boldsymbol{\theta}} \log \prod_{i=1}^{n} \mathbb{P}_{\boldsymbol{\theta}}(\boldsymbol{x}^{(i)}, y^{(i)}), \tag{7.5}$$

ms. p.19 the product being the *likelihood* $\mathcal{L}_{\boldsymbol{\theta}}$.

Example 7.5.1 (Flipping a coin)

Let the data be $\mathcal{D} = \{H, H, T, H, T\}$, so $\boldsymbol{x}^{(i)} = y^{(i)}$ and the outcome space is finite. Model $\mathbb{P}_{\boldsymbol{\theta}}$ as a binomial:

$$\mathbb{P}_{\boldsymbol{\theta}}(x = H) = \theta, \qquad \mathbb{P}_{\boldsymbol{\theta}}(x = T) = 1 - \theta, \qquad \Theta = [0, 1].$$

The log-likelihood is

$$\mathcal{L}_\theta = \log \prod_{i=1}^{5} \mathbb{P}_{\boldsymbol{\theta}}(x^{(i)}) = \log \theta^{\#H}(1-\theta)^{\#T} = \log \theta^3 (1-\theta)^2.$$

Setting $\mathrm{d}\mathcal{L}_\theta/\mathrm{d}\theta = 0$ gives

$$\frac{\#H\, \theta^{\#H-1}(1-\theta)^{\#T} - \#T\, \theta^{\#H}(1-\theta)^{\#T-1}}{\theta^{\#H}(1-\theta)^{\#T}} = 0 \quad \Longleftrightarrow \quad \#H(1-\theta) - \#T\, \theta = 0,$$

discarding the spurious roots $\theta = 0$ and $\theta = 1$, and hence

$$\boxed{\theta^\star = \frac{\#H}{\#H + \#T}}$$

ms. p.19,20 As the number of flips grows, $\theta^\star \to \frac{1}{2}$ for a fair coin.

Let us see this geometrically. The estimate $\theta^\star = \#H/(\#H + \#T)$ is a single number, but the likelihood is a whole function of θ. Its shape carries more information than its peak does — and watching it change with sample size shows what "learning" looks like.

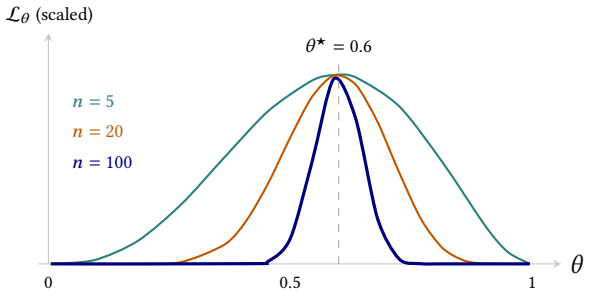


Figure 7.3: **The likelihood sharpens as data accumulates.** The likelihood $\mathcal{L}_\theta = \theta^{\#H}(1-\theta)^{\#T}$ for a coin landing heads 60% of the time, rescaled to unit peak, for three sample sizes. The maximizer sits at 0.6 in every case — more data does not move the estimate — but the peak narrows, and it is the narrowing that represents growing confidence. A point estimate reports only the location of the peak and discards the width; the Bayesian treatment of Section 7.8 keeps both.

§ 7.6 Generative versus predictive modeling

Two distinct objects can be learned from the same data: ms. p.20,27

Generative modeling learns the joint $\mathbb{P}_{\boldsymbol{\theta}}(\boldsymbol{x}, y)$. Having $\boldsymbol{\theta}^\star$, one can *generate* new samples $(\boldsymbol{x}, y) \sim \mathbb{P}(\boldsymbol{x}, y \mid \boldsymbol{\theta}^\star)$.

Predictive modeling learns the conditional $\mathbb{P}_{\boldsymbol{\theta}}(y \mid \boldsymbol{x})$, which is all that is needed to predict a label for a new input.

7.6.1 Conditional maximum likelihood

For predictive modeling the recipe is unchanged, with the conditional in place of the joint: ms. p.20

$$J(\boldsymbol{\theta}) = D_{\mathrm{KL}}(\mathbb{P}_{\text{data}}(y \mid \boldsymbol{x}) \,\|\, \mathbb{P}_{\boldsymbol{\theta}}(y \mid \boldsymbol{x}))\,, \qquad \boldsymbol{\theta}^\star = \arg\max_{\boldsymbol{\theta}} \log \prod_{i=1}^{n} \mathbb{P}_{\boldsymbol{\theta}}(y^{(i)} \mid \boldsymbol{x}^{(i)}).$$

This substitution deserves more than a sentence, because a dataset consists of *pairs* and the likelihood of a pair is not the conditional. Factoring the joint,

$$\mathbb{P}_{\boldsymbol{\theta}}\big(\boldsymbol{x}^{(i)}, y^{(i)}\big) = \underbrace{\mathbb{P}_{\boldsymbol{\theta}}\big(y^{(i)} \mid \boldsymbol{x}^{(i)}\big)}_{\text{what we model}} \cdot \underbrace{\mathbb{P}_{\boldsymbol{\theta}}\big(\boldsymbol{x}^{(i)}\big)}_{\text{what we do not}}\,, \tag{7.6}$$

so the full log-likelihood of the data is $\sum_i \log \mathbb{P}_{\boldsymbol{\theta}}(y^{(i)} \mid \boldsymbol{x}^{(i)}) + \sum_i \log \mathbb{P}_{\boldsymbol{\theta}}(\boldsymbol{x}^{(i)})$, and the second sum has simply been dropped.

The licence to drop it is an assumption, and it is worth stating plainly.

Key idea

The predictive assumption. The distribution of the inputs does not depend on $\boldsymbol{\theta}$:

$$\mathbb{P}_{\boldsymbol{\theta}}(\boldsymbol{x}) = \mathbb{P}(\boldsymbol{x}) \quad \text{for every } \boldsymbol{\theta}.$$

Then $\sum_i \log \mathbb{P}_{\boldsymbol{\theta}}(\boldsymbol{x}^{(i)})$ is a constant as far as $\boldsymbol{\theta}$ is concerned, it contributes nothing to $\arg\max_{\boldsymbol{\theta}}$, and (7.6) lets us maximize the conditional alone. Nothing has been proved — we have declared that our parameters describe how y follows from $\boldsymbol{x}$ and say nothing about which $\boldsymbol{x}$ arise.

This is exactly the line between the two programs above. A generative model *does* parameterize the inputs — Chapter 9 builds $\mathbb{P}_{\boldsymbol{\theta}}(\boldsymbol{x} \mid y)$ explicitly — and there the dropped term is not constant and cannot be discarded. A predictive model buys its simplicity by refusing to model $\mathbb{P}(\boldsymbol{x})$ at all, and pays for it by having nothing to say when the input distribution shifts, which is the failure mode Definition 6.1.1 warned about.

§ 7.7 Conditional maximum likelihood recovers the L_2 loss

Take the linear model $f_\theta(x) = \theta^\top x$ and assume Gaussian noise:

$$\mathbb{P}_\theta(y \mid x) = \frac{1}{\sqrt{2\pi}\,\sigma} \exp\!\Big(-\frac{1}{2\sigma^2}(y - \theta^\top x)^2\Big) = \mathcal{N}(\theta^\top x, \sigma^2). \tag{7.7}$$

Then

$$\log \mathbb{P}_\theta(y^{(i)} \mid x^{(i)}) = -\frac{1}{2\sigma^2}(y^{(i)} - \theta^\top x^{(i)})^2 + \text{const},$$

so summing over i and dropping the constant,

$$\theta^\star = \arg\max_\theta \sum_i \log \mathbb{P}_\theta(y^{(i)} \mid x^{(i)}) = \arg\min_\theta \frac{1}{2n} \sum_{i=1}^{n} (y^{(i)} - \theta^\top x^{(i)})^2,$$

ms. p.21 which is exactly the L_2 loss of Chapter 3.

Key idea

The squared-error loss is not an arbitrary choice. It is the negative log-likelihood of a Gaussian noise model. A different noise model gives a different loss — Laplace noise gives the L_1 loss.

Note:-

The predictive model is a distribution, not a curve. The trained object is $\mathbb{P}_{\theta^\star}(y \mid x) \propto \exp(-\frac{1}{2\sigma^2}(y - \theta^\star \cdot x)^2)$. The regression line $f_{\theta^\star}(x) = \theta^\star \cdot x$ is its mean; the width σ around that line is part of the answer and is discarded if one ms. p.21 reports only the line.

Let us see this geometrically. The note above is easier to believe once drawn. Plot the fitted conditional density at a few inputs, standing up out of the page, and the regression line is visibly just the ridge running along their peaks.

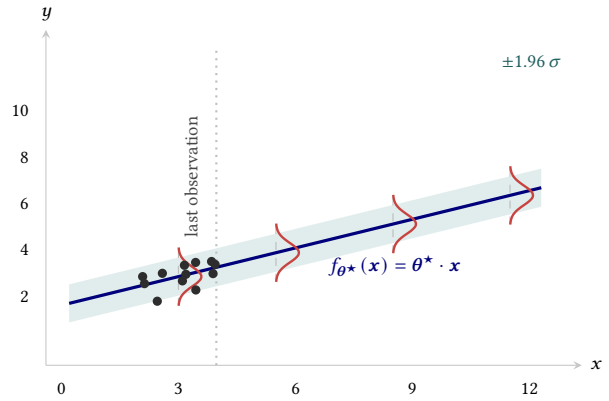


Figure 7.4: **What is actually fitted.** Twelve observations, every one taken between $x = 2$ and $x = 4$ — a calibration run over a single narrow operating range — with the least-squares line $\theta^\star = (1.594, 0.413)$ and known noise scale $\sigma = 0.42$. Four conditional densities $\mathbb{P}_{\theta^\star}(y \mid x)$ are drawn sideways, each Gaussian by (7.7) and centred on the line. The line is not the model; it is the *mean* of the model, the ridge joining the peaks, and the width is the rest of it. The $\pm 1.96\sigma$ band should hold about 95% of observations and holds 11 of the 12. Now look past the dotted line, where there is no data at all. The bell at $x = 11.5$ is *identical* to the one at $x = 3$: a point estimate has one width and uses it everywhere, and is exactly as confident about an extrapolation as about the middle of its data. That is not a property of the data — it follows from having replaced the posterior by a single $\theta^\star$, and Figure 7.6 redraws this same picture without doing so.

§ 7.8 Bayesian inference

Definition 7.8.1: Bayes' formula

$$\mathbb{P}(\boldsymbol{\theta} \mid z) = \frac{\mathbb{P}(z \mid \boldsymbol{\theta})\,\mathbb{P}(\boldsymbol{\theta})}{\mathbb{P}(z)}, \qquad \mathbb{P}(z) = \int_{\Theta} \mathbb{P}(z \mid \boldsymbol{\theta})\mathbb{P}(\boldsymbol{\theta})\,\mathrm{d}\boldsymbol{\theta},$$

with $\mathbb{P}(\boldsymbol{\theta})$ the prior, $\mathbb{P}(z \mid \boldsymbol{\theta})$ the likelihood of the measurement z given $\boldsymbol{\theta}$, and $\mathbb{P}(\boldsymbol{\theta} \mid z)$ the posterior. ms. p.21,22

Let us see this geometrically. Bayes' formula multiplies two curves and renormalizes. The prior says what we believed before the data; the likelihood says what the data prefers; the posterior is the compromise — and it is narrower than either, because two sources of information are better than one.

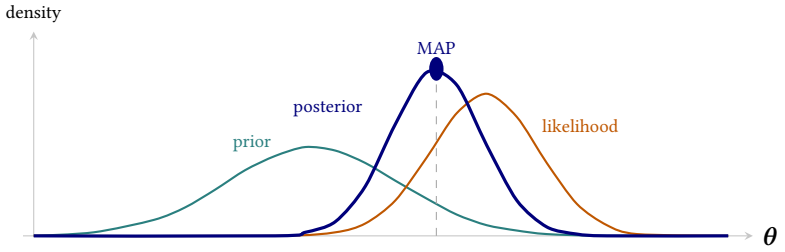


Figure 7.5: **One Bayesian update.** A Gaussian prior centered at 0 (teal), a Gaussian likelihood centered at 1.6 (orange), and the resulting posterior (navy). The posterior mean falls *between* the two — pulled from the likelihood back toward the prior, which is exactly the shrinkage that ridge regression performs — and its spread is *smaller* than either input. The MAP estimate is the location of the posterior's peak, and reporting only that number discards the width, which is the whole difference between Sections 7.9 and 7.10.

Since the evidence $\mathbb{P}(z)$ does not depend on $\boldsymbol{\theta}$, it may be dropped for the purpose of optimization: $\mathbb{P}(\boldsymbol{\theta} \mid z) \propto \mathbb{P}(z \mid \boldsymbol{\theta})\mathbb{P}(\boldsymbol{\theta})$. Methods such as Markov chain Monte Carlo exploit exactly this, requiring no knowledge of the normalizing constant. ms. p.22

§ 7.9 MAP estimation and ridge regression

Definition 7.9.1: MAP point

The point $\boldsymbol{\theta}^{\star}$ at which $\mathbb{P}(\boldsymbol{\theta} \mid \boldsymbol{y}, \boldsymbol{X})$ is largest is the maximum a posteriori *estimate. The inputs $\boldsymbol{X}$ are conditioned on throughout, never modelled — see §7.6.1.* ms. p.23

The data are pairs, so begin with the joint likelihood and split it by (7.6), assuming the samples i.i.d.:

$$\mathbb{P}(\mathcal{D} \mid \boldsymbol{\theta}) = \prod_{i=1}^{n} \mathbb{P}\big(y^{(i)} \mid \boldsymbol{x}^{(i)}, \boldsymbol{\theta}\big) \cdot \prod_{i=1}^{n} \mathbb{P}\big(\boldsymbol{x}^{(i)} \mid \boldsymbol{\theta}\big).$$

The second product is the one we have no model for, and by the predictive assumption of §7.6.1 we do not need one: it does not involve $\boldsymbol{\theta}$, so it is a constant multiplier that survives into the posterior unchanged and cancels against the same factor in the normalizer $\mathbb{P}(\mathcal{D})$. Equivalently, and more honestly in notation, condition on the inputs from the start and work with $\mathbb{P}(\boldsymbol{\theta} \mid \boldsymbol{y}, \boldsymbol{X})$, which is the object Definition 7.9.1 names. Either way what is left is the Gaussian conditional likelihood (7.7),

$$\mathbb{P}(\boldsymbol{y} \mid \boldsymbol{X}, \boldsymbol{\theta}) \propto \exp\Big(-\frac{1}{2\sigma^2}\sum_{i=1}^{n}(y^{(i)} - \boldsymbol{\theta}^{\mathsf{T}}\boldsymbol{x}^{(i)})^2\Big),$$

together with the Gaussian prior

$$\mathbb{P}(\boldsymbol{\theta}) = \mathcal{N}(\mathbf{0}, \gamma^2\mathrm{I}) \propto \exp\Big(-\frac{1}{2\gamma^2}\sum_{j}\theta_j^2\Big) = \exp\Big(-\frac{1}{2\gamma^2}\|\boldsymbol{\theta}\|^2\Big).$$

Multiplying the two, Bayes' formula gives

$$\mathbb{P}(\boldsymbol{\theta} \mid \boldsymbol{y}, \boldsymbol{X}) \propto \exp\Big(-\frac{1}{2\sigma^2}\sum_{i=1}^{n}(y^{(i)} - \boldsymbol{\theta}^{\mathsf{T}}\boldsymbol{x}^{(i)})^2 - \frac{1}{2\gamma^2}\|\boldsymbol{\theta}\|^2\Big),$$

and maximizing is minimizing the negative exponent:

$$\boldsymbol{\theta}^{\star} = \arg\min_{\boldsymbol{\theta}} \ \frac{1}{2\sigma^2}\sum_{i=1}^{n}(y^{(i)} - \boldsymbol{\theta}^{\mathsf{T}}\boldsymbol{x}^{(i)})^2 + \frac{1}{2\gamma^2}\|\boldsymbol{\theta}\|^2, \tag{7.8}$$

ms. p.23,24 which is (6.1) with $\lambda = \sigma^2/\gamma^2$.

Key idea

Ridge regression is MAP estimation with a Gaussian prior. The regularization weight is not a free knob after all — it is the ratio of noise variance to prior variance. The lasso, correspondingly, is MAP with a Laplace prior.

§ 7.10 The full Bayesian predictive model

MAP collapses the posterior to a single point, which is convenient but throws away everything the posterior knew about uncertainty. The parameter $\boldsymbol{\theta}$ is in any case an artifact of our modeling choice; the full Bayesian treatment marginalizes ms. p.24 it out:

$$\mathbb{P}(y \mid \boldsymbol{x}) = \int_{\Theta} \mathbb{P}(y \mid \boldsymbol{x}, \boldsymbol{\theta})\, \mathbb{P}(\boldsymbol{\theta} \mid \mathcal{D})\, \mathrm{d}\boldsymbol{\theta} = \mathbb{E}_{\mathbb{P}(\boldsymbol{\theta}\mid\mathcal{D})}\big[\mathbb{P}(y \mid \boldsymbol{x}, \boldsymbol{\theta})\big] \approx \frac{1}{M}\sum_{i=1}^{M}\mathbb{P}(y \mid \boldsymbol{x}, \boldsymbol{\theta}^i),$$

the last step being a Monte Carlo estimate over posterior samples $\boldsymbol{\theta}^i \sim \mathbb{P}(\boldsymbol{\theta} \mid \mathcal{D})$.

Let us see this geometrically. Redraw Figure 7.4 without collapsing the posterior — same data, same axes, same four inputs — and compare the two pictures bell by bell.

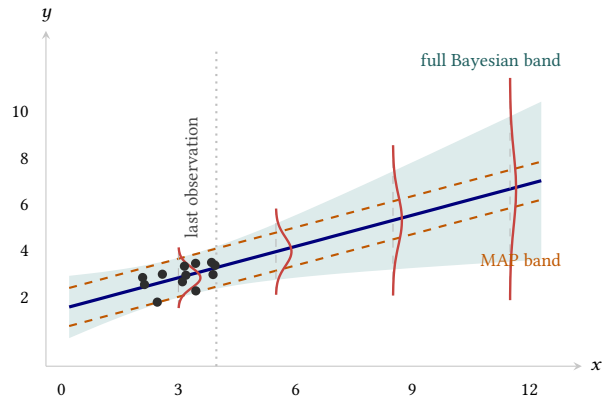


Figure 7.6: **The predictive distribution widens where the data runs out.** The same twelve observations as Figure 7.4, on the same axes and with the density drawn at the same four inputs, now with a Gaussian prior $\mathcal{N}(\mathbf{0}, \gamma^2\mathrm{I})$, $\gamma = 2$, so $\lambda = \sigma^2/\gamma^2 = 0.0441$. That λ is small enough that the posterior mean $(1.470, 0.451)$ is barely distinguishable from the least-squares line of the previous figure: the two share a line and differ in nothing but width. Likelihood and prior are both Gaussian, so the predictive $\mathbb{P}(y \mid \boldsymbol{x})$ is Gaussian too, with variance $\sigma^2 + \boldsymbol{\varphi}(\boldsymbol{x})^{\mathsf{T}}\Sigma\boldsymbol{\varphi}(\boldsymbol{x})$ — observation noise plus what the posterior still does not know about $\boldsymbol{\theta}$. At $x = 3$ the second term contributes 5% and the standard deviation is 0.437, barely above the $\sigma = 0.42$ of the previous figure; at $x = 11.5$ it contributes 93% and the deviation is 1.601 — nearly four times the noise scale, and the width is now almost entirely ignorance about $\boldsymbol{\theta}$ rather than observation noise. Set the two figures side by side: the four bells there are congruent, here they are not, and the MAP band (orange, dashed) runs on at $\pm 1.96\sigma$ forever. Having collapsed the posterior to a point, it knows about observation noise and nothing about its own ignorance.

Key idea

Compare the three: maximum likelihood returns a point and ignores the prior; MAP returns a point and uses the prior; full Bayes returns a distribution. Only the third propagates parameter uncertainty into the prediction — which in a scientific computing context is usually the quantity of interest.

Note:-

The generalization framework of Chapter 6 is the statistical counterpart of this chapter's probabilistic one: there we assumed data was drawn IID from $\mathbb{P}$ and asked when a fitted model generalizes; here we assume the same and ask what $\mathbb{P}$ itself is. Definition 6.1.1 and Definition 6.4.1 are used without further comment from now on.

§ 7.11 Exercises

The code problems use the datasets of Chapter 1; see Section C.5 for loading them and Section C.3 for why `x64` matters when exponentiating log-likelihoods.

Warm-up

1. [Theory] Write down $D_{\mathrm{KL}}(p \,\|\, q)$ for the two-outcome distributions $p = (0.5, 0.5)$ and $q = (0.9, 0.1)$, then compute $D_{\mathrm{KL}}(q \,\|\, p)$. Confirm numerically that the two differ, and say which of Figures 7.2a and 7.2b each corresponds to in spirit.

2. [Theory] For a single Gaussian observation $y \sim \mathcal{N}(\boldsymbol{\theta}^{\mathsf{T}}\boldsymbol{x}, \sigma^2)$, write $-\log \mathbb{P}_{\boldsymbol{\theta}}(y \mid \boldsymbol{x})$ and identify which term depends on $\boldsymbol{\theta}$ and which does not. Why may the second be discarded?

3. [Theory] Using Figure 7.3, explain what changes and what does not as n grows. Which feature of the curve does a point estimate report, and which does it throw away?

4. [Theory] In Figure 7.5, the posterior mean lies between the prior mean and the likelihood mean. Show this must always happen for two Gaussians by writing the posterior mean as a weighted average, and identify the weights.

5. [Theory] State the difference between $\mathbb{P}_{\boldsymbol{\theta}}(\boldsymbol{x}, y)$ and $\mathbb{P}_{\boldsymbol{\theta}}(y \mid \boldsymbol{x})$ in words, and give one task for which you would want each. Which does Chapter 9 learn?

6. [Code] Load the Combined Cycle Power Plant data, fit the affine model by least squares, and compute the residual standard deviation. Interpret it as the σ of the Gaussian noise model of Section 7.7, and plot a histogram of the residuals against the fitted Gaussian density.

Core

7. [Theory] Give the Jensen's-inequality proof of Proposition 7.4.1(i): apply Jensen to the convex function $-\log$ and the random variable $\mathbb{P}_{\boldsymbol{\theta}}/\mathbb{P}_{\mathrm{data}}$. Compare it with the proof in the text and say what each route costs — in particular, what extra work Jensen leaves you to do before you can conclude Proposition 7.4.1(ii), which is the part Theorem 7.5.1 needs.

8. [Theory] The proof of Proposition 7.4.1 rests on $\log t \le t - 1$. Show that this is the first-order Taylor bound for a concave function, and deduce the sharper two-sided estimate $1 - 1/t \le \log t \le t - 1$. Which side gives a lower bound on $D_{\mathrm{KL}}(p \,\|\, q)$, and is it ever useful?

9. [Theory] Show that KL divergence is not symmetric by exhibiting two distributions with $D_{\mathrm{KL}}(p \,\|\, q) \neq D_{\mathrm{KL}}(q \,\|\, p)$. Is it a metric? Which metric axiom fails first?

10. [Theory] Theorem 5.6.2 gives the mean square error of a Monte Carlo estimate. Restate it for the estimator (7.3) of $\mathbb{E}[g(x)]$, and say what plays the role of $\mathrm{Var}[g]$ when $g = \log \mathbb{P}_{\boldsymbol{\theta}}$ is the log-likelihood.

11. [Theory] In Example 7.5.1, verify that $\theta^\star = \#H/(\#H + \#T)$ is a *maximum* and not merely a stationary point, and explain why $\theta = 0$ and $\theta = 1$ arise as roots and why they are discarded.

12. [Theory] Complete the derivation of the L_2 loss from the Gaussian conditional likelihood, keeping track of every term that is discarded and justifying each discard.

13. [Theory] Carry out the MAP derivation (7.8) in full, starting from the joint likelihood $\mathbb{P}(\mathcal{D} \mid \boldsymbol{\theta})$ rather than the conditional, and track the factor $\prod_i \mathbb{P}(\boldsymbol{x}^{(i)} \mid \boldsymbol{\theta})$ all the way to the end. Show explicitly where it cancels, and confirm $\lambda = \sigma^2/\gamma^2$. What happens to the estimator as $\gamma \to \infty$, and how does that relate to maximum likelihood?

14. [Theory] Suppose the predictive assumption of §7.6.1 fails, because $\boldsymbol{\theta}$ contains parameters describing the inputs as well — write $\boldsymbol{\theta} = (\boldsymbol{\theta}_y, \boldsymbol{\theta}_x)$ with $\mathbb{P}_{\boldsymbol{\theta}}(\boldsymbol{x}, y) = \mathbb{P}(y \mid \boldsymbol{x}, \boldsymbol{\theta}_y)\mathbb{P}(\boldsymbol{x} \mid \boldsymbol{\theta}_x)$. Show that if the two blocks share no components, maximum likelihood separates into two independent problems and the conditional estimate $\boldsymbol{\theta}_y^\star$ is unchanged. Then say what goes wrong when they *do* share a component, and name the chapter where that happens.

15. [Code] Implement the coin likelihood of Example 7.5.1 in JAX and reproduce Figure 7.3 for $n = 5, 20, 100$. Measure the width of each curve at half its peak and check that it shrinks like $n^{-1/2}$, as Theorem 5.6.2 would suggest.

Challenge

16. [Theory] Repeat the MAP derivation with a Laplace prior $\mathbb{P}(\boldsymbol{\theta}) \propto \exp(-\|\boldsymbol{\theta}\|_1/b)$ and show that the lasso (6.3) results. What is the correspondence between b and λ, and why does this explain the non-differentiability noted in Chapter 6?

17. [Theory] Show that assuming Laplace *noise* rather than Gaussian noise turns the L_2 loss into the L_1 loss of Chapter 2. Together with the previous exercise this gives four combinations of noise and prior; tabulate the four losses.

18. [Theory] Reproduce Figure 7.2 analytically for a mixture of two Gaussians approximated by one. Show that minimizing $D_{\mathrm{KL}}(p \,\|\, q)$ matches the first two moments of p, and argue why minimizing $D_{\mathrm{KL}}(q \,\|\, p)$ instead concentrates on a single mode.

19. [Theory] For Gaussian likelihood and Gaussian prior, derive the full Bayesian predictive distribution in closed form and show that its variance is $\sigma^2 + \boldsymbol{\varphi}(\boldsymbol{x})^{\mathsf{T}}\Sigma\boldsymbol{\varphi}(\boldsymbol{x})$. Identify which term is observation noise and which is parameter uncertainty, and hence explain why the shaded band of Figure 7.6 widens away from the data while the dashed one does not.

20. [Code] Reproduce Figure 7.6. Fit a ridge model to twelve points on $x \in [2, 4]$ with $\sigma = 0.42$ and $\gamma = 2$, compute the posterior covariance $\Sigma = \sigma^2(\Phi^{\mathsf{T}}\Phi + \lambda \mathrm{I})^{-1}$, and plot both bands. Confirm the standard deviations quoted in the caption at $x = 3$ and $x = 11.5$. Then extrapolate to $x = 20$ and report the ratio of the two widths. At what x does the ratio exceed two, and why is that so much further out than the last observation?

21. [Code] Sample the posterior $\mathbb{P}(\boldsymbol{\theta} \mid \mathcal{D})$ for the same problem and build the predictive distribution by Monte Carlo instead of in closed form, using (7.3). Plot the Monte Carlo band against the analytic one for $M = 10, 100, 1000$ posterior samples, and check the discrepancy falls like $M^{-1/2}$.

Chapter 8

Classification

Goal of this chapter

Chapter 3 predicted numbers. This chapter predicts *labels*, and the change is less cosmetic than it looks: the target space $\mathcal{Y}$ becomes a finite set, and almost everything follows from that.
The chapter answers:

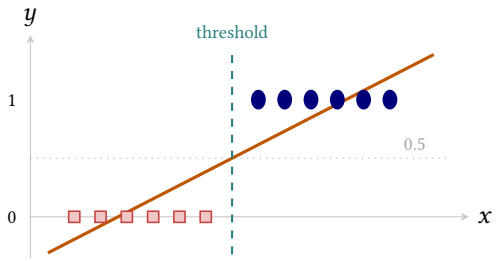
1. Why can we not simply fit a line and threshold it?
2. What model produces a number that may honestly be read as a probability?
3. What loss does Chapter 7 then hand us, without our choosing anything?
4. What shape is the resulting decision rule, and what happens with more than two classes?

The pattern of Chapter 7 repeats exactly: fix a noise model, take the negative log-likelihood, and the loss is determined. Only the noise model changes — Gaussian there, Bernoulli here.

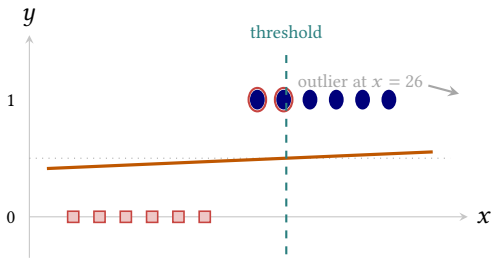
§ 8.1 Why linear regression is the wrong tool

Take $\mathcal{D} = \{(\boldsymbol{x}^{(i)}, y^{(i)})\}_{i=1}^{n}$ with $y^{(i)} \in \{0, 1\}$, fit the linear model $y = \boldsymbol{\theta}^{\mathsf{T}}\boldsymbol{x}$ by least squares, and threshold the output at 0.5. Consider two samples with $\boldsymbol{x}^{(1)} = 1$, $y^{(1)} = 10$ and $\boldsymbol{x}^{(2)} = 1$, $y^{2} = 0.01$: the fitted value is unbounded, so the meaning of the threshold drifts with the data, and a single distant point drags the boundary arbitrarily. We need a model whose output is confined to $[0, 1]$ and can be read as a probability. ms. p.24

Let us see this geometrically. The objection is not that thresholding a line *never* works. On clean, well-separated data it works fine. The objection is that it works for the wrong reason, and a single distant point — one that any human would call *easy* — can break it.



(a) Clean data: thresholding works.



(b) One far-away point, correctly labeled.

Figure 8.1: **Least squares plus a threshold is not a classifier.** Squares are class 0, circles class 1; the orange line is the least-squares fit and the dashed line where it crosses 0.5. **(a)** On separated data the rule is correct. **(b)** Adding a single point at $x = 26$ with label 1 — a point that is not even ambiguous — flattens the fit and drags the threshold from 2.30 to 2.93, so the two ringed points are now misclassified *despite being correctly labeled in the training set.* The squared-error loss penalizes the model for predicting 2.4 when the label is 1, though from a classification standpoint that prediction is emphatically right. Nothing is wrong with the optimizer; the loss is measuring the wrong thing.

Note:-

Three standard approaches to binary classification are logistic regression, support vector machines, and deep neural networks. This chapter develops the first; the third is the subject of Chapters 11 and 12.

ms. p.24

§ 8.2 The logistic function

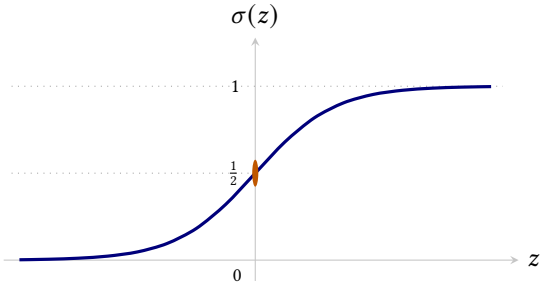
Definition 8.2.1: Logistic, or sigmoid, function

$$\sigma(z) \coloneqq \frac{1}{1+e^{-z}}, \qquad \sigma : \mathbb{R} \to (0,1).$$

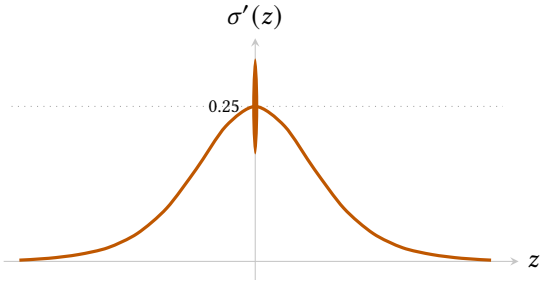
ms. p.25

The range $(0,1)$ is exactly what the previous section asked for.

Let us see this geometrically. Two features of σ matter, and only one of them is the reason we introduced it. The range $(0,1)$ is what we asked for. The shape of its *derivative* is what will come back to haunt us in Chapter 12.



(a) σ squashes $\mathbb{R}$ into $(0,1)$.



(b) σ' never exceeds $\frac{1}{4}$.

Figure 8.2: **The logistic function and its derivative. (a)** σ maps the whole real line into $(0,1)$, so $\boldsymbol{\theta} \cdot \boldsymbol{x}$ may be any size and the output is still readable as a probability. It crosses $\frac{1}{2}$ at $z = 0$, which is what makes the decision boundary of Section 8.4 a hyperplane. **(b)** Its derivative $\sigma' = \sigma(1-\sigma)$ peaks at $\sigma'(0) = \frac{1}{4}$ and decays to zero in both directions. Two consequences: gradients vanish when the model is confident, so a badly initialized logistic model learns slowly; and stacking L such layers multiplies L factors each at most $\frac{1}{4}$, which is the vanishing-gradient problem of Chapter 12 in embryo.

Example 8.2.1 (A confident unit stops learning)

The derivative in Figure 8.2b is not a curiosity; it decides whether training makes progress. Tabulating $\sigma'(z) = \sigma(z)\big(1-\sigma(z)\big)$:

z	$\sigma(z)$	$\sigma'(z)$	below peak
0	0.500	2.50×10^{-1}	1×
4	0.982	1.77×10^{-2}	14×
8	0.9997	3.35×10^{-4}	746×
10	0.99996	4.54×10^{-5}	5507×

A unit driven to $z = 10$ is 99.996% confident and has a gradient five thousand times smaller than one sitting at $z = 0$. Gradient descent will barely move it, whether it is right or wrong.

The arithmetic compounds with depth. Chapter 12 shows that backpropagating through L sigmoid layers multiplies L such factors together. At best, every unit at $z = 0$, that is $0.25^{10} \approx 10^{-6}$ for ten layers. With units sitting at $z = 8$ it is $(3.35 \times 10^{-4})^{10} \approx 10^{-35}$, which is zero in float64. This single table is why Chapter 11 eventually abandons the sigmoid for ReLU in hidden layers, and it is worth carrying forward from here.

§ 8.3 The logistic model

Model the conditional distribution as

ms. p.25

$$\mathbb{P}(y = 1 \mid \boldsymbol{\theta}, \boldsymbol{x}) = \sigma(\boldsymbol{\theta} \cdot \boldsymbol{x}) = \frac{1}{1 + \exp(-\boldsymbol{\theta} \cdot \boldsymbol{x})}, \qquad \mathbb{P}(y = 0 \mid \boldsymbol{\theta}, \boldsymbol{x}) = 1 - \sigma(\boldsymbol{\theta} \cdot \boldsymbol{x}).$$

Both cases combine into the single expression

$$\mathbb{P}(y \mid \boldsymbol{\theta}, \boldsymbol{x}) = \sigma(\boldsymbol{\theta} \cdot \boldsymbol{x})^{y} \big[1 - \sigma(\boldsymbol{\theta} \cdot \boldsymbol{x})\big]^{1-y},$$

valid for $y \in \{0, 1\}$.

8.3.1 Cross-entropy loss

Applying conditional maximum likelihood (Theorem 7.5.1), the log likelihood is

$$\log \prod_{i=1}^{n} \mathbb{P}(y^{(i)} \mid \boldsymbol{\theta}, \boldsymbol{x}^{(i)}) = \sum_{i=1}^{n} y^{(i)} \log\big(\sigma(\boldsymbol{\theta} \cdot \boldsymbol{x}^{(i)})\big) + (1 - y^{(i)}) \log\big[1 - \sigma(\boldsymbol{\theta} \cdot \boldsymbol{x}^{(i)})\big],$$

and $\boldsymbol{\theta}^{\star}$ maximizes it. The negative of this quantity is the *cross-entropy loss*:

$$J(\boldsymbol{\theta}) \coloneqq -\sum_{i=1}^{n} \Big[y^{(i)} \log \sigma(\boldsymbol{\theta} \cdot \boldsymbol{x}^{(i)}) + (1 - y^{(i)}) \log\big(1 - \sigma(\boldsymbol{\theta} \cdot \boldsymbol{x}^{(i)})\big) \Big]. \tag{8.1}$$

ms. p.25

Let us see this geometrically. The shape of the cross-entropy penalty is worth a moment. It is *unbounded*: a model that is confidently wrong is not merely penalized, it is punished without limit.

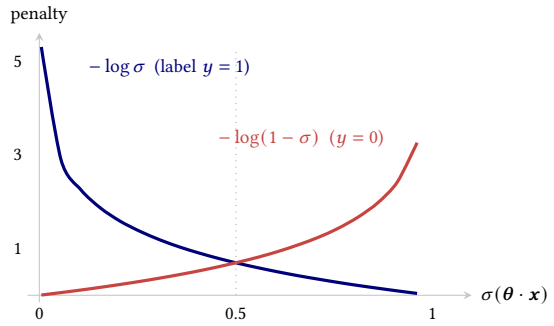


Figure 8.3: **Cross-entropy punishes confident mistakes without bound.** The two terms of (8.1), each plotted against the predicted probability. If the true label is 1, the penalty is $-\log\sigma$: near zero when the model says 0.99, but growing without limit as the prediction approaches 0. Contrast the squared error of Figure 8.1, whose penalty for a confidently wrong prediction is bounded by 1 — which is exactly why a single distant point could tilt that fit while barely moving this loss. Unboundedness is a feature: it forbids the model from being certain and wrong.

Key idea

Cross-entropy is not a new invention. It is the negative log-likelihood of a Bernoulli model, exactly as the L_2 loss was the negative log-likelihood of a Gaussian one. The pattern from Chapter 7 repeats without modification.

Note:-

Unlike linear regression, (8.1) has no closed-form minimizer. It must be solved by gradient descent (Chapter 4), which is why that chapter precedes this one.

Example 8.3.1 (What it costs to be confidently wrong)

Take one sample whose true label is 1 and watch the penalty as the model grows more sure it is 0:

prediction σ	squared error $(1-\sigma)^2$	cross-entropy $-\log\sigma$
0.1	0.810	2.303
0.01	0.980	4.605
10^{-3}	0.998	6.908
10^{-6}	1.000	13.816

Becoming a hundred thousand times more confident, from 0.1 to 10^{-6}, multiplies the squared error by 1.23 and the cross-entropy by 6.

The decisive quantity is not the penalty but its *derivative*, since that is what gradient descent sees. For squared error, $\partial_\sigma(1-\sigma)^2 = -2(1-\sigma)$, which runs from -1.8 at $\sigma = 0.1$ to -2 as $\sigma \to 0$: it saturates. For cross-entropy $\partial_\sigma(-\log\sigma) = -1/\sigma$, which is -10, then -100, then -1000 at the same three predictions.

So a confidently wrong prediction produces almost no corrective signal under squared error and an overwhelming one under cross-entropy. That, rather than anything about the shape of the curve, is the mechanism behind §8.1: the outlier in Figure 8.1 tilted the least-squares fit because squared error could not tell the difference between slightly wrong and absurdly wrong.

§ 8.4 Prediction and the decision boundary

Given $\theta^\star$ and a new input x, the predictive model returns $\mathbb{P}(\hat{y}=1 \mid \theta^\star, x) = \sigma(\theta^\star \cdot x)$, and we assign

$$\hat{y} = \begin{cases} 1 & \text{if } \mathbb{P}(\hat{y}=1 \mid \theta^\star, x) \geq 0.5, \\ 0 & \text{otherwise.} \end{cases}$$

Proposition 8.4.1 The boundary is a hyperplane

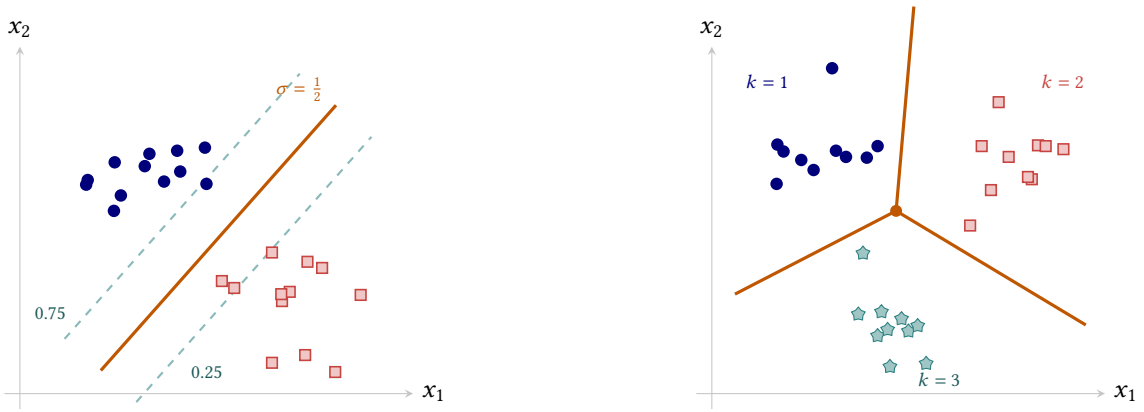
The decision boundary — the set on which the two class probabilities agree — is $\{\boldsymbol{x} : \boldsymbol{\theta}^\star \cdot \boldsymbol{x} = 0\}$.

Proof: The boundary is where $\mathbb{P}(y = 1 \mid \boldsymbol{\theta}^\star, \boldsymbol{x}) = \mathbb{P}(y = 0 \mid \boldsymbol{\theta}^\star, \boldsymbol{x})$, that is

$$\log \frac{\sigma(\boldsymbol{\theta}^\star \cdot \boldsymbol{x})}{1 - \sigma(\boldsymbol{\theta}^\star \cdot \boldsymbol{x})} = 0.$$

Substituting $\sigma(z) = 1/(1 + e^{-z})$ and $1 - \sigma(z) = e^{-z}/(1 + e^{-z})$, the ratio collapses to $e^{\boldsymbol{\theta}^\star \cdot \boldsymbol{x}}$, so the condition reads $\boldsymbol{\theta}^\star \cdot \boldsymbol{x} = 0$. ms. p.26
■

Let us see this geometrically. The boundary is a hyperplane, but the *model* is not a hyperplane: it assigns a probability to every point, and those probabilities vary smoothly across parallel lines. The boundary is just the one line where the probability equals a half.



(a) Binary: one hyperplane.

(b) Three classes: convex polyhedral regions.

Figure 8.4: **What the decision rule looks like. (a)** The logistic model assigns a probability everywhere; the dashed lines are the level sets $\sigma = 0.25$ and 0.75, parallel to the boundary because σ depends on $\boldsymbol{x}$ only through $\boldsymbol{\theta} \cdot \boldsymbol{x}$. The orange line is where $\sigma = \frac{1}{2}$, which is the hyperplane $\boldsymbol{\theta}^\star \cdot \boldsymbol{x} = 0$. **(b)** With $K = 3$ classes, softmax gives one hyperplane per pair, $(\boldsymbol{\theta}^j - \boldsymbol{\theta}^k) \cdot \boldsymbol{x} = 0$. Only the nearest portion of each survives, and the three meet at a single point where all three probabilities are equal. The resulting decision regions are convex polyhedra — a real restriction, and the reason Chapter 11 is needed at all.

Example 8.4.1 (The half is a convention, not a result)

Nothing in Proposition 8.4.1 says the threshold must be 0.5. That value minimizes the number of mistakes, which is the right objective only when the two kinds of mistake cost the same.

Return to the weld inspection of Example 5.3.5. Missing a crack risks a structural failure; a false alarm costs one re-inspection. If a miss costs c_{FN} and a false alarm c_{FP}, flagging a weld is worth it when $c_{\text{FN}}\mathbb{P}(\text{crack} \mid \boldsymbol{x}) > c_{\text{FP}}\big(1 - \mathbb{P}(\text{crack} \mid \boldsymbol{x})\big)$, that is when

$$\mathbb{P}(\text{crack} \mid \boldsymbol{x}) \;>\; t^\star = \frac{c_{\text{FP}}}{c_{\text{FP}} + c_{\text{FN}}}.$$

cost ratio $c_{\text{FN}} : c_{\text{FP}}$	threshold $t^\star$
1 : 1	0.500
10 : 1	0.091
100 : 1	0.0099
500 : 1	0.0020

At 500 : 1 you should flag any weld with more than a 0.2% chance of a crack. The fitted model does not change —

$\theta^\star$ is the same, and so is the probability it reports. Only the line drawn through those probabilities moves, and it moves because of engineering economics, not statistics.

Example 8.4.2 (Why accuracy is the wrong number to report)

Keep the 1% crack rate and inspect 10,000 welds.

A classifier that always answers "sound" scores 99.0% accuracy. It finds none of the 100 cracks, and it required no data.

A detector with sensitivity 0.92 and specificity 0.85 scores 85.1% accuracy — substantially *worse* by that measure. It finds 92 of the 100 cracks, at the price of 1485 false alarms. Its precision is only 5.8%: nineteen of every twenty flagged welds are sound, which is the base-rate arithmetic of Example 5.3.5 seen again.

The worse-accuracy model is the useful one. Accuracy weights the 9900 sound welds and the 100 cracked ones equally, and the whole value of the exercise lives in the 100. Report sensitivity and precision at the operating threshold you actually intend to use — the one Example 8.4.1 computes — rather than a single number that hides which mistakes are being made.

§ 8.5 Multi-class classification and softmax

Definition 8.5.1: Softmax

For $K \geq 2$ classes with parameter vectors $\theta^1, \dots, \theta^K$,

$$\mathbb{P}(y = k \mid \boldsymbol{x}, \theta^1, \dots, \theta^K) := \frac{\exp(\theta^k \cdot \boldsymbol{x})}{\sum_{j=1}^{K} \exp(\theta^j \cdot \boldsymbol{x})}.$$

ms. p.26,27

Prediction assigns the class of largest probability. For instance if $\mathbb{P}(y = 1 \mid \cdot) = 0.2$, $\mathbb{P}(y = 2 \mid \cdot) = 0.5$, and $\mathbb{P}(y = 3 \mid \cdot) = 0.3$, then $\hat{y} = 2$.

ms. p.27

8.5.1 Identifiability

The parameterization is redundant: adding a common vector to every θ^k leaves the softmax unchanged. One therefore fixes a reference class, taking $\theta^K = \mathbf{0}$, which makes the model well defined.

ms. p.27

For $K = 2$ this recovers logistic regression exactly. Dividing numerator and denominator by $\exp(\theta^1 \cdot \boldsymbol{x})$,

$$\mathbb{P}(y = 1 \mid \cdot) = \frac{1}{1 + \exp\big((\theta^2 - \theta^1) \cdot \boldsymbol{x}\big)},$$

so only the difference $\theta^2 - \theta^1$ is identifiable.

ms. p.26

8.5.2 Decision boundaries

The boundary between classes j and k is the set of $\boldsymbol{x}$ at which $\mathbb{P}(y = j \mid \boldsymbol{x}, \theta^\star) = \mathbb{P}(y = k \mid \boldsymbol{x}, \theta^\star)$, which reduces to the hyperplane $(\theta^j - \theta^k) \cdot \boldsymbol{x} = 0$.

ms. p.27

§ 8.6 Exercises

The code problems use the **Palmer Penguins** data (`https://allisonhorst.github.io/palmerpenguins/`, or `seaborn.load_dataset("penguins")`) and the datasets of Chapter 1. Section C.5 covers loading them.

Warm-up

1. [Theory] Example 8.2.1 tabulates σ' at $z = 0, 4, 8, 10$. Verify two of those entries by hand, then find the z at which σ' has fallen to one hundredth of its peak. How confident, as a probability, is a unit sitting there?

2. [Theory] Using Example 8.4.1, compute the optimal threshold when a false alarm costs *more* than a miss, say $c_{\text{FN}} : c_{\text{FP}} = 1 : 4$. Give an engineering situation where that ratio is the right one.

3. [Theory] Show that $\sigma(-z) = 1 - \sigma(z)$, and hence that $\mathbb{P}(y = 0 \mid \boldsymbol{\theta}, \boldsymbol{x}) = \sigma(-\boldsymbol{\theta} \cdot \boldsymbol{x})$. What does this say about the symmetry of the two classes under the model?

4. [Theory] Verify that $\mathbb{P}(y \mid \boldsymbol{\theta}, \boldsymbol{x}) = \sigma^y (1 - \sigma)^{1-y}$ reproduces both cases $y = 0$ and $y = 1$, and derive (8.1) from it.

5. [Theory] Show that $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ and that $\max_z \sigma'(z) = \sigma'(0) = \frac{1}{4}$, confirming Figure 8.2b. Retain the constant: it is the key quantity in Chapter 12.

6. [Theory] Using Figure 8.3, state the penalty incurred when the model predicts 0.01 and the true label is 1, and compare it with the squared error for the same prediction. Which loss would an optimizer work harder to fix?

7. [Theory] In Figure 8.4a the dashed lines are parallel to the boundary. Explain why, using only the fact that σ depends on $\boldsymbol{x}$ through $\boldsymbol{\theta} \cdot \boldsymbol{x}$ alone.

8. [Code] Load the Palmer Penguins data, drop missing rows, and build a binary problem: Gentoo against the other two species, using flipper length and bill depth. Plot the two classes and estimate by eye whether a single straight line could separate them.

Core

9. [Theory] Example 8.3.1 compares the two losses through their derivatives. Show that for the squared error $\partial_\sigma (1 - \sigma)^2 \to -2$ as $\sigma \to 0$ while for cross-entropy $\partial_\sigma (-\log \sigma) \to -\infty$, and explain in one sentence why a bounded derivative is the precise defect that §8.1 illustrates with an outlier.

10. [Code] Reproduce Example 8.4.2 on the Palmer Penguins data, made deliberately imbalanced: keep every sample of one species and subsample a second down to 1%. Fit logistic regression, then report accuracy, sensitivity and precision at thresholds 0.5, 0.1 and 0.01. Which threshold would you ship, and what additional information did you need to answer that?

11. [Theory] Compute $\nabla_\theta J(\boldsymbol{\theta})$ for the cross-entropy loss and show it takes the form $\sum_i (\sigma(\boldsymbol{\theta} \cdot \boldsymbol{x}^{(i)}) - y^{(i)}) \boldsymbol{x}^{(i)}$ — structurally identical to the L_2 gradient of Chapter 3 with the model output replaced by $\sigma(\boldsymbol{\theta} \cdot \boldsymbol{x})$. What is the practical consequence for implementation?

12. [Theory] Show that (8.1) is convex in $\boldsymbol{\theta}$. Why does this matter, given that no closed-form minimizer exists? Which result of Chapter 2 does it let you invoke?

13. [Theory] Show that softmax is invariant under a common shift $\boldsymbol{\theta}^k \mapsto \boldsymbol{\theta}^k + \boldsymbol{c}$, and hence that fixing $\boldsymbol{\theta}^K = \mathbf{0}$ loses no generality. How many free parameters does the K-class model actually have?

14. [Theory] Show that the multi-class decision regions are convex polyhedra, as drawn in Figure 8.4b. Deduce that no softmax model can carve out the annulus of Figure 1.6a without a feature map.

15. [Theory] Make the argument of Section 8.1 precise: construct a dataset on which least squares plus thresholding misclassifies a point that logistic regression classifies correctly, and identify the property of the squared-error loss responsible.

16. [Theory] On perfectly separable data the cross-entropy minimizer does not exist: show that $J(\boldsymbol{\theta})$ can be driven toward zero by scaling $\boldsymbol{\theta} \to c\boldsymbol{\theta}$ with $c \to \infty$, so $\|\boldsymbol{\theta}^\star\|$ diverges. What does Chapter 6 offer as a remedy?

17. [Code] Fit logistic regression on your penguin problem in JAX, training with Algorithm 2. Plot the decision boundary over the scatter and report accuracy, precision, and recall. Then plot the fitted probability as a shaded background and compare with Figure 8.4a.

Challenge

18. [Theory] Derive the softmax cross-entropy loss for K classes from the categorical likelihood, and show it reduces to (8.1) when $K = 2$ and $\boldsymbol{\theta}^2 = \mathbf{0}$.

19. [Theory] Show that the Hessian of the cross-entropy loss is $\boldsymbol{\Phi}^\mathsf{T} \boldsymbol{D} \boldsymbol{\Phi}$ with $\boldsymbol{D}$ diagonal and $D_{ii} = \sigma_i(1 - \sigma_i)$. Using Figure 8.2b, explain why the Hessian becomes nearly singular once the model fits the data confidently, and what that does to the convergence rate of Chapter 4.

20. [Theory] Logistic regression models $\mathbb{P}(y \mid \boldsymbol{x})$ directly. Chapter 9 instead models $\mathbb{P}(\boldsymbol{x}, y)$ and derives the conditional. State what each approach assumes and what each can do that the other cannot, and predict which will need less data.

21. [Theory] Add an L_2 penalty to (8.1) and show the minimizer now exists even on separable data. Interpret the result as MAP estimation with a Gaussian prior, following Section 7.9, and say what the prior is doing here in words.

22. [Code] Reproduce Figure 8.1. Fit least squares to a one-dimensional two-class dataset, then move a single correctly labeled point steadily to the right and record the threshold as a function of its position. At what distance does the first misclassification occur? Repeat with logistic regression and plot both curves.

23. [Code] Extend to three-class softmax on the full penguin dataset. Plot the polyhedral decision regions, verify the shift-invariance of Exercise 9 numerically, and locate the point where all three predicted probabilities are equal. Compare with Figure 8.4b.

Chapter 9

Generative Classification: Gaussian Discriminant Analysis

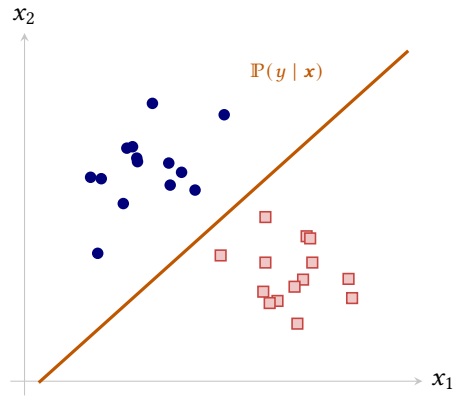
Goal of this chapter

Chapter 8 learned $\mathbb{P}(y \mid \boldsymbol{x})$ directly: it drew a boundary and never asked where the data came from. This chapter takes the opposite route — model the joint $\mathbb{P}(\boldsymbol{x}, y)$, then *derive* the conditional from it.
The chapter answers:

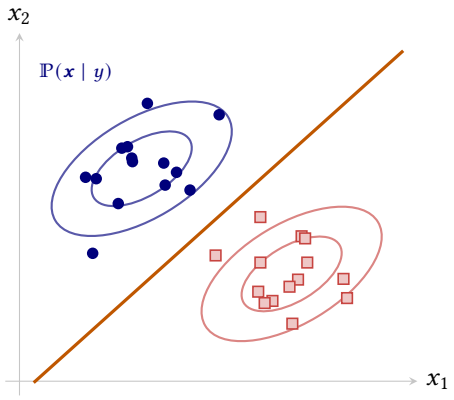
1. What must we assume to model the joint, and what does that cost?
2. What does the resulting model say about $\boldsymbol{x}$ alone — and why does a mixture appear without our putting one there?
3. How are the parameters fitted, and why does the calculation split into independent pieces?

The payoff is that a generative model can do something no discriminative model can: *generate*. Having fitted it, we can draw new samples. The price is a much stronger assumption about the data, and the trade between those two is the subject.

Let us see this geometrically. The distinction is easiest to see before any formulas. Both pictures below classify the same points equally well. They differ in what else they know.



(a) Discriminative: a boundary, and nothing else.



(b) Generative: a model of the data itself.

Figure 9.1: **Two ways to classify the same points. (a)** Logistic regression fits the boundary directly. Ask it where the data lives and it has no answer — it never modeled that. **(b)** Gaussian discriminant analysis fits a density to *each class* (contours shown), and the boundary is a consequence rather than the object of the fit. That extra structure is what lets the model be run backwards to produce new data, and it is also the extra assumption that can be wrong: if the classes are not remotely Gaussian, (b) is committed to a falsehood that (a) never had to state.

§ 9.1 The postulates

ms. p.28 Gaussian discriminant analysis postulates:

1. a categorical class prior,

$$\mathbb{P}(y = k \mid \boldsymbol{\theta}) = \phi_k, \qquad 0 \le \phi_k \le 1, \qquad \sum_{k=1}^{K} \phi_k = 1;$$

2. a Gaussian class-conditional density,

$$\mathbb{P}(\boldsymbol{x} \mid y = k, \boldsymbol{\theta}) = \mathcal{N}(\boldsymbol{\mu}_k, \Sigma_k) \propto \exp\Big(-\tfrac{1}{2}(\boldsymbol{x} - \boldsymbol{\mu}_k)^{\mathsf{T}} \Sigma_k^{-1} (\boldsymbol{x} - \boldsymbol{\mu}_k)\Big).$$

The parameters are $\boldsymbol{\theta} \coloneqq \{\phi_1, \dots, \phi_K,\ \boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_K,\ \Sigma_1, \dots, \Sigma_K\}$.

Proposition 9.1.1 Parameter count

For $\boldsymbol{x} \in \mathbb{R}^d$ the model has

$$K + dK + \frac{d^2}{2}(K + 1)$$

ms. p.28 *free parameters, the last term reflecting the symmetry of each Σ_k.*

Example 9.1.1 (Knowing when not to answer)

The second postulate buys something logistic regression never has: a density for $\boldsymbol{x}$ itself. Take two classes in the plane at $\boldsymbol{\mu}_1 = (0, 0)$ and $\boldsymbol{\mu}_2 = (4, 0)$ with $\Sigma = 0.6\mathrm{I}$ and equal priors, and feed four inputs to both kinds of model.

input $\boldsymbol{x}$	$\mathbb{P}(y = 2 \mid \boldsymbol{x})$	$\mathbb{P}(\boldsymbol{x})$
$(3.6,\ 0.2)$	1.000	1.1×10^{-1}
$(2.0,\ 0.0)$	0.500	9.5×10^{-3}
$(4.2,\ 9.0)$	1.000	6.2×10^{-31}
$(25,\ 0.0)$	1.000	3.3×10^{-161}

Read the two columns against each other. The posterior is all a discriminative model reports, and by that column the last two inputs are ordinary: class 2, with complete confidence. They are nothing of the sort. The point

$(4.2,\ 9.0)$ has a marginal density 5×10^{-30} times that at a class centre, and $(25,\ 0)$ is beyond absurd. Both lie far outside anything the training data resembled, and the posterior cannot say so — it is a *ratio*, and a ratio of two negligible numbers is perfectly well behaved.
This is what $\mathbb{P}(\boldsymbol{x})$ is for. Thresholding it gives novelty detection for free: a structural monitor can refuse to classify a reading rather than confidently misreport it, which on an instrumented vehicle is the difference between a flagged sensor fault and a wrong answer delivered with authority. Nothing in Chapter 8 can do this, and the price is the modeling assumption of the second postulate — §9.4 and Example 9.4.1 are about what that assumption costs.

§ 9.2 The induced marginal is a Gaussian mixture

Definition 9.2.1: Total probability

If $B_1, \dots, B_K$ partition the sample space, $\sum_k \mathbb{P}(B_k) = 1$, then $\mathbb{P}(A) = \sum_k \mathbb{P}(A \mid B_k)\mathbb{P}(B_k)$. ms. p.28

Example 9.2.1 (Example)

Let B_1 be sunny and B_2 rain, with $\mathbb{P}(B_1) = 0.6$ and $\mathbb{P}(B_2) = 0.4$, and let A be the event that the instructor comes to class, with $\mathbb{P}(A \mid B_1) = 0.9$ and $\mathbb{P}(A \mid B_2) = 0.1$. Then $\mathbb{P}(A) = 0.9(0.6) + 0.1(0.4) = 0.58$. ms. p.28

Applying this to the postulates,

$$\mathbb{P}(\boldsymbol{x}) = \sum_{k=1}^{K} \mathbb{P}(\boldsymbol{x} \mid y = k)\mathbb{P}(y = k) = \sum_{k=1}^{K} \phi_k \, \mathcal{N}(\boldsymbol{\mu}_k, \Sigma_k), \tag{9.1}$$

a *Gaussian mixture*. The marginal density of $\boldsymbol{x}$ is a weighted sum of the class-conditional Gaussians. ms. p.28

Let us see this geometrically. Nothing in the postulates mentioned a mixture. It appeared because Theorem 5.3.3 was applied to a model that specifies the data class-by-class — and forgetting the labels leaves the weighted sum behind.

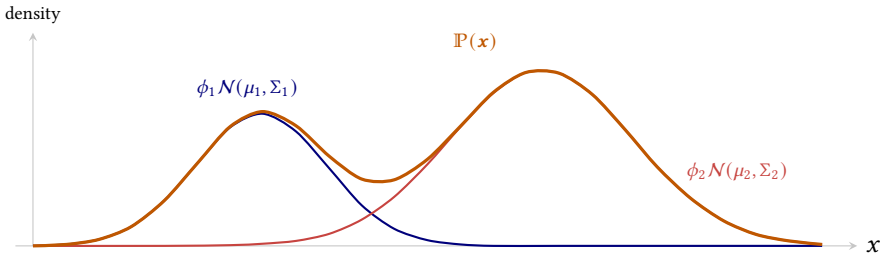


Figure 9.2: **The marginal is a mixture.** Two class-conditional Gaussians, each scaled by its class prior ϕ_k (here 0.35 and 0.65), and their sum (9.1) in orange. The mixture is not Gaussian — it has two modes and a visible valley between them — yet every ingredient was. This is what the model believes about $\boldsymbol{x}$ *before* any label is seen, and it is the object Chapter 14 will learn to sample from.

§ 9.3 Maximum likelihood

Given labeled data $\mathcal{D} = \{(\boldsymbol{x}^{(i)}, y^{(i)})\}_{i=1}^{n}$,

$$\boldsymbol{\theta}^{\star} = \arg\max_{\boldsymbol{\theta}} \ \log \prod_{i=1}^{n} \mathbb{P}(\boldsymbol{x}^{(i)}, y^{(i)} \mid \boldsymbol{\theta}),$$

and factoring the joint,

$$\begin{aligned}\mathcal{L} &= \sum_{i=1}^{n}\Big[\log \mathbb{P}(\boldsymbol{x}^{(i)} \mid y^{(i)}, \boldsymbol{\theta}) + \log \mathbb{P}(y^{(i)} \mid \boldsymbol{\theta})\Big] \\ &= \underbrace{\sum_{k=1}^{K} \sum_{i\,:\,y^{(i)}=k} \log \mathcal{N}(\boldsymbol{x}^{(i)}; \boldsymbol{\mu}_k, \Sigma_k)}_{\text{depends only on } \boldsymbol{\mu}_k,\ \Sigma_k} \\ &\quad + \underbrace{\sum_{i=1}^{n}\Big(\log \phi_{y^{(i)}} - \log \sum_{j=1}^{K} \phi_j\Big)}_{\text{depends only on } \phi_k}.\end{aligned} \tag{9.2}$$

ms. p.29,30 The inner sum in the first term runs over all samples whose label equals k.

Key idea

The log-likelihood *separates*: the mixture weights ϕ_k appear only in the second term and the Gaussian parameters only in the first. Each group can therefore be optimized independently, which is why closed forms exist here and not for the unlabeled case.

9.3.1 The mixture weights

Differentiating the second term of (9.2) with respect to ϕ_ℓ,

$$\frac{\partial \mathcal{L}}{\partial \phi_\ell} = \sum_{i\,:\,y^{(i)}=\ell} \frac{1}{\phi_{y^{(i)}}} - \sum_{i=1}^{n} \frac{1}{\sum_j \phi_j} = \frac{n_\ell}{\phi_\ell} - \frac{n}{\sum_j \phi_j},$$

where n_ℓ is the number of samples carrying label ℓ. Setting this to zero,

$$\boxed{\mathbb{P}(y = \ell \mid \boldsymbol{\theta}) = \frac{\phi_\ell}{\sum_j \phi_j} = \frac{n_\ell}{n}} \tag{9.3}$$

ms. p.29 The estimated class prior is simply the empirical class frequency.

9.3.2 The Gaussian parameters

The first term of (9.2) expands to

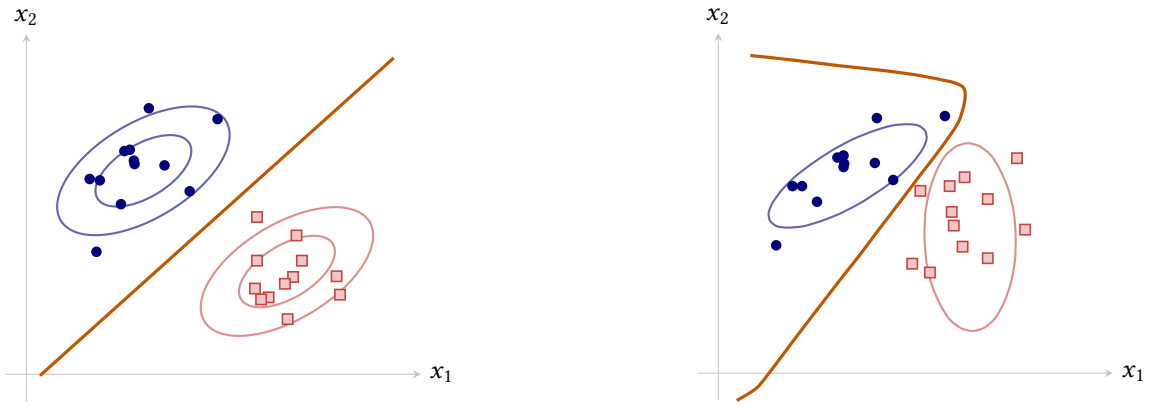
$$\sum_{k=1}^{K} \sum_{i\,:\,y^{(i)}=k} \log\left[\frac{1}{\sqrt{(2\pi)^d \det \Sigma_k}} \exp\Big(-\tfrac{1}{2}(\boldsymbol{x}^{(i)} - \boldsymbol{\mu}_k)^{\top} \Sigma_k^{-1} (\boldsymbol{x}^{(i)} - \boldsymbol{\mu}_k)\Big)\right],$$

ms. p.30 and differentiating with respect to $\boldsymbol{\mu}_\ell$ and Σ_ℓ requires the matrix identities of Appendix A. The results are the class-conditional sample mean and covariance; the derivation is Exercise 12.

§ 9.4 Shared or separate covariances

One modeling choice remains, and it changes the geometry entirely: are the Σ_k allowed to differ?

Let us see this geometrically. Constrain all classes to share one covariance and the boundary is a straight line. Let them differ and it bends. Nothing else about the model changes — the same postulates, the same maximum likelihood.



(a) $\Sigma_k = \Sigma$: linear boundary.

(b) Σ_k free: quadratic boundary.

Figure 9.3: **The covariance assumption decides the shape of the boundary. (a)** With a common covariance the quadratic terms of the two log-densities cancel, leaving a linear boundary — and the model turns out to be exactly the logistic form of Chapter 8, reached from entirely different assumptions. This is *linear discriminant analysis.* **(b)** Letting the covariances differ leaves the quadratic terms in place, and the boundary becomes a conic — here a curve that wraps around the tighter, more elongated class. This is *quadratic discriminant analysis.* The price is parameters: each extra covariance costs $d(d+1)/2$ numbers, so (b) overfits sooner than (a).

Note:-

The remainder of this chapter is not in the handwritten manuscript, which treats only the labeled case. It is included because (9.1) is already written down as a mixture, and because the argument that makes it work is the one Chapter 14 needs.

Example 9.4.1 (Why the choice is usually made for you)

The shared-covariance question looks like a modeling preference. For anything but small d it is a data budget. A single covariance in d dimensions carries $d(d+1)/2$ free entries, and separate ones multiply that by K:

d	per class	separate, $K=3$	shared
4	10	30	10
10	55	165	55
20	210	630	210
50	1275	3825	1275

Counting is the easy part. The sample covariance of class k is *singular* unless that class contributes more than d samples, and Σ_k^{-1} appears in the second postulate, so the model does not merely fit badly — it fails to evaluate. Nor is $n_k > d$ enough. Drawing from $\Sigma = \mathrm{I}$ with $d = 10$ and measuring the condition number of the estimate:

n_k	median $\kappa(\widehat{\Sigma}_k)$
11	1163
15	50.4
30	9.6
100	3.0
1000	1.4

The truth has $\kappa = 1$. At $n_k = 11$, one sample past the threshold, the estimate is off by three orders of magnitude; it takes several times d samples per class before the inverse means anything.

So with $d = 20$ and a few dozen samples per class, separate covariances are not available at any price, and the straight boundary of Figure 9.3 is not a simplifying assumption — it is the only thing the data can support. This is Corollary 6.3.2 again, in a setting where the failure is a singular matrix rather than a widening generalization gap.

§ 9.5 The unlabeled case: expectation–maximization

Everything so far assumed the labels $y^{(i)}$ were observed. Suppose they are not — we have only $\{\boldsymbol{x}^{(i)}\}_{i=1}^{n}$ and wish to fit the mixture (9.1) anyway. This is now *unsupervised* learning: the clustering problem of Figure 1.2a, posed probabilistically.

9.5.1 Why the closed forms disappear

With the labels unknown, the likelihood must marginalize over them:

$$\mathcal{L}(\boldsymbol{\theta}) = \sum_{i=1}^{n} \log\Big(\underbrace{\sum_{k=1}^{K} \phi_k \, \mathcal{N}(\boldsymbol{x}^{(i)}; \boldsymbol{\mu}_k, \Sigma_k)}_{\mathbb{P}(\boldsymbol{x}^{(i)} \mid \boldsymbol{\theta})}\Big). \tag{9.4}$$

Common pitfall

Compare (9.4) with (9.2). There the logarithm sat *inside* the sum over classes, because each sample's class was known and only one term applied. Here the logarithm sits *outside* a sum, and a log of a sum does not separate. The tidy split that produced the closed forms of Section 9.3 is gone, and setting $\nabla_{\boldsymbol{\theta}} \mathcal{L} = \mathbf{0}$ yields equations in which the estimators appear on both sides.

9.5.2 Responsibilities

The obstruction is that we do not know which component produced $\boldsymbol{x}^{(i)}$. We do, however, know what the current parameters *believe* about it.

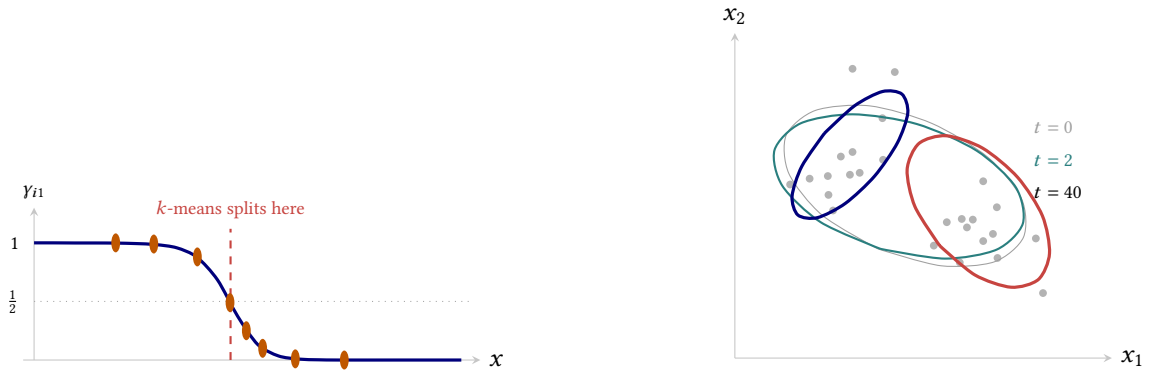
Definition 9.5.1: Responsibility

Given parameters $\boldsymbol{\theta}$, the responsibility *of component k for sample $\boldsymbol{x}^{(i)}$ is the posterior class probability*

$$\gamma_{ik} := \mathbb{P}(y^{(i)} = k \mid \boldsymbol{x}^{(i)}, \boldsymbol{\theta}) = \frac{\phi_k \, \mathcal{N}(\boldsymbol{x}^{(i)}; \boldsymbol{\mu}_k, \Sigma_k)}{\sum_{j=1}^{K} \phi_j \, \mathcal{N}(\boldsymbol{x}^{(i)}; \boldsymbol{\mu}_j, \Sigma_j)}, \qquad \sum_{k=1}^{K} \gamma_{ik} = 1.$$

The denominator is Bayes' formula in action: prior ϕ_k times likelihood, normalized. A responsibility is a *soft* label — a sample may be 70% the first component and 30% the second.

Let us see this geometrically. Compare this with k-means, which assigns each point to exactly one cluster. Responsibilities interpolate: far from the boundary they are almost 0 or 1, and only near it does a point genuinely belong to both.



(a) Soft assignment, not hard.

(b) Components converging from a poor start.

Figure 9.4: **Expectation–maximization. (a)** Responsibility γ_{i1} of the first component along the line. Away from the crossing it is essentially 0 or 1; only within about one standard deviation does a point belong appreciably to both. The dashed line is where k-means would cut, discarding exactly the information EM keeps. **(b)** Both components initialized as identical unit circles at the center of the data (gray) — a deliberately poor start. After two iterations (teal) they have begun to separate; by iteration 40 they have locked onto the two clusters (navy and red). No labels were used at any point.

9.5.3 The algorithm

Key idea

If we knew the labels, fitting would be the closed-form calculation of Section 9.3. If we knew the parameters, computing the labels would be Definition 9.5.1. We know neither — so we alternate, using each to improve the other. That is the whole of the algorithm.

Algorithm 5: Expectation–maximization for a Gaussian mixture

Input: data $\{\boldsymbol{x}^{(i)}\}_{i=1}^{n}$, number of components K, initial $\boldsymbol{\theta}$
Output: fitted $\phi_k, \boldsymbol{\mu}_k, \Sigma_k$

repeat

`// E step: what the current parameters believe`

$$\gamma_{ik} \leftarrow \frac{\phi_k \, \mathcal{N}(\boldsymbol{x}^{(i)}; \boldsymbol{\mu}_k, \Sigma_k)}{\sum_j \phi_j \, \mathcal{N}(\boldsymbol{x}^{(i)}; \boldsymbol{\mu}_j, \Sigma_j)} \quad \text{for all } i, k$$

`// M step: the closed forms of Section 9.3, with counts replaced by weights`

$$n_k \leftarrow \sum_{i=1}^{n} \gamma_{ik}$$

$$\phi_k \leftarrow n_k / n$$

$$\boldsymbol{\mu}_k \leftarrow \frac{1}{n_k} \sum_{i=1}^{n} \gamma_{ik} \, \boldsymbol{x}^{(i)}$$

$$\Sigma_k \leftarrow \frac{1}{n_k} \sum_{i=1}^{n} \gamma_{ik} \, (\boldsymbol{x}^{(i)} - \boldsymbol{\mu}_k)(\boldsymbol{x}^{(i)} - \boldsymbol{\mu}_k)^{\mathsf{T}}$$

until $\mathcal{L}(\boldsymbol{\theta})$ *has stopped increasing*

Note:-

Compare the M step with (9.3) and Exercise 12. They are the *same formulas*, with the hard count "number of samples with $y^{(i)} = k$" replaced by the soft weight $\sum_i \gamma_{ik}$. If every γ_{ik} were 0 or 1, the M step would reduce exactly to the labeled case — and the algorithm would reduce to k-means.

9.5.4 Why it works

The algorithm looks like a heuristic. It is not: each sweep provably does not decrease the likelihood, and the reason is an inequality we will meet again in Chapter 14.

Theorem 9.5.1 EM never decreases the likelihood

Let $\boldsymbol{\theta}^t$ and $\boldsymbol{\theta}^{t+1}$ be successive iterates of Algorithm 5. Then $\mathcal{L}(\boldsymbol{\theta}^{t+1}) \geq \mathcal{L}(\boldsymbol{\theta}^t)$.

Proof: [Proof sketch] For any distribution q over the unknown labels, Jensen's inequality applied to the concave log gives

$$\mathcal{L}(\boldsymbol{\theta}) = \sum_i \log \sum_k \mathbb{P}(\boldsymbol{x}^{(i)}, y^{(i)}{=}k \mid \boldsymbol{\theta}) \;\geq\; \underbrace{\sum_i \sum_k q_{ik} \log \frac{\mathbb{P}(\boldsymbol{x}^{(i)}, y^{(i)}{=}k \mid \boldsymbol{\theta})}{q_{ik}}}_{\text{a lower bound } \mathcal{F}(q,\boldsymbol{\theta})}, \tag{9.5}$$

with equality precisely when $q_{ik} = \mathbb{P}(y^{(i)}{=}k \mid \boldsymbol{x}^{(i)}, \boldsymbol{\theta})$, that is, when q is the responsibilities. So:

- the **E step** sets $q = \gamma$, making the bound *tight*: $\mathcal{F}(\gamma, \boldsymbol{\theta}^t) = \mathcal{L}(\boldsymbol{\theta}^t)$;
- the **M step** maximizes $\mathcal{F}(\gamma, \cdot)$ over $\boldsymbol{\theta}$, so $\mathcal{F}(\gamma, \boldsymbol{\theta}^{t+1}) \geq \mathcal{F}(\gamma, \boldsymbol{\theta}^t)$.

Chaining these with (9.5),

$$\mathcal{L}(\boldsymbol{\theta}^{t+1}) \;\geq\; \mathcal{F}(\gamma, \boldsymbol{\theta}^{t+1}) \;\geq\; \mathcal{F}(\gamma, \boldsymbol{\theta}^t) \;=\; \mathcal{L}(\boldsymbol{\theta}^t).$$

■

Key idea

The quantity $\mathcal{F}(q, \boldsymbol{\theta})$ in (9.5) is the *evidence lower bound*, and this is its first appearance in these notes. EM is the strategy of pushing a lower bound up rather than the intractable quantity itself: tighten the bound, maximize it, repeat. Chapter 14 does exactly this again for the variational autoencoder — the same inequality, the same two moves, with a neural network in place of the responsibilities.

Common pitfall

Theorem 9.5.1 guarantees the likelihood does not *decrease*. It says nothing about reaching a global maximum, and two failures are routine. The likelihood surface has many local maxima, so the result depends on initialization — as in Figure 4.3. Worse, by Challenge Exercise 20 the likelihood is *unbounded above*: a component can collapse onto a single point with $\Sigma_k \to 0$ and drive $\mathcal{L} \to \infty$. Practical implementations restart from several initializations and add $\varepsilon \mathrm{I}$ to each covariance — which is exactly the ridge remedy of Chapter 6.

§ 9.6 Exercises

The code problems use the **Palmer Penguins** data and `sklearn.datasets.make_blobs`; see Section C.5. `make_blobs` generates data that satisfies the postulates of Section 9.2 exactly, which makes it the right first test of any implementation.

Warm-up

1. [Theory] Verify the parameter count of Proposition 9.2.1, explaining the factor $d^2/2$. How many parameters does a 3-class model on $d = 4$ features have, and how does that compare with softmax regression on the same data?

2. [Theory] Example 9.1.1 gives one question only the generative model can answer. Using Figure 9.1, name a *second* — one involving a feature whose value is missing at prediction time — and say how $\mathbb{P}(\boldsymbol{x} \mid y)$ supplies it. Then name one assumption the generative model makes that the discriminative one avoids, and point to the example that prices it.

3. [Theory] In Figure 9.2 the mixture has two modes although both components are Gaussian. Show that a mixture of two Gaussians is never itself Gaussian unless the components coincide.

4. [Theory] Write out Theorem 5.3.3 for $K = 2$ and confirm it gives (9.1). Which of ϕ_k, $\boldsymbol{\mu}_k$, Σ_k appear in the marginal, and which do not?

5. [Theory] Explain why (9.2) splits into two independent pieces, and what would go wrong with the argument if the class label $y^{(i)}$ were unknown.

6. [Theory] For a two-component mixture, compute γ_{i1} by hand at a point equidistant from both means when $\phi_1 = \phi_2$ and $\Sigma_1 = \Sigma_2$. Now set $\phi_1 = 0.9$. Which way does the responsibility move, and why is that the right behavior?

7. [Theory] Example 9.4.1 says a class covariance is singular unless $n_k > d$. Prove it: show that $\widehat{\Sigma}_k$ is a sum of n_k rank-one terms with one linear constraint among them, so its rank is at most $n_k - 1$. What does that say about the $n_k = 11$, $d = 10$ row of the table?

8. [Code] Generate three well-separated blobs with `make_blobs`, and estimate ϕ_k, $\boldsymbol{\mu}_k$ and Σ_k by the closed forms of Section 9.4. Compare with the parameters you asked `make_blobs` for.

Core

9. [Code] Reproduce Example 9.1.1. Fit both a softmax classifier and a two-class GDA to the same well-separated blobs, then evaluate each at a point far outside the data. Report the posterior from both and the marginal $\mathbb{P}(\boldsymbol{x})$ from the GDA, and confirm the posteriors agree while only one model registers anything unusual. Then set a density threshold that rejects 1% of the training data and report how far from the means a point must lie to be rejected.

10. [Code] Reproduce the conditioning table of Example 9.4.1. Draw n_k samples from $\mathcal{N}(\mathbf{0}, \mathrm{I})$ in $d = 10$, form the sample covariance, and record its condition number, repeating enough times for a stable median. Confirm the singular case at $n_k \le d$ and the decay thereafter. How large must n_k/d be before $\kappa < 2$?

11. [Theory] Derive (9.3) in full. Why does the normalization $\sum_j \phi_j$ appear, and what happens if one instead imposes $\sum_j \phi_j = 1$ as a constraint and uses a Lagrange multiplier (Chapter 13)? Confirm the two routes agree.

12. [Theory] Derive the maximum likelihood estimators $\hat{\boldsymbol{\mu}}_k$ and $\hat{\Sigma}_k$, showing they are the class-conditional sample mean and covariance. Use $\partial \log\det \Sigma / \partial \Sigma = \Sigma^{-1}$.

13. [Theory] Show that if all classes share a covariance, $\Sigma_k = \Sigma$, the quadratic terms cancel and the boundary $\mathbb{P}(y = j \mid \boldsymbol{x}) = \mathbb{P}(y = k \mid \boldsymbol{x})$ is a hyperplane, as in Figure 9.3a. Show further that the induced $\mathbb{P}(y \mid \boldsymbol{x})$ has exactly the logistic form of Chapter 8.

14. [Theory] Now let the covariances differ and show the boundary is the zero set of a quadratic form, as in Figure 9.3b. Give a choice of Σ_1, Σ_2 for which the boundary is an ellipse, and one for which it is a hyperbola.

15. [Theory] Show that a mixture (9.1) with $\Sigma_k = \varepsilon \mathrm{I}$ and $\varepsilon \to 0$ recovers k-means. What does the class prior ϕ_k become in that limit?

16. [Theory] $\hat{\Sigma}_k$ is a sum of n_k rank-one matrices. Deduce that it is singular whenever $n_k \le d$, and say what this implies for applying quadratic discriminant analysis to a class with few examples. Which remedy from Chapter 6 applies?

17. [Theory] Show that the M step of Algorithm 5 reduces exactly to the closed forms of Section 9.3 when every $\gamma_{ik} \in \{0, 1\}$, and hence that EM contains the labeled case as a special case.

18. [Theory] Take $\Sigma_k = \varepsilon \mathrm{I}$ with $\varepsilon \to 0$ in Definition 9.5.1 and show that γ_{ik} tends to a hard assignment to the nearest mean. Conclude that k-means is the zero-variance limit of Algorithm 5, and say what information is discarded in that limit.

19. [Code] Fit both variants on the Palmer Penguins data using two features: once with a shared covariance and once with separate ones. Plot the two boundaries over the data, reproducing Figure 9.3, and compare held-out accuracy. Which wins, and does the answer change as you reduce the training set size?

Challenge

20. [Theory] Contrast this chapter with Chapter 8 precisely. Both produce $\mathbb{P}(y \mid \boldsymbol{x})$, and Exercise 9 shows they can produce the *same* $\mathbb{P}(y \mid \boldsymbol{x})$. State what the extra information in $\mathbb{P}(\boldsymbol{x}, y)$ buys, what it costs in assumptions, and predict which needs less data to reach a given accuracy.

21. [Theory] Suppose the true class-conditionals are not Gaussian — say heavy-tailed. Argue that maximum likelihood still returns a well-defined $\hat{\boldsymbol{\mu}}_k, \hat{\Sigma}_k$, but that the resulting boundary may be arbitrarily bad. Relate this to the *misspecification* discussion of Figure 7.2: which KL order is being minimized here?

22. [Theory] Derive the *naive Bayes* classifier as the special case Σ_k diagonal, and count its parameters. Explain why this drastic assumption often works better than full GDA on high-dimensional data, using Exercise 12.

23. [Theory] Show that the GDA log-likelihood is unbounded above if one class is allowed $\Sigma_k \to 0$ around a single data point. What does this say about maximum likelihood for mixture models, and why does the *labeled* case of this chapter escape the problem?

24. [Theory] Prove Theorem 9.5.1 in full. Derive (9.5) from Jensen's inequality, and verify that equality holds exactly when q_{ik} equals the responsibility. *Hint:* the gap between $\mathcal{L}$ and $\mathcal{F}$ is a KL divergence; identify it, and compare with Proposition 7.4.1.

25. [Theory] Theorem 9.5.1 shows the likelihood never decreases, yet Challenge Exercise 23 shows it is unbounded above. Reconcile the two. What must therefore be true of the sequence $\{\boldsymbol{\theta}^t\}$ in the degenerate case, and what does a practical implementation do about it?

26. [Code] Use the fitted model *generatively*: sample new $(\boldsymbol{x}, y)$ pairs from (9.1) and plot them alongside the training data. Then fit logistic regression to the same data and confirm that no analogous sampling is possible. This is the practical content of Figure 9.1.

27. [Code] Compare GDA and logistic regression as the training set shrinks. For $n = 20, 40, \dots, 400$ fit both and plot held-out accuracy against n, averaging over 50 resamples. Which degrades more gracefully, and how does your answer relate to the assumption each model makes? Then repeat with a deliberately non-Gaussian class and comment.

Chapter 10

Stochastic Gradient Descent

Goal of this chapter

Chapter 4 built an algorithm that steps downhill along $-\nabla_\theta J(\theta)$. Computing that gradient requires a pass over all n training samples, and for the model classes of Chapter 11 that is the dominant cost of training. Chapter 7 has already said what that objective is: an expectation over the data distribution, estimated by sampling. The same idea, applied a second time, is what makes each step cheap.

This chapter replaces the gradient by an estimate of it. It answers:

1. In what precise sense is a gradient computed from a handful of samples a substitute for the true one?
2. What does the resulting sequence converge to, and under what condition on the step sizes?
3. How large should a mini-batch be, and what does enlarging it buy?
4. Why do the optimizers used in practice — RMSProp, Adam — look nothing like plain gradient descent?

Throughout, $J(\theta)$ is assumed differentiable. Penalties such as the L_1 term of Chapter 6 are not, and the methods that handle them are outside our scope; Section 10.8 says what they are.

§ 10.1 The objective, and what it approximates

Section 7.1 established what this chapter is allowed to approximate. The quantity we care about is the risk (7.1), an expectation over a distribution we do not have; what we can actually minimize is its Monte Carlo estimate (7.2),

$$J(\theta) = \frac{1}{n}\sum_{i=1}^{n} \ell_i(\theta), \qquad \ell_i(\theta) := L\big(y^{(i)}, f_\theta(x^{(i)})\big).$$

That substitution is already an approximation, and Chapter 6 is about what it costs in accuracy. This chapter takes it as given and asks a narrower question: given that we are minimizing J, how do we do it cheaply?

§ 10.2 The cost of an exact gradient

Differentiating (7.2) term by term gives the gradient the same finite-sum structure as the objective:

$$\nabla_\theta J(\theta) = \frac{1}{n}\sum_{i=1}^{n} \nabla_\theta \ell_i(\theta). \tag{10.1}$$

One step of Algorithm 2 therefore costs n gradient evaluations. With $n = 10^6$ and a per-sample gradient taking a millisecond, a single step takes a quarter of an hour, and Chapter 4 needed hundreds of steps to converge.

The observation that rescues us is that we have already seen how to handle a quantity of this shape. Equation (7.2) replaced an expectation over $\mathbb{P}$ by an average over n samples because $\mathbb{P}$ was unavailable. Equation (10.1) is again an average — this time over samples we *do* hold, but too many of them to touch on every step. The same instrument applies a second time.

§ 10.3 The mini-batch gradient

Definition 10.3.1: Mini-batch gradient

Let $\mathcal{B} \subseteq \{1, \dots, n\}$ be a set of $|\mathcal{B}|$ indices drawn uniformly at random with replacement. The mini-batch gradient *is*

$$g_{\mathcal{B}}(\theta) := \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \nabla_\theta \ell_i(\theta).$$

The case $|\mathcal{B}| = 1$ is stochastic gradient descent *proper.*

Common pitfall

Taking $|\mathcal{B}| = n$ does *not* recover (10.1). Drawing n indices with replacement repeats some samples and omits others: the probability that a given sample is missed entirely is $(1 - 1/n)^n \to 1/e$, so about 37% of the data is absent from a typical full-size batch. The estimate is still unbiased, but it is not exact — Proposition 10.3.2 gives it variance σ^2/n, which is small but not zero. The full gradient is recovered by *enumerating* all n indices, which is a different operation from sampling n of them, or equivalently by sampling without replacement with $|\mathcal{B}| = n$, where the finite-population correction of Exercise 7 vanishes.

Key idea

Monte Carlo is now being applied at two different levels, and it is worth separating them. The first, (7.2), was *forced*: the expectation over $\mathbb{P}$ is not available at any price, so we estimate it from the n samples we were given. The second, Definition 10.3.1, is *chosen*: the average over those n samples is perfectly computable, and we sub-sample it anyway because computing it exactly on every step is too slow. The first approximation costs accuracy we can never recover; the second costs only accuracy we can buy back by taking more steps.

Two facts make the second approximation work, and both are read straight off Chapter 5.

Proposition 10.3.1 The mini-batch gradient is unbiased

For every θ and every batch size, $\mathbb{E}\big[g_{\mathcal{B}}(\theta)\big] = \nabla_\theta J(\theta)$.

Proof: Let I be a single index drawn uniformly from $\{1, \dots, n\}$, so $\mathbb{P}[I = i] = 1/n$ for each i. By the definition of expectation for a discrete random variable,

$$\mathbb{E}\big[\nabla_\theta \ell_I(\theta)\big] = \sum_{i=1}^{n} \frac{1}{n} \nabla_\theta \ell_i(\theta) = \nabla_\theta J(\theta),$$

which is exactly (10.1). The batch gradient is an average of $|\mathcal{B}|$ such draws, and expectation is linear by Lemma 5.5.1, so its expectation is the same. ∎

Key idea

The estimate is not merely close to the gradient; it is *correct on average.* Any individual mini-batch gradient may point in a poor direction, and with $|\mathcal{B}| = 1$ most of them do. What Proposition 10.3.1 guarantees is that those errors have mean zero, so they accumulate like a random walk rather than like a bias — and a random walk superimposed on a downhill drift still goes downhill.

Let us see this geometrically. Run both methods from the same start on the same problem. The full gradient traces a smooth curve to the minimizer; the stochastic one does not converge to a point at all, and yet it reaches the same neighborhood in the same number of steps at a fraction of the cost.

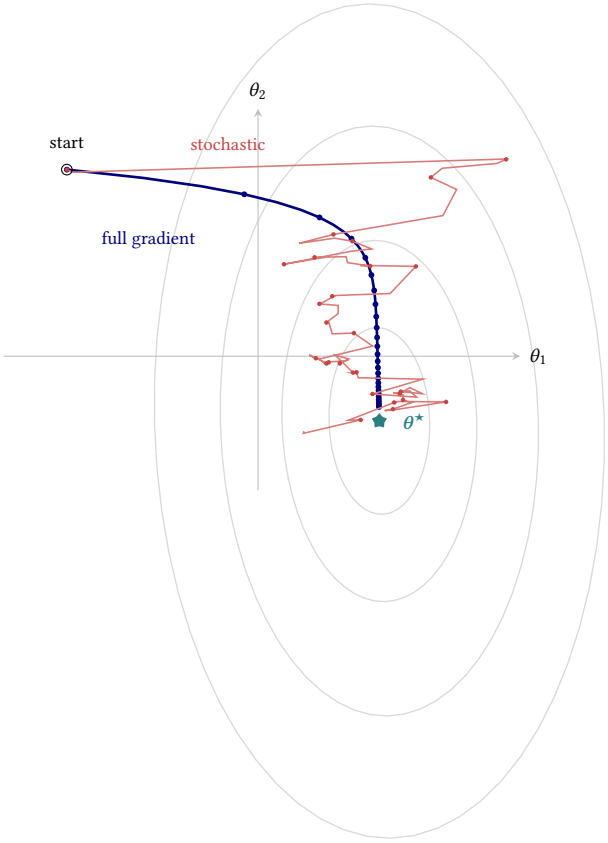


Figure 10.1: **One sample is enough to point downhill.** Eighty steps of full-batch gradient descent (navy) and of stochastic gradient descent with $|\mathcal{B}| = 1$ (red) on the same least-squares problem, $n = 400$ samples in two dimensions, from the same start and with the same step size $\alpha = 1/(4L)$. Gray curves are level sets of $J(\boldsymbol{\theta})$, elongated because $\kappa(\boldsymbol{H}) = 6.83$. The full gradient costs n sample evaluations per step and traces the smooth descending path of Chapter 4. The stochastic gradient costs *one*, and its path is visibly a random walk with a downhill drift. After eighty steps the full-batch iterate is 0.15 from $\boldsymbol{\theta}^\star$ and still closing; the stochastic iterate is 0.64 away and will never settle. But it has spent 80 sample evaluations against 32,000, and it reached the vicinity of the minimizer in about the same number of *steps*. That trade — accuracy in the limit for progress per unit of work — is the whole subject of this chapter.

Proposition 10.3.2 Variance falls inversely with batch size

Write $\sigma^2(\boldsymbol{\theta}) := \mathbb{E}\big\|\nabla_{\boldsymbol{\theta}}\ell_I(\boldsymbol{\theta}) - \nabla_{\boldsymbol{\theta}}J(\boldsymbol{\theta})\big\|^2$ *for the per-sample gradient variance. Then*

$$\mathbb{E}\big\|\boldsymbol{g}_{\mathcal{B}}(\boldsymbol{\theta}) - \nabla_{\boldsymbol{\theta}}J(\boldsymbol{\theta})\big\|^2 = \frac{\sigma^2(\boldsymbol{\theta})}{|\mathcal{B}|}.$$

Proof: The $|\mathcal{B}|$ draws are independent and identically distributed, and each has mean $\nabla_{\boldsymbol{\theta}}J(\boldsymbol{\theta})$ by Proposition 10.3.1. Apply Theorem 5.6.2 to each of the $d + 1$ components separately and sum. ■

Let us see this geometrically. The variance of the estimate is what makes the path wander, so it is worth measuring exactly how much a larger batch buys.

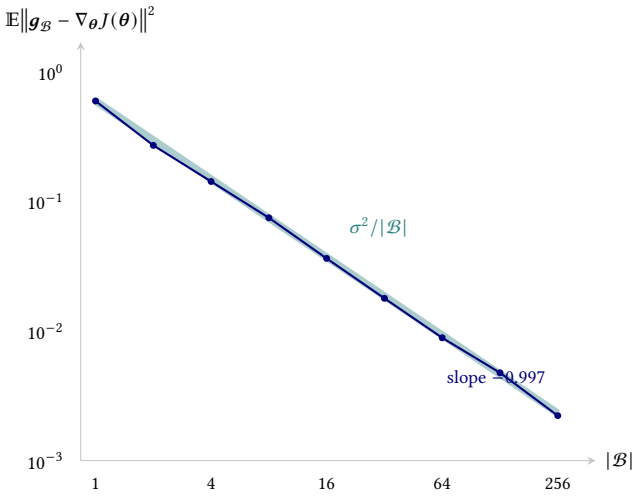


Figure 10.2: **Variance falls like $1/|\mathcal{B}|$, and no faster.** Mean squared deviation of the mini-batch gradient from the full gradient at a fixed point, measured over 4000 resamplings at each batch size on the problem of Figure 10.1. The pale line is $\sigma^2/|\mathcal{B}|$ from Proposition 10.3.2, anchored at $|\mathcal{B}| = 1$; the measured points sit on it, with a fitted slope of -0.997 against the predicted -1. Read the consequence carefully: going from $|\mathcal{B}| = 1$ to 256 costs 256 times the work per step and reduces the *standard deviation* of the estimate by a factor of 16, since it is the variance and not its square root that falls linearly. This is the same $O(1/\sqrt{N})$ wall as Theorem 5.6.2, met here in the cost of an optimizer rather than in the cost of an integral, and it is why very large batches are rarely worth their price.

§ 10.4 The algorithm

Algorithm 6: Stochastic gradient descent

Input: initial guess θ^0, step sizes $\alpha_j > 0$, batch size b
Output: approximate minimizer $\theta^\dagger$

$j \leftarrow 0$
while *stopping criteria not met* **do**
  draw $\mathcal{B}_j \subseteq \{1, \dots, n\}$ of size b at random
  $\theta^{j+1} \leftarrow \theta^j - \alpha_j \, g_{\mathcal{B}_j}(\theta^j)$
  $j \leftarrow j + 1$
$\theta^\dagger \leftarrow \theta^j$

Note:-

In practice the batches are not drawn independently. The indices are shuffled once and consumed in order, n/b batches making one *epoch*, after which they are reshuffled. This samples *without* replacement within an epoch, which slightly reduces the variance of Proposition 10.3.2 and guarantees every sample is used equally often. The analysis below assumes the simpler with-replacement model.

Common pitfall

The stopping criteria of Section 4.3 do not transfer. $\|g_{\mathcal{B}}\|$ is not small at the minimizer — it is a noisy quantity whose size is set by σ, not by proximity to $\theta^\star$ — and $J(\theta^{j+1}) < J(\theta^j)$ fails constantly on individual steps even when training is going well. Progress must be judged on a held-out set, evaluated once an epoch rather than once a step.

§ 10.5 What the iterates converge to

A constant step size does not give convergence to $\theta^\star$. Figure 10.1 already shows why: near the minimizer the true gradient vanishes but the estimate does not, so the iterate is kicked away from $\theta^\star$ at exactly the moment it should be settling. The

following makes that quantitative.

Proposition 10.5.1 The noise floor

Suppose J is twice differentiable with $\mu \mathrm{I} \preceq \boldsymbol{H} \preceq L\mathrm{I}$ for constants $0 < \mu \le L$, and that the per-sample gradient variance is bounded by σ^2. Run Algorithm 6 with a constant step $\alpha \le \mu/L^2$. Then

$$\limsup_{j\to\infty} \mathbb{E}\left\|\boldsymbol{\theta}^j - \boldsymbol{\theta}^\star\right\|^2 \;\le\; \frac{\alpha\,\sigma^2}{\mu\,|\mathcal{B}|}.$$

Proof: Write $d_j \coloneqq \mathbb{E}\|\boldsymbol{\theta}^j - \boldsymbol{\theta}^\star\|^2$ and abbreviate $\boldsymbol{g}_j = \boldsymbol{g}_{\mathcal{B}_j}(\boldsymbol{\theta}^j)$. Expanding the square,

$$d_{j+1} = \mathbb{E}\left\|\boldsymbol{\theta}^j - \boldsymbol{\theta}^\star - \alpha \boldsymbol{g}_j\right\|^2 = d_j - 2\alpha\,\mathbb{E}\left\langle \boldsymbol{g}_j,\; \boldsymbol{\theta}^j - \boldsymbol{\theta}^\star \right\rangle + \alpha^2\,\mathbb{E}\|\boldsymbol{g}_j\|^2.$$

Take the inner-product term first. Conditioning on $\boldsymbol{\theta}^j$ and applying Proposition 10.3.1 replaces $\boldsymbol{g}_j$ by $\nabla_\theta J(\boldsymbol{\theta}^j)$. Since $\nabla_\theta J(\boldsymbol{\theta}^\star) = \mathbf{0}$ and $\boldsymbol{H} \succeq \mu\mathrm{I}$,

$$\left\langle \nabla_\theta J(\boldsymbol{\theta}^j),\; \boldsymbol{\theta}^j - \boldsymbol{\theta}^\star \right\rangle = \left\langle \nabla_\theta J(\boldsymbol{\theta}^j) - \nabla_\theta J(\boldsymbol{\theta}^\star),\; \boldsymbol{\theta}^j - \boldsymbol{\theta}^\star \right\rangle \;\ge\; \mu\|\boldsymbol{\theta}^j - \boldsymbol{\theta}^\star\|^2,$$

so that term is at most $-2\alpha\mu\, d_j$. For the last term, split the estimate into its mean and its fluctuation: the cross term vanishes in expectation, leaving

$$\mathbb{E}\|\boldsymbol{g}_j\|^2 = \mathbb{E}\|\nabla_\theta J(\boldsymbol{\theta}^j)\|^2 + \mathbb{E}\left\|\boldsymbol{g}_j - \nabla_\theta J(\boldsymbol{\theta}^j)\right\|^2 \;\le\; L^2 d_j + \frac{\sigma^2}{|\mathcal{B}|},$$

where the first bound is $\boldsymbol{H} \preceq L\mathrm{I}$ applied as $\|\nabla_\theta J(\boldsymbol{\theta})\| = \|\nabla_\theta J(\boldsymbol{\theta}) - \nabla_\theta J(\boldsymbol{\theta}^\star)\| \le L\|\boldsymbol{\theta} - \boldsymbol{\theta}^\star\|$, and the second is Proposition 10.3.2. Collecting,

$$d_{j+1} \;\le\; \underbrace{\left(1 - 2\alpha\mu + \alpha^2 L^2\right)}_{\coloneqq\,\rho}\, d_j \;+\; \frac{\alpha^2\sigma^2}{|\mathcal{B}|}. \tag{10.2}$$

Because $\alpha \le \mu/L^2$ we have $\alpha^2 L^2 \le \alpha\mu$, hence $\rho \le 1 - \alpha\mu < 1$ and the recursion is a contraction plus a constant. Iterating it drives d_j to the fixed point $\alpha^2\sigma^2/\big(|\mathcal{B}|(1-\rho)\big)$, and $1 - \rho = 2\alpha\mu - \alpha^2 L^2 \ge \alpha\mu$ by the same inequality, giving the stated bound. ∎

The bound says two useful things at once. The iterates do not converge, but they do not escape either: they end up in a ball whose radius is proportional to α and to $\sigma/\sqrt{|\mathcal{B}|}$. Halving the step halves the radius. So does quadrupling the batch.

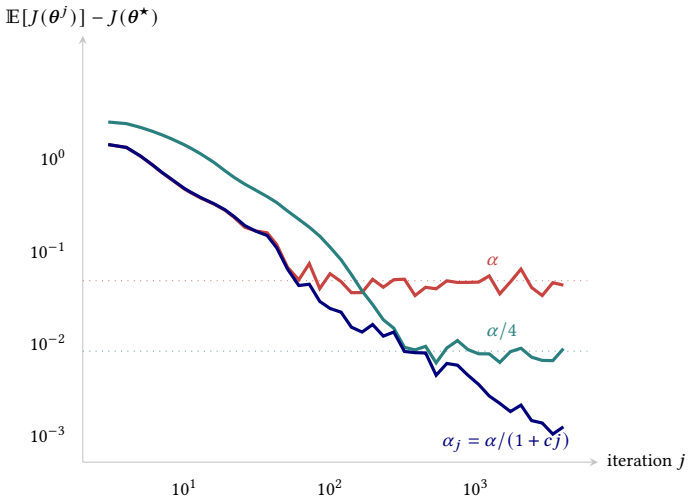


Figure 10.3: **A constant step size buys speed and then stops.** Expected suboptimality against iteration for stochastic gradient descent on the problem of Figure 10.1, averaged over 40 independent runs with $|\mathcal{B}| = 1$. Both constant step sizes descend rapidly and then flatten onto a *noise floor* (dotted): at step α the floor is 4.6×10^{-2}, at $\alpha/4$ it is 8.0×10^{-3}. Quartering the step lowers the floor by a factor of 5.8, which is Proposition 10.5.1 up to the constant, and buys it with a correspondingly slower approach. Neither run improves after roughly 500 iterations, however long it is left to run. The decaying schedule $\alpha_j = \alpha/(1 + cj)$ of Theorem 10.5.2 has no floor: it is slower than the constant steps at first, passes both of them, and is still descending at $j = 4000$, where it has reached 1.2×10^{-3} — more than six times below the better of the two floors.

Theorem 10.5.2 Convergence with a decaying step size

Under the assumptions of Proposition 10.5.1, suppose the step sizes satisfy

$$\sum_{j=0}^{\infty} \alpha_j = \infty \qquad \text{and} \qquad \sum_{j=0}^{\infty} \alpha_j^2 < \infty. \tag{10.3}$$

Then $\mathbb{E}\|\theta^j - \theta^\star\|^2 \to 0$.

Note:-

The recursion (10.2) holds verbatim with α replaced by α_j, and it already shows where each condition acts. The accumulated contraction across j steps is roughly $\exp(-\mu \sum_k \alpha_k)$, which vanishes precisely when the first sum diverges; the accumulated noise is controlled by $\sum_k \alpha_k^2$, which must therefore converge. Turning that observation into a proof requires showing the two effects can be separated in the limit, which is a supermartingale argument we do not give. Exercise 17 asks you to carry out the decomposition itself.

Key idea

The two conditions in (10.3) pull in opposite directions and both are necessary. The first says the steps must not shrink so fast that their total length is finite — otherwise the iterate can stall before reaching $\theta^\star$, having simply run out of travel. The second says they must shrink fast enough that the accumulated noise, which enters through α_j^2 in (10.2), is summable. Between them sits $\alpha_j \sim 1/j$: harmonic, so divergent, but square-summable. Any $\alpha_j \sim j^{-p}$ with $\frac{1}{2} < p \le 1$ will do.

Common pitfall

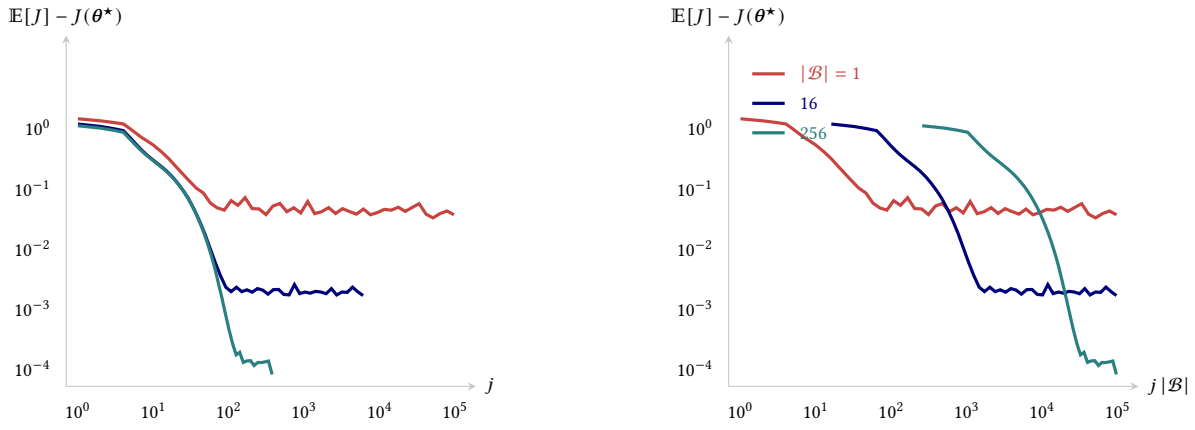
Satisfying (10.3) is necessary, not sufficient, for good behavior in practice. A schedule that decays correctly but starts too small spends its whole budget crawling; one that decays correctly but too late spends it bouncing on the noise floor. The schedules actually used — step decay, cosine annealing, a warm-up followed by decay — are chosen by

validation, exactly as λ is in Section 6.8.

§ 10.6 Mini-batching as a cost–variance trade

Combining Proposition 10.3.2 with Proposition 10.5.1: enlarging the batch by a factor of b divides the noise floor by b and multiplies the work per step by b. For the floor that is a fair exchange. It is not a fair exchange for the *approach* to the floor, which is governed by α and the conditioning and which a larger batch does not accelerate at all.

Let us see this geometrically. Run the same problem at three batch sizes and plot it twice: once against the iteration counter, and once against the number of sample gradients actually computed. The two plots tell opposite stories, and the second is the one that corresponds to a bill.



(a) against iteration

(b) against sample evaluations

Figure 10.4: **The same three runs, billed two ways.** Stochastic gradient descent at a fixed step $\alpha = 1/(4L)$ on the problem of Figure 10.1, averaged over 30 runs, at $|\mathcal{B}| = 1$, 16 and 256. **(a)** Against the iteration counter, a larger batch is uniformly better: the floors measure 4.4×10^{-2}, 2.0×10^{-3} and 1.1×10^{-4}, falling slightly faster than the $1/|\mathcal{B}|$ of Proposition 10.5.1 for the reason Exercise 24 pursues. The three curves descend at nearly the same rate before they flatten — the larger batch is not faster per step, it simply stops later. **(b)** Against sample gradients computed, which is what the batch actually costs, the ordering is no longer uniform. After 4000 sample evaluations $|\mathcal{B}| = 16$ leads at 1.9×10^{-3}, against 4.5×10^{-2} for $|\mathcal{B}| = 1$, which is already sitting on its floor, and 2.0×10^{-1} for $|\mathcal{B}| = 256$, which has managed only 15 iterations. There is an interior optimum, and it moves right as the budget grows.

Take-home message

A larger batch buys a lower floor, not a faster descent.

Per iteration, more samples always help. Per unit of work they help only until the batch is large enough that the estimate is no longer the bottleneck, after which the extra samples are reducing a standard deviation at $O(1/\sqrt{|\mathcal{B}|})$ — the same unfavorable rate that made Monte Carlo expensive in Theorem 5.6.2.

Common pitfall

Figure 10.4 is run at a *constant* step size, which is what makes the floor the binding constraint and hence what makes a larger batch pay. Under a schedule satisfying (10.3) there is no floor to lower, and the argument for a large batch largely evaporates: the decaying step already removes the variance in the limit, so batching is buying a second time what the schedule provides for free. This is why the practical advice — take the smallest batch that saturates a vectorized gradient evaluation, and no larger — is usually right despite panel (b), and why it is advice about *throughput* rather than about statistics.

§ 10.7 Adaptive step sizes

Chapter 4 diagnosed the zig-zag of Figure 4.2 as a conditioning problem: a single step size must serve directions whose curvatures differ by a factor of κ. Momentum, Section 4.5, attacked it by averaging successive directions. The methods below attack it by giving each coordinate its own step size, inferred from the gradients already seen.

10.7.1 RMSProp

Maintain a running average of the squared gradient, componentwise:

$$\underbrace{v_k^j}_{\substack{\text{mean square}\\ \text{of coord. } k}} = \underbrace{\rho\, v_k^{j-1}}_{\substack{\text{what we}\\ \text{believed}}} + \underbrace{(1-\rho)\big(g_k^j\big)^2}_{\substack{\text{new}\\ \text{evidence}}}, \qquad \theta_k^{j+1} = \theta_k^j - \underbrace{\frac{\alpha}{\sqrt{v_k^j+\epsilon}}}_{\substack{\textit{per-coordinate}\\ \text{step size}}} \underbrace{g_k^j}_{\substack{\text{same direction}\\ \text{as before}}}, \tag{10.4}$$

where g_k^j is the k-th component of $\boldsymbol{g}_{\mathcal{B}_j}$, with $\rho \approx 0.9$ and $\epsilon \approx 10^{-8}$.

Take the terms in turn.

- $\rho\, v_k^{j-1}$ **carries the past forward**, with a weight that decays geometrically — see (10.5) below. It is a *running* average deliberately: the curvature along coordinate k changes as the iterate moves, and an average over all history would keep responding to a landscape the iterate has left.
- $(1-\rho)(g_k^j)^2$ **admits the newest gradient**, squared so that only magnitude counts. Sign is irrelevant here; a coordinate oscillating between $+1$ and -1 and one marching steadily at $+1$ produce the same v_k, and are stepped identically.
- $\alpha/(\sqrt{v_k^j}+\epsilon)$ **is the step size, and it is now indexed by** k. This is the entire departure from everything before. In Algorithm 6 the scalar α multiplied every component of $\boldsymbol{g}$; here each component gets its own divisor. The ϵ only prevents division by zero for a coordinate whose gradient has been identically zero.
- g_k^j **is unchanged**. RMSProp does not alter the search direction, only how far to go along each axis of it.

The first bullet is worth making explicit, since "running average" hides what is actually being averaged. Apply the recursion to itself: substituting $v_k^{j-1} = \rho v_k^{j-2} + (1-\rho)(g_k^{j-1})^2$ and continuing all the way down gives

$$v_k^j = (1-\rho)\sum_{\ell=0}^{j-1} \rho^\ell \big(g_k^{j-\ell}\big)^2 \qquad \text{(taking } v_k^0 = 0\text{)}. \tag{10.5}$$

So v_k^j *is* a weighted average of every squared gradient so far, with the gradient ℓ steps back carrying weight $(1-\rho)\rho^\ell$. Those weights are positive and sum to $1-\rho^j \to 1$, so nothing is discarded; the old terms are merely suppressed geometrically.

Note:-

"Roughly the last $1/(1-\rho)$ gradients" is a rule of thumb, and it is worth knowing what it approximates. The weights in (10.5) fall by a factor of e every $1/\log(1/\rho)$ steps, which is 9.49 at $\rho = 0.9$; the familiar $1/(1-\rho) = 10$ is the first-order approximation to that, since $\log(1/\rho) \approx 1-\rho$ for ρ near 1. Other reasonable definitions give other numbers: the mean lag $\sum_\ell \ell(1-\rho)\rho^\ell$ is $\rho/(1-\rho) = 9$, and the effective sample size $(\sum w)^2/\sum w^2$ is $(1+\rho)/(1-\rho) = 19$. Concretely, at $\rho = 0.9$ the ten most recent gradients carry 65% of the weight and the twenty most recent carry 88%. "Ten" is a scale, not a window.

Since $\sqrt{v_k^j}$ is a running root-mean-square of g_k itself, the ratio $g_k/\sqrt{v_k}$ is dimensionless: multiplying every ℓ_i by a constant leaves the update unchanged. A coordinate receiving large gradients is stepped *less*, one receiving small gradients *more*, and α sets only the overall scale.

Key idea

RMSProp is *preconditioned* stochastic gradient descent. Writing $G := \mathbb{E}[\boldsymbol{g}\boldsymbol{g}^\mathsf{T}]$ for the second moment of the gradient, the update is $\boldsymbol{\theta} \mapsto \boldsymbol{\theta} - \alpha\, \widehat{G}^{-1/2}\boldsymbol{g}$ with $\widehat{G}$ the diagonal running estimate $\mathrm{diag}(v_k)$. That is a description of the algorithm, not an approximation claim, and it is the only part of this section that is not in dispute.

It is tempting to go further and call the scaling a cheap stand-in for the curvature correction of Newton's method. Resist it. The quickest objection is dimensional: curvature $\boldsymbol{H}_{kk}$ carries units of $[J]/[\theta]^2$, whereas g_k and therefore $\sqrt{v_k}$ carry $[J]/[\theta]$. A gradient magnitude is not a curvature, and averaging will not make it one. Newton's step $\boldsymbol{H}^{-1}\nabla_\theta J$ is a displacement in $[\theta]$; RMSProp's is α times the dimensionless $g_k/\sqrt{v_k}$, so α carries the whole displacement. This mismatch is exactly what motivated AdaDelta [7].

A second objection is that $\sqrt{v_k}$ measures different things in different places. On the quadratic model $g_k = \boldsymbol{H}_{kk}(\theta_k - \theta_k^\star) + \xi_k$ with noise variance s_k^2, we have $\mathbb{E}[g_k^2] = \boldsymbol{H}_{kk}^2(\theta_k - \theta_k^\star)^2 + s_k^2$. Far from the minimizer the first term dominates, $\sqrt{v_k} \to \boldsymbol{H}_{kk}|\theta_k - \theta_k^\star|$, and the step degenerates to $\alpha\,\mathrm{sgn}(\theta_k - \theta_k^\star)$ — a fixed-size move with the curvature cancelling out entirely. Near the minimizer the first term vanishes and $\sqrt{v_k} \to s_k$, which is gradient *noise*.

Note:-

Curvature interpretations of RMSProp do exist in the literature, and it is worth knowing what they claim and how firmly. Dauphin et al. [2] show that the preconditioner best suited to indefinite problems is built from the *absolute values* of the Hessian's eigenvalues — which is explicitly not what Newton or the Jacobi preconditioner does — and that RMSProp approximates that *equilibration* preconditioner, albeit as a biased estimator of it. A separate line of argument routes through the empirical Fisher matrix to a Gauss–Newton reading, but Kunstner et al. [4] show the resulting preconditioner is often far from the true Hessian. Neither line supports "RMSProp approximates Newton", and this chapter does not rely on either.

Key idea

What can be said without qualification is narrower, and enough for our purposes. The scaling is *per coordinate*, so like any diagonal preconditioner it addresses coordinates of differing size and does nothing about a $\boldsymbol{H}$ whose eigenvectors are tilted relative to the axes — Exercise 10. It is built from gradients already computed, so it costs nothing beyond the gradient itself, against the $O(d^3)$ of forming and applying $\boldsymbol{H}^{-1}$. And it is scale-free: multiplying every ℓ_i by a constant leaves the update unchanged.

Let us see this geometrically. The whole point of the method is that the two coordinates stop sharing a step size. Plotting the effective step of each against iteration shows them separating within a few dozen steps and staying apart.

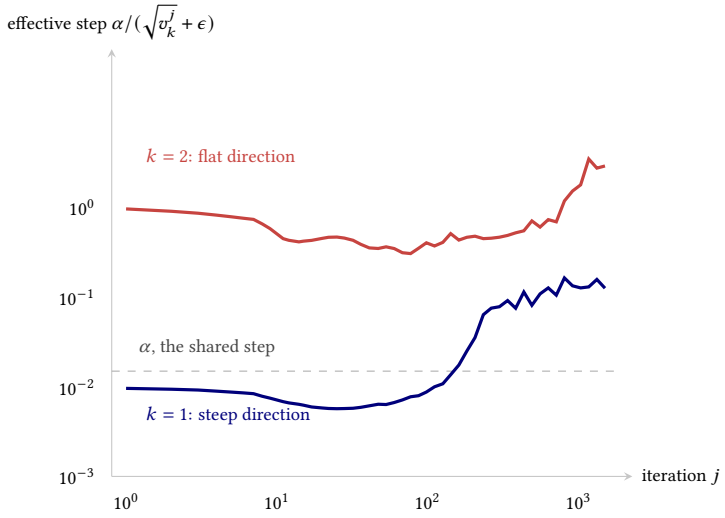


Figure 10.5: **RMSProp gives every coordinate its own step size.** The two effective steps $\alpha/(\sqrt{v_k^j}+\epsilon)$ from (10.4), on a deliberately badly scaled least-squares problem with $\kappa(\boldsymbol{H}) = 392$ and diagonal curvatures differing by a factor of 392. A single tuned α (dashed) would be applied to both. Instead the flat coordinate, whose gradients are small, accumulates a small v_2 and is stepped between 7 and 130 times further than the steep one — always in the right direction, though never by the full factor the curvature ratio would call for. Note also what the curves do *not* do: they do not settle. v_k^j is a running average over roughly the last $1/(1-\rho) = 10$ gradients, so the effective step keeps responding to the local landscape rather than converging to a schedule.

10.7.2 Adam

Adam combines the momentum of Section 4.5 with the RMSProp scaling, and corrects both for their initialization at zero:

$$\begin{aligned} m_k^j &= \beta_1 m_k^{j-1} + (1-\beta_1) g_k^j, & \hat{m}_k^j &= \frac{m_k^j}{1-\beta_1^j}, \\ v_k^j &= \beta_2 v_k^{j-1} + (1-\beta_2)\left(g_k^j\right)^2, & \hat{v}_k^j &= \frac{v_k^j}{1-\beta_2^j}, \end{aligned} \tag{10.6}$$

with the update

$$\theta_k^{j+1} = \theta_k^j - \alpha \underbrace{\hat{m}_k^j}_{\substack{\textit{which way,} \\ \text{averaged}}} \Big/ \underbrace{\left(\sqrt{\hat{v}_k^j} + \epsilon\right)}_{\substack{\textit{how far,} \\ \text{per coordinate}}} \tag{10.7}$$

and defaults $\beta_1 = 0.9$, $\beta_2 = 0.999$. The four quantities in (10.6) divide cleanly into two jobs and one repair:

- m_k^j, the **first moment**, is the momentum of Section 4.5 written as a running average instead of a running sum. It smooths the *direction*: successive stochastic gradients disagree, and averaging them cancels the disagreement while preserving the drift. Its weights decay as in (10.5), with time constant $1/(1-\beta_1) = 10$ iterations.
- v_k^j, the **second moment**, is exactly RMSProp's accumulator with ρ renamed β_2. It sets the *magnitude*, per coordinate. Time constant $1/(1-\beta_2) = 1000$ iterations, in the same sense — a hundred times longer than m's, which matters in a moment.
- $\hat{m}$ and $\hat{v}$ are the **bias corrections**. Both accumulators are initialized at $\mathbf{0}$, so both begin biased toward zero; dividing by $1-\beta_1^j$ and $1-\beta_2^j$ removes that bias exactly, for the reason derived below.
- The quotient $\hat{m}/\sqrt{\hat{v}}$ is the whole method in one symbol. Numerator decides *which way*; denominator decides *how far*. Momentum supplies only the first and RMSProp only the second; Adam is the observation that they are independent and can be done at once.

Common pitfall

The superscript on β is an exponent, not an index. In (10.6) the symbol j means two different things: m_k^j and v_k^j are the accumulators *at iteration* j, following the convention used for $\boldsymbol{\theta}^j$ throughout these notes, while β_1^j and β_2^j are genuine powers — β_1 multiplied by itself j times. At $\beta_1 = 0.9$ the divisor $1-\beta_1^j$ is 0.100 at $j=1$, 0.651 at $j=10$ and 0.999973 at $j=100$.

That it must be a power is forced by the algebra. Unrolling the first line of (10.6) exactly as (10.5) was unrolled gives $m_k^j = (1-\beta_1)\sum_{\ell=0}^{j-1} \beta_1^\ell g_k^{j-\ell}$, so if the gradient has a stationary mean then

$$\mathbb{E}\left[m_k^j\right] = (1-\beta_1)\mathbb{E}[g_k] \sum_{\ell=0}^{j-1} \beta_1^\ell = (1-\beta_1)\mathbb{E}[g_k]\,\frac{1-\beta_1^j}{1-\beta_1} = \left(1-\beta_1^j\right)\mathbb{E}[g_k], \tag{10.8}$$

using the finite geometric sum. The factor $1-\beta_1^j$ is exactly what $\hat{m}$ divides out, and it is a power because a geometric series of j terms sums that way. Read as an index it could not tend to 1, and the estimator would stay biased no matter how long training ran.

Common pitfall

It is tempting to read the bias correction as a small fix to the first few steps. It is not. Because m and v recover from their zero initialization at rates differing by a factor of a hundred, the ratio of an uncorrected step to a corrected one is $(1-\beta_1^j)/\sqrt{1-\beta_2^j}$, which *grows* before it decays: it is 3.2 at $j=1$, peaks near 6.6 at $j \approx 12$, is still 3.2 at $j=100$, and does not fall below 1.1 until several thousand iterations have passed (Figure 10.6). Dropping the correction does not destabilize the method — it silently multiplies your chosen α by a slowly decaying factor for the first few thousand steps, so that α means something different early in training than it does late.

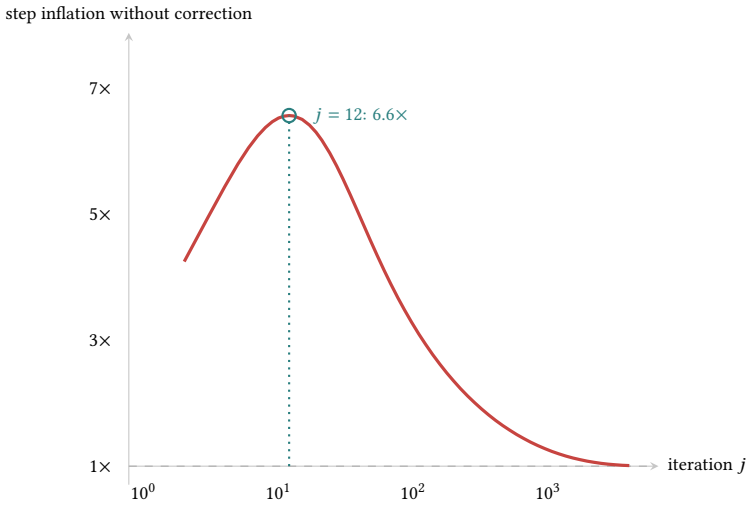


Figure 10.6: **What the bias correction is actually for.** The factor by which an uncorrected Adam step exceeds a corrected one, $(1-\beta_1^j)/\sqrt{1-\beta_2^j}$, at the standard $\beta_1 = 0.9$, $\beta_2 = 0.999$. It is *not* a transient of the first few steps. Both m and v start at zero and are biased low, but they recover at very different speeds: m has a time constant of about 10 iterations and v of about 1000, so the ratio first grows, peaks near $j = 12$ at 6.6×, and only returns to 1 after some thousands of steps. Omitting the correction therefore does not merely perturb the start; it imposes an unrequested decaying schedule on top of whatever schedule you chose, and it makes α mean different things at different times. That, rather than instability, is the reason to keep it.

Note:-

Adam is the default in most deep learning software and is usually the right first choice, but it is not uniformly better than well-tuned SGD with momentum. On several standard image classification benchmarks plain SGD with a tuned schedule generalizes measurably better, and the gap is a subject of active research. The honest summary is that Adam is far more forgiving of a badly chosen α, which is worth a great deal in practice.

10.7.3 How these differ from what came before

It is worth setting the four update rules side by side, because they differ in exactly one respect each. Writing $\boldsymbol{g}$ for the current stochastic gradient:

Method	Direction	Magnitude	New machinery
SGD	$\boldsymbol{g}$	α, one scalar	—
Momentum	averaged $\boldsymbol{g}$	α, one scalar	first moment
RMSProp	$\boldsymbol{g}$	$\alpha/\sqrt{v_k}$, per coordinate	second moment
Adam	averaged $\boldsymbol{g}$	$\alpha/\sqrt{v_k}$, per coordinate	both, plus debiasing

Two independent axes, then. Momentum and RMSProp each move along one of them; Adam moves along both. Neither axis changes what is being estimated — Proposition 10.3.1 and Proposition 10.3.2 hold verbatim for all four, since none of them touches how $\boldsymbol{g}$ is formed.

Key idea

None of these methods reduces the variance of the gradient estimate, so none of them removes the noise floor of Proposition 10.5.1. They change how efficiently a given estimate is *used*, not how good it is. A decaying schedule is still required for convergence, and Adam is normally run with one on top of its adaptivity.

Common pitfall

α is not comparable across these methods. In SGD it multiplies a gradient; in RMSProp and Adam it multiplies a dimensionless ratio, so it has different units and a different useful range. Transferring a tuned α from one method to another is meaningless — on the problem of Figure 10.7 the tuned values span a factor of fifty.

Let us see this geometrically. With each method given its own tuned step size — which is the only fair comparison, since α does not mean the same thing once the update is rescaled — the differences between them are visible in a single plot.

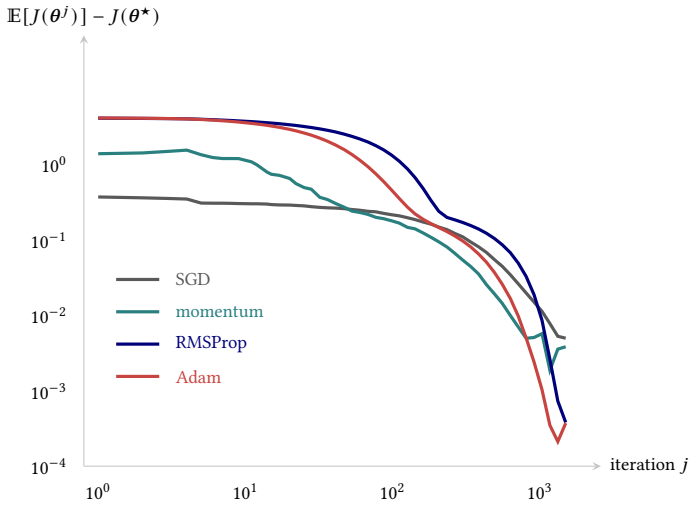


Figure 10.7: **Four optimizers on a badly scaled problem.** Expected suboptimality against iteration at $|\mathcal{B}| = 16$, averaged over 12 runs, on the $\kappa(\boldsymbol{H}) = 392$ problem of Figure 10.5. Each method was given its own α, chosen by sweeping thirty values over four decades and keeping the best: $\alpha^\star = 0.76$ for SGD, 0.127 for momentum, 0.0149 for RMSProp and 0.0304 for Adam. The tuned values differ by a factor of fifty, which is itself the point — α is not a comparable quantity across these methods. Plain SGD is quickest to get within an order of magnitude and then stalls, because one step size cannot serve two curvatures. The three others all beat it by the end, Adam by a factor of 20. The ordering is specific to this problem: on a well-conditioned one, tuned SGD is competitive with all of them, which is the caveat the note below records.

§ 10.8 What this chapter does not cover

Every method above assumes J is differentiable. The lasso penalty of Section 6.7 is not differentiable at zero — which is precisely why it produces exact zeros — so none of these algorithms applies to it unmodified. The standard remedy is a *proximal* step: take a gradient step on the differentiable part, then apply the soft-thresholding operator of Exercise 13 to the result. Proximal gradient methods, and the subgradient calculus they rest on, are the natural sequel to this chapter and belong to a course on convex optimization.

Take-home message

Trade an exact gradient for a cheap unbiased one, then manage the variance.

Everything here follows from Proposition 10.3.1 and Proposition 10.3.2: the estimate is right on average, and its error shrinks like $1/|\mathcal{B}|$. Decaying steps remove the variance in the limit, larger batches suppress it directly, and adaptive methods reshape it per coordinate. None of them removes the underlying $1/\sqrt{N}$ wall of Theorem 5.6.2.

§ 10.9 Exercises

The code problems use the datasets of Chapter 1; the `optax` library is introduced in Appendix B, Section B.1.

Warm-up

1. [Theory] Verify Proposition 10.3.1 by hand for $n = 3$ and $|\mathcal{B}| = 1$: write out the three equally likely values of $\boldsymbol{g}_{\mathcal{B}}$ and average them.

2. [Theory] A per-sample gradient variance $\sigma^2 = 4$ is measured at some $\boldsymbol{\theta}$. What batch size makes the standard deviation of $\boldsymbol{g}_{\mathcal{B}}$ equal to 0.1, and what does Proposition 10.3.2 give for the variance there?

3. [Theory] Which of $\alpha_j = 1/\sqrt{j}$, $1/j$, $1/j^2$, and $\alpha_j = 0.1$ satisfy (10.3)? For each failure, say which condition fails and what goes wrong as a result.

4. [Theory] Using Proposition 10.5.1, by what factor must the batch size grow to absorb a doubling of the step size, if the noise floor is to stay where it is?

5. [Theory] In (10.6), evaluate $1 - \beta_2^j$ at $j = 1$, $j = 100$ and $j = 5000$ with $\beta_2 = 0.999$. At which of these is the bias correction still doing significant work?

6. [Theory] Explain in two sentences why $\|\boldsymbol{g}_{\mathcal{B}}\| < \tau$ is a poor stopping criterion, using Proposition 10.3.2.

Core

7. [Theory] Prove Proposition 10.3.2 in full, stating where independence of the draws is used and what changes if the batch is drawn *without* replacement. Confirm that the correction vanishes at $|\mathcal{B}| = n$, and say what that means. *Hint:* the finite-population correction factor is $(n - |\mathcal{B}|)/(n - 1)$.

8. [Theory] Take $J(\theta) = \frac{1}{2}\theta^2$ in one dimension with gradient estimate $g = \theta + \xi$, $\xi \sim \mathcal{N}(0, s^2)$ drawn independently at each step. Show that $\mathbb{E}[(\theta^{j+1})^2] = (1-\alpha)^2\mathbb{E}[(\theta^j)^2] + \alpha^2 s^2$ and deduce the exact noise floor $\alpha s^2/(2-\alpha)$. Compare with Proposition 10.5.1, whose bound is not tight here — by how much?

9. [Theory] Show that the RMSProp update (10.4) is invariant under $J \mapsto cJ$ for $c > 0$, and that plain gradient descent is not. Why is this desirable?

10. [Theory] Show that the diagonal scaling of (10.4) cannot fix the zig-zag of Figure 4.2 when the Hessian's eigenvectors are not aligned with the coordinate axes. Construct a 2×2 example.

11. [Code] Investigate the curvature claim for yourself, in the one setting where it is cleanest. Build a least-squares problem whose Hessian is diagonal with entries spanning three decades, and evaluate the per-sample gradients at $\boldsymbol{\theta}^\star$. Show first that $\mathrm{Cov}(\nabla_{\boldsymbol{\theta}}\ell) = \sigma^2\boldsymbol{H}$ there, so that $\sqrt{\mathbb{E}[g_k^2]} = \sigma\sqrt{\boldsymbol{H}_{kk}}$, and confirm it numerically. Then correlate the implied step $1/\sqrt{v_k}$ against $1/\boldsymbol{H}_{kk}$ and against $\boldsymbol{H}_{kk}^{-1/2}$ and report which it matches. Finally — and this is the point of the exercise — say why this calculation does *not* establish the general claim the note above declines to make. Which of its ingredients survive away from a minimizer, or for a loss that is not a log-likelihood?

12. [Theory] Far from the minimizer the text claims RMSProp degenerates to $\alpha\,\mathrm{sgn}(g_k)$. Show this for a one-dimensional quadratic with no noise, and explain what it implies about the method's behaviour during the first few iterations from a poor initialization. Is that a defect?

13. [Theory] An epoch is n/b steps. Using Proposition 10.5.1, compare the noise floor reached after one epoch at batch size b against batch size $2b$, holding α fixed. Which is further along, and does the answer depend on n?

14. [Code] Implement Algorithm 6 from scratch in JAX for least squares on the Combined Cycle Power Plant data. Compare it against the closed-form solution of Chapter 3: how many epochs at $|\mathcal{B}| = 32$ before the iterate is within 1% of it, and how many full-batch gradient steps would the same number of sample evaluations have bought?

15. [Code] On the Airfoil Self-Noise data, measure the per-sample gradient variance $\sigma^2(\boldsymbol{\theta})$ at the least-squares solution and at a point far from it. Use Proposition 10.5.1 to predict the noise floor at $\alpha = 10^{-3}$ and $|\mathcal{B}| = 16$, then run Algorithm 6 and check the prediction. Is the bound tight?

Challenge

16. [Code] Compare your implementation against `optax.sgd`, `optax.sgd` with momentum, and `optax.adam` on the same ill-conditioned quadratic. Plot loss against iteration for all four, and comment on which of the three weaknesses named in the take-home message of Chapter 4 each optimizer attacks.

17. [Theory] Rederive the recursion (10.2) with α replaced by α_j, and separate the result into a contraction term and a noise term. Identify which condition of (10.3) controls which term, and explain why neither alone suffices.

18. [Code] Batch size and learning rate are widely held to scale together — double the batch, double the step. Test this on MNIST with a small network: for $|\mathcal{B}| \in \{32, 64, 128, 256\}$ find the α giving the best validation loss after a fixed budget of *sample evaluations*, not steps. Does the empirical relationship match the linear rule, and does Proposition 10.5.1 predict it?

19. [Theory] Adam's $\hat{v}$ averages squared gradients, so $\sqrt{\hat{v}}$ estimates a root-mean-square rather than a standard deviation — it never subtracts the mean. Explain what this means near a minimizer, where the true gradient vanishes and the fluctuation is everything, versus far from one, where the mean dominates.

20. [Code] Implement a proximal gradient step for the lasso of Section 6.7: a plain SGD step on the least-squares term followed by soft thresholding. Verify that coefficients reach *exactly* zero, which plain SGD on the L_1 objective will not achieve, and explain the discrepancy in terms of Section 10.8.

21. [Theory] Suppose the per-sample gradient variance is not bounded but grows like $\|\nabla_\theta J(\boldsymbol{\theta})\|^2$ — the *multiplicative noise* regime. Argue from (10.2) that a constant step size can then converge after all, and say why this regime is plausible for an over-parameterized model able to interpolate its training data.

Reproducing the figures

[TODO: This subsection is the prototype for a set that should appear in every chapter: one problem per numerical figure, sharing a stated setup, asking the reader to regenerate the figure and check the caption's measured values. Chapters 6 and 7 are done, all their figures and examples covered; Chapters 1–5 and 8–14 remain. Chapter 12 (PCA/POD) is the next natural one, since Definition 12.8.1 already specifies its model problem in full.]

Every numerical figure in these notes was computed, not drawn, and the values quoted in the captions are measurements. The problems below ask you to regenerate the three figures of this chapter and check those numbers yourself. They share a single setup, which is the one the figures were made from:

$$\begin{gathered} n = 400, \quad d = 2, \quad \boldsymbol{x}^{(i)} = \boldsymbol{A}\boldsymbol{z}^{(i)} \ \text{ with } \ \boldsymbol{z}^{(i)} \sim \mathcal{N}(\boldsymbol{0}, \mathrm{I}), \quad \boldsymbol{A} = \begin{pmatrix} 1 & 0 \\ 0 & 0.38 \end{pmatrix}, \\ y^{(i)} = \big(\boldsymbol{x}^{(i)}\big)^{\mathsf{T}} \boldsymbol{\theta}^\star + 0.7\, \varepsilon^{(i)}, \quad \boldsymbol{\theta}^\star = (1, -0.8)^{\mathsf{T}}, \quad \varepsilon^{(i)} \sim \mathcal{N}(0, 1), \end{gathered} \tag{10.9}$$

with $\boldsymbol{H} = \boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi}/n$, $L = \lambda_{\max}(\boldsymbol{H})$, and $\alpha = 1/(4L)$ throughout. Your random draw will not be ours, so expect agreement to two significant figures rather than exact equality; a quantity that differs in the first figure is a bug, not a seed.

22. [Code] **Figure 10.1.** Using (10.9), confirm $\kappa(\boldsymbol{H}) \approx 6.8$. From $\boldsymbol{\theta}^0 = (-1.6,\ 2.2)$ run 80 steps of full-batch gradient descent and 80 of Algorithm 6 at $|\mathcal{B}| = 1$, both at α, and plot the two paths over level sets of J. Report $\|\boldsymbol{\theta}^{80} - \boldsymbol{\theta}^\star\|$ for each — roughly 0.15 and 0.6 — and count the sample evaluations each consumed. Then answer: is the stochastic path's failure to settle a property of the step size, of the batch size, or of both?

23. [Code] **Figure 10.2.** Fix the probe point $\boldsymbol{\theta} = \boldsymbol{\theta}^\star + (0.35,\ -0.25)$. For $|\mathcal{B}| = 1, 2, 4, \ldots, 256$ estimate $\mathbb{E}\|\boldsymbol{g}_{\mathcal{B}} - \nabla_\theta J\|^2$ by resampling the batch at least 4000 times, and plot it against $|\mathcal{B}|$ on log–log axes. Fit the slope; you should get -1.00 to two figures. Overlay $\sigma^2/|\mathcal{B}|$ anchored at $|\mathcal{B}| = 1$ and confirm the measured points lie on it. How few resamplings can you use before the fitted slope becomes unreliable?

24. [Code] **Figure 10.3.** Run Algorithm 6 at $|\mathcal{B}| = 1$ for 4000 iterations under three schedules — constant α, constant $\alpha/4$, and $\alpha_j = \alpha/(1 + 0.008j)$ — averaging $J(\boldsymbol{\theta}^j) - J(\boldsymbol{\theta}^\star)$ over at least 40 independent runs. Plot all three on log–log axes. Confirm the two constant schedules flatten onto floors near 4.6×10^{-2} and 8.0×10^{-3} while the decaying one keeps descending, reaching about 1.2×10^{-3} at $j = 4000$. Proposition 10.5.1 suggests the floors should fall in proportion to α, so the measured ratio ought to be 4; it is closer to 5.8. Account for the discrepancy. *Hint:* two separate effects contribute,

and you can separate them numerically. First, even for a quadratic with noise of *fixed* covariance the exact stationary floor is $\frac{\alpha}{2}\sum_k s_k^2/(2-\alpha\lambda_k)$ over the eigendirections of $\boldsymbol{H}$, which is not quite linear in α; simulating that case gives a ratio near 4.4. Second, the gradient noise here is not of fixed covariance at all — measure $\sigma^2(\boldsymbol{\theta})$ at $\boldsymbol{\theta}^\star$ and at a point a distance 2 away and you will find it grows by more than an order of magnitude. The larger step size settles further out, where the noise is stronger, which feeds back. Proposition 10.5.1 assumes a single bound σ^2 valid everywhere, and that assumption is what is being strained. Exercise 21 pursues the consequence.

25. [Code] The figures of this chapter all use $\alpha = 1/(4L)$ rather than the $\alpha = 1/L$ that Chapter 4 uses for full-batch descent. Find out why: rerun Exercise 22 at $\alpha = 1/L$ and report what happens to each of the two paths. Which hypothesis of Proposition 10.5.1 fails, and what does that say about transferring a step size from the deterministic setting to the stochastic one?

Figures 10.5–10.7 use a second, deliberately worse-conditioned problem, since the adaptive methods have little to show on a well-scaled one. Its setup is

$$n = 500, \quad \boldsymbol{A} = \begin{pmatrix} 1 & 0 \\ 0 & 0.05 \end{pmatrix}, \quad \boldsymbol{\theta}^\star = (1, -14)^\mathsf{T}, \quad \text{noise } 0.4, \\ \boldsymbol{\theta}^0 = (-2, 2)^\mathsf{T}, \quad |\mathcal{B}| = 16, \tag{10.10}$$

otherwise as in (10.9). It has $\kappa(\boldsymbol{H}) = 393$, with the two diagonal curvatures differing by a factor of 392.

26. [Code] **Figure 10.4.** Using (10.9), run Algorithm 6 at $|\mathcal{B}| = 1$, 16 and 256 with constant α, averaging over at least 30 runs, and give every batch size the *same budget of sample evaluations* rather than the same number of iterations. Plot the loss gap twice, against j and against $j|\mathcal{B}|$. Confirm the floors near 4.4×10^{-2}, 2.0×10^{-3} and 1.1×10^{-4}. Then locate the interior optimum in the second plot: at a budget of 4000 evaluations which batch size is ahead, and at 96,000? Explain the crossing.

27. [Code] **Figure 10.5.** Using (10.10), implement (10.4) and record the two effective steps $\alpha/(\sqrt{v_k^j}+\epsilon)$ at every iteration. Plot both against j on log–log axes. Confirm that the flat coordinate is stepped between roughly 7 and 130 times further than the steep one, and that neither curve settles. Then answer: the curvature ratio is 392, so why does the step ratio never approach it? *Hint:* v_k tracks the gradient, and the gradient in coordinate k is λ_k times the error in that coordinate, not λ_k.

28. [Code] **Figure 10.6.** This one needs no simulation. Plot $(1-\beta_1^j)/\sqrt{1-\beta_2^j}$ against j for $\beta_1 = 0.9$, $\beta_2 = 0.999$, and confirm the peak of 6.6 near $j = 12$. Show analytically why the curve rises before it falls, and find where the peak moves if β_2 is lowered to 0.99. Then run Adam on (10.10) with and without the correction and report the loss gap at $j = 50$ for each — you may be surprised which is ahead, and §10.7.2 explains why that is not an argument for dropping it.

29. [Code] **Figure 10.7.** Using (10.10), run SGD, momentum, RMSProp and Adam for 1500 iterations, averaged over 12 runs. Tune α separately for each by sweeping at least 30 values over four decades and keeping the best final loss — do *not* share one α, and explain in one sentence why sharing would invalidate the comparison. You should find tuned values near 0.76, 0.13, 0.015 and 0.030. Reproduce the four curves and confirm the ordering at $j = 1500$. Finally, rerun the whole comparison on the well-conditioned (10.9) and report how the ranking changes.

Part III

Deep Learning

Chapter 11

Neural Networks: What They Can Represent

Goal of this chapter

Every model class so far has been chosen for tractability. Affine models gave closed forms; the logistic model gave a convex loss. Neither can bend far. Neural networks can — but before asking how to train one, we should ask what one can *represent*, and how big it must be.
The chapter answers:

1. What is a network, formally?
2. Which functions can a ReLU network reproduce *exactly*?
3. How well can it approximate the rest, and how many neurons does a given accuracy cost?

The argument is *constructive* throughout: we build the network by hand and read its size off the construction. That is unusual — most universal approximation theorems assert existence without producing anything — and it is possible because the tools are the ones a numerical analyst already owns. Sections 11.3 and 11.4 are finite element theory wearing a different hat.

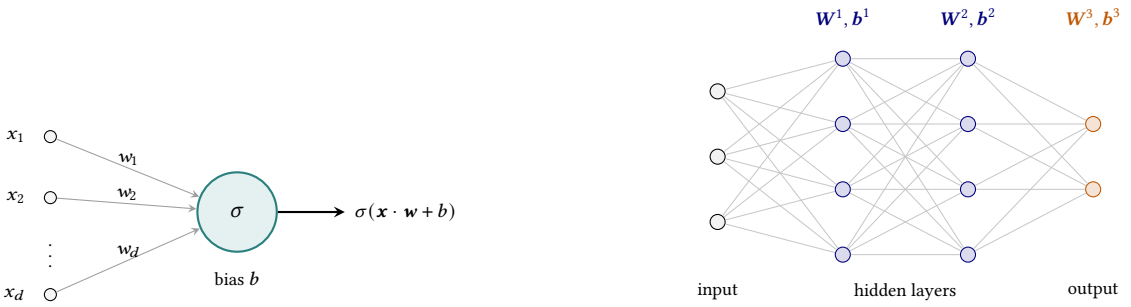
§ 11.1 The neuron and the network

A single neuron applies an affine transformation followed by a scalar nonlinearity:

$$x \longmapsto \sigma(x \cdot w + b),$$

with w the *weights* and b the *bias*. ms. p.31

Let us see this geometrically. A neuron is an affine map followed by a scalar nonlinearity, and a network is neurons composed in layers. Both fit in one picture.



(a) One neuron. (b) A fully connected network.

Figure 11.1: **Neurons and networks. (a)** A neuron computes $\sigma(\boldsymbol{x}\cdot\boldsymbol{w}+b)$: weights, a bias, and one scalar nonlinearity. Without σ it would be an affine map, and composing affine maps gives an affine map — so the nonlinearity is the only reason depth buys anything. **(b)** Stacking gives (11.1). The parameters are the weight matrices and bias vectors; every arrow is one number, and training adjusts all of them at once.

Definition 11.1.1: Activation functions

ms. p.31

Common choices for $\sigma : \mathbb{R} \to \mathbb{R}$*:*

- *the step (Heaviside) function;*
- *the sigmoid* $\sigma(z) := 1/(1+e^{-z})$*;*
- *the rectified linear unit* $\mathrm{ReLU}(z) := \max\{0, z\}$*;*
- tanh.

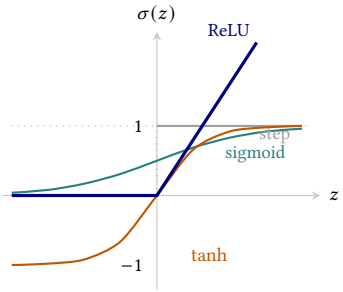


Figure 11.2: **Four activation functions.** The step function is the historical original but has zero derivative everywhere it is defined, so no gradient-based method can train through it. The sigmoid and tanh repair that but saturate at both ends, with the consequences seen in Figure 8.2b. ReLU is not differentiable at the origin and yet is the default in practice: on its active branch the derivative is exactly 1, so gradients pass through undiminished. It is also the only one of the four that is piecewise linear, which is what makes the construction of Section 11.2 possible.

Conventionally the same activation is used for every neuron, and σ applied to a vector acts componentwise: $\sigma(\boldsymbol{y}) = (\sigma(y_1), \dots, \sigma(y_p))^{\mathsf{T}}$.

ms. p.31

11.1.1 Layers

Write $\boldsymbol{W}^i$ for the weight matrix of layer i, whose columns are the weight vectors of the neurons in that layer, and $\boldsymbol{b}^i$ for the bias vector. Composing layers,

$$\boldsymbol{x} \longmapsto \sigma(\boldsymbol{W}^1\boldsymbol{x}+\boldsymbol{b}^1) \longmapsto \sigma\big(\boldsymbol{W}^2\sigma(\boldsymbol{W}^1\boldsymbol{x}+\boldsymbol{b}^1)+\boldsymbol{b}^2\big) \longmapsto \cdots$$

Writing $A^i(\boldsymbol{y}) := \boldsymbol{W}^i\boldsymbol{y}+\boldsymbol{b}^i$ for the affine map of layer i, an L-layer network is the composition

$$f_{\boldsymbol{\theta}} = \sigma \circ A^L \circ \sigma \circ A^{L-1} \circ \cdots \circ \sigma \circ A^1, \tag{11.1}$$

and we write $\boldsymbol{y}^i = \sigma(A^i(\boldsymbol{y}^{i-1}))$ for the output of layer i, with $\boldsymbol{y}^0 = \boldsymbol{x}$. ms. p.31,33

Note:-

A network is *fully connected* when every neuron feeds every neuron of the next layer. The parameters $\{\boldsymbol{W}^i, \boldsymbol{b}^i\}$ are what training adjusts — in the lecture's phrase, the weights are the network's memory. ms. p.32

§ 11.2 ReLU networks represent piecewise-linear functions exactly

The question is whether a network can approximate any function well. We answer a sharper question first: which functions can a ReLU network reproduce *exactly*? ms. p.32,33

11.2.1 Building blocks

Example 11.2.1 (A single kink)

For $r \in \mathbb{R}$,

$$f(x) = \begin{cases} r(x-a) & x > a \\ 0 & x \le a \end{cases} \qquad \Longleftrightarrow \qquad f(x) = r\max\{0, x-a\} = r\,\sigma(x-a),$$

a one-neuron network with weight 1, bias $-a$, and output weight r. ms. p.33

Example 11.2.2 (An affine function)

For $f(x) = ax + b$,

$$f(x) = \max\{ax + b,\ 0\} - \max\{-(ax+b),\ 0\},$$

a two-neuron network with output weights $+1$ and -1. ms. p.33

11.2.2 The general construction

Given nodes $a_1 < a_2 < \cdots < a_m$, values $g(a_j)$, and prescribed slopes S_L and S_R outside the interval, set

$$f(x) = h + \sum_{i=1}^{m} c_i\,\sigma(x - a_i) + c_0\,\sigma(a_1 - x). \tag{11.2}$$

Imposing $f(a_j) = g(a_j)$ for $j = 1, \ldots, m$ together with the two slope conditions

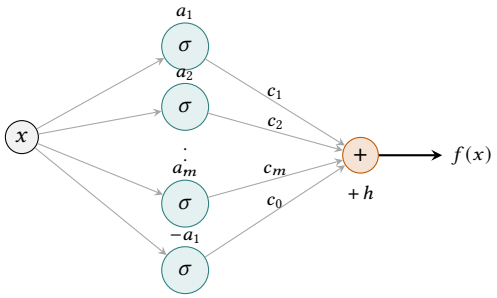
$$x < a_1 :\ f'(x) = -c_0 = S_L, \qquad x > a_m :\ f'(x) = \sum_{i=1}^{m} c_i = S_R,$$

gives a lower-triangular linear system for $(h, c_1, \ldots, c_m, c_0)$, which is therefore uniquely solvable. ms. p.33,34,35

Theorem 11.2.1 Exact representation

Every continuous piecewise-linear function g on $[a, b]$ with finitely many breakpoints is represented exactly by a single-hidden-layer ReLU network of the form (11.2).

Let us see this geometrically. The construction is worth seeing as a network. Every $\sigma(x - a_i)$ is one hidden neuron; the coefficients c_i are the output weights; and the resulting system is triangular because $\sigma(a_j - a_i) = 0$ whenever $i \ge j$ — the i-th kink has not yet switched on at the j-th node.



(a) One hidden neuron per node.

$$\begin{bmatrix} 1 & 0 & 0 & \cdots & 0 & * \\ 1 & \sigma(a_2-a_1) & 0 & \cdots & 0 & * \\ 1 & \sigma(a_3-a_1) & \sigma(a_3-a_2) & \cdots & 0 & * \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 1 & \sigma(a_m-a_1) & \sigma(a_m-a_2) & \cdots & 0 & * \\ 0 & 1 & 1 & \cdots & 1 & 0 \\ 0 & 0 & 0 & \cdots & 0 & -1 \end{bmatrix} \begin{bmatrix} h \\ c_1 \\ c_2 \\ \vdots \\ c_m \\ \\ c_0 \end{bmatrix} = \begin{bmatrix} g(a_1) \\ g(a_2) \\ g(a_3) \\ \vdots \\ g(a_m) \\ S_R \\ S_L \end{bmatrix}$$

(b) The resulting system is triangular.

Figure 11.3: **The construction, as a network and as a linear system. (a)** Each term $c_i\,\sigma(x-a_i)$ of (11.2) is a single hidden neuron with input weight 1 and bias $-a_i$; the output layer sums them and adds h. So the network has one hidden layer with $m+1$ neurons. **(b)** Imposing $f(a_j)=g(a_j)$ and the two slope conditions gives the displayed system. It is lower triangular because $\sigma(a_j-a_i)=0$ for $i\geq j$: the kink at a_i contributes nothing to the left of itself. A triangular system with nonzero diagonal is solvable by forward substitution, so the coefficients exist and are unique — which is Theorem 11.2.1.

§ 11.3 Piecewise-linear interpolation

To go from exact representation of piecewise-linear functions to approximation of general functions, we need interpolation theory.

Definition 11.3.1: Nodal basis

Partition $[a,b]$ by $x_0<x_1<\cdots<x_{N+1}$, write $I_i:=[x_{i-1},x_i]$ and $h_i:=x_i-x_{i-1}$, and define the hat functions

$$\varphi_i(x):=\begin{cases}\dfrac{x-x_{i-1}}{h_i} & x\in[x_{i-1},x_i],\\[2mm] \dfrac{x_{i+1}-x}{h_{i+1}} & x\in[x_i,x_{i+1}],\\[2mm] 0 & \text{otherwise.}\end{cases}$$

ms. p.35

Proposition 11.3.1 Nodal basis properties

(a) $\varphi_i(x_j)=\delta_{ij}$;

(b) $\displaystyle\sum_{i=0}^{N+1}\varphi_i(x)=1$ *for all* $x\in[a,b]$ *(partition of unity).*

ms. p.36

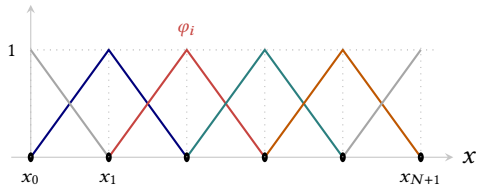
Definition 11.3.2: Piecewise-linear interpolant

The interpolant of v on the nodes is, restricted to I_i,

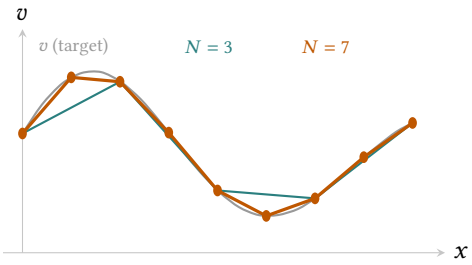
$$g(x)\big|_{I_i}:=\frac{x_i-x}{h_i}v(x_{i-1})+\frac{x-x_{i-1}}{h_i}v(x_i)=\varphi_{i-1}(x)v(x_{i-1})+\varphi_i(x)v(x_i).$$

ms. p.36

Let us see this geometrically. The hat functions are the standard finite element basis, and the two properties of Proposition 11.3.1 are visible at a glance: each hat is 1 at its own node and 0 at every other, and at any x the hats present sum to 1.



(a) $\varphi_i(x_j) = \delta_{ij}$, and $\sum_i \varphi_i \equiv 1$.



(b) More nodes, closer interpolant.

Figure 11.4: **Piecewise-linear interpolation. (a)** The nodal basis. Each φ_i is a hat peaking at its own node; the sum over all i is identically 1, which is the partition of unity used in the proof of Theorem 11.4.1. **(b)** The interpolant of a smooth target on $N + 2$ nodes. By Theorem 11.2.1 each of these piecewise-linear curves *is* a single-hidden-layer ReLU network, so bounding the distance between the two curves bounds the error of a neural network — using nothing from learning theory at all.

Key idea

By Theorem 11.2.1, g *is* a single-hidden-layer ReLU network. So bounding $|v - g|$ bounds the error of a neural network, and the bound comes from interpolation theory rather than from anything specific to learning.

§ 11.4 The error bound

Definition 11.4.1: Lipschitz continuity

v *defined on* $[a, b]$ *is* Lipschitz continuous *if there exists* $L > 0$ *with*

$$|v(x) - v(y)| \le L|x - y| \qquad \text{for all } x, y \in [a, b].$$

ms. p.37

Theorem 11.4.1 Interpolation error

Let v be Lipschitz with constant L on $[a, b]$ and let g be its piecewise-linear interpolant. Set $h = \max_i h_i$. Then

$$|v(x) - g(x)| \le Lh \qquad \text{for all } x \in [a, b].$$

Proof: Fix $x \in I_i$. Using the partition of unity (Proposition 11.3.1(b)) to write $v(x) = (\varphi_{i-1}(x) + \varphi_i(x))v(x)$,

$$v(x) - g(x)\big|_{I_i} = \varphi_{i-1}(x)\big[v(x) - v(x_{i-1})\big] + \varphi_i(x)\big[v(x) - v(x_i)\big].$$

By the triangle inequality and Lipschitz continuity, with $|x - x_{i-1}| \le h_i$ and $|x - x_i| \le h_i$,

$$|v(x) - g(x)\big|_{I_i}| \le \varphi_{i-1}(x)Lh_i + \varphi_i(x)Lh_i = Lh_i\big(\varphi_{i-1}(x) + \varphi_i(x)\big) = Lh_i,$$

using the partition of unity once more. Taking $h = \max_i h_i$ and noting the bound holds on every I_i gives the result on all of $[a, b]$. ■ ms. p.36,37

Corollary 11.4.2 Convergence

If $N \to \infty$ so that $h \to 0$, then $\lim_{h\to 0}|v(x) - g(x)| = 0$: with sufficiently many training nodes we may approximate v to any accuracy we like.

ms. p.37

§ 11.5 How many neurons?

Corollary 11.5.1 Neuron count

With uniform intervals $h_1 = \cdots = h_{N+1} = (b-a)/(N+1)$,

$$|v(x) - g(x)| \le L\frac{b-a}{N+1} \qquad \text{for all } x \in [a,b],$$

where $N+1$ *is approximately the number of neurons in the ReLU network* g. *To achieve accuracy* ε *it therefore suffices that*

$$L\frac{b-a}{N+1} = O(\varepsilon) \qquad \Longrightarrow \qquad N = O\left(\frac{1}{\varepsilon}\right),$$

ms. p.38,39 *and conversely* $\varepsilon = O(1/N)$. *For* $\varepsilon = 0.01$ *this gives* $N \approx 100$.

Note:-

Two caveats from lecture.

1. The construction assumes we control the sample locations $\{x_i, v(x_i)\}$. In practice we are handed a fixed and ms. p.39 noisy dataset $\{x_i, v(x_i) + \text{noise}\}$.
2. The theory prescribes $O(N)$ neurons for *one* hidden layer. How many are needed for two or more layers is not known. The working belief is that deeper is more expressive — it covers a larger set of functions — so for a deep network one takes the *total* neuron count to be around $O(N)$. In practice one chooses far more than N, which ms. p.40 is *over-parameterization*.

Let us see this geometrically. Corollary 11.5.1 is a *worst-case* statement: it assumes only Lipschitz continuity. It is worth checking numerically, and worth seeing what happens when the target is nicer than the theorem requires.

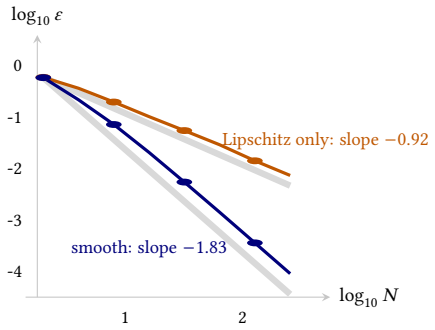


Figure 11.5: **The bound is sharp, but only in the worst case.** Maximum interpolation error against the number of interior nodes, log–log, with the reference slopes -1 and -2 drawn pale beneath. For a target that is Lipschitz but not differentiable — a single kink, averaged over its position — the measured slope is -0.92, matching Corollary 11.5.1: halving the error costs twice the neurons. For a smooth target the measured slope is -1.83, so the same accuracy is bought with far fewer neurons than the corollary promises. The theorem is not wrong; it is stating what can be guaranteed knowing *only* that v is Lipschitz. Assume more and you get more.

Key idea

The neuron count scales as $1/\varepsilon$ in one dimension. In d dimensions the same tensor-product argument gives $O(\varepsilon^{-d})$ — the curse of dimensionality — and the fact that deep networks empirically evade this is the central open question of the subject.

§ 11.6 Exercises

The code problems need only `jax.numpy`; nothing here is trained until Chapter 12. Enable `x64` (Section C.3) — the triangular solve of Figure 11.3b loses digits quickly in single precision.

Warm-up

1. [Theory] Show that a network with no activation function — σ the identity — collapses to a single affine map however many layers it has. Which of the four functions in Figure 11.2 would therefore be useless, and why is the step function useless for a different reason?

2. [Theory] Write $f(x) = \max\{0, 3x - 6\}$ as $r\,\sigma(x - a)$ and identify r and a. Then write $f(x) = 2x + 1$ as a difference of two ReLUs, as in Section 10.2.1.

3. [Theory] Prove that Lipschitz continuity implies continuity. ms. p.37

4. [Theory] Show that if v is differentiable with $|v'(x)| \le L$ on $[a, b]$, then v is Lipschitz with constant L. ms. p.37

5. [Theory] Using Figure 11.4a, verify Proposition 11.3.1 at the three points $x = x_i$, $x = (x_i + x_{i+1})/2$, and $x = x_{i+1}$. How many hats are nonzero at a generic x?

6. [Code] Implement the two building blocks of Section 10.2.1 in JAX and plot them: a single kink $r\,\sigma(x - a)$, and an affine function written as a difference of two ReLUs. Confirm the second reproduces $2x + 1$ to machine precision on $[-3, 3]$.

Core

7. [Theory] Solve the triangular system of Figure 11.3b explicitly for $m = 3$ and verify that the resulting network interpolates a given piecewise-linear function exactly.

8. [Theory] Show the coefficient matrix is lower triangular by proving $\sigma(a_j - a_i) = 0$ whenever $i \ge j$, and explain why a nonzero diagonal then guarantees a unique solution.

9. [Theory] Prove Proposition 11.3.1, both parts. Where in the proof of Theorem 11.4.1 is part (b) used, and what would fail without it?

10. [Theory] Prove Theorem 11.4.1.

11. [Theory] Count the parameters of the network in Figure 11.3a as a function of m, and compare with the number of interpolation conditions imposed. Why must the two match?

12. [Theory] Show that an L-layer ReLU network of width w produces a piecewise-linear function, and give an upper bound on its number of linear pieces. Compare the growth in L against the growth in w: which buys more pieces per parameter?

13. [Code] Implement the construction (11.2) in JAX for a target piecewise-linear function. Do *not* train — solve the triangular system and set the weights directly. Verify exact interpolation at the nodes to machine precision, and check what happens in `float32` as m grows.

Challenge

14. [Theory] Extend Corollary 11.5.1 to d dimensions using a tensor-product grid and show $N = O(\varepsilon^{-d})$. For $\varepsilon = 0.01$ and $d = 10$, how many neurons does the bound demand? Compare with the number of atoms available to you.

15. [Theory] Figure 11.5 shows a smooth target achieving slope -2 where the theorem guarantees only -1. Prove the sharper bound $|v - g| \le \frac{1}{8}h^2 \max|v''|$ for $v \in C^2$, and explain why Theorem 11.4.1 does not contradict it.

16. [Theory] The construction uses the nodes $\{a_i\}$ we chose. Suppose instead the nodes are given, noisy, and unevenly spaced, as Remark 10.5.1 warns. Show that the error bound then depends on the *largest* gap $h = \max_i h_i$, and construct a node distribution for which N nodes give error no better than $O(1)$.

17. [Theory] Prove that no ReLU network of finite width and depth can represent $v(x) = x^2$ exactly on $[0, 1]$, but that Theorem 11.4.1 still applies. What does this say about the difference between *representation* and *approximation*?

18. [Code] Reproduce Figure 11.5. For a smooth target and for $v(x) = |x - c|$ with c drawn at random, build the interpolant for $N = 2, 4, \ldots, 256$, measure the maximum error, and fit the slope on log–log axes. Average the kink case over 60 values of c and explain why a single c gives an erratic curve.

19. [Code] Now *train* a single-hidden-layer ReLU network of the same width from random initialization on the same smooth target, using Algorithm 2. Compare the achieved error to the constructive bound at each width. Does optimization reach what approximation theory guarantees exists? Then repeat with noisy samples, per Remark 10.5.1, and comment on which failure dominates.

Chapter 12

Training Deep Networks

Goal of this chapter

Chapter 11 showed that a network of the right size *exists*, and even built one by hand. This chapter is about finding it by optimization instead — and about the fact that the obvious approach stops working once the network is deep. The chapter answers:

1. How is $\nabla_\theta J(\boldsymbol{\theta})$ computed for a network, without doing L times more work than necessary?
2. Why does that computation degrade with depth, and by exactly how much?
3. What architectural changes repair it — and what do they turn the network into?

The answer to the second question is a single number, $\sigma'(0) = \frac{1}{4}$ from Figure 8.2b, raised to the power of the depth. The answers to the third lead somewhere unexpected: a deep residual network is a numerical integrator.

§ 12.1 Backpropagation

Write the network of (11.1) layer by layer,

$$\boldsymbol{y}^\ell = \sigma\big(\boldsymbol{W}^\ell \boldsymbol{y}^{\ell-1} + \boldsymbol{b}^\ell\big), \qquad \ell = 1, \dots, L, \qquad \boldsymbol{y}^0 = \boldsymbol{x}, \tag{12.1}$$

and take the L_2 loss on the network output, $J = \|\boldsymbol{y}^L - \boldsymbol{y}\|^2$, with $\boldsymbol{y}$ the data. The parameters are $\boldsymbol{\theta} = \{\boldsymbol{b}^1, \dots, \boldsymbol{b}^L, \boldsymbol{W}^1, \dots, \boldsymbol{W}^L\}$. To run Algorithm 2 we need $\partial J / \partial \boldsymbol{\theta}_\ell$ for the parameters of every layer. ms. p.40

Since $\boldsymbol{y}^L$ depends on $\boldsymbol{\theta}_\ell$ only through the chain $\boldsymbol{y}^\ell \to \boldsymbol{y}^{\ell+1} \to \cdots \to \boldsymbol{y}^L$, the chain rule gives

$$\frac{\partial J}{\partial \boldsymbol{\theta}_\ell} = \frac{\partial J}{\partial \boldsymbol{y}^L} \frac{\partial \boldsymbol{y}^L}{\partial \boldsymbol{y}^{L-1}} \cdots \frac{\partial \boldsymbol{y}^{\ell+1}}{\partial \boldsymbol{y}^\ell} \frac{\partial \boldsymbol{y}^\ell}{\partial \boldsymbol{\theta}_\ell}. \tag{12.2}$$

ms. p.40

Let us see this geometrically. The whole point of backpropagation is bookkeeping. Reading the chain rule left to right recomputes the same products L times; reading it right to left computes each once and reuses it.

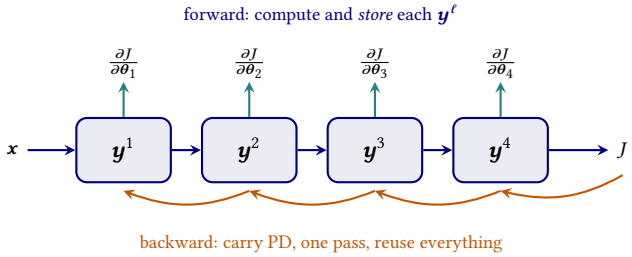


Figure 12.1: **Backpropagation is the chain rule, ordered correctly.** The forward pass evaluates the network and stores every intermediate $\boldsymbol{y}^\ell$ — this storage is the memory cost of training. The backward pass then sweeps once from the loss to the input, carrying the running product PD of Algorithm 7 and peeling off one parameter gradient (teal) at each layer. Computing the L gradients independently would cost $O(L^2)$ layer-derivative evaluations; this costs $O(L)$.

Key idea

The leading factors of (12.2) for layer ℓ are exactly the factors already computed for layer $\ell+1$. Evaluating from the output backward therefore reuses everything, turning what looks like L separate chain-rule computations into a single sweep. Backpropagation is nothing more than the chain rule with the multiplications ordered correctly — a smart chain rule.

ms. p.40,43

Algorithm 7: Backpropagation

Input: activations $\boldsymbol{y}^0, \dots, \boldsymbol{y}^L$ stored by a forward pass of (12.1); parameters $\{\boldsymbol{W}^\ell, \boldsymbol{b}^\ell\}_{\ell=1}^{L}$
Output: $\partial J/\partial \boldsymbol{\theta}_\ell$ for every layer ℓ

```
// PD = previous derivative, carried backwards from the output
```

PD $\leftarrow \partial J/\partial \boldsymbol{y}^L$
for $\ell = L, L-1, \dots, 1$ **do**
  compute the local derivative $\partial \boldsymbol{y}^\ell/\partial \boldsymbol{\theta}_\ell$
  $\partial J/\partial \boldsymbol{\theta}_\ell \leftarrow$ PD $\times\, \partial \boldsymbol{y}^\ell/\partial \boldsymbol{\theta}_\ell$
  PD $\leftarrow$ PD $\times\, \partial \boldsymbol{y}^\ell/\partial \boldsymbol{y}^{\ell-1}$

ms. p.41

Note:-

The forward pass is not part of the algorithm; it is an ordinary evaluation of (12.1), and its only role here is to leave behind the activations the backward loop consumes. That is worth noticing rather than burying in a loop, because it is the whole source of backpropagation's memory cost: every $\boldsymbol{y}^\ell$ must survive until the backward pass reaches layer ℓ, so storage grows as $O(L)$. Section 12.3.4 removes exactly this requirement.

ms. p.41 The two local derivatives needed are

$$\frac{\partial \boldsymbol{y}^\ell}{\partial \boldsymbol{\theta}_\ell} = \begin{cases} \sigma'(\boldsymbol{W}^\ell \boldsymbol{y}^{\ell-1} + \boldsymbol{b}^\ell) & \boldsymbol{\theta}_\ell = \boldsymbol{b}^\ell, \\ \boldsymbol{y}^{\ell-1}\, \sigma'(\boldsymbol{W}^\ell \boldsymbol{y}^{\ell-1} + \boldsymbol{b}^\ell) & \boldsymbol{\theta}_\ell = \boldsymbol{W}^\ell, \end{cases} \qquad \frac{\partial \boldsymbol{y}^\ell}{\partial \boldsymbol{y}^{\ell-1}} = \boldsymbol{W}^\ell\, \mathrm{d}\sigma^\ell,$$

writing $\mathrm{d}\sigma^\ell := \sigma'(\boldsymbol{W}^\ell \boldsymbol{y}^{\ell-1} + \boldsymbol{b}^\ell)$.

Note:-

This is reverse-mode automatic differentiation, which is what `jax.grad` implements. Exercise 6 asks you to check the two against each other; deriving the recursion by hand once is worth doing before trusting the library forever.

§ 12.2 Vanishing and exploding gradients

Substituting the local derivatives into (12.2) for $\boldsymbol{\theta}_\ell = \boldsymbol{b}^\ell$,

$$\frac{\partial J}{\partial b^\ell} = \frac{\partial J}{\partial y^L}\left(\prod_{i=\ell+1}^{L} w^i\,\mathrm{d}\sigma^i\right)\mathrm{d}\sigma^\ell, \qquad \text{so} \qquad \left|\frac{\partial J}{\partial b^\ell}\right| = \left|\frac{\partial J}{\partial y^L}\right| \prod_{i=\ell+1}^{L} |w^i|\,|\mathrm{d}\sigma^i|\,|\mathrm{d}\sigma^\ell|.$$

ms. p.42,43

For the sigmoid, $\sigma' = \sigma(1-\sigma)$ attains its maximum $\sigma'(0) = 0.25$. With the common initialization $w^i \sim \mathcal{N}(0,1)$, $b^i \sim \mathcal{N}(0,1)$, so that $|w^i| \approx 1$, the *best* case is

$$\left|\frac{\partial J}{\partial b^\ell}\right| \approx \left|\frac{\partial J}{\partial y^L}\right| (0.25)^{L-\ell+1}. \tag{12.3}$$

ms. p.42,43

Example 12.2.1 (Example)
Take $L = 8$ and $\ell = 1$. Then $|\partial J/\partial b^1| \approx |\partial J/\partial y^L|(1/4)^8 \approx 10^{-2L} = 10^{-16}$ — at the level of double-precision roundoff. The update

$$(b^\ell)^{i+1} = (b^\ell)^i - \alpha \frac{\partial J}{\partial b^\ell} \approx (b^\ell)^i$$

leaves the early layers unchanged: no learning at all.

ms. p.42,43

Let us see this geometrically. The bound (12.3) is easy to check. Build a deep network, initialize it, and measure how big the gradient actually is at each layer — before any training at all.

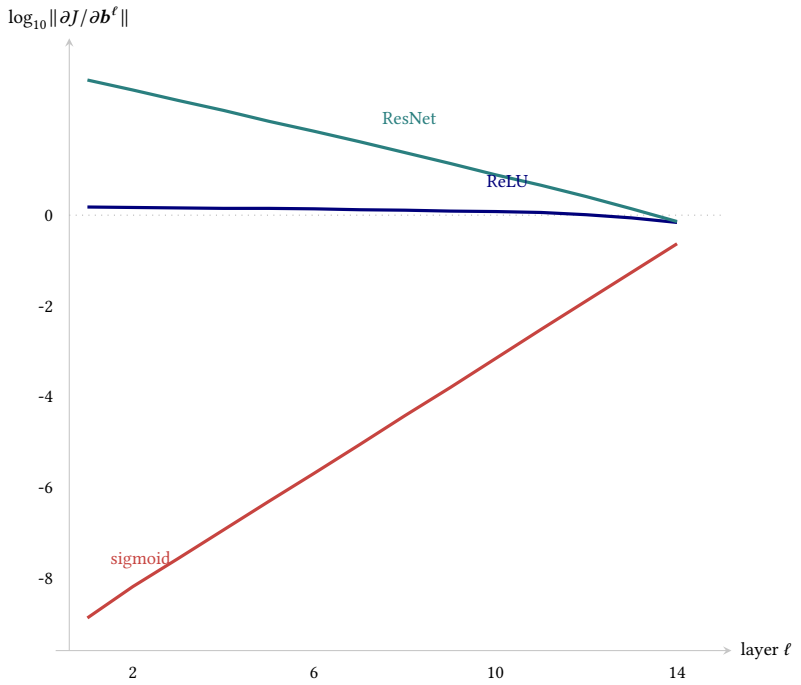


Figure 12.2: **Vanishing gradients, measured.** Mean gradient magnitude at each layer of a 14-layer network of width 32 at initialization, averaged over 120 draws, before any training. With sigmoid activations (red) the gradient falls by 0.635 decades per layer — compare $-\log_{10}(0.25) = 0.602$ predicted by (12.3) — so layer 1 receives a gradient eight orders of magnitude smaller than layer 14 and does not move. ReLU (navy) is almost flat, because its derivative is exactly 1 on the active branch. The residual network (teal) also decays gently but starts far higher, since the skip connection contributes a 1 to every factor. Only the first curve represents a network that cannot be trained.

Key idea

The factor $0.25^{L-\ell+1}$ is why sigmoid activations do not scale with depth, and it is the strongest argument for ReLU, whose derivative is 1 on the active branch.

§ 12.3 ResNet

The residual network changes the layer map to

$$\boldsymbol{y}^{\ell} = \boldsymbol{y}^{\ell-1} + \sigma\big(\boldsymbol{W}^{\ell}\boldsymbol{y}^{\ell-1} + \boldsymbol{b}^{\ell}\big), \tag{12.4}$$

ms. p.43 adding a skip connection. The local derivative becomes

$$\frac{\partial y^{i}}{\partial y^{i-1}} = 1 + w^{i}\,\mathrm{d}\sigma^{i},$$

so that

$$\Big|\frac{\partial J}{\partial b^{\ell}}\Big| = \Big|\frac{\partial J}{\partial y^{L}}\Big| \underbrace{\prod_{i=\ell+1}^{L} \underbrace{\big(1 + w^{i}\mathrm{d}\sigma^{i}\big)}_{O(1)}}_{O(1)}\ \mathrm{d}\sigma^{\ell},$$

ms. p.44 and the product no longer collapses geometrically.

Note:-

Two further claims from lecture.

(a) ResNet cures the gradient vanishing problem.

(b) The loss landscape of a ResNet is much smoother than that of a plain deep network — see the loss-landscape ms. p.44 visualizations from the University of Maryland group.

12.3.1 From ResNet to neural ODE

Rewriting (12.4) as $\boldsymbol{y}^{\ell} = \boldsymbol{y}^{\ell-1} + \Delta t\,\sigma(\boldsymbol{W}^{\ell}\boldsymbol{y}^{\ell-1} + \boldsymbol{b}^{\ell})$ exhibits it as the forward Euler discretization of

$$\frac{d\boldsymbol{y}}{dt} = \sigma\big(\boldsymbol{W}(t)\boldsymbol{y} + \boldsymbol{b}(t)\big), \qquad t \in [0, T], \tag{12.5}$$

ms. p.44 sampled at $t_{\ell} = \ell\Delta t$. The observation is due independently to Haber and Ruthotto [3] and to several others, and it is the starting point for Chen et al. [1].

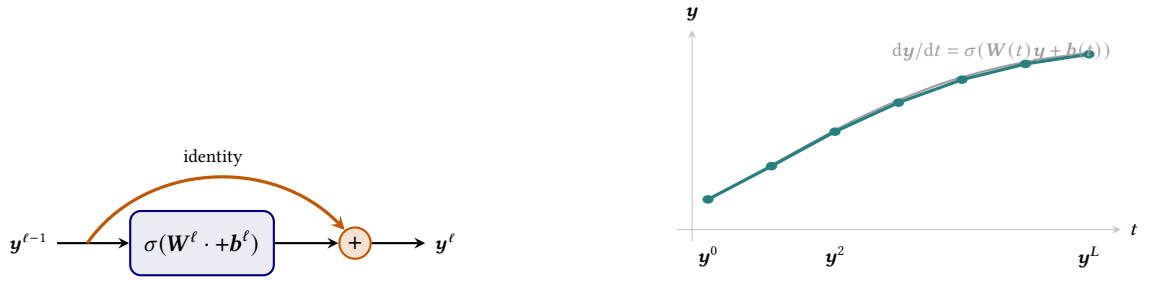
Common pitfall

The correspondence needs $\Delta t = T/L$, not $\Delta t = 1$. A ResNet as usually written takes unit steps, so adding layers extends the integration *horizon* rather than refining the grid, and the $L \to \infty$ limit is then an ODE integrated for infinite time rather than a fixed one solved accurately. To get a continuum limit the residual branch must be scaled by $1/L$. This is a real condition, not bookkeeping: an unscaled hundred-layer ResNet is not a discretization of any fixed (12.5), and statements proved about the ODE do not automatically transfer to it.

Key idea

Depth is a time discretization. A deep residual network is a numerical integrator for (12.5), which means the apparatus of ODE theory — stability, stiffness, adaptive stepping, higher-order schemes — becomes available for reasoning about architectures. The rest of this section takes three of those and cashes them.

Let us see this geometrically. The skip connection is a small change to draw and a large change to the gradient. Reading the same picture as a time step turns depth into duration.

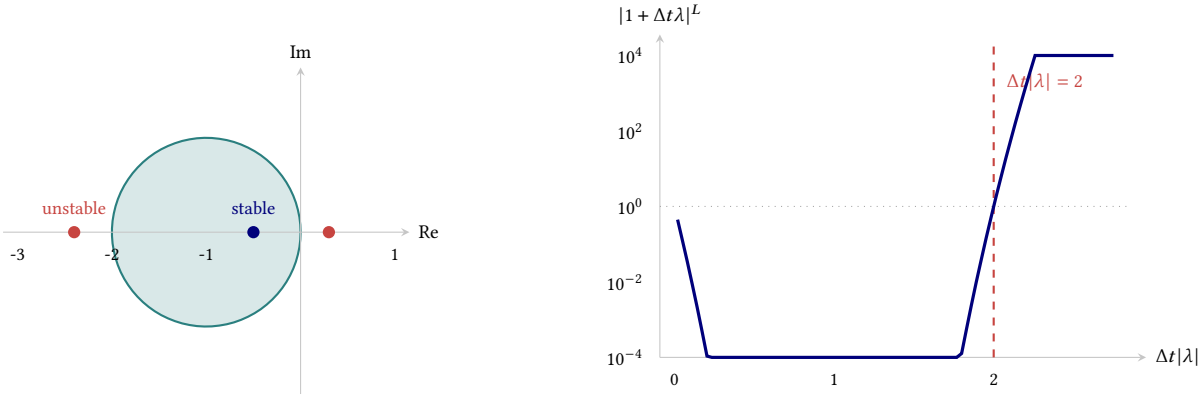


(a) A residual block.

(b) Layers as forward Euler steps.

Figure 12.3: **ResNet, and what it becomes in the limit.** **(a)** The block computes $\boldsymbol{y}^\ell = \boldsymbol{y}^{\ell-1} + \sigma(\cdot)$: the orange path carries the input through untouched. Differentiating gives $\partial \boldsymbol{y}^\ell / \partial \boldsymbol{y}^{\ell-1} = 1 + w^\ell \mathrm{d}\sigma$, and it is that 1 — contributed by the skip, not the weights — that keeps the product in (12.2) from collapsing. **(b)** With a step size attached, (12.4) is exactly forward Euler for (12.5): the teal points are layer outputs and the gray curve the underlying trajectory. Depth is duration, and the layer index is a time index.

Let us see this geometrically. Forward Euler is only conditionally stable, and the condition is a statement about the weights. Drawing its stability region turns a numerical analysis fact into an architectural constraint.



(a) The region $|1 + z| \le 1$, $z = \Delta t\, \lambda$.

(b) Amplification over $L = 40$ layers.

Figure 12.4: **Stability is a constraint on the weights.** Applying (12.4) to a linearized block, the propagated state is multiplied by $1 + \Delta t\lambda$ at each layer, where λ runs over the eigenvalues of the block Jacobian. **(a)** Forward Euler is stable exactly when $\Delta t\lambda$ lies in the unit disc centred at -1; for real λ that is $0 \le \Delta t|\lambda| \le 2$. An eigenvalue with positive real part (right marker) is outside it however small Δt is made. **(b)** The same statement over $L = 40$ layers. Inside the region the state decays, often violently: at $\Delta t|\lambda| = 0.5$ the factor is already 9×10^{-13}. At $\Delta t|\lambda| = 2$ it is exactly 1, and past it the network explodes — 4.5×10^1 at 2.1 and 1.1×10^7 at 2.5. Note that both failures of Section 12.2 appear here as one picture, read at different points of the same axis.

12.3.2 Stability, and what it forbids

The first thing ODE theory supplies is a constraint nobody had to invent. Linearize a block about a state and let λ run over the eigenvalues of the Jacobian $\boldsymbol{J} = \partial\sigma/\partial\boldsymbol{y}$. Forward Euler propagates each mode by the factor $1 + \Delta t\lambda$ per step, so after L layers the state has been multiplied by $(1 + \Delta t\lambda)^L$, and the discretization is stable exactly when $\Delta t\lambda$ lies in the unit disc centred at -1 (Figure 12.4a). For real λ that reads

$$0 \;\le\; \Delta t\,|\lambda| \;\le\; 2. \tag{12.6}$$

Read (12.6) as a statement about weights and it becomes an initialization rule: the spectral radius of $\boldsymbol{W}^\ell$, scaled by the step, may not exceed a threshold that depends only on the integrator. Exceed it and the forward pass diverges —

1.1×10^7 over forty layers at $\Delta t|\lambda| = 2.5$, from Figure 12.4b. Worse, an eigenvalue with *positive* real part is outside the region for every $\Delta t > 0$, so no amount of careful stepping rescues it.

Key idea

Section 12.2 presented vanishing and exploding gradients as two separate pathologies. They are one picture read at two points of the same axis: $|1 + \Delta t\lambda| < 1$ shrinks the signal geometrically with depth, > 1 grows it, and only the boundary preserves it. That is why the cure in both cases is a statement about the *spectrum* of the layer map rather than about its size.

Note:-

Stability and trainability are not the same condition, and it is worth seeing a case where they part. The network measured in Figure 12.2 lost 0.635 decades of gradient per layer, a per-layer factor of 0.232, which corresponds to $\Delta t\lambda = -0.768$ — comfortably inside (12.6). That network was perfectly stable and completely untrainable. Stability forbids blow-up; it does not promise that anything reaches the early layers.

Haber and Ruthotto [3] turn this into architecture. If the block is built so that $\boldsymbol{J}$ is antisymmetric, its eigenvalues are purely imaginary, every mode sits on the boundary of the stability region, and the forward map neither grows nor decays no matter how deep the network is. Their construction replaces $\boldsymbol{W}$ by $\frac{1}{2}(\boldsymbol{W} - \boldsymbol{W}^\mathsf{T})$, at the cost of restricting what the layer can represent — the usual trade.

12.3.3 Other discretizations, other architectures

Once (12.4) is recognized as *a* scheme rather than *the* scheme, the obvious question is what the other schemes correspond to. Several architectures proposed independently turn out to be classical integrators.

architecture	update	scheme
ResNet	$\boldsymbol{y}^{\ell+1} = \boldsymbol{y}^\ell + f(\boldsymbol{y}^\ell)$	forward Euler
PolyNet	$\boldsymbol{y}^{\ell+1} = \boldsymbol{y}^\ell + f(\boldsymbol{y}^\ell) + f(f(\boldsymbol{y}^\ell))$	approximate backward Euler
LM-ResNet	$\boldsymbol{y}^{\ell+1} = (1-k)\boldsymbol{y}^\ell + k\boldsymbol{y}^{\ell-1} + f(\boldsymbol{y}^\ell)$	linear multistep
Second-order CNN	$\boldsymbol{y}^{\ell+1} = 2\boldsymbol{y}^\ell - \boldsymbol{y}^{\ell-1} + \Delta t^2 f(\boldsymbol{y}^\ell)$	leapfrog

The reading is due to Lu et al. [5] and to Haber and Ruthotto [3]. It is worth more than a curiosity: each scheme brings its known stability region and order of accuracy with it, so a question like "should the skip connection reach back one layer or two" stops being a matter of taste and becomes a question about multistep methods, which has answers.

12.3.4 The adjoint equation, or backpropagation in the limit

The third payoff is the one that changed practice. Backpropagation through L layers stores every intermediate activation, so memory grows as $O(L)$ — Section 12.1 needed exactly this. Take the continuum limit and the storage disappears.

Let J depend on $\boldsymbol{y}(T)$ and define the *adjoint* $\boldsymbol{a}(t) \coloneqq \partial J/\partial \boldsymbol{y}(t)$, the sensitivity of the loss to a perturbation of the state at time t. Writing $f(\boldsymbol{y}, t) = \sigma(\boldsymbol{W}(t)\boldsymbol{y} + \boldsymbol{b}(t))$, the adjoint satisfies its own ODE, integrated *backwards* from $t = T$:

$$\frac{d\boldsymbol{a}}{dt} = -\boldsymbol{a}^\mathsf{T}\frac{\partial f}{\partial \boldsymbol{y}}, \qquad \frac{dJ}{d\boldsymbol{\theta}} = -\int_T^0 \boldsymbol{a}(t)^\mathsf{T}\frac{\partial f}{\partial \boldsymbol{\theta}}\,\mathrm{d}t. \tag{12.7}$$

Key idea

Compare (12.7) with (12.2). The backward recursion of backpropagation is forward Euler applied to the adjoint equation, run in reverse time; the product of Jacobians that caused all the trouble in Section 12.2 is the discrete trace of the integrating factor $\exp\!\big(-\int \partial f/\partial \boldsymbol{y}\,\mathrm{d}t\big)$. Backpropagation is not merely analogous to an adjoint method — in the limit it *is* one, and the adjoint-state method it becomes was standard in optimal control and PDE-constrained optimization long before it was used to train networks.

The practical consequence is that nothing need be stored. Since (12.5) can be integrated backwards from $\boldsymbol{y}(T)$ to recover $\boldsymbol{y}(t)$ on the fly, the backward pass carries only the current state and the current adjoint: memory is $O(1)$ in depth rather than $O(L)$. That is the central contribution of Chen et al. [1], and it buys depth that would otherwise exhaust the device.

Common pitfall

The $O(1)$ memory is not free; it is memory traded for arithmetic and for accuracy. The backward solve costs a second integration, roughly doubling the work, and reconstructing $\boldsymbol{y}(t)$ by integrating backwards is only as accurate as the scheme — for a dissipative forward problem the reverse integration is the unstable direction, and errors grow exactly where the forward solve damped them. Checkpointing a few intermediate states, the classical compromise in PDE-constrained optimization, remains the practical answer.

§ 12.4 Convolutional networks

Definition 12.4.1: Convolution

For f, g on $\mathbb{R}$,

$$(f * g)(x) \coloneqq \int_{\mathbb{R}} f(t)\, g(x-t)\, \mathrm{d}t,$$

with f the filter. *In the discrete case, for $f \in \mathbb{R}^n$ and $g \in \mathbb{R}^m$ with $n < m$,*

$$\text{1D:}\quad (f * g)(p) \coloneqq \sum_{r=1}^{n} f(r)\, g(p-r), \qquad \text{2D:}\quad (f * g)(p,q) \coloneqq \sum_{s=1}^{n}\sum_{r=1}^{n} f(r,s)\, g(p-r, q-s).$$

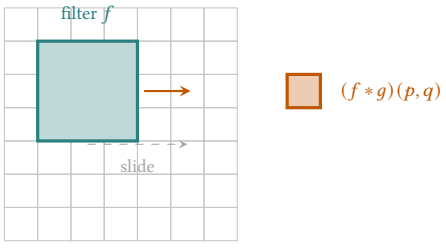
Many filters of different sizes may be applied. ms. p.44

A convolutional layer applies $\sigma(f * g)$ — convolution followed by a nonlinear activation — and *pooling* reduces the spatial size of the input, retaining only the most important features. ms. p.45

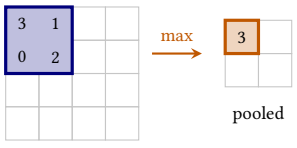
Note:-

LeNet for MNIST digit recognition is the canonical example. CNNs are used as universal visual feature extractors, and it is common to reuse an ImageNet-trained CNN rather than train from scratch, since CNNs carry large computational and data requirements. ms. p.45

Let us see this geometrically. A convolution slides one small filter across the whole input. Two consequences: the same weights are reused everywhere, so the parameter count no longer grows with the image size; and the operation is local, so nearby pixels are combined before distant ones.



(a) One filter, applied everywhere.



(b) Pooling keeps only the strongest response.

Figure 12.5: **Convolution and pooling. (a)** The filter f is much smaller than the input g and is applied at every position, so a 3×3 filter has nine weights regardless of the image size — a fully connected layer on a 64×64 image would need thousands per neuron. **(b)** Pooling reduces the spatial size by keeping only the largest response in each window, discarding *where* within the window it occurred. Stacking the two alternately builds features that are increasingly global and increasingly indifferent to exact position.

12.4.1 What weight sharing buys

The figure's claim about parameter counts is worth making numerical, because the gap is not a factor of a few.

image	dense layer	3×3 filter	ratio
32×32	1.05×10^{6}	9	1.2×10^{5}
64×64	1.68×10^{7}	9	1.9×10^{6}
128×128	2.68×10^{8}	9	3.0×10^{7}
256×256	4.29×10^{9}	9	4.8×10^{8}

A dense layer mapping an $N\times N$ image to another has N^4 weights; the filter has nine, whatever N is. Real layers carry many channels, so a 3×3 layer with C channels in and out holds $9C^2$ weights — 36,864 at $C=64$ — but that count still does not grow with the image.

Key idea

Weight sharing is a *prior*, not a compression trick. Writing the layer as a matrix makes this plain: a convolution is multiplication by a banded Toeplitz matrix whose diagonals are constant. A dense layer could represent that matrix exactly; the convolutional layer differs in refusing to represent anything else. What it asserts is that the useful features of an image do not depend on *where* they occur — which is true of edges and textures, and false of, say, a plot axis label. When the assumption holds it converts Corollary 6.3.2 in our favour by shrinking H enormously; when it fails the network cannot express the thing it needs.

12.4.2 A filter is a stencil

Readers who have discretized a PDE have met convolution already under another name. The discrete Laplacian on a uniform grid is

$$(\Delta_h u)_{ij} = \frac{u_{i+1,j}+u_{i-1,j}+u_{i,j+1}+u_{i,j-1}-4u_{ij}}{h^2},$$

and the right-hand side is exactly a 3×3 convolution with the filter

$$f = \frac{1}{h^2}\begin{pmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{pmatrix}. \tag{12.8}$$

Convolving $u=\sin(2\pi x)\sin(3\pi y)$ on a 64×64 grid with (12.8) reproduces Δu to a relative error of 1.5×10^{-3}, which is the $O(h^2)$ truncation error of the stencil and nothing to do with the network.

Key idea

A convolutional layer can represent a differential operator exactly, and the first layers of a trained vision network routinely learn filters that are recognizably first- and second-derivative stencils. Read the other way: a CNN is a *learned* stencil applied repeatedly, with the activation between applications supplying the nonlinearity. That is a useful way to think about why convolutional architectures have been adapted so readily to solving PDEs — the operator they are built from is the one those problems are written in.

12.4.3 Equivariance, and what pooling does to it

Convolution commutes with translation. Writing T_v for a shift by v,

$$f * (T_v\, g) \;=\; T_v\,(f * g), \tag{12.9}$$

exactly, for periodic boundaries — verified to machine zero — and in the interior for any other boundary treatment. Translate the input and the feature map translates with it.

Common pitfall

This is *equivariance*, not invariance, and the two are often conflated. An equivariant map still knows where the feature was: it has moved the response, not ignored the move. Convolution alone therefore does not make a classifier indifferent to position.

Pooling is what converts the one into the other. Taking a maximum over a window discards the location of the response within that window, so the composition is invariant to shifts smaller than the window — and only to those. A global maximum is fully invariant, which is why Figure 12.5b is not merely a size reduction: it is where the architecture's claim about position is actually made.

Note also where (12.9) fails: at the boundary. Zero padding breaks the symmetry exactly on the edge pixels, which is why boundary handling in a CNN is the same nuisance it is in a finite-difference code, and is dealt with by the same devices.

12.4.4 Depth and the receptive field

How far can information travel? A 3×3 filter combines a pixel with its immediate neighbours, so after L stacked layers of stride one an output depends on an input patch of width $2L + 1$.

layers L	stride-one field	with 2×2 pooling each layer
1	3	4
2	5	10
5	11	94
10	21	≈ 3000

Stride one grows the field *linearly*: spanning a 64-pixel image needs 32 layers before any output can see the whole input. Pooling makes the growth geometric, and that — rather than the saving in arithmetic — is its structural purpose.

Key idea

The receptive field is the numerical *domain of dependence*, and the correspondence with Section 12.3.2 is exact rather than decorative. There the layer index was time and stability bounded how much a mode could grow per step; here the layer index is again time and the stencil width bounds how far information can travel per step. A CNN that must relate two distant pixels needs enough layers for their domains of dependence to overlap, exactly as an explicit PDE solver needs enough steps for a signal to cross the grid. Depth buys reach, and pooling buys it faster.

§ 12.5 Recurrent networks

For time-series data $\{x_t, y_t\}$ with $x_t \in \mathbb{R}^{M_1}$, the task is to predict $y_{t_{N+1}}$. A recurrent network maintains a hidden state $h_t \in \mathbb{R}^m$:

$$h_t = \sigma\big(\boldsymbol{W}_h h_{t-1} + \boldsymbol{W}_x x_t + \boldsymbol{b}_h\big), \qquad y_t = \sigma\big(\boldsymbol{W}_y h_t + \boldsymbol{c}_y\big), \tag{12.10}$$

the role of h_t being to keep some of the memory. For a time series the parameters $\boldsymbol{W}_h, \boldsymbol{b}_h, \boldsymbol{W}_y, \boldsymbol{c}_y$ are held fixed across t. ms. p.45,47

With the L_2 loss $J = \frac{1}{n}\sum_t \|y_t - \hat{y}_t\|^2$, backpropagation through time gives

$$\frac{\partial h_t}{\partial b_h} = \mathrm{d}\sigma^t + \frac{\partial h_t}{\partial h_{t-1}}\frac{\partial h_{t-1}}{\partial b_h}, \qquad \prod_{s=1}^{t}\frac{\partial h_s}{\partial h_{s-1}} = \prod_{s=1}^{t}\boldsymbol{W}_h\,\mathrm{d}\sigma^s,$$

so the vanishing/exploding problem returns, now controlled by repeated multiplication by the *same* matrix $\boldsymbol{W}_h$. ms. p.46,47

Let us see this geometrically. Unrolling the recurrence in time turns it into a deep network — one whose depth is the length of the sequence, and whose weights are the *same* at every layer. That repetition is exactly why the gradient pathology returns.

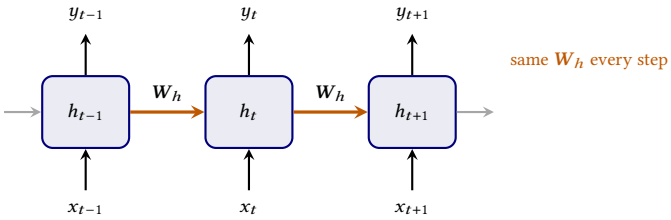


Figure 12.6: **A recurrent network, unrolled.** Each step takes the input x_t and the previous hidden state, and produces an output and a new hidden state. Unrolled over T steps this is a network of depth T — but unlike Figure 12.1, every layer shares the *same* matrix $\boldsymbol{W}_h$. Backpropagation through time therefore multiplies T copies of one matrix, and the product is governed by its spectral radius: less than one and the gradient vanishes, greater than one and it explodes. Figure 12.2 showed the same effect with distinct matrices; here there is not even the averaging that independent draws provide.

12.5.1 Long short-term memory

ms. p.47 LSTM introduces gates:

$$
\begin{aligned}
f_t &= \sigma\big(\boldsymbol{W}_f[h_{t-1}, x_t] + b_f\big) && \text{(forget gate)}\\
i_t &= \sigma\big(\boldsymbol{W}_i[h_{t-1}, x_t] + b_i\big) && \text{(input gate)}\\
o_t &= \sigma\big(\boldsymbol{W}_o[h_{t-1}, x_t] + b_o\big) && \text{(output gate)}\\
\tilde{C}_t &= \tanh\big(\boldsymbol{W}_C[h_{t-1}, x_t] + b_C\big) && \text{(candidate cell state)}
\end{aligned}
$$

and updates

$$\boxed{C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t, \qquad h_t = o_t \odot \tanh(C_t)}$$

ms. p.47,48 where $\odot$ is the componentwise (Hadamard) product.

Note:-

ms. p.48 **Reading the gates.** Since σ is the sigmoid, every gate lies in $(0, 1)$:

- $f_t = 0$ forgets the previous cell state C_{t-1} entirely; $f_t = 1$ retains all of it; intermediate values remember partially.
- $i_t = 0$ admits no new information; $i_t = 1$ admits all of it.

Proposition 12.5.1 Why LSTM vanishes less

From the cell-state update, $\partial C_t / \partial C_{t-1} = \operatorname{diag}(f_t)$, so

$$\prod_{s=1}^{t} \frac{\partial C_s}{\partial C_{s-1}} = \prod_{s=1}^{t} \operatorname{diag}(f_s).$$

ms. p.48 *The network can therefore* learn *to hold f_s near 1 along paths where memory must persist, keeping the product $O(1)$.*

Note:-

ms. p.48 The architecture above is one-to-one. There are also one-to-many, many-to-one, and many-to-many variants.

§ 12.6 Exercises

The code problems need `jax.grad` and `jax.jit`; the last two use MNIST via `fetch_openml` (Section C.5) and are the first in these notes for which a GPU is worth requesting (Section C.3 and Appendix C).

Warm-up

1. [Theory] Verify the two local derivative formulas following Algorithm 7, for $\boldsymbol{\theta}_\ell = \boldsymbol{b}^\ell$ and $\boldsymbol{\theta}_\ell = \boldsymbol{W}^\ell$.

2. [Theory] Show $\sigma' = \sigma(1-\sigma)$ and $\max \sigma' = 0.25$, and use it to evaluate (12.3) for $L = 14$, $\ell = 1$. Compare with the leftmost point of Figure 12.2.

3. [Theory] Using Figure 12.1, state what must be held in memory during the forward pass and why. For a network of L layers and width w, how does that memory grow?

4. [Theory] In Figure 12.3a, which of the two paths contributes the 1 in $\partial \boldsymbol{y}^{\ell} / \partial \boldsymbol{y}^{\ell-1} = 1 + w^{\ell} \mathrm{d}\sigma$? What would the derivative be without the skip?

5. [Theory] A 3×3 convolutional filter is applied to a 64×64 image. How many weights does the layer have? How many would a fully connected layer producing the same output size need?

6. [Code] Implement a two-layer network in JAX and verify `jax.grad` against a finite-difference gradient to five digits. State which precision you used and why it matters.

Core

7. [Theory] Show that a one-dimensional convolution with a filter of width n is multiplication by a banded Toeplitz matrix, and write that matrix for $n = 3$. How many distinct entries does it contain, and how many would a dense matrix of the same size contain? Relate your answer to the parameter table of §12.4.1.

8. [Code] Verify §12.4.2 numerically. Convolve $u = \sin(2\pi x)\sin(3\pi y)$ on a uniform $N \times N$ grid with the stencil (12.8) and compare against the exact Δu, for $N = 32, 64, 128$. Confirm the relative error is 1.5×10^{-3} at $N = 64$ and that it falls at $O(h^2)$. Whose error is this — the network's or the discretization's?

9. [Code] Test (12.9) for yourself. Convolve a random array with a random 3×3 filter, and compare $f * (T_v g)$ with $T_v(f * g)$ under periodic padding and under zero padding. Report the maximum discrepancy over the whole array and over the interior only, and explain the difference. Then apply a global maximum to both and report what happens.

10. [Theory] Derive the receptive-field formula $2L + 1$ for L stacked 3×3 stride-one layers, and the corresponding recursion $r \mapsto 2r + 2$ when 2×2 pooling follows each layer. How many pooled layers are needed before an output sees a 256-pixel image, and how many unpooled ones?

11. [Theory] Derive (12.6). For the scalar test equation $\mathrm{d}y/\mathrm{d}t = \lambda y$ with λ real and negative, show that forward Euler satisfies $|y^L| \le |y^0|$ exactly when $\Delta t|\lambda| \le 2$. Then redo it for complex λ and confirm the region is the unit disc centred at -1. Which part of the complex plane is excluded for *every* $\Delta t > 0$, and what does a layer whose Jacobian has eigenvalues there do?

12. [Theory] An antisymmetric matrix has purely imaginary eigenvalues. Use this to show that the Haber–Ruthotto block of §12.3.2 sits exactly on the boundary of the stability region, so the forward map neither grows nor decays with depth. What does the network give up in exchange? *Hint:* count the degrees of freedom in $\frac{1}{2}(\boldsymbol{W} - \boldsymbol{W}^{\mathsf{T}})$ against those in $\boldsymbol{W}$.

13. [Theory] Verify that backpropagation is forward Euler on (12.7). Discretize the adjoint equation with step $-\Delta t$ and show that the resulting recursion for $\boldsymbol{a}^{\ell}$ is (12.2). Where does the product of Jacobians appear, and what is it the discrete version of?

14. [Code] Reproduce Figure 12.4b. For a scalar linear block, plot $|1 + \Delta t\lambda|^L$ against $\Delta t|\lambda|$ at $L = 40$ on log axes, and confirm the values 9×10^{-13} at 0.5, exactly 1 at 2, and 1.1×10^{7} at 2.5. Then repeat at $L = 10$ and $L = 200$ and describe what changes — and what does not.

15. [Theory] Derive (12.2) carefully, stating what is stored in the forward pass and what is required in the backward pass.

16. [Theory] Show that computing all L gradients by naive forward application of the chain rule costs $O(L^2)$ layer-derivative evaluations, while Algorithm 7 costs $O(L)$.

17. [Theory] Derive (12.3) in full, stating every assumption. Which assumption is violated by the He initialization used to produce Figure 12.2, and does it change the conclusion?

18. [Theory] Repeat the analysis for tanh and for ReLU. What is the analogous constant in each case, and what does it predict about trainable depth? Check your prediction against Figure 12.2.

19. [Theory] Derive $\partial y^i / \partial y^{i-1} = 1 + w^i \mathrm{d}\sigma^i$ for the ResNet layer (12.4) and explain precisely why the resulting product is $O(1)$ rather than geometrically small.

20. [Theory] Show that (12.4) is forward Euler applied to (12.5) with $\Delta t = 1$. What would the layer map look like for a second-order Runge–Kutta scheme, and what architecture does it correspond to?

21. [Code] Reproduce Figure 12.2. Build 14-layer networks of width 32 with sigmoid, ReLU, and residual layers, and at initialization measure $\|\partial J / \partial \boldsymbol{b}^\ell\|$ for each ℓ, averaged over 120 draws. Plot on a log scale, fit the sigmoid slope, and compare with $-\log_{10}(0.25)$.

Challenge

22. [Code] Train a small CNN on MNIST and inspect the first-layer filters. Compare them against the finite-difference stencils for ∂_x, ∂_y and Δ, measuring the angle between each learned filter and each stencil when both are flattened to vectors. How many of the learned filters are close to a derivative operator, and does that change if you train on a dataset whose features are not edges?

23. [Theory] The key idea of §12.4.1 calls weight sharing a prior. Make that precise using Corollary 6.3.2: estimate $\log H$ for a dense layer and for a convolutional layer on an $N \times N$ image, and give the resulting ratio of sample requirements. Then describe a task for which the prior is actively wrong, and say what the bound has to say about it — if anything.

24. [Code] Implement the adjoint method of §12.3.4 for a small neural ODE and compare it against ordinary backpropagation. Measure peak memory against depth for both, and confirm $O(1)$ versus $O(L)$. Then measure the *accuracy* of the gradient: by how much do the two disagree, and does the disagreement grow with depth? Relate your answer to the pitfall about reverse integration following (12.7).

25. [Theory] The continuum reading requires $\Delta t = T/L$. Take a trained ResNet with unit steps and L layers, and ask what ODE it approximates if the horizon is held at $T = 1$ rather than $T = L$. Show that the required weights scale like $1/L$, and comment on whether standard initialization schemes produce weights of that size.

26. [Theory] Show that the RNN gradient problem reduces to a statement about the spectral radius of $\boldsymbol{W}_h$, and state the condition for gradients to explode rather than vanish. Why does Figure 12.6 make this worse than Figure 12.2, even at equal depth?

27. [Theory] Prove Proposition 11.6.1 and explain in what sense LSTM does not *eliminate* the vanishing-gradient problem but makes it learnable. What must the network learn about f_t for a memory to survive T steps?

28. [Theory] Interpreting depth as time (Figure 12.3b), forward Euler is stable only for sufficiently small Δt. State the analogous condition on a residual network in terms of the eigenvalues of $\boldsymbol{W}^\ell$, and say what an unstable network would look like in practice.

29. [Theory] A convolution is a linear map, so a convolutional layer is a fully connected layer with tied and mostly-zero weights. Write the 1-D convolution of Definition 11.5.1 as a matrix and identify its structure. Using Exercise 5, explain the parameter saving in terms of that structure.

30. [Code] Train the plain network and the ResNet of Exercise 13 on MNIST for $L = 4, 8, 16, 32$. Plot final training loss against depth for both and identify the depth at which the plain network stops training altogether. Does the crossover match what Figure 12.2 predicts?

31. [Code] Implement (12.5) with an adaptive ODE solver in place of fixed unit steps, and compare accuracy and cost against the equivalent ResNet on the same task. Then halve Δt while doubling the number of layers and report what happens — this is a convergence study of the kind you would run on any discretized differential equation.

Part IV

Dimension Reduction and Generative Models

Chapter 13

Constrained Optimization, PCA, and Reduced-Order Modeling

Goal of this chapter

Every optimization so far has been unconstrained. This chapter adds constraints — and the first thing they buy is principal component analysis, which turns out to be an eigenvalue problem in disguise.
The chapter answers:

1. How does one minimize subject to a constraint?
2. Which directions in a dataset carry the most information, and in what precise sense?
3. How is the same computation used to replace a large differential equation by a small one?

The third question is why this chapter belongs in a scientific computing course. Principal component analysis, the singular value decomposition, and proper orthogonal decomposition are the same construction under three names, and the third is the standard tool for reducing the dimension of a discretized PDE. The chapter ends by observing that the whole apparatus is a *linear* autoencoder — which is where Chapter 14 begins.

§ 13.1 Lagrange multipliers

Theorem 13.1.1 Lagrange multiplier theorem

Consider

$$\min_{\boldsymbol{x}\in\mathbb{R}^n} f(\boldsymbol{x}) \qquad \textit{subject to} \qquad \boldsymbol{c}(\boldsymbol{x}) = \boldsymbol{0}, \qquad \boldsymbol{c}:\mathbb{R}^n \to \mathbb{R}^m,\ m < n,$$

and suppose a solution $\boldsymbol{x}^\star$ exists. Then there exists $\boldsymbol{\lambda} \in \mathbb{R}^m$ such that

$$\nabla_{\boldsymbol{x}} f(\boldsymbol{x}^\star) = \left[\nabla_{\boldsymbol{x}} \boldsymbol{c}(\boldsymbol{x}^\star)\right]^{\mathsf{T}} \boldsymbol{\lambda}, \qquad \textit{where} \qquad \nabla_{\boldsymbol{x}} \boldsymbol{c} = \begin{bmatrix} -\nabla_{\boldsymbol{x}} c_1 - \\ \vdots \\ -\nabla_{\boldsymbol{x}} c_m - \end{bmatrix} \in \mathbb{R}^{m\times n}.$$

ms. p.49

Equivalently, defining the *Lagrangian*

$$L(\boldsymbol{x}, \boldsymbol{\lambda}) := f(\boldsymbol{x}) + \boldsymbol{\lambda}^{\mathsf{T}} \boldsymbol{c}(\boldsymbol{x}),$$

which converts the constrained problem into an unconstrained one in $(\boldsymbol{x}, \boldsymbol{\lambda})$, stationarity reads

$$\nabla_{\boldsymbol{x}} L = \boldsymbol{0} \iff \nabla_{\boldsymbol{x}} f(\boldsymbol{x}^\star) + \left[\nabla_{\boldsymbol{x}} \boldsymbol{c}(\boldsymbol{x}^\star)\right]^{\mathsf{T}} \boldsymbol{\lambda}^\star = \boldsymbol{0}, \qquad \nabla_{\boldsymbol{\lambda}} L = \boldsymbol{0} \iff \boldsymbol{c}(\boldsymbol{x}^\star) = \boldsymbol{0}.$$

Geometrically, at the optimum the gradient of the objective is parallel to the gradient of the constraint. ms. p.49,50

Example 13.1.1 (Example)

For $\min_{x,y} f(x,y)$ subject to $c(x,y) = x^2 + y^2 - 1 = 0$, the solution is confined to the unit circle and satisfies $\nabla f(x^\star, y^\star) = \lambda \nabla c(x^\star, y^\star)$. ms. p.49

Let us see this geometrically. The theorem says the two gradients are parallel at the optimum. That is worth seeing: walk along the constraint curve, and you can keep descending until the objective's contour is *tangent* to it. At the tangency there is nowhere left to go without leaving the curve.

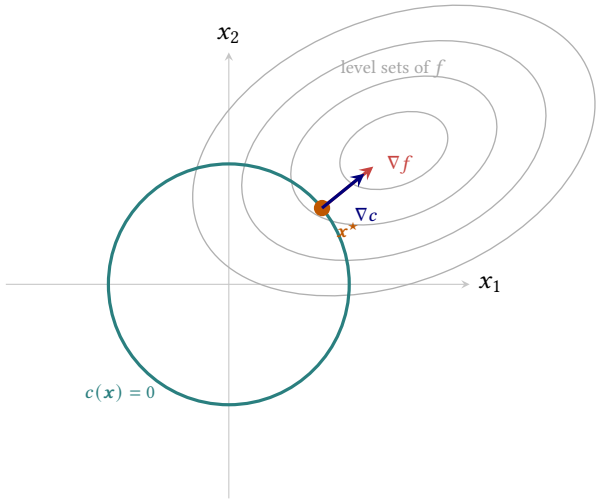


Figure 13.1: **At a constrained optimum the gradients are parallel.** Gray curves are level sets of the objective f, teal is the constraint set $c(x) = 0$. Moving along the constraint decreases f until the level set through the current point becomes *tangent* to it. Past that point any further motion along the constraint climbs again. Tangency means the two gradients point along the same line, $\nabla f = \lambda \nabla c$, which is Theorem 13.1.1 — and λ is precisely the ratio of their lengths.

§ 13.2 PCA: the first principal direction

We seek the direction $\hat{x}$ along which the data varies most. Assume the data has been centered, $\mathbb{E}[x^{(i)}] = 0$, so that $\mathbb{E}[x^{(i)} \cdot \hat{x}] = 0$ and hence $\mathbb{E}[(x^{(i)} \cdot \hat{x})^2]$ is precisely the variance of the projection of the data onto $\hat{x}$. Its Monte Carlo estimate is $\frac{1}{n}\sum_i (x^{(i)} \cdot \hat{x})^2$, so we solve ms. p.49

$$\max_{\hat{x}} \ \frac{1}{2}\sum_{i=1}^{n} |x^{(i)} \cdot \hat{x}|^2 = \max_{\hat{x}} \ \frac{1}{2}\|X^\mathsf{T}\hat{x}\|^2 \qquad \text{subject to} \qquad \|\hat{x}\|^2 = 1, \tag{13.1}$$

where $X = [x^{(1)}\ x^{(2)}\ \cdots\ x^{(n)}] \in \mathbb{R}^{d\times n}$ has the data as its columns, so that its column space is the data space. ms. p.48,49

The Lagrangian is $L(\hat{x}, \lambda) = \frac{1}{2}\|X^\mathsf{T}\hat{x}\|^2 - \frac{1}{2}\lambda(\hat{x}\cdot\hat{x} - 1)$, and the first-order conditions give

$$\nabla_{\hat{x}} L = 0 \iff XX^\mathsf{T}\hat{x} = \lambda\hat{x}, \qquad \nabla_\lambda L = 0 \iff \|\hat{x}\|^2 = 1.$$

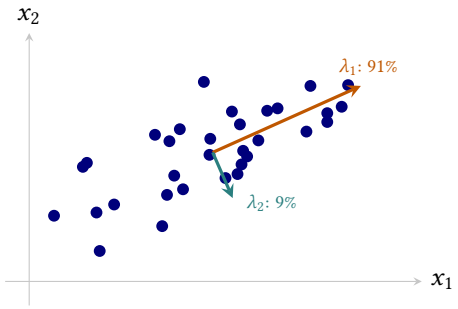
So $(\lambda^\star, \hat{x}^\star)$ is an eigenpair of the data covariance matrix $XX^\mathsf{T} \in \mathbb{R}^{d\times d}$. ms. p.50,51

Evaluating the objective at the optimum,

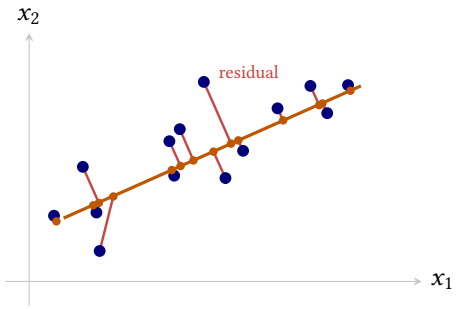
$$J(\hat{x}^\star) = \tfrac{1}{2}\hat{x}^{\star\mathsf{T}} XX^\mathsf{T}\hat{x}^\star = \tfrac{1}{2}\lambda^\star\|\hat{x}^\star\|^2 = \tfrac{1}{2}\lambda^\star,$$

so maximizing variance selects the *largest* eigenvalue. ms. p.50,51

Let us see this geometrically. Two readings of the same answer. Maximizing the spread of the projections is the same as minimizing the distance to the line — because by Pythagoras the two add up to a constant.



(a) The two principal directions.



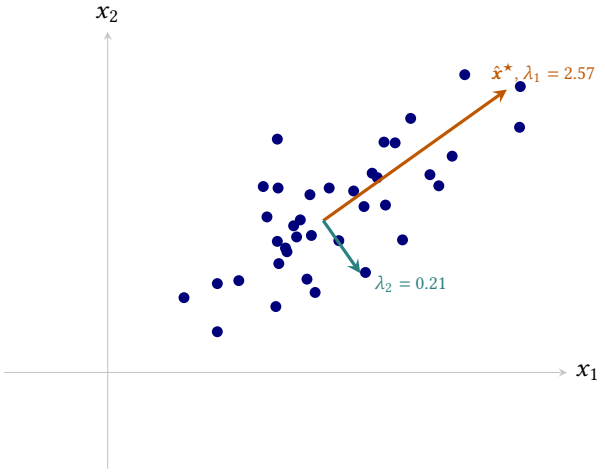
(b) Projection onto the first component.

Figure 13.2: **Principal component analysis, two ways. (a)** The eigenvectors of the data covariance, scaled by $\sqrt{\lambda_i}$. The first captures 91% of the variance and the second 9%: the cloud is nominally two-dimensional but nearly one-dimensional in content. **(b)** Projecting each point onto the first direction leaves the red residuals. Maximizing the spread *along* the orange line and minimizing the total squared length of the red segments are the same problem, because for each point the two are the legs of a right triangle whose hypotenuse — the distance to the mean — does not depend on the line. This is why (13.1) may be read either as a variance maximization or as a reconstruction-error minimization.

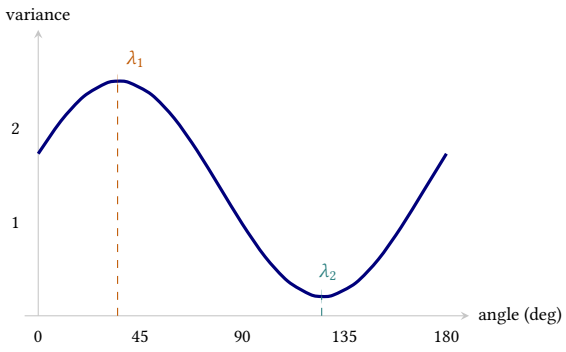
Key idea

The principal directions are the eigenvectors of the data covariance matrix, and the eigenvalues *are* the variances captured. The loss function and the eigenvalue are the same number.

Let us see this geometrically. The maximization (13.1) is over a single angle in the plane, so it can simply be plotted. Doing that shows the eigenvector is not merely *a* stationary point — it is the maximum, and the eigenvalue is the value attained there.



(a) Principal directions of a tilted cloud.



(b) Variance captured, against direction.

Figure 13.3: **PCA maximizes a variance, and the maximum is an eigenvalue. (a)** Forty points and the two principal directions, drawn with lengths proportional to $\sqrt{\lambda_i}$. **(b)** The objective of (13.1) evaluated at every unit direction: the peak sits at 35°, which is where the leading eigenvector points, and its height is $\lambda_1 = 2.57$. The trough at 125° — exactly perpendicular — is λ_2. So the eigenvalues are not a device for *finding* the answer; they *are* the answer, measured in units of variance.

§ 13.3 PCA: r principal directions

In practice we want $\hat{X} \in \mathbb{R}^{d\times r}$ with $r \ll n$ and $r < d$, solving

$$\max_{\hat{X}} \sum_{i=1}^{r} \|X^{\mathsf{T}}\hat{x}^{(i)}\|^2 = \|X^{\mathsf{T}}\hat{X}\|_F^2 \qquad \text{subject to} \qquad \hat{x}^{(i)}\cdot\hat{x}^{j} = \delta_{ij}, \quad \text{i.e.} \quad \hat{X}^{\mathsf{T}}\hat{X} = \mathrm{I},$$

with $\|\cdot\|_F$ the Frobenius norm. The Lagrangian carries one multiplier per constraint,

$$L(\hat{X}, \Lambda) = \|X^{\mathsf{T}}\hat{X}\|_F^2 - \sum_{i=1}^{r}\sum_{j=1}^{r} \lambda_{i,j}\big(\hat{x}^{(i)}\cdot\hat{x}^{j} - \delta_{ij}\big),$$

and stationarity yields

$$\boxed{XX^{\mathsf{T}}\hat{X} = \hat{X}\Lambda} \tag{13.2}$$

with Λ diagonal: each column of $\hat{X}$ is an eigenvector of XX^{T}, and the optimal value is the sum of the r largest eigenvalues. ms. p.51

Example 13.3.1 (PCA rediscovers the mode shapes)

A simply supported beam is instrumented with $d = 200$ displacement sensors and loaded $n = 400$ times with different random static loads. Nobody tells the analysis anything about beams. It receives a 400×200 table of numbers.

The physics, unknown to the method, is that the deflection is a combination of mode shapes $\sin(k\pi x)$ whose amplitudes fall off with k. Solving (13.2) on the measured data gives

r	λ_r	cumulative
1	1.01×10^{2}	83.06%
2	1.35×10^{1}	94.19%
3	3.44	97.03%
6	4.37×10^{-1}	99.31%
11	7.83×10^{-2}	99.95%

Two hundred sensors, and two directions already carry 94% of everything the beam does. But the striking part is not the compression. Comparing each computed direction against the physical mode it ought to be,

$$\big|\langle \hat{x}^{(k)},\, \sin(k\pi x)\rangle\big| = 0.9997,\ 0.9994,\ 0.9975,\ 0.9941,\ 0.9742 \qquad k = 1, \dots, 5,$$

where the vectors are normalized. *The eigenvectors of XX^{T} are the mode shapes*, recovered to four digits from measurements alone. PCA was given no differential equation, no boundary conditions and no notion of a mode; it found the modes because they are the directions along which this structure actually varies, and that is what (13.2) asks for.

This is the recurring reason PCA is worth computing on physical data even when storage is not the concern. The principal directions are frequently interpretable — modes, coherent structures, dominant reaction pathways — and the spectrum tells you how many of them matter before any model is built. Section 13.6 returns to this under the name proper orthogonal decomposition.

§ 13.4 Feature extraction and storage

Once $\hat{X}$ is computed, each sample is stored in reduced coordinates:

$$x^{(i)} \in \mathbb{R}^{d} \quad \longmapsto \quad x_r^{(i)} := \hat{X}^{\mathsf{T}}x^{(i)} \in \mathbb{R}^{r}.$$

Storage drops from $O(dn)$ for X to $O(dr + rn) = O(d + n)$ when $r = O(1)$. ms. p.51,52

Reconstruction is $x^{(i)} \approx \hat{X}\hat{X}^{\mathsf{T}}x^{(i)}$, with error

$$e^{(i)} := x^{(i)} - \hat{X}\hat{X}^{\mathsf{T}}x^{(i)} = (\mathrm{I} - \hat{X}\hat{X}^{\mathsf{T}})x^{(i)}, \qquad \sum_{i=1}^{n}\|e^{(i)}\|^2 = \|(\mathrm{I} - \hat{X}\hat{X}^{\mathsf{T}})X\|_F^2.$$

ms. p.52 If XX^{T} has only r nonzero eigenvalues the reconstruction is exact.

Example 13.4.1 (What the reduction is worth)

The $O(dr + rn)$ above is easier to feel with simulation-sized numbers. A three-dimensional unsteady solve on a modest mesh writes a double-precision field at every grid point at every saved step:

d	n	r	full X	basis + coefficients	factor
10^6	5000	20	40 GB	0.16 GB	249×
5×10^6	20 000	30	800 GB	1.20 GB	664×

Eight hundred gigabytes becomes something that fits in memory. Look at where the remaining cost sits, though: at $d = 5 \times 10^6$ the basis occupies 1.2 GB and the twenty thousand coefficient vectors occupy 4.8 MB between them. The basis is essentially the whole bill.

$$\text{Compression } \frac{dn}{dr + rn} \approx \frac{n}{r} \text{ whenever } d \gg r.$$

That ratio is worth committing to memory, because it says what the method is for. The saving does not come from each snapshot being cheaper to describe — it comes from describing *many* snapshots against one basis. Reducing a single field buys nothing; reducing a long time history, a parameter sweep, or an ensemble buys a factor of n/r. Once the basis is paid for, each additional snapshot costs r numbers, which is why a reduced-order model can afford to store trajectories that the full simulation could not.

Example 13.4.2 (The error you accepted when you chose r)

The reconstruction error above is not an estimate; it is the part of the spectrum you declined to keep. For the beam of Example 13.3.1,

$$\frac{1}{n}\sum_{i=1}^{n} \|\boldsymbol{e}^{(i)}\|^2 = \sum_{j>r} \lambda_j,$$

verified numerically at $r = 6$: both sides equal 8.3609×10^{-1}, agreeing to six digits. In relative terms the discarded tail is $1 - 99.31\% = 0.69\%$ of the energy, so the root-mean-square reconstruction error is $\sqrt{0.0069} = 8.3\%$ of the signal.

Key idea

Note which quantity is which. Choosing r by cumulative *energy* is choosing the squared error; the error itself is the square root, and is therefore much larger than the energy figure suggests. Keeping 99.3% of the energy leaves an 8.3% error, and keeping 99% leaves 10%. A spectrum plot that looks reassuring on an energy axis can correspond to visibly wrong reconstructions, which is why the error, and not the energy, is the thing to report.

§ 13.5 Singular value decomposition

ms. p.53 Forming and diagonalizing $XX^{\mathsf{T}} \in \mathbb{R}^{d\times d}$ costs $O(d^3)$, impractical when $d \gg 1$.

Theorem 13.5.1 SVD

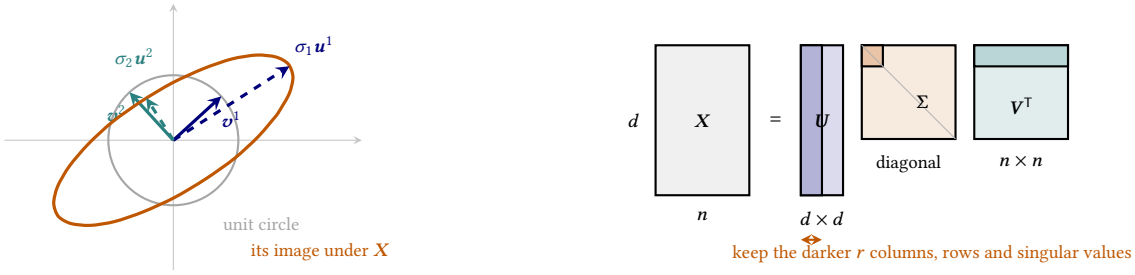
Any $X \in \mathbb{R}^{d\times n}$ factors as

$$X = U\Sigma V^{\mathsf{T}}, \qquad U^{\mathsf{T}}U = \mathrm{I}, \quad V^{\mathsf{T}}V = \mathrm{I},$$

ms. p.53 *with Σ diagonal and $\sigma_i \geq 0$.*

Let us see this geometrically. The theorem is a statement about three matrices, but it is also a statement about geometry:

every matrix, however it was built, acts by rotating, then stretching along perpendicular axes, then rotating again. The singular values are the stretches.



(a) Every matrix maps a circle to an ellipse.

(b) Truncating to rank r.

Figure 13.4: **The singular value decomposition, two ways. (a)** X carries the unit circle to an ellipse. The right singular vectors v^i are the directions on the circle that land on the ellipse's axes, and the axes themselves are $\sigma_i u^i$. Here $\sigma_1 = 2.12$ and $\sigma_2 = 0.77$, so the map stretches nearly three times as much along u^1 as along u^2: the ratio σ_1/σ_2 is the condition number of Figure 3.3b, read off geometrically. **(b)** The same statement as block shapes. Keeping only the darker r columns of U, the leading r singular values and the top r rows of $V^\top$ gives the best rank-r approximation of X — which is the truncation performed in Figure 13.6b, and the reason storage falls from $O(dn)$ to $O(r(d+n))$.

Consequently

$$XX^\top = U\Sigma V^\top V\Sigma^\top U^\top = U\Sigma\Sigma^\top U^\top,$$

so the eigenvectors of $XX^\top$ are the left singular vectors of X and its eigenvalues are σ_i^2. The r principal components are the r left singular vectors,

$$\sum_{i=1}^{r} u^i \, \sigma_i^2 \, (u^i)^\top,$$

and r is chosen by inspecting where the singular value spectrum $\sigma_1 \geq \sigma_2 \geq \cdots$ falls off. ms. p.53

Key idea

PCA $\iff$ SVD $\iff$ Proper Orthogonal Decomposition (POD).

Three names, three communities, one computation. ms. p.53

§ 13.6 Proper orthogonal decomposition and reduced-order modeling

Consider a linear dynamical system

$$\frac{dx}{dt} = Ax + Bu, \tag{13.3}$$

with u a control input.

Example 13.6.1 (Where such a system comes from)

Systems of the form (13.3) are not exotic — discretizing a linear PDE in space and leaving time continuous produces one automatically. This is the *method of lines*.

Take the advection–diffusion problem of Definition 13.8.1. Replacing the spatial derivatives by centered differences on $d = 198$ interior nodes, but *not* discretizing time, gives

$$\frac{du_i}{dt} = \nu \frac{u_{i+1} - 2u_i + u_{i-1}}{\Delta x^2} - c \frac{u_{i+1} - u_{i-1}}{2\,\Delta x}, \qquad i = 1, \ldots, d,$$

which is exactly $\frac{d\boldsymbol{x}}{dt} = \boldsymbol{A}\boldsymbol{x}$ with $\boldsymbol{x} = (u_1, \dots, u_d)^{\mathsf{T}}$ and $\boldsymbol{A}$ the tridiagonal matrix

$$\boldsymbol{A} = \begin{bmatrix} \beta & \gamma & & & \\ \alpha & \beta & \gamma & & \\ & \ddots & \ddots & \ddots & \\ & & \alpha & \beta \end{bmatrix}, \qquad \begin{aligned} \alpha &= \frac{\nu}{\Delta x^2} + \frac{c}{2\Delta x} = 247.95, \\ \beta &= -\frac{2\nu}{\Delta x^2} = -316.81, \\ \gamma &= \frac{\nu}{\Delta x^2} - \frac{c}{2\Delta x} = 68.85. \end{aligned}$$

Here $\boldsymbol{B}\boldsymbol{u} = \boldsymbol{0}$; a source term, a boundary actuator, or an imposed inflow would supply it. Applying forward Euler to this system recovers (13.5), so the scheme used to generate Figure 13.6 is one particular time discretization of (13.3).

Note:-

Note that $\alpha \neq \gamma$: the advection term is *antisymmetric* and breaks the symmetry that pure diffusion would have. So $\boldsymbol{A}$ is not normal, its eigenvalues are complex (here with real parts in $[-607, -26]$ and imaginary parts up to ± 125), and the stability guarantee of Exercise 18 does not apply. The larger c/ν, the worse this gets — which is why convection-dominated problems are the hard case for reduced-order modeling.

Solve (13.3) for a few representative controls and collect the solution snapshots

$$\boldsymbol{X} \coloneqq \begin{bmatrix} \boldsymbol{x}(t_1) & \boldsymbol{x}(t_2) & \boldsymbol{x}(t_3) & \cdots \end{bmatrix}.$$

Extract the r most dominant components by PCA/SVD/POD, giving $\boldsymbol{V} \in \mathbb{R}^{d \times r}$ with $r \ll d$, and write $\boldsymbol{x} \approx \boldsymbol{V}\boldsymbol{x}_r$ with $\boldsymbol{x}_r \in \mathbb{R}^r$.

Substituting into (13.3) leaves a residual, which the Galerkin condition requires to be orthogonal to the reduced basis:

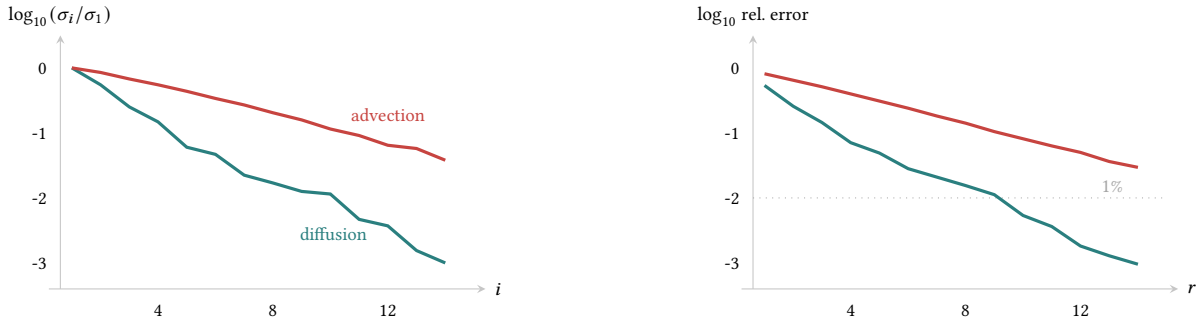
$$\boldsymbol{V}^{\mathsf{T}}\Big(\frac{d}{dt}\big(\boldsymbol{V}\boldsymbol{x}_r(t)\big) - \boldsymbol{A}\boldsymbol{V}\boldsymbol{x}_r - \boldsymbol{B}\boldsymbol{u}\Big) = \boldsymbol{0},$$

and since $\boldsymbol{V}^{\mathsf{T}}\boldsymbol{V} = \mathrm{I}$,

$$\frac{d\boldsymbol{x}_r}{dt} = \boldsymbol{V}^{\mathsf{T}}\boldsymbol{A}\boldsymbol{V}\boldsymbol{x}_r + \boldsymbol{V}^{\mathsf{T}}\boldsymbol{B}\boldsymbol{u}.$$

ms. p.54 This is a system of dimension r rather than d: a *reduced-order model*.

Let us see this geometrically. The whole method rests on the singular values decaying. Whether they do is a property of the physics, not of the algorithm — and for the two regimes below the answer is very different.

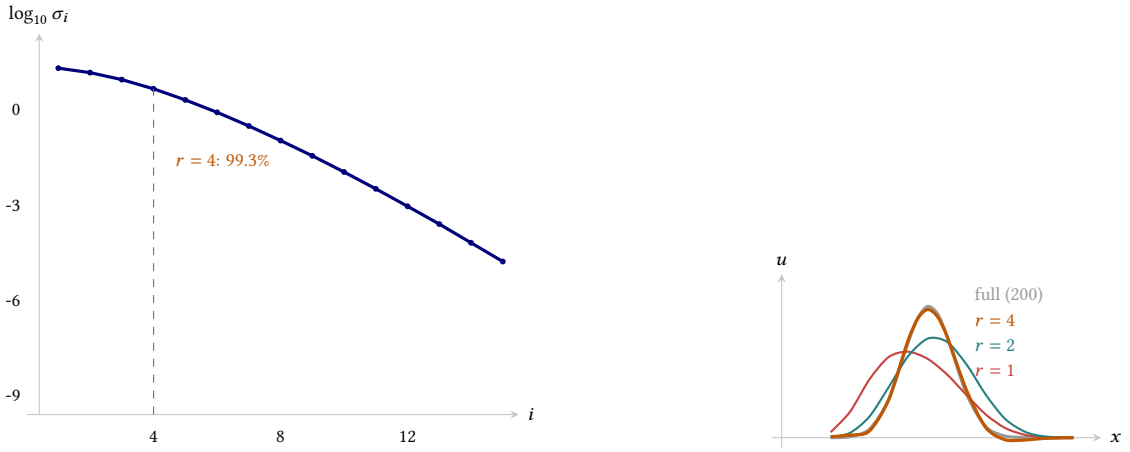


(a) Singular value spectra.

(b) Reconstruction error against r.

Figure 13.5: **When does model reduction pay?** A one-dimensional advection–diffusion equation solved for twelve initial conditions, giving a snapshot matrix with 200 rows. **(a)** The singular values, scaled by σ_1. When diffusion dominates (teal) they fall three decades in fourteen modes; when advection dominates (red) they fall barely one and a half. **(b)** The consequence: reaching 1% relative error needs $r = 4$ modes in the diffusive case and $r = 10$ in the advective one. A travelling feature must be represented by many global modes, since no small set of fixed spatial profiles can encode a moving peak. This is a genuine limitation of linear reduction, not a defect of the implementation — and it is the standard motivation for the *nonlinear* reduction of Section 13.7.

Let us see this geometrically. This is the payoff, and it is worth a concrete case. Take a one-dimensional advection–diffusion problem, discretize it on 200 grid points, march it forward, and keep 70 snapshots. The snapshot matrix is 200×70. How many modes does it really need?



(a) The singular values collapse.

(b) One snapshot, reconstructed from r modes.

Figure 13.6: **Proper orthogonal decomposition of a travelling pulse.** A one-dimensional advection–diffusion equation on 200 grid points, sampled at 70 times. **(a)** The singular values of the snapshot matrix fall off geometrically: by $i = 8$ they are below 10^{-1} of the first, and the first four already carry 99.3% of the energy $\sum \sigma_i^2$. **(b)** A representative snapshot reconstructed from the leading r modes. One mode gets the location roughly right and the shape wrong; two are still too broad; four are visually indistinguishable from the full solution, at a relative error of 0.08. A state vector of 200 numbers has been replaced by 4 — and it is the *same* computation as Figure 13.3, applied to solution snapshots instead of data points.

§ 13.7 Autoencoders

The maps $\boldsymbol{x} \mapsto \boldsymbol{V}^{\mathsf{T}}\boldsymbol{x} = \boldsymbol{x}_r$ and $\boldsymbol{x}_r \mapsto \boldsymbol{V}\boldsymbol{x}_r$ are a *linear encoder* and *linear decoder*. Replacing them by deep networks $E_{\boldsymbol{\phi}}$ and $D_{\boldsymbol{\theta}}$ gives the *autoencoder*, trained by minimizing the reconstruction error

$$\min_{\boldsymbol{\theta},\boldsymbol{\phi}} \frac{1}{n} \sum_{i=1}^{n} \|\boldsymbol{x}^{(i)} - D_{\boldsymbol{\theta}}\big(E_{\boldsymbol{\phi}}(\boldsymbol{x}^{(i)})\big)\|^2. \tag{13.4}$$

This is unsupervised learning: no labels appear. Applications include data reduction and denoising.

ms. p.54,55

Let us see this geometrically. Drawn as a diagram, the two are the same shape. Only the boxes differ.

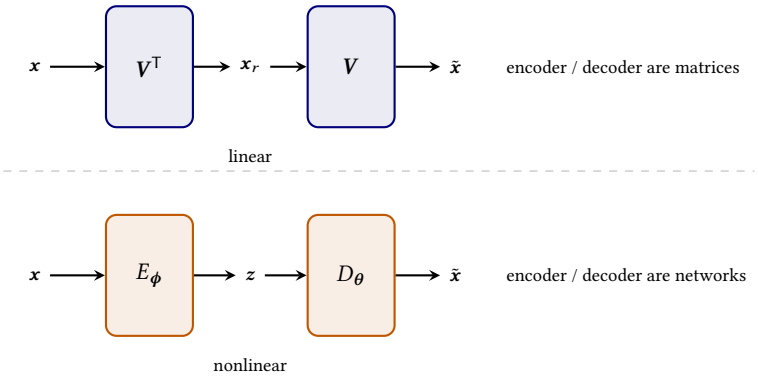


Figure 13.7: **PCA is the linear autoencoder.** Both diagrams compress $\boldsymbol{x}$ to a small code and reconstruct it, and both are trained by minimizing the same reconstruction error (13.4). In the linear case the encoder and decoder are the matrices $\boldsymbol{V}^\top$ and $\boldsymbol{V}$, and the optimum can be written down — it is the top r principal components. Replace them by networks and the optimum can only be found by gradient descent, but the model is no longer confined to a *subspace*: it can follow a curved manifold, which is exactly what the travelling feature of Figure 13.5b needs.

Let us see this geometrically. An autoencoder is the same squeeze drawn as a network: force the data through a narrow layer and ask that it survive the trip. When the two maps are linear, the narrow layer is a subspace — and the best subspace is the one PCA already found.

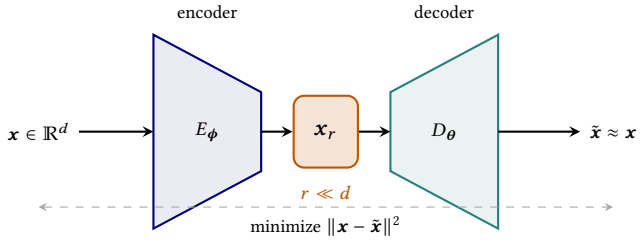


Figure 13.8: **The autoencoder.** Data is compressed to r numbers and expanded back, and the loss (13.4) asks only that the round trip be faithful. No labels appear anywhere — this is unsupervised learning. If E_ϕ and D_θ are constrained to be linear, the optimum reproduces the PCA projection of Section 12.4, and Figure 13.6 *is* such an autoencoder with $E = \boldsymbol{V}^\top$ and $D = \boldsymbol{V}$. Letting them be networks buys curved latent coordinates, at the cost of the closed form and of the orthogonality that made r easy to choose.

Key idea

PCA is the linear autoencoder. Everything in Chapter 14 follows from asking what happens when the latent code is made probabilistic rather than deterministic.

§ 13.8 Exercises

The code problems use `sklearn.datasets.fetch_california_housing` (Section C.5) and the model problem below. Enable `x64` (Section C.3) — forming $\boldsymbol{X}\boldsymbol{X}^\top$ squares the condition number, exactly as in Figure 3.3b.

The model problem

Several exercises below build on one advection–diffusion problem. It is the problem used to produce Figures 13.6 and 13.5, and the parameters are given in full so that those figures can be reproduced exactly.

Definition 13.8.1: Advection–diffusion model problem

Find $u(x,t)$ on $x \in [0,1]$, $t \in [0,T]$ satisfying

$$\frac{\partial u}{\partial t} + c\,\frac{\partial u}{\partial x} = \nu\,\frac{\partial^2 u}{\partial x^2}, \qquad u(0,t) = u(1,t) = 0, \qquad u(x,0) = \exp\!\left(-\left(\frac{x-0.18}{0.055}\right)^2\right),$$

with advection speed $c = 0.9$ and diffusivity $\nu = 0.004$. The initial condition is a narrow Gaussian pulse; it is transported to the right at speed c while spreading at a rate set by ν.

Discretize with $n_x = 200$ equally spaced grid points, $\Delta x = 1/199$, using centered differences in space and forward Euler in time:

$$u_i^{k+1} = u_i^k + \Delta t\left[\nu\,\frac{u_{i+1}^k - 2u_i^k + u_{i-1}^k}{\Delta x^2} - c\,\frac{u_{i+1}^k - u_{i-1}^k}{2\,\Delta x}\right], \qquad u_0^k = u_{199}^k = 0. \tag{13.5}$$

Take $\Delta t = 2 \times 10^{-4}$ and march 1400 steps, so $T = 0.28$ and the pulse travels a distance $cT = 0.252$. Store every twentieth step, giving a snapshot matrix $\boldsymbol{X} \in \mathbb{R}^{200\times 70}$.

Common pitfall

Scheme (13.5) is explicit and therefore only conditionally stable. Three numbers must be checked before trusting any output:

$$\underbrace{\frac{\nu\,\Delta t}{\Delta x^2} = 0.032,}_{\le 1/2} \qquad \underbrace{\frac{c\,\Delta t}{\Delta x} = 0.036,}_{\le 1} \qquad \underbrace{\frac{c\,\Delta x}{\nu} = 1.13\,.}_{\le 2}$$

The first two are the usual diffusion and Courant conditions. The third is the *cell Péclet number*: centered differencing of the advection term produces spurious oscillations once it exceeds 2, however small Δt is. The parameters above satisfy all three with margin, but a student who coarsens the grid to $n_x = 50$ will violate the third and see the solution develop wiggles — which is a failure of the discretization, not of POD.

Warm-up

1. [Theory] State the constraint qualification under which Theorem 13.1.1 holds, and give an example where it fails.

2. [Theory] Using Figure 13.1, explain in one sentence why a point at which ∇f and ∇c are *not* parallel cannot be optimal. What does the multiplier λ measure?

3. [Theory] For $\boldsymbol{X}\boldsymbol{X}^{\mathsf{T}} = \left[\begin{smallmatrix} 5 & 2 \\ 2 & 2 \end{smallmatrix}\right]$, compute both eigenvalues and eigenvectors by hand. Which direction is the first principal component, and how much of the total variance does it carry?

4. [Theory] In Figure 13.3b the maximum and minimum of the variance occur exactly 90° apart. Prove that this must happen in two dimensions, and say what replaces the statement in $\mathbb{R}^d$.

5. [Theory] From Figure 13.6a, estimate how many modes are needed for 99.9% of the energy. Why is the vertical axis logarithmic, and what would the plot look like if the snapshots were random noise instead of PDE solutions?

6. [Code] Perform PCA on the California housing features by eigendecomposition of the covariance matrix and by SVD of the centered data matrix. Confirm the two agree to machine precision, and time both as the number of features grows.

Core

7. [Code] Reproduce Example 13.3.1. Build snapshots as random combinations of $\sin(k\pi x)$ on $d = 200$ points with amplitudes falling like $k^{-3/2}$, add a little sensor noise, and solve (13.2). Confirm the cumulative energies 83.1%, 94.2%, 97.0% and the inner products against the true modes. Then break it: give two modes nearly equal amplitudes and report what happens to the corresponding principal directions. Why does the method stop recovering them individually?

8. [Theory] Prove the identity used in Example 13.4.2, that $\sum_i \|\boldsymbol{e}^{(i)}\|^2 = n\sum_{j>r}\lambda_j$, from (13.2) and the orthogonality of $\hat{\boldsymbol{X}}$. Where is it used that the columns are *eigenvectors* rather than merely orthonormal?

9. [Theory] Example 13.4.1 claims the compression ratio is $\approx n/r$ when $d \gg r$. Derive it, then find the d at which the approximation is wrong by more than 10% for $n = 5000$, $r = 20$. For what kind of problem would you be in that regime?

10. [Theory] Complete the derivation of (13.1), including the step showing the optimal value equals $\frac{1}{2}\lambda^\star$.

11. [Theory] Show that the second principal direction is the eigenvector of the second largest eigenvalue, by adding orthogonality to the first as a constraint.

ms. p.51 12. [Theory] Derive (13.2) and show that $\boldsymbol{\Lambda}$ may be taken diagonal. *This was assigned as homework in lecture.*

13. [Theory] Show that the reconstruction error equals the sum of the discarded eigenvalues, $\sum_{i>r} \lambda_i$, and hence that PCA minimizes it among all rank-r projections (Eckart–Young). Check the claim numerically against Figure 13.6a.

14. [Theory] Verify $\boldsymbol{X}\boldsymbol{X}^\mathsf{T} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{\Sigma}^\mathsf{T}\boldsymbol{U}^\mathsf{T}$ and deduce the relationship between singular values and eigenvalues. Explain why computing the SVD of $\boldsymbol{X}$ is numerically preferable to forming $\boldsymbol{X}\boldsymbol{X}^\mathsf{T}$, and connect this to Figure 3.3b.

15. [Theory] Prove that a linear autoencoder (13.4) with latent dimension r has an optimum whose encoder spans the same subspace as the top r principal components. Explain why the optimum is not unique, and what that non-uniqueness means for interpreting the learned weights.

16. [Code] Implement Definition 13.8.1 with the discretization (13.5) and verify the three stability numbers before running. Collect the 70 snapshots and reproduce Figure 13.6: plot $\log_{10}\sigma_i$, the cumulative energy $\sum_{i\le r}\sigma_i^2/\sum_i\sigma_i^2$, and one snapshot reconstructed from $r = 1, 2, 4$ modes. You should find 99.3% of the energy in the first four modes and a relative error of about 0.08 at $r = 4$; report your own numbers and say whether they agree.

Challenge

17. [Code] The key idea of Example 13.4.2 warns that energy and error are not the same number. Make the point visually: on the beam data, reconstruct a single snapshot at r chosen to retain 90%, 99% and 99.9% of the energy, and plot each against the truth. Report the relative error in each case alongside the energy retained, and decide which of the three you would be willing to call a reconstruction.

18. [Theory] Show that Galerkin projection preserves stability when $\boldsymbol{A}$ is symmetric negative definite. The operator of Definition 13.8.1 is *not* symmetric — the advection term contributes an antisymmetric part — so give a counterexample for non-normal $\boldsymbol{A}$ and explain why this is a genuine hazard in convection-dominated problems. What happens to the danger as $\nu \to 0$ with c fixed?

19. [Theory] Return to Exercise 14 of Chapter 6 and complete it using Theorem 13.1.1: prove the equivalence of the penalized and constrained forms of ridge regression, and give the relationship between λ and r.

20. [Theory] The reduced model $\frac{d\boldsymbol{x}_r}{dt} = \boldsymbol{V}^\mathsf{T}\boldsymbol{A}\boldsymbol{V}\boldsymbol{x}_r + \boldsymbol{V}^\mathsf{T}\boldsymbol{B}\boldsymbol{u}$ is exact only if the true trajectory lies in $\operatorname{span}(\boldsymbol{V})$. Bound the error committed when it does not, in terms of the discarded singular values. What does this say about using a basis built from one set of controls $\boldsymbol{u}$ to predict the response to a different one?

21. [Theory] PCA is a *linear* method, yet Figure 1.2b showed data lying near a line and Figure 1.6a showed data lying on a circle. Explain why PCA succeeds on the first and fails on the second, and describe what a nonlinear autoencoder could do that PCA cannot. What does it lose in exchange?

22. [Code] Train a linear autoencoder and a deep autoencoder on the snapshot matrix of Definition 13.8.1 across a range of latent dimensions. Plot reconstruction error against r for both, and confirm the linear one matches PCA exactly. At which r does the nonlinear model overtake it, and how much training does that require?

23. [Code] Build the Galerkin reduced-order model of Section 13.6 for Definition 13.8.1. Write (13.5) as $\frac{d\boldsymbol{x}}{dt} = \boldsymbol{A}\boldsymbol{x}$ by assembling the 200×200 matrix $\boldsymbol{A}$ explicitly, project it to $\boldsymbol{A}_r = \boldsymbol{V}^\mathsf{T}\boldsymbol{A}\boldsymbol{V}$, and march the reduced system with the same Δt. Compare reduced and full solutions for $r = 2, 4, 8$, plot error against r alongside the discarded singular-value energy, and report the speedup. Then change the initial pulse width from 0.055 to 0.11 — a state not represented among the snapshots — and comment on what happens to the reduced model.

Chapter 14

Generative Models

Goal of this chapter

Chapter 13 ended with an autoencoder: compress, then reconstruct. That model can *recover* data it is shown. It cannot produce data it has never seen, because nothing tells it which latent codes are plausible.
This chapter fixes that, and answers:

1. What must be added to an autoencoder before it can generate?
2. The resulting likelihood is intractable — what do we optimize instead, and how much do we lose?
3. How is the resulting objective differentiated, given that it contains a sampling step?
4. What happens when the single latent variable is replaced by a long chain of them?

The answer to the second is the evidence lower bound, which Chapter 9 already used once: EM tightened it in the E step and raised it in the M step. Here the same inequality is optimized by gradient descent instead, and the last section pushes it to its limit — a chain of hundreds of latent variables, which is a diffusion model.

§ 14.1 From autoencoder to VAE

The autoencoder of Chapter 13 maps $\boldsymbol{x} \to E_{\boldsymbol{\phi}}(\boldsymbol{x}) = \boldsymbol{z} \to D_{\boldsymbol{\theta}}(\boldsymbol{z}) = \tilde{\boldsymbol{x}}$ deterministically. The variational autoencoder replaces both maps by distributions: ms. p.55

$$q_{\boldsymbol{\phi}}(\boldsymbol{z} \mid \boldsymbol{x}) \text{ (encoder)}, \qquad p_{\boldsymbol{\theta}}(\boldsymbol{x} \mid \boldsymbol{z}) \text{ (decoder)},$$

with the specific choices

$$p(\boldsymbol{z}) = \mathcal{N}(\boldsymbol{z}; \mathbf{0}, \mathrm{I}), \qquad q_{\boldsymbol{\phi}}(\boldsymbol{z} \mid \boldsymbol{x}) = \mathcal{N}\big(\boldsymbol{z};\ \boldsymbol{\mu}_{\boldsymbol{\phi}}(\boldsymbol{x}),\ \sigma^2_{\boldsymbol{\phi}}(\boldsymbol{x})\mathrm{I}\big),$$

where $\boldsymbol{\mu}_{\boldsymbol{\phi}}$ and $\sigma_{\boldsymbol{\phi}}$ are themselves neural networks. The quantity $p(\boldsymbol{x})$ is called the *evidence*. ms. p.55

Let us see this geometrically. The change from Chapter 13 is one arrow. Where the autoencoder produced a *point* in latent space, the VAE produces a *distribution* — and that is what makes the latent space something we can sample from.



(a) Autoencoder: z is a point.

(b) VAE: z is a distribution, then sampled.

Figure 14.1: **One arrow separates compression from generation. (a)** The autoencoder of Figure 13.8 maps each x to a single code. Feed it a code it has never seen and the output is unconstrained: nothing in the training told it which codes are legitimate. **(b)** The VAE maps x to a *density* over codes, and the training objective additionally pulls that density toward a fixed prior $p(z) = \mathcal{N}(0, I)$. Once it has, sampling $z \sim \mathcal{N}(0, I)$ and decoding gives new data — the encoder is not needed at all. Everything in this chapter is machinery for making that second picture trainable.

§ 14.2 The evidence lower bound

Theorem 14.2.1 ELBO

$$\log p(x) \;\geq\; \mathbb{E}_{q_\phi(z|x)}\left[\log \frac{p(x,z)}{q_\phi(z \mid x)}\right] \;=:\; \text{ELBO}.$$

Proof: [First proof: easy but not intuitive] Insert the encoder into the marginalization and apply Jensen's inequality $f(\int g) \leq \int f(g)$ for concave log:

$$\log p(x) = \log \int p(x,z)\,dz = \log \int \frac{p(x,z)}{q_\phi(z \mid x)} q_\phi(z \mid x)\,dz = \log \mathbb{E}_{q_\phi}\left[\frac{p(x,z)}{q_\phi}\right] \;\geq\; \mathbb{E}_{q_\phi}\left[\log \frac{p(x,z)}{q_\phi}\right].$$

ms. p.55,56 ■

Proof: [Second proof: easy and intuitive] Since $\int q_\phi(z \mid x)\,dz = 1$,

$$\log p(x) = \int \log p(x)\, q_\phi(z \mid x)\,dz = \int \log \frac{p(x,z)}{p(z \mid x)} q_\phi\,dz = \int \log\left[\frac{p(x,z)}{q_\phi} \cdot \frac{q_\phi}{p(z \mid x)}\right] q_\phi\,dz,$$

that is,

$$\log p(x) = \underbrace{\mathbb{E}_{q_\phi}\left[\log \frac{p(x,z)}{q_\phi}\right]}_{\text{ELBO}} + \underbrace{D_{\mathrm{KL}}\big(q_\phi(z \mid x) \,\|\, p(z \mid x)\big)}_{\geq\, 0}. \tag{14.1}$$

ms. p.56 The inequality follows immediately, and the second proof implies the first. ■

Note:-

This is the second evidence lower bound in these notes. The first was (9.5) in Chapter 9, where maximizing it *was* the EM algorithm: the E step tightened the bound, the M step raised it. The two moves below are the same, with a neural network in place of the responsibilities.

Key idea

Equation (14.1) says the gap in Theorem 14.2.1 is *exactly* $D_{\mathrm{KL}}\big(q_\phi(z \mid x) \,\|\, p(z \mid x)\big)$. Hence

$$\max_\phi \text{ ELBO} \quad\Longleftrightarrow\quad \min_\phi D_{\mathrm{KL}}\big(q_\phi(z \mid x) \,\|\, p(z \mid x)\big),$$

ms. p.56 so maximizing the bound is the same as pushing the encoder as close as possible to the true posterior.

Let us see this geometrically. Equation (14.1) is a decomposition, and decompositions are worth drawing. The evidence is fixed — it does not depend on $\boldsymbol{\phi}$ at all — so anything the encoder gains, the gap loses.

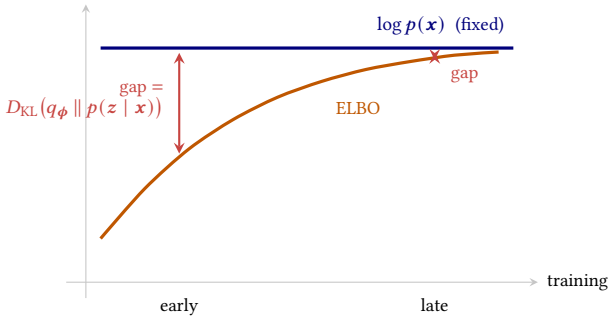


Figure 14.2: **Maximizing the bound closes the gap.** The evidence $\log p(\boldsymbol{x})$ (navy) is a property of the data and the decoder, not of the encoder. By (14.1) it splits exactly into the ELBO plus $D_{\mathrm{KL}}\big(q_{\boldsymbol{\phi}}(\boldsymbol{z} \mid \boldsymbol{x}) \,\|\, p(\boldsymbol{z} \mid \boldsymbol{x})\big)$, and the KL term is non-negative. So raising the ELBO over $\boldsymbol{\phi}$ can only shrink the gap: the encoder is being pushed toward the true posterior, which we could not compute directly. This is why an intractable problem became a tractable one — and it is the same manoeuvre as the E step of Algorithm 5, which sets the gap to zero exactly rather than descending toward it.

§ 14.3 Training the VAE

Assuming $p(\boldsymbol{x}, \boldsymbol{z}) = p_{\boldsymbol{\theta}}(\boldsymbol{x} \mid \boldsymbol{z})p(\boldsymbol{z})$, the ELBO splits:

$$\mathrm{ELBO} = \underbrace{\mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}\big[\log p_{\boldsymbol{\theta}}(\boldsymbol{x} \mid \boldsymbol{z})\big]}_{\text{reconstruction}} - \underbrace{D_{\mathrm{KL}}\big(q_{\boldsymbol{\phi}}(\boldsymbol{z} \mid \boldsymbol{x}) \,\|\, p(\boldsymbol{z})\big)}_{\text{prior matching}}. \tag{14.2}$$

Training maximizes this over $\boldsymbol{\phi}$ and $\boldsymbol{\theta}$ jointly, the expectation being estimated by Monte Carlo over M samples per data point. ms. p.56

14.3.1 The reparameterization trick

The obstruction is that $\boldsymbol{z}^j(\boldsymbol{x}^{(i)}) \sim \mathcal{N}(\boldsymbol{\mu}_{\boldsymbol{\phi}}(\boldsymbol{x}^{(i)}), \sigma^2_{\boldsymbol{\phi}}(\boldsymbol{x}^{(i)})\mathrm{I})$ depends on $\boldsymbol{\phi}$ through the *sampling operation*, which is not differentiable. Instead define

$$\boxed{\boldsymbol{z}^j(\boldsymbol{x}^{(i)}) \coloneqq \boldsymbol{\mu}_{\boldsymbol{\phi}}(\boldsymbol{x}^{(i)}) + \sigma_{\boldsymbol{\phi}}(\boldsymbol{x}^{(i)})\,\boldsymbol{\varepsilon}^j, \qquad \boldsymbol{\varepsilon}^j \sim \mathcal{N}(\mathbf{0}, \mathrm{I})} \tag{14.3}$$

which has the same distribution — if $\boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \mathrm{I})$ and $\boldsymbol{z} = \boldsymbol{\mu} + \sigma\boldsymbol{\varepsilon}$ then $\boldsymbol{z} \sim \mathcal{N}(\boldsymbol{\mu}, \sigma^2)$ — but moves all randomness into a parameter-free variable, restoring differentiability with respect to $\boldsymbol{\phi}$. ms. p.57

Let us see this geometrically. The obstruction is easiest to see as a graph. Gradients flow backwards along arrows; a sampling step is not a function of its inputs, so there is no arrow to flow along.



(a) Sampling breaks the path.

(b) Reparameterizing restores it.

Figure 14.3: **The reparameterization trick, as a computational graph.** **(a)** Drawing $\boldsymbol{z} \sim \mathcal{N}(\boldsymbol{\mu}_{\boldsymbol{\phi}}, \sigma^2_{\boldsymbol{\phi}})$ is a *sampling* operation, not a differentiable function of $\boldsymbol{\mu}$ and σ, so reverse-mode differentiation (Chapter 12) has nothing to traverse and $\partial J/\partial\boldsymbol{\phi}$ is unavailable. **(b)** Writing $\boldsymbol{z} = \boldsymbol{\mu}_{\boldsymbol{\phi}} + \sigma_{\boldsymbol{\phi}}\boldsymbol{\varepsilon}$ with $\boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \mathrm{I})$ leaves the distribution of $\boldsymbol{z}$ unchanged but moves all randomness into a leaf node that carries no parameters. The path from J back to $\boldsymbol{\phi}$ is now made of ordinary arithmetic.

14.3.2 Generation

Once trained, new data is produced by sampling $z^k \sim p(z) = \mathcal{N}(\mathbf{0}, \mathrm{I})$ and decoding through $p_\theta(x \mid z^k)$ — no encoder required.

ms. p.57

§ 14.4 Hierarchical VAE

Replace the single latent by a Markov chain $z_1, \dots, z_T$:

$$q_\phi(z_{1:T} \mid x) = q_\phi(z_1 \mid x) \prod_{t=2}^{T} q_\phi(z_t \mid z_{t-1}), \qquad p(x, z_{1:T}) = p(z_T) \Big[\prod_{t=2}^{T} p_\theta(z_{t-1} \mid z_t) \Big] p_\theta(x \mid z_1),$$

and the same bound applies with z replaced by $z_{1:T}$:

$$\log p(x) \;\geq\; \mathbb{E}_{q_\phi(z_{1:T} \mid x)} \left[\log \frac{p(x, z_{1:T})}{q_\phi(z_{1:T} \mid x)} \right].$$

ms. p.58

§ 14.5 Variational diffusion models

The development below follows Luo [6], which derives the variational diffusion model as a restricted Markovian HVAE and is the source these lectures were built from.

ms. p.59 A VDM is a special case of the HVAE with four additional postulates:

1. $\dim(z_t) = \dim(x)$;
2. $x_0 = x$ and $x_t = z_t$ for $t = 1, \dots, T$;
3. $p(x_T) = \mathcal{N}(x_T; \mathbf{0}, \mathrm{I})$;
4. the encoder is *fixed*, not learned:

$$q(x_t \mid x_{t-1}) = \mathcal{N}\big(x_t;\ \sqrt{\alpha_t}\, x_{t-1},\ (1-\alpha_t)\mathrm{I}\big), \qquad 0 \leq \alpha_t \leq 1,\ \ \alpha_t \to 0 \text{ as } t \to T.$$

ms. p.59,60 The forward direction adds Gaussian noise (*noisification*); the reverse direction, parameterized by θ, denoises.

Proposition 14.5.1 Closed form for $q(x_t \mid x_0)$
Writing $\bar\alpha_t := \prod_{i=1}^{t} \alpha_i$,

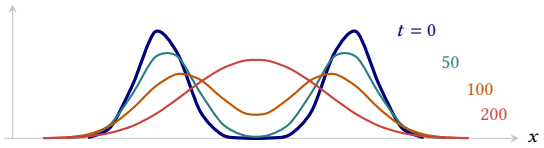
$$x_t = \sqrt{\bar\alpha_t}\, x_0 + \sqrt{1-\bar\alpha_t}\, \varepsilon_0, \qquad \text{so} \qquad x_t \sim \mathcal{N}\big(\sqrt{\bar\alpha_t}\, x_0,\ (1-\bar\alpha_t)\mathrm{I}\big).$$

Proof: Use that $x_1 \sim \mathcal{N}(\mu_1, \sigma_1^2 \mathrm{I})$ and $x_2 \sim \mathcal{N}(\mu_2, \sigma_2^2 \mathrm{I})$ independent implies $x_1 + x_2 \sim \mathcal{N}(\mu_1 + \mu_2, (\sigma_1^2 + \sigma_2^2)\mathrm{I})$. Applying the reparameterization (14.3) twice,

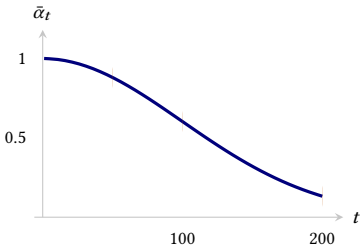
$$\begin{aligned} x_t &= \sqrt{\alpha_t}\big(\sqrt{\alpha_{t-1}} x_{t-2} + \sqrt{1-\alpha_{t-1}}\varepsilon_{t-1}\big) + \sqrt{1-\alpha_t}\,\varepsilon_t \\ &= \sqrt{\alpha_t\alpha_{t-1}}\, x_{t-2} + \underbrace{\sqrt{\alpha_t - \alpha_t\alpha_{t-1}}\,\varepsilon_{t-1} + \sqrt{1-\alpha_t}\,\varepsilon_t}_{\sim\, \mathcal{N}(\mathbf{0},\ (1-\alpha_t\alpha_{t-1})\mathrm{I})}, \end{aligned}$$

ms. p.61 so $x_t \sim \mathcal{N}(\sqrt{\alpha_t\alpha_{t-1}} x_{t-2}, (1-\alpha_t\alpha_{t-1})\mathrm{I})$. Induction gives the claim. ∎

Let us see this geometrically. Proposition 14.5.1 says $q(x_t \mid x_0)$ is Gaussian with a mean that shrinks and a variance that grows. Applying it to a bimodal data distribution shows what the forward process actually destroys, and how slowly.



(a) $q(\boldsymbol{x}_t \mid \boldsymbol{x}_0)$ marginalized over the data.



(b) The signal fraction $\bar{\alpha}_t$.

Figure 14.4: **The forward process, and how fast it forgets. (a)** A bimodal data distribution pushed through $q(\boldsymbol{x}_t \mid \boldsymbol{x}_0)$ of Proposition 14.5.1 with a linear schedule, β_t from 10^{-4} to 0.02 over $T = 200$ steps. The two modes are still clearly separate at $t = 50$, merging at $t = 100$, and gone at $t = 200$, where the density is nearly $\mathcal{N}(0, \mathrm{I})$. **(b)** The reason: $\bar{\alpha}_t = \prod_{i \le t} \alpha_i$ is the fraction of the original signal surviving, and it decays smoothly rather than abruptly. That gradualness is the design: each reverse step has only a little structure to restore, so $p_\theta(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t)$ can be a simple Gaussian and still be accurate — which is what makes the denoising-matching terms of (14.4) tractable.

14.5.1 Decomposing the ELBO

Starting from

$$\mathrm{ELBO} = \mathbb{E}_{q(\boldsymbol{x}_{1:T} \mid \boldsymbol{x}_0)} \left[\log \frac{p(\boldsymbol{x}_{0:T})}{q(\boldsymbol{x}_{1:T} \mid \boldsymbol{x}_0)} \right]$$

and applying Bayes' rule in the form

$$q(\boldsymbol{x}_t \mid \boldsymbol{x}_{t-1}, \boldsymbol{x}_0) = \frac{q(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t, \boldsymbol{x}_0)\, q(\boldsymbol{x}_t \mid \boldsymbol{x}_0)}{q(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_0)},$$

the telescoping sum collapses and yields

$$\begin{aligned} \mathrm{ELBO} = \underbrace{-D_{\mathrm{KL}}(q(\boldsymbol{x}_T \mid \boldsymbol{x}_0) \,\|\, p(\boldsymbol{x}_T))}_{\text{prior matching}} + \underbrace{\mathbb{E}_{q(\boldsymbol{x}_1 \mid \boldsymbol{x}_0)} \left[\log p_\theta(\boldsymbol{x}_0 \mid \boldsymbol{x}_1) \right]}_{\text{reconstruction}} \\ - \sum_{t=2}^{T} \underbrace{\mathbb{E}_{q(\boldsymbol{x}_t \mid \boldsymbol{x}_0)} \left[D_{\mathrm{KL}}(q(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t, \boldsymbol{x}_0) \,\|\, p_\theta(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t)) \right]}_{\text{denoising matching}}. \end{aligned} \tag{14.4}$$

ms. p.60,61

The posterior $q(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t, \boldsymbol{x}_0)$ is obtained in closed form by completing the square in $\boldsymbol{x}_{t-1}$, giving

$$q(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t, \boldsymbol{x}_0) = \mathcal{N}\left(\boldsymbol{x}_{t-1};\ \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})\boldsymbol{x}_t + \sqrt{\bar{\alpha}_{t-1}}(1 - \alpha_t)\boldsymbol{x}_0}{1 - \bar{\alpha}_t},\ \frac{(1 - \alpha_t)(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} \mathrm{I} \right),$$

so every term of (14.4) is a Gaussian KL and is therefore tractable.

ms. p.61

Note:-

There is an *earlier* decomposition, obtained by telescoping without first applying Bayes' rule, in which the middle term is a *consistency* term $D_{\mathrm{KL}}(q(\boldsymbol{x}_t \mid \boldsymbol{x}_{t-1}) \,\|\, p_\theta(\boldsymbol{x}_t \mid \boldsymbol{x}_{t+1}))$: a denoising step from a noisier state should match the noising step from a cleaner one. It is correct, but worse. That term is an expectation over *two* random variables $\{\boldsymbol{x}_{t-1}, \boldsymbol{x}_{t+1}\}$, whereas the denoising-matching term of (14.4) involves only $\boldsymbol{x}_t$. By Theorem 5.6.2 the Monte Carlo variance of the estimate scales with $\mathrm{Var}[g]$, and summing $T - 1$ high-variance terms is ruinous for large T. Conditioning on $\boldsymbol{x}_0$ is what buys the lower-variance form.

Key idea

Fixing the encoder is what makes diffusion models work. Because q carries no parameters, $q(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t, \boldsymbol{x}_0)$ is known analytically, and the denoising-matching term reduces to a closed-form Gaussian KL. All the learning is concentrated

in p_θ.

§ 14.6 What the network actually learns

Equation (14.4) leaves the denoising-matching term as a KL divergence between two distributions we have not yet written down. Doing so collapses it to something startlingly simple.

14.6.1 The KL reduces to matching means

Both arguments of the KL are Gaussian. The target $q(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t, \boldsymbol{x}_0)$ was computed by completing the square above, with mean and variance

$$\boldsymbol{\mu}_q(\boldsymbol{x}_t, \boldsymbol{x}_0) = \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})\,\boldsymbol{x}_t + \sqrt{\bar{\alpha}_{t-1}}(1-\alpha_t)\,\boldsymbol{x}_0}{1-\bar{\alpha}_t}, \qquad \sigma_q^2(t) = \frac{(1-\alpha_t)(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}. \tag{14.5}$$

Every quantity in $\sigma_q^2(t)$ is a fixed schedule coefficient, so we may give the model $p_\theta(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t)$ that *same* variance and leave only its mean $\boldsymbol{\mu}_\theta(\boldsymbol{x}_t, t)$ to be learned.

Proposition 14.6.1 Equal-covariance Gaussian KL

For $\Sigma_x = \Sigma_y = \sigma^2 \mathrm{I}$,

$$D_{\mathrm{KL}}\Big(\mathcal{N}(\boldsymbol{\mu}_x, \sigma^2\mathrm{I}) \,\|\, \mathcal{N}(\boldsymbol{\mu}_y, \sigma^2\mathrm{I})\Big) = \frac{1}{2\sigma^2}\|\boldsymbol{\mu}_x - \boldsymbol{\mu}_y\|^2.$$

The general Gaussian KL carries a log-determinant, a trace, and a dimension term; with matched covariances all three cancel, leaving a squared distance between means. So the denoising-matching term is

$$\arg\min_\theta \; D_{\mathrm{KL}}(q(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t, \boldsymbol{x}_0) \,\|\, p_\theta(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t)) = \arg\min_\theta \; \frac{1}{2\sigma_q^2(t)}\|\boldsymbol{\mu}_\theta(\boldsymbol{x}_t, t) - \boldsymbol{\mu}_q(\boldsymbol{x}_t, \boldsymbol{x}_0)\|^2. \tag{14.6}$$

Key idea

An objective that began as a KL divergence between distributions over sequences of length T has become an L_2 loss between two vectors — the same L_2 loss as Chapter 3. Everything from here is a matter of *what* we choose to have the network output.

14.6.2 Three equivalent objectives

Comparing (14.6) with (14.5): the only term in $\boldsymbol{\mu}_q$ that the network cannot already see is $\boldsymbol{x}_0$. So set

$$\boldsymbol{\mu}_\theta(\boldsymbol{x}_t, t) = \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})\,\boldsymbol{x}_t + \sqrt{\bar{\alpha}_{t-1}}(1-\alpha_t)\,\hat{\boldsymbol{x}}_\theta(\boldsymbol{x}_t, t)}{1-\bar{\alpha}_t},$$

with $\hat{\boldsymbol{x}}_\theta$ a network predicting the clean image. Substituting into (14.6), everything cancels except the $\boldsymbol{x}_0$ terms:

$$\boxed{\arg\min_\theta \; \tfrac{1}{2}\big(\mathrm{SNR}(t-1) - \mathrm{SNR}(t)\big)\|\hat{\boldsymbol{x}}_\theta(\boldsymbol{x}_t, t) - \boldsymbol{x}_0\|^2} \tag{14.7}$$

where the *signal-to-noise ratio* of Proposition 14.5.1 is

$$\mathrm{SNR}(t) \coloneqq \frac{\bar{\alpha}_t}{1-\bar{\alpha}_t}. \tag{14.8}$$

Note:-

The weight in (14.7) is the *drop* in signal-to-noise ratio across one step. Steps that destroy a lot of signal are weighted heavily; steps that destroy little count for little. Figure 14.4b plots $\bar{\alpha}_t$, and SNR is a monotone function of it — requiring

the SNR to decrease monotonically is exactly the third postulate, restated.

There are two other ways to say the same thing. Rearranging Proposition 14.5.1 for $\boldsymbol{x}_0$ and substituting gives a network that predicts the *noise* instead:

$$\boldsymbol{\mu}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t) = \frac{1}{\sqrt{\alpha_t}}\boldsymbol{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}\sqrt{\alpha_t}}\,\hat{\boldsymbol{\varepsilon}}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t), \qquad \arg\min_{\boldsymbol{\theta}} \frac{(1-\alpha_t)^2}{2\sigma_q^2(t)(1-\bar{\alpha}_t)\alpha_t}\|\boldsymbol{\varepsilon}_0 - \hat{\boldsymbol{\varepsilon}}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t)\|^2. \tag{14.9}$$

The third route needs one classical result.

Theorem 14.6.2 Tweedie's formula

For $\boldsymbol{z} \sim \mathcal{N}(\boldsymbol{\mu}_z, \Sigma_z)$,

$$\mathbb{E}[\boldsymbol{\mu}_z \mid \boldsymbol{z}] = \boldsymbol{z} + \Sigma_z \nabla_{\boldsymbol{z}} \log p(\boldsymbol{z}).$$

Applied to $q(\boldsymbol{x}_t \mid \boldsymbol{x}_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}\boldsymbol{x}_0, (1-\bar{\alpha}_t)\mathrm{I})$, whose mean is $\sqrt{\bar{\alpha}_t}\boldsymbol{x}_0$, this gives $\boldsymbol{x}_0 = \big(\boldsymbol{x}_t + (1-\bar{\alpha}_t)\nabla \log p(\boldsymbol{x}_t)\big)/\sqrt{\bar{\alpha}_t}$. Comparing with the reparameterized form $\boldsymbol{x}_0 = (\boldsymbol{x}_t - \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\varepsilon}_0)/\sqrt{\bar{\alpha}_t}$ yields the identity

$$\boxed{\nabla_{\boldsymbol{x}_t} \log p(\boldsymbol{x}_t) = -\frac{1}{\sqrt{1-\bar{\alpha}_t}}\,\boldsymbol{\varepsilon}_0} \tag{14.10}$$

Take-home message

Predicting the image, predicting the noise, and predicting the score are the same task.

By (14.10) the score $\nabla \log p(\boldsymbol{x}_t)$ and the added noise $\boldsymbol{\varepsilon}_0$ differ only by the factor $-1/\sqrt{1-\bar{\alpha}_t}$, and by Proposition 14.5.1 either determines $\boldsymbol{x}_0$. So the three objectives (14.7), (14.9) and the score-matching version differ only in parameterization and in the weight attached to each t. This is the unification of Luo [6]: the variational route of this chapter and the score-based route of the wider literature arrive at one algorithm. In practice $\hat{\boldsymbol{\varepsilon}}_{\boldsymbol{\theta}}$ is usually preferred, on empirical grounds rather than theoretical ones.

Note:-

The score has an independent meaning worth keeping: $\nabla \log p(\boldsymbol{x})$ points in the direction of steepest increase of log-likelihood, so it is a vector field over data space pointing toward the modes. Following it with added noise — *Langevin dynamics* — generates samples without ever computing a normalizing constant, which is why score-based models were developed independently of anything in this chapter.

Note:-

Two sanity checks. Setting $T = 1$ reduces (14.4) to the vanilla VAE objective (14.2), so the diffusion model really does contain the VAE as a special case. And only the Markov property was used in deriving (14.4), so that decomposition holds for *any* Markovian HVAE, not just for the Gaussian one.

§ 14.7 Exercises

The code problems use MNIST via `fetch_openml` and the `make_moons` generator (Section C.5). These are the most computationally demanding exercises in the notes; request a GPU runtime (Section C.3).

Warm-up

1. [Theory] Using Figure 14.1, state in one sentence what an autoencoder cannot do that a VAE can, and identify the single structural change responsible.

2. [Theory] Compute $D_{\mathrm{KL}}\big(\mathcal{N}(\boldsymbol{\mu}, \sigma^2\mathrm{I}) \,\|\, \mathcal{N}(\mathbf{0}, \mathrm{I})\big)$ in closed form. Verify it is zero exactly when $\boldsymbol{\mu} = \mathbf{0}$ and $\sigma = 1$, and hence write the VAE loss (14.2) with no expectations remaining except the reconstruction term.

3. [Theory] In Figure 14.2 the evidence is drawn as a horizontal line. Explain why it does not depend on $\boldsymbol{\phi}$, and what that implies about the effect of training the encoder.

4. [Theory] Verify that $\boldsymbol{z} = \boldsymbol{\mu} + \sigma\boldsymbol{\varepsilon}$ with $\boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \mathrm{I})$ has distribution $\mathcal{N}(\boldsymbol{\mu}, \sigma^2\mathrm{I})$. Which node of Figure 14.3b carries no parameters, and why does that matter?

5. [Theory] From Figure 14.4b, read off $\bar{\alpha}_t$ at $t = 100$ and compute the standard deviation of $q(\boldsymbol{x}_t \mid \boldsymbol{x}_0)$ there. At roughly what t is half the original signal gone?

6. [Theory] Compute $\mathrm{SNR}(t)$ from (14.8) at $t = 50, 100, 200$ using the $\bar{\alpha}_t$ values of Figure 14.4b. Verify it decreases monotonically, and say which postulate that enforces.

7. [Code] Implement the forward process of Proposition 14.5.1 for a one-dimensional bimodal dataset with a linear β_t schedule, and reproduce Figure 14.4a by histogramming samples at $t = 0, 50, 100, 200$.

Core

8. [Theory] Give both proofs of Theorem 14.2.1 in full, and show explicitly that the second implies the first.

9. [Theory] Derive (14.2) from Theorem 14.2.1 using $p(\boldsymbol{x}, \boldsymbol{z}) = p_{\boldsymbol{\theta}}(\boldsymbol{x} \mid \boldsymbol{z})p(\boldsymbol{z})$, and interpret each of the two terms. Which one would be optimized by an ordinary autoencoder, and what does the other add?

10. [Theory] Explain precisely why sampling breaks differentiability with respect to $\boldsymbol{\phi}$ but not with respect to $\boldsymbol{\theta}$. Why does the decoder need no analogue of the reparameterization trick?

11. [Theory] Prove Proposition 14.5.1 by induction, in full.

12. [Theory] Derive $q(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t, \boldsymbol{x}_0)$ by completing the square, and confirm the mean and variance stated in the text. Why is it essential that this posterior be available in closed form?

13. [Theory] Show that as $t \to T$ with $\alpha_t \to 0$, $q(\boldsymbol{x}_T \mid \boldsymbol{x}_0)$ approaches $\mathcal{N}(\mathbf{0}, \mathrm{I})$, so the prior-matching term of (14.4) vanishes and carries no learnable parameters. Check against Figure 14.4b.

14. [Theory] Prove Proposition 14.6.1 from the general Gaussian KL formula, showing that the log-determinant, trace, and dimension terms cancel when the covariances agree. Why are we entitled to give $p_{\boldsymbol{\theta}}$ the same variance as q?

15. [Theory] Substitute the parameterization of $\boldsymbol{\mu}_{\boldsymbol{\theta}}$ into (14.6) and verify (14.7), including the SNR weight. Which steps t does that weight emphasize, and does that match your intuition about where the hard work lies?

16. [Code] Implement a VAE in JAX on MNIST with a two-dimensional latent space. Plot the latent codes of the test set colored by digit class, and generate samples by decoding a grid of points from $\mathcal{N}(\mathbf{0}, \mathrm{I})$. Compare reconstructions against a plain autoencoder of identical architecture.

Challenge

17. [Theory] The ELBO appeared once before, as (9.5) in Chapter 9. State precisely what plays the role of $q_{\boldsymbol{\phi}}(\boldsymbol{z} \mid \boldsymbol{x})$ there, and explain why EM can set the gap to *zero* at every iteration while a VAE cannot. What is the VAE buying in exchange?

18. [Theory] Carry out the telescoping argument leading to (14.4), showing where each of the three terms comes from.

19. [Theory] Show that a VAE with linear encoder and decoder and vanishing noise reduces to PCA, connecting back to Chapter 13. What does the prior-matching term become in that limit?

20. [Theory] In the VAE loss (14.2) the two terms compete. Show that if the decoder is powerful enough to reconstruct $\boldsymbol{x}$ without using $\boldsymbol{z}$, the optimum drives $q_{\boldsymbol{\phi}}(\boldsymbol{z} \mid \boldsymbol{x}) \to p(\boldsymbol{z})$ and the latent code carries no information. (This is *posterior collapse*.) What does Figure 14.2 look like when it happens?

21. [Theory] Derive (14.9) from (14.7) by eliminating $\boldsymbol{x}_0$ with Proposition 14.5.1, and confirm the stated weight. Then prove (14.10) using Theorem 14.6.2, and hence that all three objectives are reparameterizations of one another.

22. [Theory] (14.7) and (14.9) carry different t-dependent weights. Show that dropping both weights — training with an unweighted $\|\boldsymbol{\varepsilon}_0 - \hat{\boldsymbol{\varepsilon}}_{\boldsymbol{\theta}}\|^2$, as is standard in practice — corresponds to optimizing a *different* objective from the ELBO. What is being traded away, and why might it improve sample quality nonetheless?

23. [Code] Ablate the reparameterization trick: attempt to train the VAE of Exercise 13 by sampling directly, and confirm that the gradients with respect to $\boldsymbol{\phi}$ are wrong or absent. Report what `jax.grad` returns and reconcile it with Figure 14.3a.

24. [Code] Train a denoising diffusion model on two-dimensional toy data from `make_moons`. Visualize the reverse process at several t, confirm the generated distribution matches the data, and plot each of the three terms of (14.4) during training. Then halve T and report which term degrades first.

Chapter A

Linear Algebra and Matrix Calculus Reference

Collected identities used throughout the notes. All are proved in the exercises of the chapter where they first appear.

Gradients

For $\theta, x \in \mathbb{R}^m$ and $A \in \mathbb{R}^{m \times m}$:

$$\begin{aligned}
\nabla_\theta (\theta^\top x) &= x \\
\nabla_\theta (x^\top \theta) &= x \\
\nabla_\theta (\theta^\top A \theta) &= A\theta + A^\top \theta \\
\nabla_\theta (\theta^\top A \theta) &= 2A\theta \qquad (A \text{ symmetric}) \\
\nabla_\theta \|\theta\|^2 &= 2\theta
\end{aligned}$$

Matrix derivatives

For symmetric positive definite Σ:

$$\begin{aligned}
\frac{\partial}{\partial \Sigma} \log\det \Sigma &= \Sigma^{-1} \\
\frac{\partial}{\partial \Sigma} \operatorname{Tr}(A\Sigma^{-1}) &= -\Sigma^{-1} A \Sigma^{-1}
\end{aligned}$$

Positive definiteness

$A \succ 0$ iff $z^\top A z > 0$ for all $z \neq 0$. For any Φ, $\Phi^\top \Phi \succeq 0$ always, and $\Phi^\top \Phi \succ 0$ iff Φ has full column rank.

Gaussian identities

$$\mathcal{N}(x; \mu, \Sigma) = \frac{1}{\sqrt{(2\pi)^d \det \Sigma}} \exp\!\left(-\tfrac{1}{2}(x-\mu)^\top \Sigma^{-1}(x-\mu)\right)$$

$$x_1 \sim \mathcal{N}(\mu_1, \sigma_1^2 \mathrm{I}),\ x_2 \sim \mathcal{N}(\mu_2, \sigma_2^2 \mathrm{I}) \implies x_1 + x_2 \sim \mathcal{N}(\mu_1 + \mu_2, (\sigma_1^2 + \sigma_2^2)\mathrm{I})$$

Singular value decomposition

Any $X \in \mathbb{R}^{d\times n}$ factors as $X = U\Sigma V^{\mathsf{T}}$ with $U^{\mathsf{T}}U = V^{\mathsf{T}}V = \mathrm{I}$ and Σ diagonal with $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$ (Theorem 13.5.1). Consequently

$$XX^{\mathsf{T}} = U\Sigma\Sigma^{\mathsf{T}}U^{\mathsf{T}}, \qquad \text{so} \qquad \lambda_i(XX^{\mathsf{T}}) = \sigma_i^2, \qquad \kappa(X) = \frac{\sigma_1}{\sigma_{\min}}.$$

Eckart–Young. Among all matrices of rank at most r, the truncation $X_r = \sum_{i=1}^{r} \sigma_i \, u^i (v^i)^{\mathsf{T}}$ minimizes both $\|X - \cdot\|_F$ and the spectral norm, with

$$\|X - X_r\|_F^2 = \sum_{i>r} \sigma_i^2.$$

This is why the discarded singular-value energy of Figure 13.6a *is* the reconstruction error.

Chapter B

A JAX Primer

The programming exercises assume familiarity with the four transformations below. Everything runs in Google Colab with `pip install jax optax`.

Automatic differentiation

```
import jax
import jax.numpy as jnp

def loss(theta, Phi, y):
    r = Phi @ theta - y
    return 0.5 * jnp.mean(r ** 2)

grad_loss = jax.grad(loss)        # gradient w.r.t. first argument
hess_loss = jax.hessian(loss)
```

`jax.grad` implements reverse-mode automatic differentiation — the same algorithm derived by hand as backpropagation in Chapter 12.

Just-in-time compilation

```
@jax.jit
def step(theta, Phi, y, alpha):
    return theta - alpha * grad_loss(theta, Phi, y)
```

Vectorization

```
feature_map = lambda x, d: x ** jnp.arange(d + 1)
Phi = jax.vmap(feature_map, in_axes=(0, None))(x, d)
```

Randomness

JAX requires explicit PRNG keys; there is no global random state.

```
key = jax.random.PRNGKey(0)
key, subkey = jax.random.split(key)
noise = jax.random.normal(subkey, shape=(n,))
```

Precision

JAX defaults to `float32`. Several exercises in these notes depend on double precision and will give misleading answers without it:

```
jax.config.update("jax_enable_x64", True)
```

The flag must be set before any array is created. See Section C.3 of Appendix C for why this matters here specifically, and what goes wrong if you forget.

§ B.1 Optimizers with `optax`

`optax` supplies the algorithms of Chapter 10 as composable transformations of a gradient. The pattern is always the same: build an optimizer, initialize its state, then alternate `update` and `apply_updates`.

```
import optax

opt   = optax.adam(learning_rate=1e-3)   # or optax.sgd(1e-2, momentum=0.9)
state = opt.init(theta)

for batch in batches:
    grads          = jax.grad(loss)(theta, batch)
    updates, state = opt.update(grads, state, theta)
    theta          = optax.apply_updates(theta, updates)
```

A decaying schedule satisfying (10.3) is passed in place of the constant learning rate:

```
sched = optax.exponential_decay(1e-2, transition_steps=1000, decay_rate=0.9)
opt   = optax.sgd(sched, momentum=0.9)
```

`optax.chain` composes transformations, which is how gradient clipping and weight decay are added without rewriting the update rule.

Chapter C

The Computing Environment: Google Colab

The programming exercises in these notes are written for JAX running in Google Colab. This appendix explains why, where that choice hurts, and how to set up a notebook so that the numerical results you report are the ones you actually computed.

§ C.1 Why Colab, and where it falls short

Colab is a hosted Jupyter environment: notebooks run on a Google machine, not on yours. For a course like this one the advantages are practical rather than intellectual.

- **Nothing to install.** JAX is preinstalled. Students on Windows, macOS, Linux, and Chromebooks all get an identical environment, and no lecture time is lost to CUDA drivers.
- **Free accelerators.** A GPU is available at no cost, which matters from Chapter 11 onward.
- **Sharing is a link.** Submitting work, and receiving worked solutions, needs no packaging.

Against that, four things are worth knowing before you rely on it.

1. **The runtime is ephemeral.** Sessions are reclaimed after a period of inactivity and have a hard lifetime. Everything in memory — variables, trained parameters, downloaded data — disappears. Only the notebook file survives.
2. **The software stack drifts.** Colab upgrades Python and its preinstalled libraries without notice. A notebook that ran last semester may not run today. Section C.2 shows how to defend against this.
3. **Notebooks permit hidden state.** Cells can be run in any order, so a notebook that appears to work may depend on a variable defined in a cell you have since edited or deleted. This is the single most common source of irreproducible results in computational coursework.
4. **It needs a Google account**, and access is not uniform worldwide. Section C.7 gives a local alternative that runs the same code.

Note:-

None of the exercises in these notes *require* Colab. They require JAX. If you prefer a local installation, a university cluster, or another hosted notebook service, use it — the code is identical.

§ C.2 A reproducible first cell

Begin every notebook for this course with the following. It pins the library versions, turns on double precision, and prints enough information that a result can be traced back to the environment that produced it.

```
# --- COE 371 standard preamble ---------------------------------
# Pin versions so this notebook still runs next semester.
!pip install -q "jax[cpu]==0.4.35" "optax==0.2.3" "ucimlrepo"

import jax, jax.numpy as jnp, numpy as np, platform

# Double precision. Read Section C.3 before removing this line.
jax.config.update("jax_enable_x64", True)

print("python  :", platform.python_version())
print("jax     :", jax.__version__)
print("devices :", jax.devices())
print("x64 on  :", jnp.zeros(1).dtype == jnp.float64)
```

Change the pinned versions only deliberately. If a notebook stops working after Colab updates, the pin is what lets you distinguish *your* bug from *their* upgrade.

§ C.3 Precision: the trap in this course

JAX defaults to `float32`, even on hardware that supports `float64`. This is a sensible default for deep learning, where 32-bit arithmetic is fast and accurate enough. It is the wrong default for numerical linear algebra.

Common pitfall

In `float32` the unit roundoff is about 10^{-7}. A matrix with condition number $\kappa \approx 10^7$ therefore has *no* correct digits in the computed solution — and the Vandermonde design matrices of Chapter 3 reach that condition number by degree 8 or so. If you run the conditioning exercise of Chapter 3 without enabling `x64`, the normal equations will appear to fail far earlier than theory predicts, and you will be measuring JAX's default precision rather than the mathematics.

Enable double precision *before* any array is created:

```
import jax
jax.config.update("jax_enable_x64", True)    # must come first
import jax.numpy as jnp
jnp.zeros(3).dtype                           # -> float64
```

Setting the flag after arrays exist leaves those arrays in `float32`, and the resulting mixture is very hard to debug.

Homework. Run the conditioning experiment of Chapter 3 twice, once with `x64` enabled and once without, and plot the coefficient error against polynomial degree for both. The gap between the two curves is the unit roundoff of `float32`, made visible.

§ C.4 When you need an accelerator

Most of this course does not need a GPU. Chapters 1 through 9 involve linear algebra on datasets of a few thousand rows; a CPU runtime is faster in practice, because it avoids the cost of moving data to the device.

Switch to a GPU (Runtime → Change runtime type) only from Chapter 11 onward, and in particular for the network training in Chapters 12 and 14. Free GPU access is rationed, so spending it on a 9568×4 least-squares problem is a poor trade.

Note:-

If you select a TPU runtime, choose TPU v2 or later; the older TPU runtime is deprecated and JAX will silently fall back to the CPU. Verify with `jax.devices()` rather than assuming.

§ C.5 Getting the course datasets

Every dataset used in these notes is public, and all of them load in a few lines. There is no need to download anything by hand.

scikit-learn: the three families

`sklearn.datasets` is the workhorse. Its functions come in three families, distinguished by prefix, and knowing which is which saves confusion.

Prefix	Behavior	Example
`load_*`	bundled with the install; works offline	`load_digits()`
`fetch_*`	downloaded on first call, then cached	`fetch_california_housing()`
`make_*`	generated on demand from a random seed	`make_moons()`

scikit-learn is preinstalled in Colab, so no `pip install` is needed.

What a loader returns

Both `load_*` and `fetch_*` return a `Bunch`, which is a dictionary that also permits attribute access. For the California housing data used in Chapter 1:

```
from sklearn.datasets import fetch_california_housing

d = fetch_california_housing()
d.data.shape        # (20640, 8)   float64
d.target.shape      # (20640,)     float64
d.feature_names     # ['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms',
                    #  'Population', 'AveOccup', 'Latitude', 'Longitude']
print(d.DESCR)      # provenance, units, caveats -- read this
```

Read DESCR at least once per dataset. It records units and known defects. For this one, for instance, the target is capped at 5.0, so the largest values are censored rather than measured — the kind of fact that changes how you interpret a residual plot.

If you would rather have a `pandas` frame, ask for one:

```
df = fetch_california_housing(as_frame=True).frame   # features + target
```

Handing the data to JAX

scikit-learn returns `numpy` arrays in `float64`. JAX will *silently* demote them:

```
jnp.asarray(d.data).dtype     # float32  -- if x64 is not enabled
jnp.asarray(d.data).dtype     # float64  -- if it is
```

Common pitfall

No warning is issued. You lose eight significant digits between `sklearn` and JAX, and nothing on screen says so. This is the same trap as Section C.3, arriving by a different route: enable `x64` in your first cell and it cannot happen.

Synthetic data

The `make_*` generators are useful when you want to control the answer rather than discover it — for testing an implementation against a known truth. They will appear from Chapter 9 onward:

```
from sklearn.datasets import make_regression, make_blobs, make_moons

X, y = make_regression(n_samples=200, n_features=1, noise=12.0, random_state=0)
X, y = make_blobs(n_samples=300, centers=3, cluster_std=0.9, random_state=0)
X, y = make_moons(n_samples=300, noise=0.08, random_state=0)
```

`make_blobs` produces exactly the picture of Figure 1.2a, and is the natural test case for the Gaussian mixture of Chapter 9. `make_moons` produces clusters that are *not* blobs, which is how you discover that a method assuming elliptical classes has limits. Always pass `random_state` so your figure is reproducible.

Splitting the data

```
from sklearn.model_selection import train_test_split
Xtr, Xte, ytr, yte = train_test_split(d.data, d.target,
                                      test_size=0.2, random_state=0)
```

Several exercises ask you to hold data out; this is how. Fit on the training part only, and touch the test part once, at the end.

Datasets not in scikit-learn

```
# UCI repository -- Airfoil Self-Noise (id 291), Power Plant (id 294)
!pip install -q ucimlrepo
from ucimlrepo import fetch_ucirepo
airfoil = fetch_ucirepo(id=291)
X = airfoil.data.features.to_numpy()
y = airfoil.data.targets.to_numpy().ravel()

# Palmer Penguins, via seaborn (downloads from GitHub on first call)
import seaborn as sns
pen = sns.load_dataset("penguins").dropna()   # 333 rows x 7 columns

# Anything on OpenML, by name
from sklearn.datasets import fetch_openml
mnist = fetch_openml("mnist_784", version=1, as_frame=False)
```

Note:-

Because the Colab runtime is ephemeral, the cache in the home directory is discarded with the session, so `fetch_*` re-downloads each time you connect. That is a few seconds for the sets used here. For anything larger, cache it on your Drive as shown below.

Common pitfall

Older tutorials and textbooks load the Boston housing data with `load_boston()`. It was removed from scikit-learn in version 1.2 for documented ethical reasons: one of its engineered variables encodes an assumption about racial segregation and house prices. Calling it now raises an `ImportError` carrying that explanation. Use the California housing or Ames housing data instead. If you find course material online that depends on it, that material predates 2023 and may be stale in other ways too.

Caching on Google Drive

```
from google.colab import drive
drive.mount('/content/drive')
# then read from and write to /content/drive/MyDrive/coe371/
```

§ C.6 Reproducibility

Two habits, both cheap, that make your results defensible.

Restart and run all before you submit. Runtime → Restart and run all. If the notebook does not run top to bottom in a fresh kernel, it is not a result; it is an anecdote. This single step catches the hidden-state problem described above.

Seed explicitly. JAX has no global random state — every random operation takes an explicit key, which is a feature, not an inconvenience. The same key gives the same numbers on any machine.

```
key = jax.random.PRNGKey(0)
key, sub = jax.random.split(key)
noise = jax.random.normal(sub, (n,))
```

Report the seed alongside any number you quote. See also the randomness section of Appendix B.

§ C.7 Working locally instead

Nothing here depends on Colab. To run the same exercises on your own machine:

```
python -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate
pip install "jax[cpu]==0.4.35" optax ucimlrepo scikit-learn matplotlib jupyterlab
jupyter lab
```

A local installation gives you a persistent filesystem, real version control (notebooks diff badly; consider writing `.py` files and importing them), and no session limits. It costs you a GPU, which matters only for the last four chapters.

[TODO: Confirm whether students have TACC allocations available through the department; if so, add a short section here, since a cluster is a better fit than Colab for the reduced-order modeling work in Chapter 13.]

§ C.8 What to submit

Unless stated otherwise, submit a single notebook per problem set containing, for each problem: the code, the output it produced, and a short written answer interpreting that output. A plot with no sentence attached is an incomplete answer — the exercises ask what the picture *means*, not merely that you produced it.

Before submitting: restart and run all, confirm the version banner from Section C.2 appears at the top, and check that every figure has axis labels.

Bibliography

[1] Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David Duvenaud. Neural ordinary differential equations. In *Advances in Neural Information Processing Systems 31*, 2018.

[2] Yann N. Dauphin, Harm de Vries, and Yoshua Bengio. Equilibrated adaptive learning rates for non-convex optimization. In *Advances in Neural Information Processing Systems 28*, pages 1504–1512, 2015.

[3] Eldad Haber and Lars Ruthotto. Stable architectures for deep neural networks. *Inverse Problems*, 34(1):014004, 2018.

[4] Frederik Kunstner, Philipp Hennig, and Lukas Balles. Limitations of the empirical Fisher approximation for natural gradient descent. In *Advances in Neural Information Processing Systems 32*, 2019.

[5] Yiping Lu, Aoxiao Zhong, Quanzheng Li, and Bin Dong. Beyond finite layer neural networks: Bridging deep architectures and numerical differential equations. In *Proceedings of the 35th International Conference on Machine Learning*, 2018.

[6] Calvin Luo. Understanding diffusion models: A unified perspective. *arXiv preprint arXiv:2208.11970*, 2022.

[7] Matthew D. Zeiler. ADADELTA: An adaptive learning rate method. *arXiv preprint arXiv:1212.5701*, 2012.